

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO THỰC TẬP CƠ SỞ

**ĐỀ TÀI: Xây dựng hệ thống tìm kiếm đồ thất lạc
BeacondFound**

Giảng viên hướng dẫn: TS.ĐỖ THỊ LIÊN

**Sinh viên thực hiện : NGUYỄN HỮU NIÊM
NGUYỄN ANH TRƯỜNG**

**Mã sinh viên : B23DCCE076
B23DCCE094**

Hà Nội - 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới TS. Đỗ Thị Liên, người đã tận tình hướng dẫn chúng em thực hiện đề tài “Xây dựng hệ thống tìm kiếm đồ thất lạc BeacondFound” trong học phần Thực tập cơ sở. Trong suốt quá trình thực hiện đề tài, cô đã định hướng, góp ý và hỗ trợ chúng em hoàn thiện nội dung nghiên cứu, từ việc xác định bài toán, phân tích yêu cầu đến thiết kế hệ thống.

Chúng em cũng xin cảm ơn Học viện Công nghệ Bưu chính Viễn thông và Khoa Công nghệ Thông tin 1 đã tạo điều kiện để chúng em có cơ hội vận dụng những kiến thức đã học vào một đề tài mang tính thực tiễn. Thông qua quá trình thực hiện, chúng em đã học hỏi thêm nhiều kinh nghiệm trong việc khảo sát, phân tích nghiệp vụ, thiết kế cơ sở dữ liệu, mô hình hóa UML và lựa chọn công nghệ phù hợp cho một hệ thống phần mềm.

Mặc dù nhóm đã cố gắng làm việc nghiêm túc và hoàn thành báo cáo trong khả năng tốt nhất, do thời gian thực hiện có hạn và kinh nghiệm thực tế còn hạn chế, bài báo cáo chắc chắn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đánh giá và góp ý của cô để bài làm được hoàn thiện hơn, đồng thời giúp chúng em tích lũy thêm kinh nghiệm cho các học phần và công việc sau này.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	1
MỤC LỤC	2
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT	5
DANH MỤC CÁC BẢNG	6
DANH MỤC CÁC HÌNH VẼ	7
BẢNG PHÂN CHIA CÔNG VIỆC	10
LỜI MỞ ĐẦU	11
Chương 1: TỔNG QUAN VỀ HỆ THỐNG TÌM KIẾM ĐỒ THẤT LẠC	12
1.1: Xác định yêu cầu:	12
- Mục tiêu và phạm vi nghiên cứu	12
1.2. Khảo sát các sản phẩm tương tự:	13
1.3. Mô tả hệ thống bằng ngôn ngữ tự nhiên	14
Người dùng và chức năng của người dùng:	14
Thông tin các đối tượng cần xử lý:	14
Quan hệ giữa các đối tượng cần xử lý:	15
Mô tả nghiệp vụ chi tiết của các chức năng:	16
1.4. Mô hình nghiệp vụ bằng UML:	18
1) Danh sách actor:	18
2) Các chức năng liên quan đến các actor:	18
3) Sơ đồ usecase tổng quan:	20
4) Phân rã chi tiết các usecase:	22
1.5. Thiết kế tương tác với sản phẩm:	33
Chương 2: NGHIÊN CỨU PHƯƠNG PHÁP TIẾP CẬN VÀ GIẢI QUYẾT VẤN ĐỀ	34
2.1. Mô hình tổng quát hệ thống:	34
2.2. Phương pháp xây dựng phần mềm	36
2.3. Mô hình phát triển phần mềm	37
2.4. Kiến trúc phần mềm được áp dụng trong triển khai lập trình	38
2.5. Lựa chọn công nghệ phù hợp để triển khai hệ thống	39
Chương 3: PHÂN TÍCH THIẾT KẾ VÀ THỰC NGHIỆM HỆ THỐNG	41
3.1 Các kịch bản	41
a. Kịch bản đăng tải bài viết	41
b. Kịch bản tìm kiếm đồ vật	42
c. Kịch bản trao đổi tin nhắn (Messages)	43
d. Kịch bản quản lý hồ sơ cá nhân	43
e. Kịch bản quản lý thông báo	44
f. Kịch bản báo cáo bài viết/người dùng	45

g. Kịch bản quản lý bài đăng	46
h. Kịch bản quản lý báo cáo vi phạm	47
i. Kịch bản quản lý danh mục	48
j. Kịch bản xem báo cáo thống kê	49
3.2. Trích các lớp thực thể	50
3.3. Phân tích chi tiết từng module	53
a. Chức năng đăng tải bài viết	54
b. Chức năng tìm kiếm đồ vật	57
c. Chức năng trao đổi tin nhắn	60
d. Chức năng quản lý hồ sơ cá nhân	65
e. Chức năng quản lý thông báo	67
f. Chức năng báo cáo bài viết	69
g. Chức năng quản lý bài đăng	71
h. Chức năng quản lý báo cáo vi phạm	73
i. Chức năng quản lý danh mục	76
j. Chức năng xem báo cáo thống kê	78
3.4. Thiết kế các lớp thực thể	81
3.5. Thiết kế cơ sở dữ liệu	82
3.6. Thiết kế giao diện và sơ đồ lớp	85
3.6.1. Chức năng đăng tải bài viết	85
3.6.2. Chức năng tìm kiếm đồ vật	91
3.6.3. Chức năng trao đổi tin nhắn	93
3.6.4. Chức năng quản lý hồ sơ cá nhân	98
3.6.5. Chức năng quản lý thông báo	101
3.6.6. Chức năng báo cáo bài viết:	104
3.6.7. Chức năng quản lý bài đăng	106
3.6.8. Chức năng quản lý báo cáo vi phạm	108
3.6.9. Chức năng quản lý danh mục	111
3.6.10. Chức năng xem báo cáo thống kê:	114
3.7. Thiết kế biểu đồ tuần tự cho các chức năng	116
3.7.1. Chức năng đăng tải bài viết	116
3.7.2. Chức năng tìm kiếm đồ vật	120
3.7.3. Chức năng trao đổi tin nhắn	121
3.7.4. Chức năng quản lý hồ sơ cá nhân	125
3.7.5. Chức năng quản lý thông báo	126
3.7.6. Chức năng báo cáo bài viết/người dùng	128
3.7.7. Chức năng quản lý bài đăng	129
3.7.8. Chức năng quản lý báo cáo vi phạm	133

3.7.9. Chức năng quản lý danh mục (Thêm danh mục)	134
3.7.10. Chức năng xem báo cáo thống kê	137
KẾT LUẬN	139
TÀI LIỆU THAM KHẢO	141
PHỤ LỤC CÀI ĐẶT, TRIỂN KHAI VÀ KIỂM THỬ:	142

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

STT	Ký hiệu / Chữ viết tắt	Ý nghĩa
1	AI	Artificial Intelligence – Trí tuệ nhân tạo
2	API	Application Programming Interface – Giao diện lập trình ứng dụng
3	CSDL	Cơ sở dữ liệu
4	DB	Database – Cơ sở dữ liệu
5	FCM	Firebase Cloud Messaging
6	HTTP	HyperText Transfer Protocol
7	JSON	JavaScript Object Notation
8	RESTful API	Kiến trúc API theo chuẩn REST
9	UI	User Interface – Giao diện người dùng
10	UML	Unified Modeling Language – Ngôn ngữ mô hình hóa thống nhất
11	UC	Use Case – Ca sử dụng
12	MVC	Model – View – Controller
13	CRUD	Create, Read, Update, Delete
14	PostGIS	Tiện ích mở rộng của PostgreSQL hỗ trợ dữ liệu không gian
15	Socket.io	Thư viện hỗ trợ giao tiếp thời gian thực
16	ReactJS	Thư viện JavaScript dùng xây dựng giao diện người dùng
17	Node.js	Môi trường chạy JavaScript phía máy chủ
18	Express.js	Framework xây dựng Backend trên Node.js
19	Leaflet.js	Thư viện bản đồ mã nguồn mở
20	Cloudinary	Dịch vụ lưu trữ và quản lý hình ảnh

DANH MỤC CÁC BẢNG

STT	Tên bảng	Trang
Bảng 1.1	So sánh ưu nhược điểm của 3 nền tảng tìm đồ thất lạc: Nhóm Facebook, iLost, ReturnMe	13
Bảng 1.2	Danh sách actor gián tiếp và use case hỗ trợ	19
Bảng 3.1	Bảng minh họa kết quả tìm kiếm theo danh sách	42
Bảng 3.2	Bảng minh họa danh sách bài đã đăng	44
Bảng 3.3	Bảng minh họa danh sách thông báo	45
Bảng 3.4	Bảng minh họa danh sách bài đăng chờ duyệt	46
Bảng 3.5	Bảng minh họa danh sách báo cáo vi phạm	47
Bảng 3.6	Bảng minh họa danh sách danh mục	48
Bảng 3.7	Bảng minh họa số liệu thống kê theo tuần	50
Bảng 3.8	Danh sách các bảng dữ liệu trong cơ sở dữ liệu hệ thống	82

DANH MỤC CÁC HÌNH VẼ

STT	Tên hình	Trang
Hình 1.1	Sơ đồ use case tổng quan của hệ thống	21
Hình 1.2	Sơ đồ use case đăng tải bài viết	23
Hình 1.3	Sơ đồ use case tìm kiếm đồ vật	24
Hình 1.4	Sơ đồ use case trao đổi thông tin	25
Hình 1.5	Sơ đồ use case quản lý hồ sơ	26
Hình 1.6	Sơ đồ use case quản lý thông báo	27
Hình 1.7	Sơ đồ use case báo cáo bài viết/người dùng	28
Hình 1.8	Sơ đồ use case quản lý bài đăng	29
Hình 1.9	Sơ đồ use case quản lý danh mục	30
Hình 1.10	Sơ đồ use case xem báo cáo thống kê	31
Hình 1.11	Sơ đồ use case quản lý báo cáo vi phạm	32
Hình 1.12	Thiết kế giao diện và tương tác các chức năng cho người quản trị	33
Hình 1.13	Thiết kế giao diện và tương tác các chức năng cho thành viên	34
Hình 2.1	Mô hình tổng quát hệ thống	35
Hình 3.1	Sơ đồ lớp thực thể của hệ thống	53
Hình 3.2	Biểu đồ lớp chức năng đăng tải bài viết	55
Hình 3.3	Biểu đồ lớp chức năng tìm kiếm đồ vật	58
Hình 3.4	Biểu đồ lớp chức năng trao đổi tin nhắn	62
Hình 3.5	Biểu đồ lớp chức năng quản lý thông báo	68
Hình 3.6	Biểu đồ lớp chức năng xét duyệt bài đăng	72

Hình 3.7	Biểu đồ lớp chức năng quản lý danh mục	77
Hình 3.8	Biểu đồ lớp chức năng xem báo cáo thống kê	79
Hình 3.9	Sơ đồ cơ sở dữ liệu của hệ thống	85
Hình 3.10	Giao diện trang chủ thành viên	86
Hình 3.11	Giao diện đăng bài bước 1	86
Hình 3.12	Giao diện đăng bài bước 2	87
Hình 3.13	Giao diện đăng bài bước 3	87
Hình 3.14	Giao diện đăng bài bước 4	88
Hình 3.15	Giao diện tìm kiếm đồ vật	91
Hình 3.16	Giao diện tìm kiếm theo bản đồ	92
Hình 3.17	Giao diện chi tiết bài đăng	94
Hình 3.18	Giao diện danh sách trò chuyện	95
Hình 3.19	Giao diện chi tiết phòng chat	96
Hình 3.20	Giao diện hồ sơ cá nhân	99
Hình 3.21	Giao diện chỉnh sửa hồ sơ	100
Hình 3.22	Giao diện danh sách thông báo	102

Hình 3.23	Giao diện báo cáo bài viết	104
Hình 3.24	Giao diện chính Admin	106
Hình 3.25	Giao diện quản lý bài đăng	106
Hình 3.26	Giao diện quản lý báo cáo vi phạm	108
Hình 3.27	Giao diện xử lý vi phạm	109
Hình 3.28	Giao diện quản lý danh mục	112
Hình 3.29	Giao diện thêm danh mục	113
Hình 3.30	Giao diện xem báo cáo thống kê	115

BẢNG PHÂN CHIA CÔNG VIỆC

Nguyễn Hữu Niêm	Nguyễn Anh Trường
Cùng nghiên cứu đi tìm hiểu về các ứng dụng, web làm về chủ đề tìm kiếm đồ thất lạc, tìm hiểu về tổng quan hệ thống, nghiên cứu phương pháp tiếp cận và giải quyết vấn đề	
Làm các chức năng : Đăng tải bài viết, trao đổi tin nhắn, quản lý thông báo, quản lý bài đăng, quản lý danh mục	Làm các chức năng: Tìm kiếm đồ vật, quản lý hồ sơ cá nhân, báo cáo bài viết, quản lý báo cáo vi phạm, xem báo cáo thống kê

LỜI MỞ ĐẦU

Trong đời sống hằng ngày, việc thất lạc tài sản cá nhân như điện thoại, ví tiền, giấy tờ tùy thân, chìa khóa hay các vật dụng có giá trị là vấn đề xảy ra khá phổ biến, đặc biệt tại những khu vực đông người như trường học, trung tâm thương mại, bến xe, khu dân cư hoặc nơi công cộng. Khi đánh mất đồ, người dùng thường gặp nhiều khó khăn trong việc tìm kiếm do thông tin bị phân tán, thiếu kênh kết nối chính thống với người nhặt được và tiềm ẩn nhiều rủi ro khi phải công khai thông tin cá nhân trên mạng xã hội.

Hiện nay, các phương thức tìm đồ thất lạc phổ biến như đăng bài trong nhóm Facebook, dán thông báo hoặc nhờ người quen chia sẻ vẫn còn nhiều hạn chế. Bài đăng dễ bị trôi, thông tin không được chuẩn hóa, thiếu công cụ lọc theo vị trí, danh mục hoặc thời gian, đồng thời người dùng có thể bị lộ số điện thoại, địa chỉ hoặc gặp rủi ro lừa đảo. Vì vậy, việc xây dựng một nền tảng chuyên biệt để kết nối người mất đồ và người nhặt được một cách nhanh chóng, an toàn và có tổ chức là cần thiết.

Từ thực tế đó, nhóm chúng em lựa chọn đề tài “Xây dựng hệ thống tìm kiếm đồ thất lạc BeacondFound”. Hệ thống hướng đến việc tạo ra một nền tảng web hỗ trợ đăng tin báo mất hoặc nhặt được đồ vật, tìm kiếm theo danh sách và bản đồ, trao đổi thông tin qua tin nhắn nội bộ, nhận thông báo thời gian thực và hỗ trợ quản trị viên kiểm duyệt nội dung. Bên cạnh đó, hệ thống còn tích hợp các công nghệ như bản đồ số, xử lý vị trí và gợi ý thẻ từ hình ảnh nhằm nâng cao hiệu quả tìm kiếm.

Nội dung báo cáo được bố cục thành ba chương chính:

Chương 1: Tổng quan về hệ thống tìm kiếm đồ thất lạc

Giới thiệu bài toán, xác định mục tiêu và phạm vi nghiên cứu, khảo sát các giải pháp tương tự, mô tả yêu cầu hệ thống và xây dựng các mô hình nghiệp vụ ban đầu.

Chương 2: Nghiên cứu phương pháp tiếp cận và giải quyết vấn đề

Trình bày mô hình tổng quát của hệ thống, phương pháp xây dựng phần mềm, mô hình phát triển, kiến trúc triển khai và lựa chọn các công nghệ phù hợp cho hệ thống.

Chương 3: Phân tích thiết kế và thực nghiệm hệ thống

Đi sâu vào phân tích các kịch bản nghiệp vụ, thiết kế lớp thực thể, thiết kế cơ sở dữ liệu, thiết kế giao diện, sơ đồ lớp và các biểu đồ trình tự cho từng chức năng chính.

Sau cùng là phần kết luận, đánh giá kết quả đạt được, hạn chế còn tồn tại và định hướng phát triển hệ thống trong tương lai.

Đặt vấn đề:

Sự cố thất lạc tài sản như thiết bị di động, giấy tờ tùy thân hay ví tiền là một vấn đề nan giải thường gặp tại các khu vực đông người (trường học, trung tâm thương mại, khu dân cư). Người đánh mất không chỉ tốn công sức tìm kiếm mà còn phải đối mặt với rủi ro bảo mật, trong khi người nhặt được lại không có cách thức chính thống nào để tìm lại chủ nhân một cách an toàn.

Thay vì dựa vào các phương pháp truyền thống kém hiệu quả và đầy rủi ro như dán tờ rơi hay đăng tin tản mạn trên mạng xã hội, xã hội hiện đại cần một cách tiếp cận ứng dụng công nghệ mạnh mẽ hơn. Đề tài xây dựng nền tảng quản lý đồ thất lạc ra đời nhằm đáp ứng nhu cầu này: tạo dựng một môi trường phần mềm trung gian chuyên biệt, kết nối nhanh chóng thông tin "mất và nhặt", từ đó tối ưu hóa khả năng hoàn trả tài sản một cách quy chuẩn và an toàn nhất.

Chương 1: TỔNG QUAN VỀ HỆ THỐNG TÌM KIẾM ĐỒ THẤT LẠC

1.1: Xác định yêu cầu:

- **Mục tiêu và phạm vi nghiên cứu**
- + **Về Mục tiêu (Objectives):**
 - Xây dựng một không gian kết nối trực tiếp, hiệu quả và an toàn giữa những người không may đánh rơi đồ vật và những người nhặt được tài sản.
 - Xóa bỏ sự rời rạc, lộn xộn của các bài đăng trên mạng xã hội, qua đó tạo ra một nền tảng chuyên biệt mang tính cộng đồng minh bạch, thao tác dễ dàng nhưng vẫn bảo mật được thông tin liên lạc cá nhân.
- + **Về Phạm vi (Scope):**
 - Nền tảng & Triển khai: Ứng dụng được xây dựng trên nền tảng web (web-based) mang tính mở, không bị giới hạn hoạt động trong bất kỳ tổ chức hay cộng đồng khép kín nào. Bất kỳ ai có mạng internet đều có thể tiếp cận không giới hạn qua trình duyệt mà không cần cài đặt.
 - Đối tượng sử dụng: Mạng lưới mở dành cho mọi đối tượng người dùng cuối tiếp cận để tìm kiếm/tạo bài đăng. Song song đó, hệ thống sẽ được duy trì và giám sát bởi một hệ thống Ban quản trị (Admin) duy nhất nhằm quản lý nội dung và xử lý vi phạm.
 - Quản lý dữ liệu: Dù người dùng truy cập từ mọi lúc, mọi nơi, qua nhiều máy tính hoặc điện thoại khác nhau, toàn bộ dữ liệu (thông tin người dùng, chi tiết bài báo cáo, dữ liệu tương tác) đều được đồng bộ và lưu trữ bảo mật tại một máy chủ (server) quản lý duy nhất.

1.2. Khảo sát các sản phẩm tương tự:

Sau khi tham khảo 3 nền tảng/phương thức thường được sử dụng để tìm đồ thất lạc gồm: Nhóm cộng đồng Facebook, ứng dụng iLost và hệ thống ReturnMe, em có các nhận xét về ưu nhược điểm như sau:

Bảng 1.1: So sánh ưu nhược điểm của 3 nền tảng tìm đồ thất lạc: Nhóm Facebook, iLost, ReturnMe.

Nền tảng	Ưu điểm	Nhược điểm
Nhóm Facebook	+ Số lượng người dùng lớn, khả năng chia sẻ thông tin rộng rãi. + Miễn phí, thao tác quen thuộc.	+ Dễ bị trôi bài viết. + Rủi ro bảo mật cao, người dùng công khai thông tin dễ bị lừa đảo giả danh.
iLost	+ Có giao diện tối ưu cho nghiệp vụ tìm lại đồ. + Hệ thống tìm kiếm theo phân loại thuộc tính rõ ràng.	+ Rào cản ngôn ngữ, chưa phổ biến ở Việt Nam. + Phù hợp với doanh nghiệp quy mô lớn thay vì cho cá nhân nhỏ lẻ.
ReturnMe	+Tính bảo mật cao. +Sử dụng hệ thống tem nhãn vật lý (Tag/QR code) gắn sẵn lên đồ vật. +Quy trình hoàn trả rõ ràng.	+Phải trả phí dịch vụ lớn để mua tem nhãn và phí hoàn trả. +Cần cài đặt và dán nhãn từ trước.

=> Từ các ưu, nhược điểm của các nền tảng trên chúng em rút ra những yếu tố cần phải có ở sản phẩm của mình là:

- Nền tảng tập trung vào tìm và trả đồ cho người đánh mất.
- Giao diện thân thiện, dễ tiếp cận.
- Có chức năng tìm kiếm linh hoạt, định vị khoanh vùng được đồ vật thông qua Bản đồ số.
- Đề cao tính bảo mật thông tin (tích hợp hộp thư riêng tư thay vì công khai số điện thoại).
- Đa dạng thuộc tính phân loại (danh mục & Tags).

1.3. Mô tả hệ thống bằng ngôn ngữ tự nhiên

Người dùng và chức năng của người dùng:

Hệ thống bao gồm 2 nhóm người dùng chính:

Người dùng cuối (Thành viên/Khách) và Người quản trị (Admin).

Người dùng cuối có thể thực hiện các chức năng:

+ Khách:

- Đăng ký: người dùng chưa có tài khoản cần đăng ký để truy cập vào hệ thống, đăng ký thông qua email hoặc số điện thoại.

+ Thành viên:

- Đăng tải bài viết: thông qua quy trình 4 bước gồm chọn loại bài đăng, mô tả đồ vật, ghim vị trí, và tải hình ảnh lên.
- Tìm kiếm đồ vật: tìm theo danh sách mở rộng (lọc theo loại đồ, trạng thái, thời gian) hoặc tìm kiếm trực quan bằng Bản đồ số (Map Search).
- Trao đổi thông tin: sử dụng hệ thống Tin nhắn trực tiếp (Messages) để liên lạc an toàn mà không lộ thông tin cá nhân.
- Quản lý hồ sơ cá nhân: chỉnh sửa thông tin cá nhân, quản lý các bài đăng của chính mình (sửa đổi, thay đổi trạng thái, xóa), và xem lại các bài đã hoàn tất trao trả.
- Quản lý thông báo: nhận các thông báo/cảnh báo theo thời gian thực về trạng thái bài đăng và phản hồi, tin nhắn mới.
- Báo cáo bài viết & người dùng: báo cáo các bài viết và người dùng lừa đảo.

Người quản trị (Admin) có thể thực hiện các chức năng:

- Quản lý bài đăng: theo dõi, ẩn, xóa hoặc thay đổi trạng thái của các bài viết báo mất/nhặt được có dấu hiệu vi phạm hoặc spam.
- Quản lý danh mục và thẻ: thêm/sửa/xóa các phân loại đồ vật (như Ví tiền, Điện tử, Thú cưng...) và thẻ (Tags).
- Xem báo cáo thống kê: thống kê số lượng truy cập trang web, lượt bài đăng phát sinh mới theo thời gian.
- Quản lý báo cáo vi phạm: xử lý các báo cáo vi phạm từ cộng đồng.

Thông tin các đối tượng cần xử lý:

- Thông tin về bài đăng: tiêu đề, loại bài đăng (Mất / Nhặt được), danh mục, mô tả chi tiết, thời gian, các thẻ (Tags), vị trí liên quan (địa chỉ, tọa độ định vị trên

bản đồ), hình ảnh đính kèm (tối đa 3 ảnh), và thông tin người đăng (Tên, ảnh đại diện chủ bài đăng).

- Thông tin về danh mục (Category): tên danh mục.
- Thông tin về người dùng: họ tên, email hoặc số điện thoại, mật khẩu, ảnh đại diện, chức vụ/quyền hạn, và số bài đăng.
- Thông tin về tin nhắn nội bộ: người gửi, người nhận, nội dung văn bản đoạn chat, thời gian nhắn.
- Thông tin về thông báo: loại sự kiện (trạng thái bài đăng/ tin nhắn), nội dung, trạng thái đọc/chưa đọc.
- Thông tin về báo cáo vi phạm: người gửi báo cáo, nội dung vi phạm, bài đăng liên quan (nếu có), trạng thái xử lý của Admin.
- Thông tin thống kê hệ thống: tổng số lượng bài viết đang hoạt động, lưu lượng người dùng theo tuần/tháng.

Quan hệ giữa các đối tượng cần xử lý:

- Trong hệ thống có nhiều danh mục lưu trữ (category), mỗi bài đăng hoặc không thuộc danh mục hoặc chỉ thuộc về một danh mục duy nhất.
- Mỗi danh mục có thể chứa nhiều bài đăng khác nhau được đăng tải tại những thời điểm khác nhau bởi nhiều người dùng.
- Mỗi người dùng (thành viên) có thể tạo nhiều bài đăng (Báo mất/Nhật được) khác nhau tại những thời điểm khác nhau.
- Mỗi bài đăng được tạo ra tính từ thời điểm khởi tạo luôn chỉ thuộc quyền sở hữu độc nhất của một thành viên xác định (người trực tiếp đăng tải).
- Đối với một bài đăng cụ thể, nó có thể được đính kèm cùng lúc nhiều Thẻ (Tags) khác nhau để hỗ trợ thông tin tìm kiếm.
- Khách vãng lai chưa có tài khoản chỉ có thể đăng ký để trở thành Thành viên.
- Một thành viên có thể nhắn tin trao đổi nội bộ nhiều lần, cho nhiều thành viên khác nhau để xác nhận thông tin về món đồ.
- Một thành viên có thể tự do chỉnh sửa, cập nhật trạng thái bài đăng của mình cho đến khi nào xóa bài đăng.
- Thành viên có thể gửi báo cáo vi phạm đối với bài viết/người dùng khác nếu có dấu hiệu lừa đảo.
- Một hệ thống thông báo chỉ gửi thông điệp đích danh đến một thành viên xác định khi bài đăng của họ được duyệt/từ chối hoặc có tin nhắn mới.
- Một Quản trị viên có thể theo dõi, xử lý, nhận báo cáo và tiến hành xóa/khóa đối với nhiều bài đăng hoặc nhiều người dùng cùng lúc.
- Một Quản trị viên có thể xem thống kê về bài viết, người dùng.

Mô tả nghiệp vụ chi tiết của các chức năng:

1. Dành cho Người dùng cuối (Thành viên)

- **Chức năng Đăng tải bài viết:** Thành viên click vào nút "Đăng bài mới" trên giao diện -> hệ thống hiển thị quy trình 4 bước -> [Bước 1] Giao diện chọn loại bài đăng hiện lên, thành viên chọn loại "Mất đồ" hoặc "Nhặt được", sau đó có thể tùy chọn Danh mục đồ vật từ dropdown (có thể none) và nhập Tiêu đề, click "Tiếp theo" -> [Bước 2] Giao diện mô tả hiện ra, thành viên nhập mô tả chi tiết nhận dạng, chọn ngày tháng xảy ra sự việc, click "Tiếp theo" -> [Bước 3] Giao diện Bản đồ số hiện ra, thành viên gõ địa chỉ và dùng chuột ghim chốt chính xác vị trí lên bản đồ, click "Tiếp theo" -> [Bước 4] Tải ảnh lên và AI phân tích: Giao diện tải ảnh hiện ra, thành viên tải tối đa 3 ảnh tĩnh. Hệ thống kích hoạt gửi hình qua API trí tuệ nhân tạo (AI) phân tích và bóc tách vật thể -> Tự động trả về các Thẻ (Tags) gợi ý hiển thị lên màn hình -> thành viên có thể xóa bỏ nếu tag sai -> click "Đăng bài" -> Hệ thống lưu bài đăng vào CSDL với trạng thái "Chờ duyệt", hiển thị thông báo "Bài viết đang chờ duyệt" chứ chưa public lên bảng tin.
- **Chức năng Tìm kiếm đồ vật:** Thành viên đang ở màn hình Bảng tin chung -> hệ thống hiển thị thanh ô tìm kiếm văn bản và nút bộ lọc. Nếu tìm theo danh sách: Thành viên gõ từ khóa (hệ thống có thể trích từ khóa để so sánh với các thẻ Tags do AI bóc tách), kết hợp bộ lọc (lọc trạng thái, lọc danh mục) -> hệ thống truy vấn CSDL và load lại kết quả danh sách bài đăng phù hợp. Nếu tìm theo bản đồ: Thành viên click nút "Tìm kiếm qua Bản đồ (Map Search)" -> giao diện đổi sang khung bản đồ lớn, thành viên nhập vị trí trọng tâm và bán kính quét (ví dụ 5km) -> hệ thống tải dữ liệu và cắm các ghim đánh dấu vị trí các bài đăng lên map -> thành viên click vào cái ghim đó để xem thẻ bài đăng chi tiết bên trong.
- **Chức năng Trao đổi thông tin (Messages):** Thành viên lướt bảng tin và ấn xem một bài đăng cụ thể -> chi tiết bài đăng hiện ra với nút "Nhắn tin" cho chủ bài viết -> thành viên click nút "Nhắn tin" -> hệ thống mở ra giao diện phòng chat nội bộ nhắn trực tiếp đến tác giả bài đăng -> thành viên nhập chữ vào ô văn bản bên dưới và ấn enter/Gửi -> hệ thống mã hóa đoạn chat, lưu lại và đồng thời đẩy thông báo sang cho người nhận, hiển thị tin nhắn vừa xong trên khung giao tiếp để 2 bên chat real-time với nhau.
- **Chức năng Quản lý hồ sơ cá nhân:** Thành viên truy cập phần "Hồ sơ cá nhân" -> giao diện hiển thị Profile và lưới Bài đã đăng. Tại đây có thể ấn "Sửa thông tin" cá nhân. Trong lưới bài đăng, click vào một bài cũ -> hiện ra các tùy chọn (Sửa đổi, Xóa) -> [TRƯỜNG HỢP 1] Nếu thành viên click "Sửa đổi", cập nhật mô tả mới rồi ấn Lưu -> hệ thống ghi nhận, tự động đổi trạng thái bài đó từ "Hoạt động" chuyển ngược về "Chờ duyệt" và ẩn khỏi bảng tin để Admin xét duyệt lại. -> [TRƯỜNG HỢP 2] Nếu món đồ đã tìm được chủ nhân hoặc

không cần tìm nữa, thành viên chủ động ấn tùy chọn "Xóa" -> hệ thống popup xác nhận xóa -> ấn Đồng ý -> hệ thống xóa bài vĩnh viễn khỏi toàn mạng lưới.

- **Chức năng Quản lý thông báo:** Thành viên đang online -> mỗi khi có sự kiện mới như có bình luận/tin nhắn từ người khác thì biểu tượng Quả chuông tự động bật thông báo màu đỏ -> thành viên click biểu tượng chuông -> thả xuống danh sách (Dropdown) các cảnh báo đã được kết rải theo thời gian -> thành viên click phần thông báo in đậm (chưa đọc) -> hệ thống xác nhận xử lý đã đọc xong và đưa người dùng trực diện vào phần màn hình chứa hành động đó (như vào phòng chat tương ứng).
- **Chức năng Báo cáo bài viết/người dùng:** Thành viên trong lúc đọc chi tiết bài đăng nghi vấn -> click vào biểu tượng 3 chấm ở góc phải -> chọn "Báo cáo vi phạm" -> hệ thống hiển ra một màn hình form nhỏ yêu cầu nhập mô tả chi tiết -> thành viên nhập thông tin đó và ấn "Gửi báo cáo" -> hệ thống ghi nhận, gửi cho thành viên lời cảm ơn vì đã đóng góp, sau đó đẩy mẫu báo cáo vừa tạo vào hàng chờ Xử lý dưới Database do Admin quản duyệt.

2. Dành cho Người Quản Trị (Admin)

- **Chức năng Quản lý bài đăng:** Admin đăng nhập tài khoản quyền quản trị -> truy cập màn hình "Quản lý Bài đăng" -> Hiện thị danh sách các bài đăng "Chờ duyệt", mỗi dòng có 2 nút "Duyệt" và "Từ chối". Đọc thấy hợp lệ, Admin bấm "Duyệt" -> hệ thống chuyển trạng thái thành "Hoạt động" và cho phép hiển thị lên bảng tin. Nếu nội dung sai, Admin bấm "Từ chối" -> hệ thống Xóa hoàn toàn bài viết đó khỏi CSDL.
- **Chức năng Quản lý Danh mục:** Vẫn ở màn quản trị, Admin click tab "Quản lý Danh mục" -> Hiện thị danh sách các danh mục -> Admin tùy chọn các thao tác "Thêm/Sửa/Xóa":
 - Nếu admin muốn "Thêm" -> admin chọn nút "+ Thêm" -> form nhập tên danh mục hiện ra -> điền tên ấn Hoàn tất -> hệ thống update vào DB làm nguồn cho người dùng sử dụng.
 - Nếu admin muốn "Sửa" -> chọn danh mục cần sửa -> click nút "Sửa" -> cập nhật lại tên danh mục -> Hệ thống update vào DB.
 - Nếu admin muốn "Xóa" -> chọn danh mục cần xóa -> click nút "Xóa" -> Hệ thống xóa khỏi DB.
- **Chức năng Xem báo cáo thống kê:** Admin ở ngay "Trang chủ Admin (Dashboard)" lúc đăng nhập -> hệ thống tự động tải và hiển thị mặc định 2 box dữ liệu thống kê (Lượng người dùng mới, lượng bài đăng mới) của tháng hiện tại. Admin có thể tùy chọn điều chỉnh công cụ Lọc theo thời gian (ví dụ: tuần, năm) -> hệ thống tải lại dữ liệu. Lúc xuất báo cáo sẽ xuất những thống kê đang hiển thị trên màn hình.
- **Chức năng Quản lý Báo cáo vi phạm:** Admin click tab "Quản lý Báo Cáo" -> giao diện hiển thị danh sách các bài báo cáo -> Admin click xem chi tiết bài

báo cáo (gồm mô tả kèm người dùng/bài viết bị báo cáo) -> Admin đọc xác minh, nếu xét thấy có tội, chọn hành động xử lý "Khóa tài khoản" hoặc "Xóa bài đăng vi phạm" hoặc không và cập nhật trạng thái report là "Đã giải quyết" -> hệ thống khóa tài khoản thủ phạm hoặc xóa bài viết ngay lập tức.

1.4. Mô hình nghiệp vụ bằng UML:

1) Danh sách actor:

Actor trực tiếp:

- **Khách (Guest):** Là người dùng chưa có tài khoản. Actor này có thể thực hiện các chức năng cơ bản như thực hiện quy trình đăng ký để trở thành thành viên.
- **Thành viên (Member):** Đây là đối tượng sử dụng chính, trực tiếp thao tác các chức năng cốt lõi: Đăng tải bài viết, Tìm kiếm (theo danh sách/bản đồ), Trao đổi qua tin nhắn, Quản lý hồ sơ và Báo cáo vi phạm.
- **Quản trị viên (Admin):** Chịu trách nhiệm vận hành và kiểm soát hệ thống thông qua các chức năng: Quản lý bài đăng, Quản lý danh mục, Xử lý vi phạm và Xem thống kê hệ thống.

Actor gián tiếp:

- **Hệ thống Bản đồ (Map & Geocoding):** Cung cấp dữ liệu địa lý và tọa độ để hỗ trợ chức năng Ghim/Tìm trên bản đồ.
- **Dịch vụ AI (AI Vision API):** Tiếp nhận hình ảnh từ quy trình đăng bài để phân tích vật thể và đưa ra các AI gợi ý Tags.
- **Hệ thống Thông báo (Notifications):** Đảm nhiệm việc đẩy các thông điệp đến người dùng khi có sự kiện mới.
- **Nền tảng Socket.io:** Đóng vai trò là actor trung gian đảm bảo tính thời gian thực (real-time) cho các chức năng Chat và Gửi thông báo.

2) Các chức năng liên quan đến các actor:

Nhóm Actor trực tiếp :

1. Đối với Actor Khách (Guest):

- **Đăng ký:** Cho phép người dùng cung cấp thông tin cá nhân (Email, số điện thoại, mật khẩu) để tạo tài khoản và trở thành thành viên chính thức.

2. Đối với Actor Thành viên (Member):

- **Đăng tải bài viết:** Chức năng cốt lõi giúp người dùng tạo bài báo mất hoặc báo nhật được đồ qua quy trình các bước (nhập thông tin, vị trí, hình ảnh).
- **Tìm kiếm đồ vật:** Ngoài tìm kiếm văn bản, thành viên có thể tìm kiếm nâng cao kết hợp với bộ lọc (danh mục, thời gian) và Ghim/Tìm trên bản đồ.
- **Trao đổi qua tin nhắn:** Kết nối trực tiếp giữa người mất và người nhặt để xác nhận thông tin món đồ qua khung chat nội bộ.
- **Quản lý hồ sơ:** Cho phép cập nhật thông tin cá nhân và quản lý danh sách các bài đăng của riêng mình (chỉnh sửa, cập nhật trạng thái hoặc xóa bài).
- **Quản lý thông báo:** Tiếp nhận và theo dõi các thông báo về tin nhắn mới hoặc trạng thái phê duyệt bài viết từ hệ thống.
- **Báo cáo bài viết/người dùng:** Gửi yêu cầu phản ánh đến Admin khi phát hiện các bài đăng có dấu hiệu lừa đảo hoặc hành vi không đúng mực.

3. Đối với Actor Quản trị viên (Admin):

- **Quản lý bài đăng:** Kiểm duyệt các bài viết mới từ Thành viên, phê duyệt để hiển thị công khai hoặc xóa bỏ các bài viết vi phạm.
- **Quản lý danh mục, thẻ:** Cập nhật các loại đồ vật (Ví, điện thoại, thú cưng...) và quản lý các từ khóa (Tags) để tối ưu hóa việc tìm kiếm.
- **Quản lý vi phạm:** Tiếp nhận và xử lý các báo cáo từ Thành viên, thực hiện các biện pháp như ẩn bài viết hoặc khóa tài khoản vi phạm.
- **Xem thống kê:** Theo dõi biểu đồ và số lượng bài viết, người dùng mới để đánh giá hiệu quả hoạt động của hệ thống.

Nhóm Actor gián tiếp :

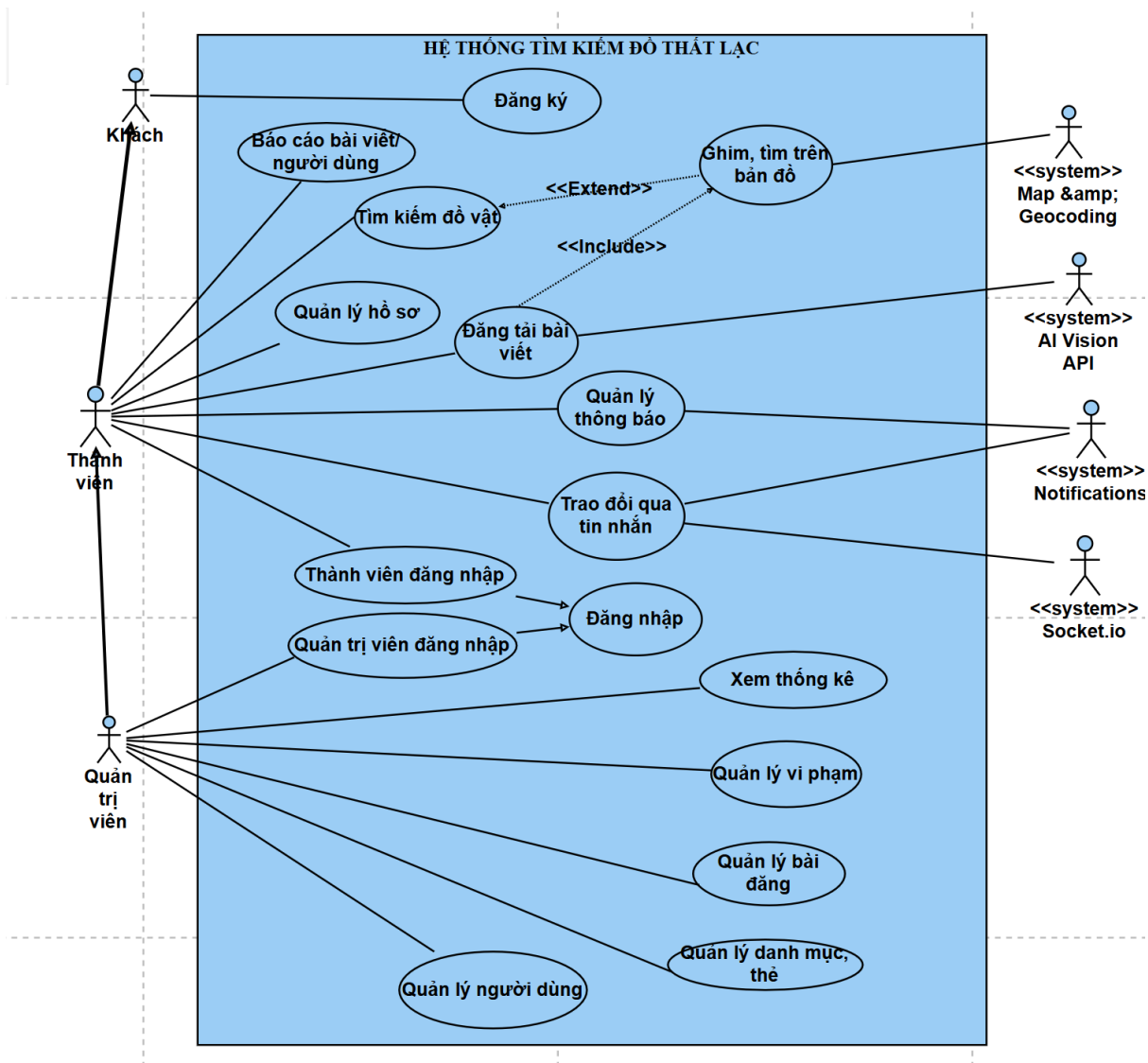
Các Use Case này được thực hiện tự động thông qua các giao thức kết nối hệ thống (API/Socket) để hỗ trợ các actor trực tiếp:

Tên Actor	Usecase hỗ trợ	Mô tả vai trò
Map & Geocoding	Ghim, tìm trên bản đồ	Chuyển đổi địa chỉ thành tọa độ địa lý và hiển thị các điểm đánh dấu trên bản đồ số.

AI Vision API	AI gợi ý Tag	Phân tích hình ảnh do Member tải lên để tự động đưa ra các nhãn dán (Tags) mô tả đồ vật.
Notifications	Gửi thông báo realtime	Đẩy thông báo đến thiết bị của người dùng ngay khi có sự kiện phát sinh (duyet bài, tin nhắn mới).
Socket.io	Chat realtime	Duy trì kết nối liên tục giúp quá trình trao đổi tin nhắn giữa hai Thành viên diễn ra tức thời.

3) Sơ đồ usecase tổng quan:

Dựa vào các actor và chức năng của các actor đã nêu ở trên, nhóm dự án thu được sơ đồ usecase tổng quan như sau:



Link UML: [USECASE](#)

Các use case được mô tả như sau:

- **Tìm kiếm đồ vật (Thành viên):** UC này cho phép người dùng tra cứu thông tin về các vật phẩm thất lạc hoặc nhặt được thông qua các bộ lọc hoặc danh sách hiển thị.
- **Báo cáo bài viết/người dùng (Thành viên):** UC này cho phép Thành viên gửi phản ánh về các nội dung nghi ngờ lừa đảo hoặc hành vi vi phạm chuẩn mực của người dùng khác.
- **Quản lý hồ sơ (Thành viên):** UC này cho phép Thành viên cập nhật thông tin cá nhân và quản lý danh sách các bài đăng (sửa, xóa) của riêng mình.
- **Đăng tải bài viết (Thành viên):** UC này cho phép Thành viên tạo các thông báo mới về việc mất đồ hoặc nhặt được đồ để chia sẻ lên hệ thống.

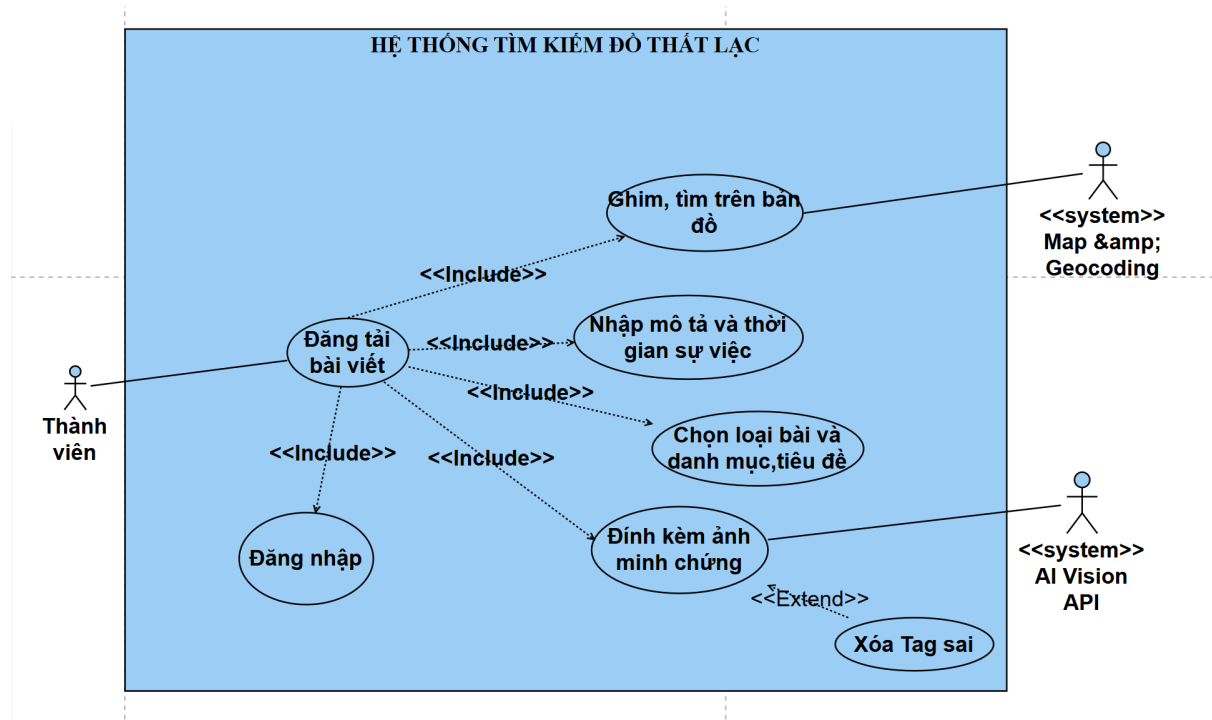
- **Quản lý thông báo (Thành viên):** UC này cho phép Thành viên tiếp nhận và theo dõi các thông báo về trạng thái bài viết, tin nhắn mới hoặc cảnh báo từ hệ thống.
- **Trao đổi qua tin nhắn (Thành viên):** UC này cho phép các Thành viên kết nối và trò chuyện trực tiếp để xác minh thông tin vật phẩm thất lạc.
- **Xem thống kê (Quản trị viên):** UC này cho phép Quản trị viên theo dõi lượng người dùng và lượng bài đăng để đánh giá hiệu quả hệ thống.
- **Quản lý bài đăng (Quản trị viên):** UC này cho phép Quản trị viên kiểm soát toàn bộ nội dung bài đăng trước khi thực hiện phê duyệt hoặc gỡ bỏ các bài viết.
- **Quản lý vi phạm (Quản trị viên):** UC này cho phép Quản trị viên tiếp nhận và xử lý các báo cáo từ cộng đồng, thực hiện khóa tài khoản hoặc xóa nội dung vi phạm.
- **Quản lý danh mục (Quản trị viên):** UC này cho phép Quản trị viên thiết lập và điều chỉnh các loại danh mục đồ vật để hỗ trợ tìm kiếm.
- **Quản lý người dùng (Quản trị viên):** UC này cho phép Quản trị viên thực hiện việc quản lý các tài khoản thông qua khóa/mở khóa tài khoản.
- **Ghim, tìm trên bản đồ (Hệ thống Map & Geocoding):** UC hỗ trợ tự động xác định tọa độ và hiển thị vị trí đồ vật trên bản đồ số để hỗ trợ các chức năng tìm kiếm và đăng bài.

4) Phân rã chi tiết các usecase:

Mục đích của bước này là mô tả chi tiết các usecase đã xác định được trong sơ đồ tổng quan.

a. Use case đăng tải bài viết:

Đăng tải bài viết bao gồm các thao tác chọn loại bài, danh mục, tiêu đề, mô tả và thời gian sự việc, ghim vị trí trên bản đồ và đính kèm ảnh minh chứng. Để đăng tải bài viết thì thành viên bắt buộc phải đăng nhập. Việc đính kèm ảnh minh chứng sẽ bao gồm chức năng AI gợi ý Tag, và từ việc AI gợi ý Tag có thể thực hiện xóa Tag sai nếu cần thiết. Sau khi hoàn tất các bước, bài viết sẽ được lưu ở trạng thái chờ duyệt.

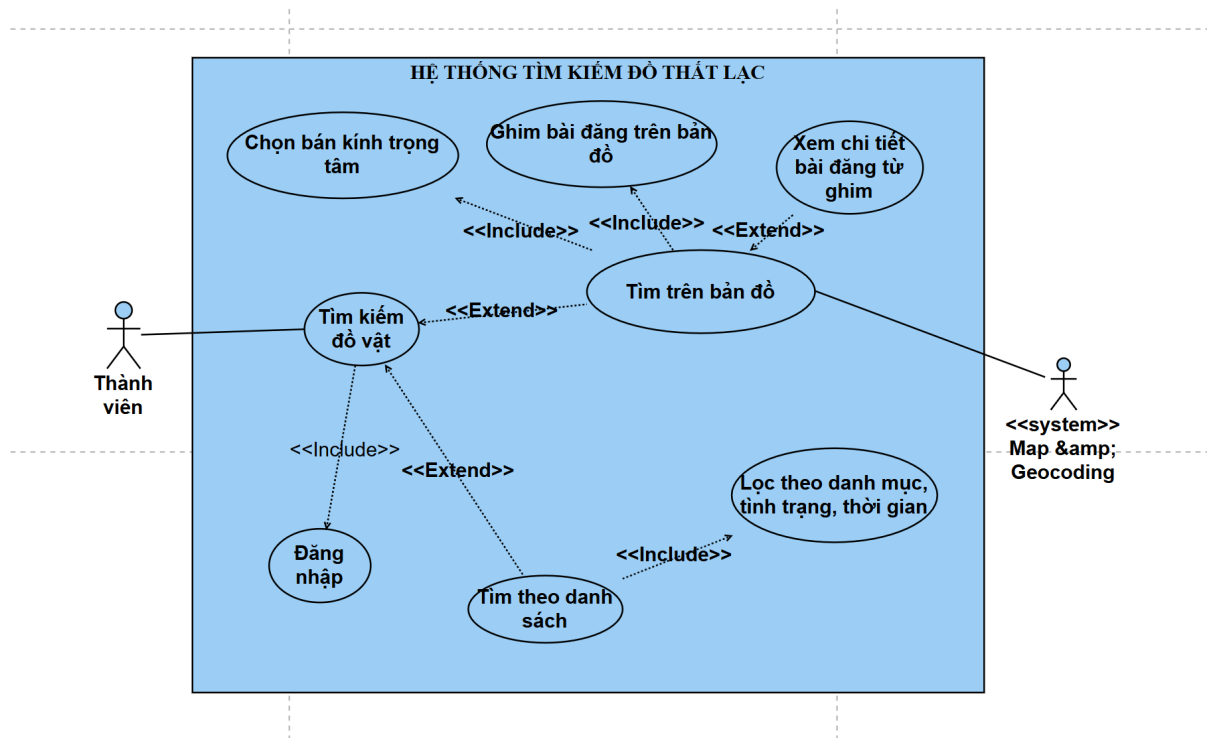


Mô tả các use case:

- **Chọn loại bài và danh mục, tiêu đề:** UC này cho phép thành viên lựa chọn loại bài (Mất/Nhặt), phân loại vào danh mục phù hợp và đặt tiêu đề cho bài viết.
- **Nhập mô tả và thời gian sự việc:** UC này cho phép thành viên cung cấp thông tin chi tiết về đặc điểm nhận dạng và thời điểm xảy ra sự việc.
- **Ghim, tìm trên bản đồ:** UC này cho phép thành viên xác định vị trí tọa độ chính xác của vật phẩm trên bản đồ số.
- **Đính kèm ảnh minh chứng:** UC này cho phép thành viên tải các hình ảnh thực tế của vật phẩm lên hệ thống.
- **Xóa Tag sai:** UC này cho phép thành viên xóa các Tag không đúng với mô tả.

b. Use case tìm kiếm đồ vật:

Tìm kiếm đồ vật bao gồm các hình thức tìm theo danh sách và tìm trên bản đồ. Để sử dụng tính năng tìm kiếm, thành viên bắt buộc phải thực hiện đăng nhập. Tìm kiếm theo danh sách bao gồm thao tác lọc theo danh mục, tình trạng và thời gian. Tìm kiếm trên bản đồ bao gồm việc quét bán kính xung quanh vị trí trọng tâm và hiển thị các ghim bài đăng trên bản đồ, từ đó có thể xem chi tiết bài đăng từ các ghim này.



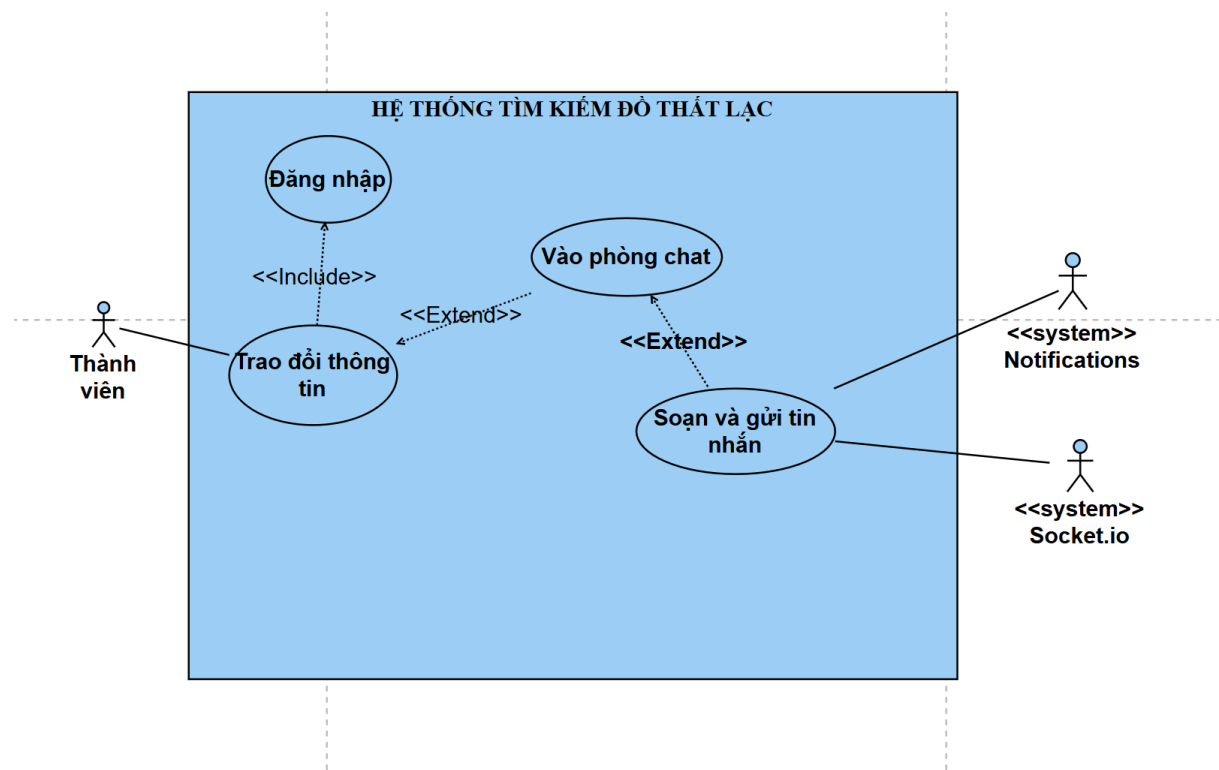
Mô tả các use case:

- **Tìm theo danh sách:** UC này cho phép thành viên tra cứu thông tin các vật phẩm bị mất hoặc nhặt được dưới dạng danh sách hiển thị truyền thống.
- **Lọc theo danh mục, tình trạng, thời gian:** UC này hỗ trợ thành viên thu hẹp phạm vi tìm kiếm dựa trên các tiêu chí cụ thể như danh mục đồ vật, tình trạng (mất/nhặt) và mốc thời gian.
- **Tìm trên bản đồ:** UC này cung cấp giao diện trực quan, cho phép thành viên tìm kiếm đồ vật dựa trên vị trí địa lý thực tế với sự hỗ trợ của dịch vụ Map.
- **Chọn bán kính trọng tâm:** UC này cho phép thành viên chọn phạm vi bán kính xung quanh để hệ thống quét các bài đăng có liên quan.
- **Ghim bài đăng trên bản đồ:** UC này cho phép thành viên ghim điểm trung tâm để tìm kiếm.
- **Xem chi tiết bài đăng từ ghim:** UC này cho phép thành viên tương tác trực tiếp với các ghim trên bản đồ để hiển thị thông tin chi tiết về vật phẩm tại vị trí đó.

c. Use case trao đổi thông tin:

Trao đổi qua tin nhắn bao gồm các thao tác vào phòng chat, soạn và gửi tin nhắn để các thành viên liên lạc với nhau. Để thực hiện trao đổi, thành viên bắt buộc phải đăng nhập. Việc vào phòng chat có thể thông qua việc tạo phòng chat mới (kết quả từ việc

xem chi tiết bài đăng) hoặc mở lại các phòng chat cũ đã tồn tại. Khi thực hiện soạn và gửi tin nhắn, hệ thống sẽ đồng thời hiển thị tin nhắn thời gian thực, mã hóa và lưu trữ đoạn chat, cũng như đẩy thông báo đến người nhận.

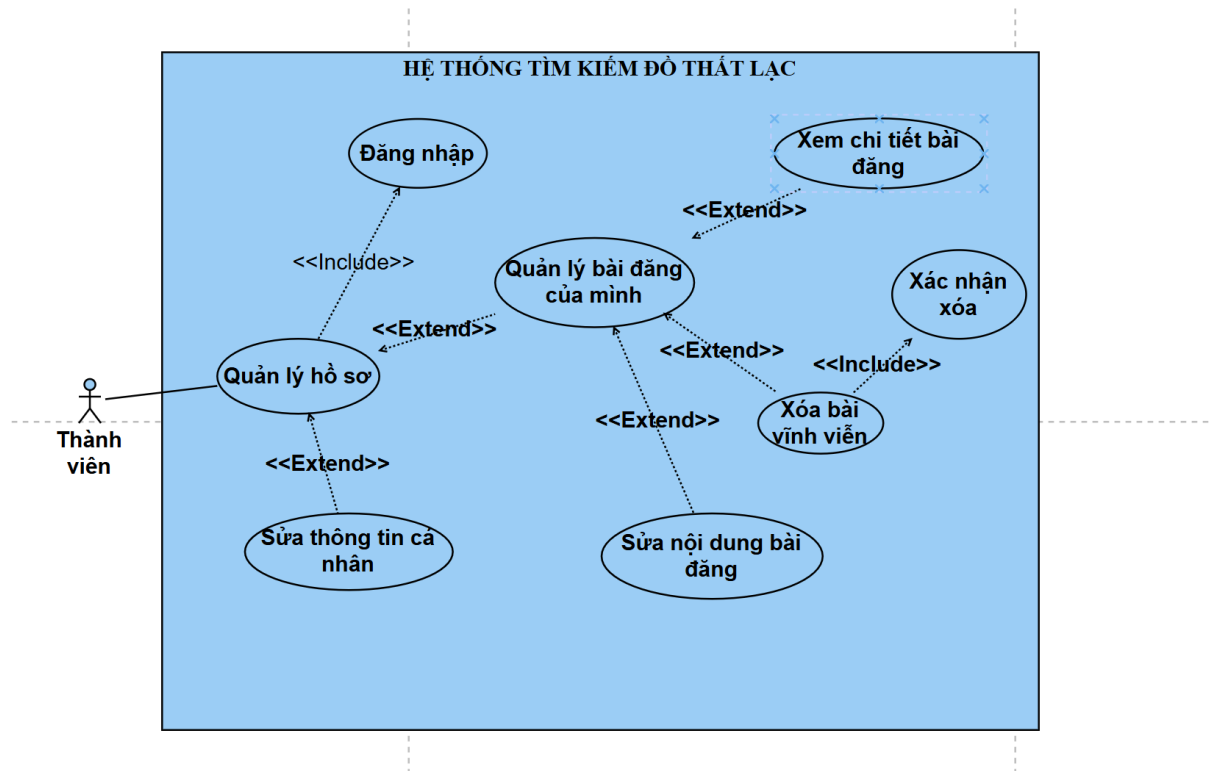


Mô tả các use case:

- **Vào phòng chat:** UC này đưa thành viên vào giao diện kết nối riêng tư để bắt đầu quá trình trao đổi thông tin về đồ vật.
- **Soạn và gửi tin nhắn:** UC này cho phép thành viên nhập nội dung văn bản và gửi đi trong phòng chat để tương tác với đối phương.

d. Use case quản lý hồ sơ:

Quản lý hồ sơ cá nhân bao gồm việc sửa thông tin cá nhân và quản lý các bài đăng của mình. Để thực hiện quản lý hồ sơ, thành viên bắt buộc phải đăng nhập. Trong phần quản lý bài đăng, thành viên có thể thực hiện xem chi tiết, sửa nội dung hoặc xóa bài vĩnh viễn. Khi sửa nội dung bài đăng, hệ thống sẽ đồng thời ẩn bài khỏi bảng tin và đưa bài về trạng thái chờ duyệt. Đối với thao tác xóa bài vĩnh viễn, thành viên cần thực hiện bước xác nhận xóa.

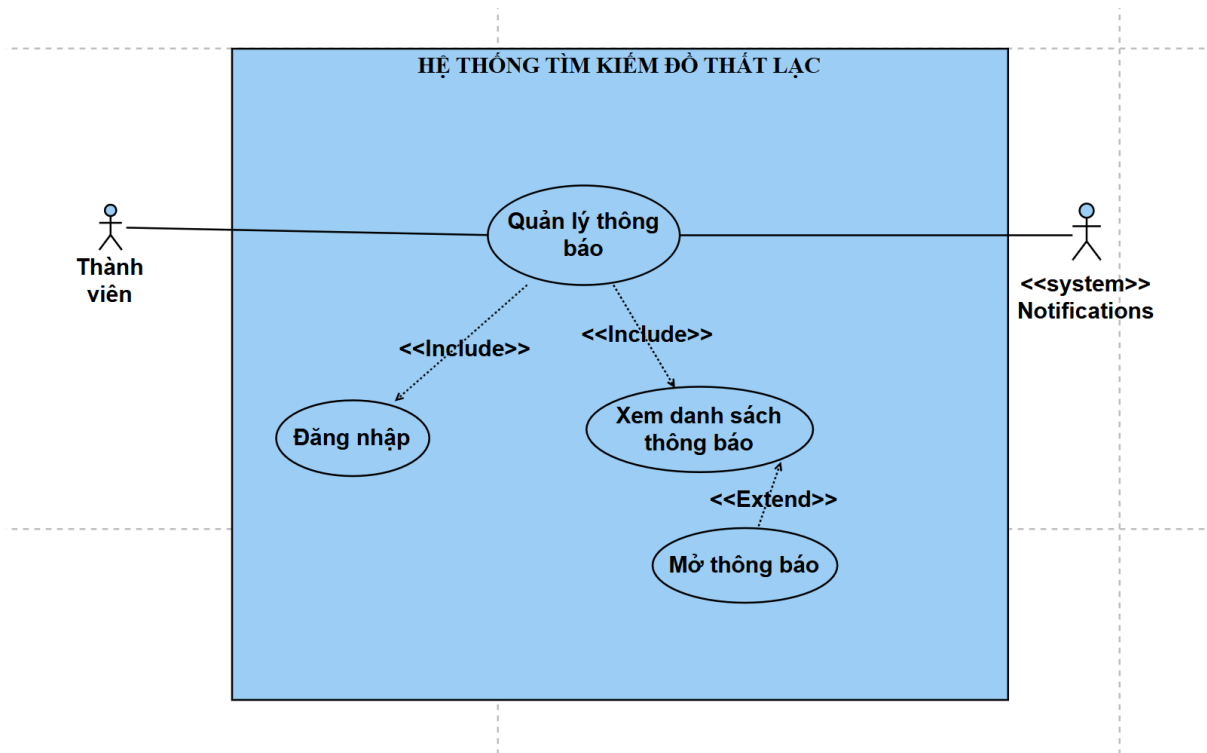


Mô tả các use case:

- **Sửa thông tin cá nhân:** UC này cho phép thành viên thay đổi các thông tin định danh như tên hiển thị, ảnh đại diện hoặc thông tin liên lạc.
- **Quản lý bài đăng của mình:** UC này cung cấp danh sách các bài viết mà thành viên đã đăng tải để thực hiện các thao tác theo dõi và điều chỉnh.
- **Xem chi tiết bài đăng:** UC này cho phép thành viên xem lại toàn bộ nội dung chi tiết của một bài viết cụ thể trong danh sách cá nhân.
- **Sửa nội dung bài đăng:** UC này cho phép thành viên cập nhật lại thông tin mô tả hoặc hình ảnh của bài viết đã đăng trước đó.
- **Xóa bài vĩnh viễn:** UC này cho phép thành viên loại bỏ hoàn toàn bài viết khỏi hệ thống khi món đồ đã được tìm thấy hoặc không còn nhu cầu đăng tin.
- **Xác nhận xóa:** UC này yêu cầu thành viên xác thực lại yêu cầu xóa để tránh các thao tác nhầm lẫn làm mất dữ liệu bài đăng.

e. Use case quản lý thông báo:

Quản lý thông báo bao gồm thao tác xem danh sách thông báo và có thể nhận thông báo thời gian thực từ hệ thống. Để thực hiện quản lý thông báo, thành viên bắt buộc phải đăng nhập. Trong quá trình xem danh sách thông báo, thành viên có thể thực hiện mở thông báo để xem chi tiết. Khi thực hiện mở thông báo, hệ thống sẽ bao gồm thao tác đánh dấu đã đọc cho thông báo đó.



Mô tả các use case:

- **Xem danh sách thông báo:** UC này hiển thị tập hợp các thông báo mà thành viên đã nhận được (như thông báo bài đăng được duyệt, có tin nhắn mới hoặc phản hồi từ quản trị viên).
- **Mở thông báo:** UC này cho phép thành viên tương tác với một thông báo cụ thể trong danh sách để chuyển hướng đến trang chức năng liên quan.

f. Use case báo cáo bài viết/người dùng:

Báo cáo bài viết/người dùng bao gồm các hình thức báo cáo bài viết hoặc báo cáo người dùng khi phát hiện dấu hiệu vi phạm hoặc lừa đảo. Để thực hiện các báo cáo này, thành viên bắt buộc phải đăng nhập vào hệ thống. Cả hai quy trình báo cáo bài viết và báo cáo người dùng đều bao gồm thao tác bắt buộc là nhập mô tả nội dung vi phạm để cung cấp thông tin chi tiết cho quản trị viên xử lý.

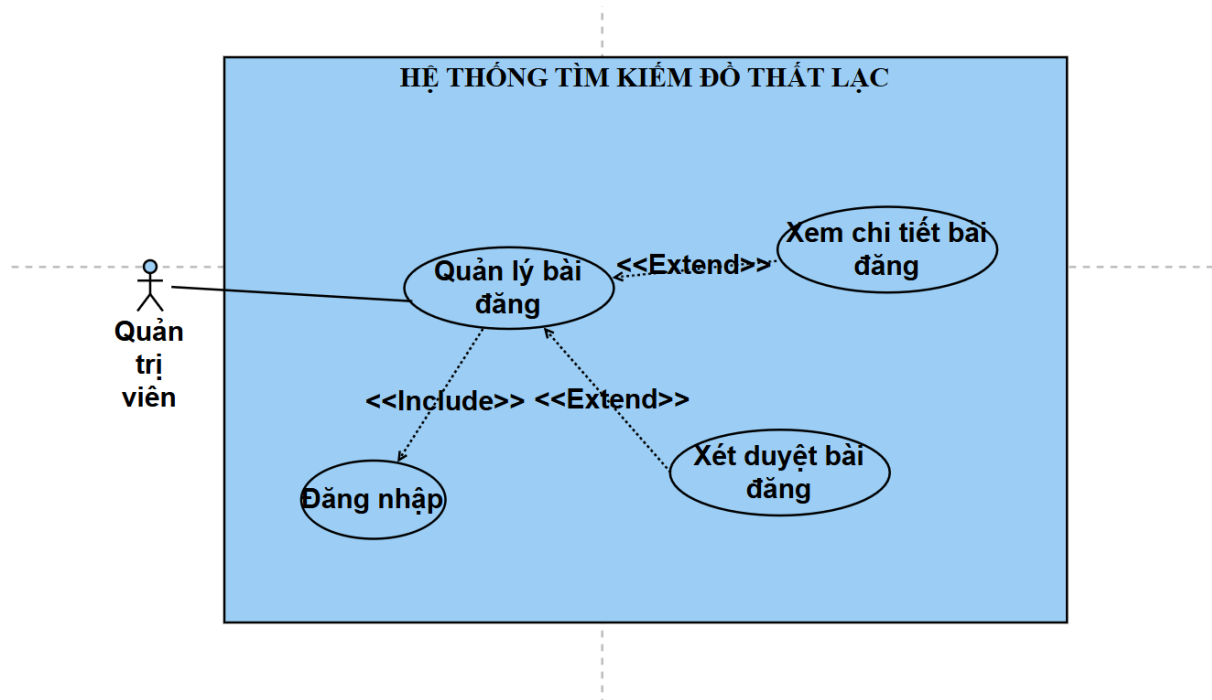


Mô tả các use case:

- **Báo cáo bài viết/người dùng:** UC này cho phép thành viên thực hiện báo cáo đối với một bài đăng cụ thể (mất đồ hoặc nhặt được đồ) mà họ nghi ngờ là sai sự thật hoặc lừa đảo.
- **Nhập mô tả nội dung vi phạm:** UC này cho phép thành viên cung cấp thông tin chi tiết, lý do vi phạm và các bằng chứng liên quan để làm cơ sở cho quản trị viên xác minh và xử lý.

g. Use case quản lý bài đăng:

Quản lý bài đăng bao gồm việc thực hiện các thao tác quản trị đối với các tin đăng trên hệ thống. Để thực hiện quản lý bài đăng, quản trị viên bắt buộc phải đăng nhập. Trong quá trình quản lý, quản trị viên có thể thực hiện xét duyệt bài đăng. Việc xét duyệt bài đăng sẽ bao gồm thao tác xem chi tiết bài đăng để làm căn cứ đánh giá và phê duyệt nội dung.

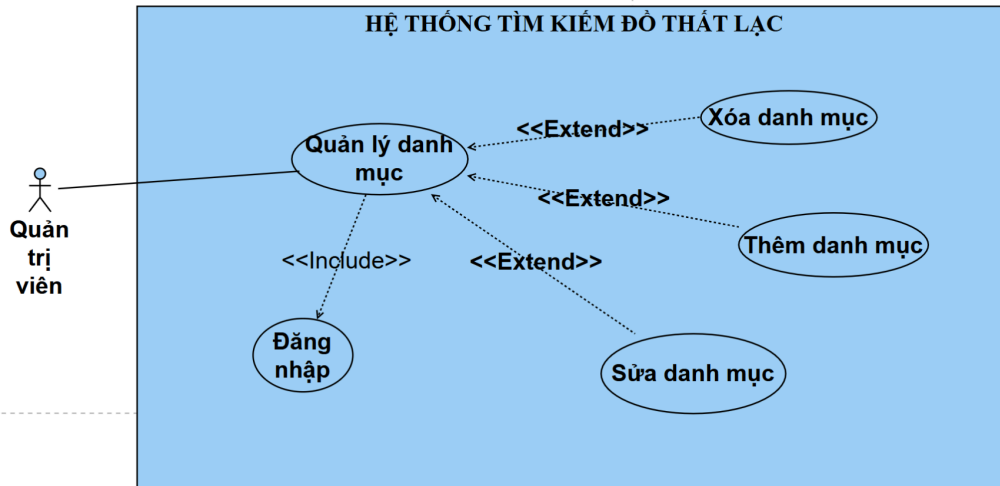


Mô tả các use case:

- **Quản lý bài đăng:** UC này cung cấp giao diện tập trung để quản trị viên theo dõi, điều phối và kiểm soát danh sách các bài đăng từ người dùng trên toàn hệ thống.
- **Xét duyệt bài đăng:** UC này cho phép quản trị viên thực hiện việc chấp nhận hoặc từ chối các bài viết mới nhằm đảm bảo nội dung hiển thị trên bảng tin là hợp lệ.
- **Xem chi tiết bài đăng:** UC này hiển thị đầy đủ các thông tin mô tả, hình ảnh và vị trí của bài viết, hỗ trợ quản trị viên có đủ dữ kiện để thực hiện bước xét duyệt bài đăng.

h. Use case quản lý danh mục:

Quản lý danh mục bao gồm việc thực hiện các thao tác quản trị đối với các phân loại đồ vật trên hệ thống. Để thực hiện quản lý danh mục, quản trị viên bắt buộc phải đăng nhập. Trong quá trình quản lý, quản trị viên có thể thực hiện thêm danh mục mới, sửa đổi thông tin danh mục hoặc xóa bỏ danh mục hiện có để phù hợp với nhu cầu của người dùng.

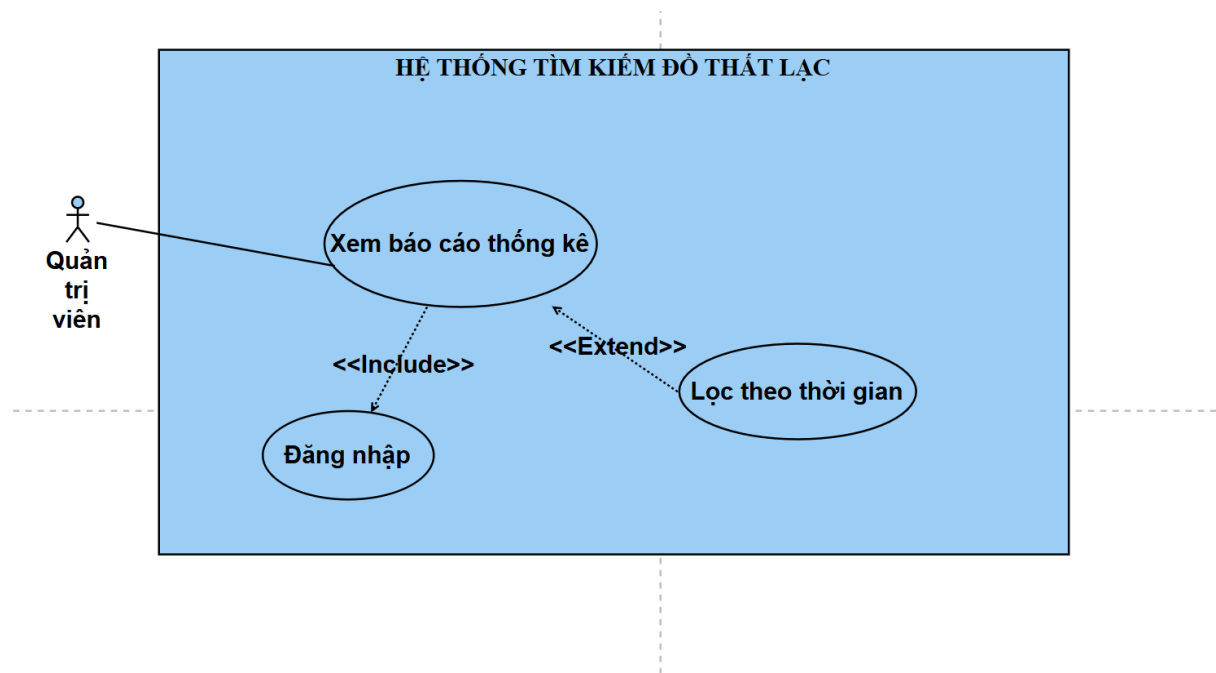


Mô tả các use case:

- **Quản lý danh mục:** UC này cung cấp giao diện tập trung để quản trị viên điều hành các loại phân loại vật phẩm, giúp hệ thống tổ chức dữ liệu một cách khoa học.
- **Thêm danh mục:** UC này cho phép quản trị viên bổ sung thêm các loại đồ vật mới vào hệ thống (ví dụ: Phụ kiện, Đồ gia dụng, Linh kiện điện tử...).
- **Sửa danh mục:** UC này cho phép quản trị viên cập nhật hoặc thay đổi tên và các thông tin liên quan của một phân loại đồ vật đã tồn tại.
- **Xóa danh mục:** UC này cho phép quản trị viên loại bỏ hoàn toàn một danh mục không còn cần thiết khỏi cơ sở dữ liệu của hệ thống.

i. Use case xem báo cáo thống kê:

Xem báo cáo thống kê bao gồm các hoạt động thống kê người dùng mới và thống kê bài đăng mới để quản trị viên theo dõi sự phát triển của hệ thống. Để thực hiện xem báo cáo thống kê, quản trị viên bắt buộc phải đăng nhập. Trong quá trình xem báo cáo, quản trị viên có thể thực hiện thao tác mở rộng là lọc theo thời gian để xem dữ liệu chi tiết trong các khoảng thời gian cụ thể.

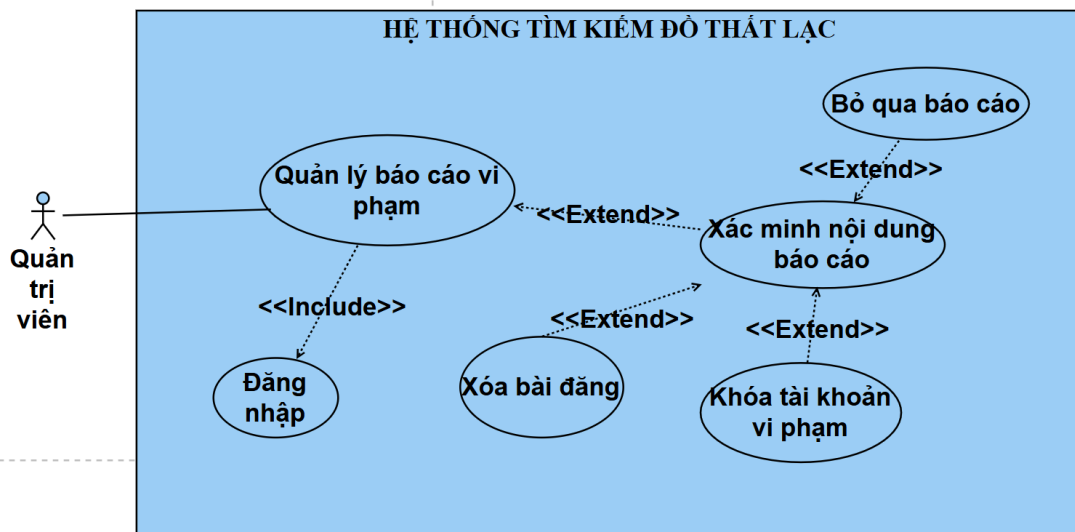


Mô tả các use case:

- **Xem báo cáo thống kê:** UC này cho phép quản trị viên xem các thống kê về người dùng, bài viết hoạt động, bài viết chờ duyệt và bài báo cáo.
- **Lọc theo thời gian:** UC này cho phép quản trị viên tùy chỉnh các mốc thời gian (theo ngày, tuần, tháng hoặc năm) để hệ thống trích xuất dữ liệu thống kê chính xác theo nhu cầu theo dõi.

j. Use case quản lý báo cáo vi phạm:

Quản lý báo cáo vi phạm bao gồm việc xác minh nội dung báo cáo từ phía người dùng gửi về hệ thống. Để thực hiện quản lý báo cáo, quản trị viên bắt buộc phải đăng nhập. Trong quá trình xác minh nội dung báo cáo, hệ thống sẽ bao gồm thao tác đánh dấu báo cáo đã giải quyết để hoàn tất quy trình xử lý. Ngoài ra, tùy thuộc vào mức độ vi phạm sau khi xác minh, quản trị viên có thể thực hiện các thao tác mở rộng như xóa bài đăng hoặc khóa tài khoản vi phạm.

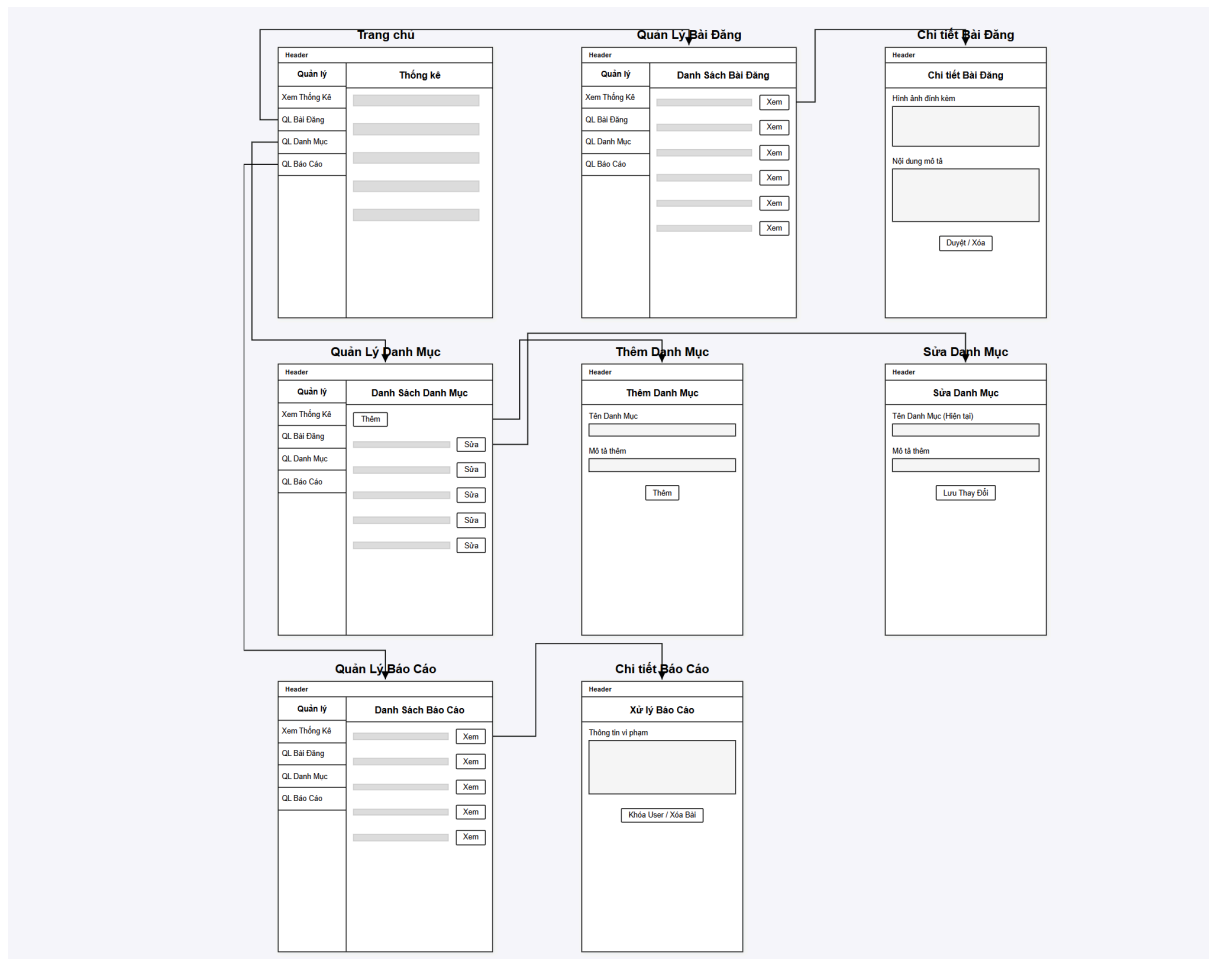


Mô tả các use case:

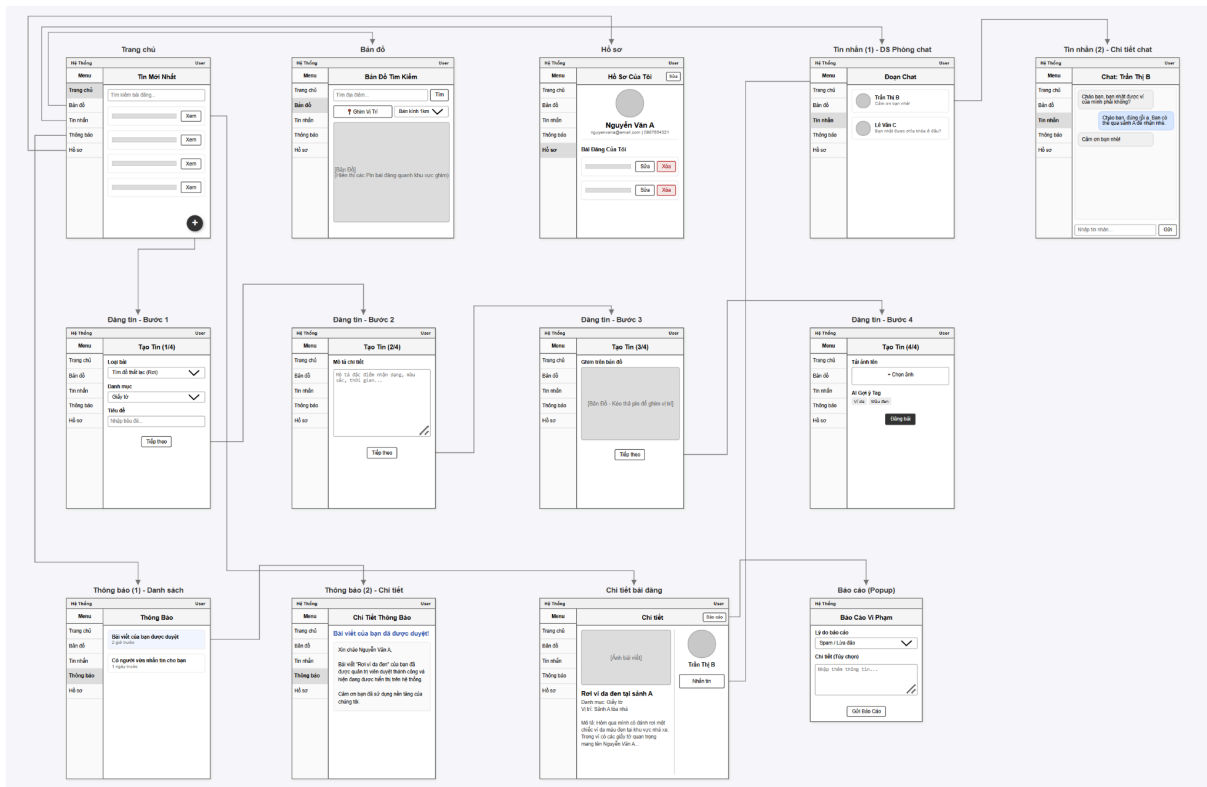
- **Xác minh nội dung báo cáo:** UC này cho phép quản trị viên xem xét chi tiết mô tả vi phạm, đối chiếu với bài viết hoặc thông tin người dùng bị báo cáo để đưa ra quyết định xử lý chính xác.
- **Xóa bài đăng:** UC này cho phép quản trị viên loại bỏ hoàn toàn bài viết vi phạm khỏi hệ thống nếu nội dung đó được xác minh là lừa đảo, sai sự thật hoặc vi phạm quy chuẩn cộng đồng.
- **Khóa tài khoản vi phạm:** UC này cho phép quản trị viên đình chỉ quyền truy cập của người dùng vi phạm nghiêm trọng hoặc tái diễn hành vi xấu trên nền tảng.
- **Bỏ qua báo cáo:** UC này cho phép quản trị viên bỏ qua những báo cáo không vi phạm, không cần xử lý.

1.5. Thiết kế tương tác với sản phẩm:

Thiết kế giao diện và tương tác các chức năng cho người quản trị và thành viên được thể hiện bởi 2 hình dưới đây:



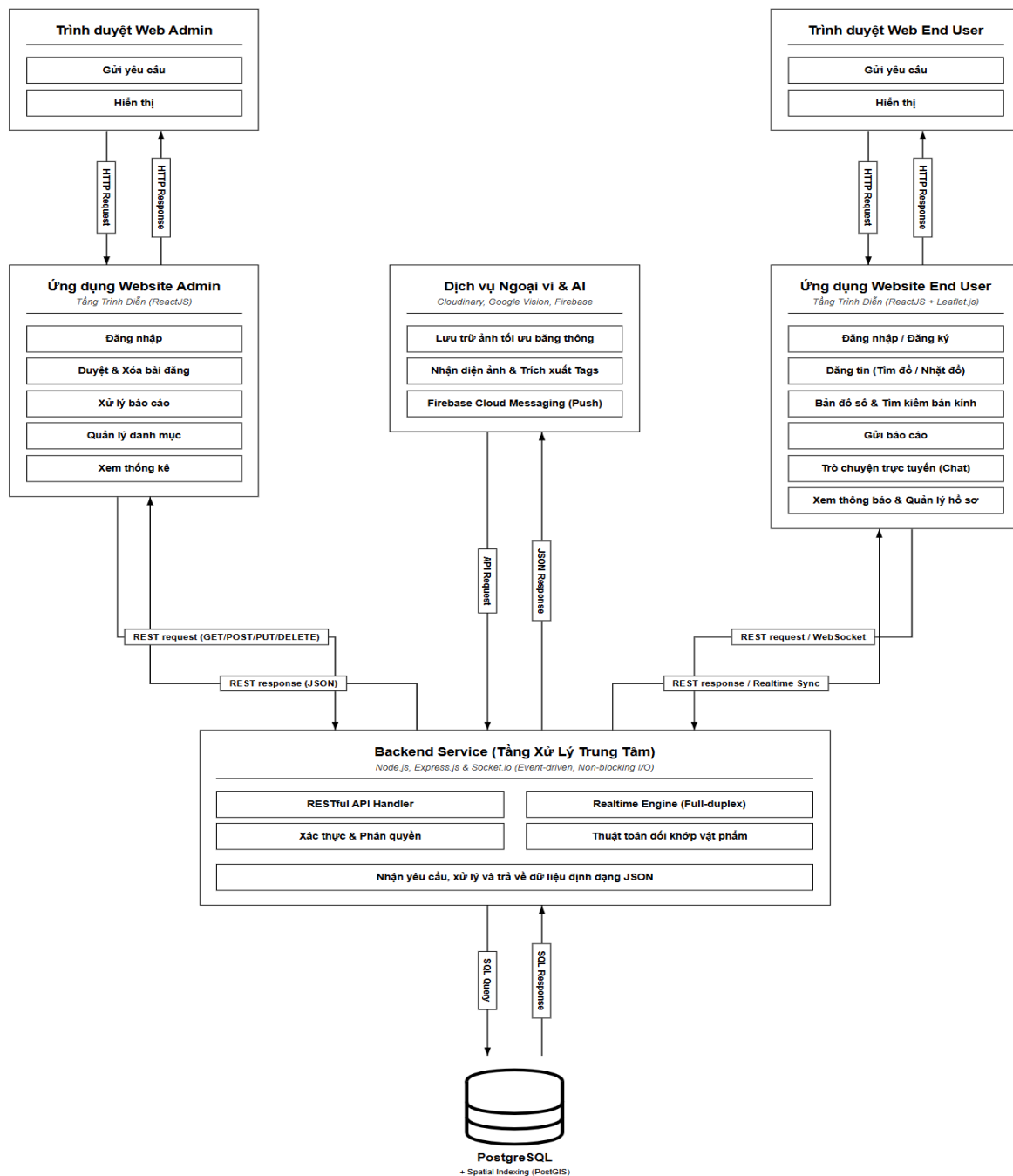
- Hình 1: Thiết kế giao diện và tương tác các chức năng cho người quản trị



- Hình 2: Thiết kế giao diện và tương tác các chức năng cho thành viên

Chương 2: NGHIÊN CỨU PHƯƠNG PHÁP TIẾP CẬN VÀ GIẢI QUYẾT VẤN ĐỀ

2.1. Mô hình tổng quát hệ thống:



Hệ thống được xây dựng trên kiến trúc **Client-Server** hiện đại, tích hợp chặt chẽ với các dịch vụ từ bên thứ ba (Third-party APIs) nhằm tối ưu hóa hiệu năng xử lý và cung cấp các tính năng thông minh. Quy trình vận hành và sự phối hợp giữa các tầng công nghệ trong mô hình sơ đồ khối được diễn giải chi tiết như sau:

- **Tầng Trình diễn (Frontend - ReactJS & Leaflet.js):** Sử dụng thư viện **ReactJS** để quản lý trạng thái giao diện (State management) và xử lý các tương tác người dùng như: đăng tin thất lạc/nhặt được, tìm kiếm và quản lý hồ sơ cá nhân. Thư viện **Leaflet.js** được tích hợp trực tiếp để trực quan hóa dữ liệu bản

đồ số, cho phép hiển thị các điểm đánh dấu (Markers) và vùng bán kính tìm kiếm một cách mượt mà, tận dụng tối đa cơ chế Virtual DOM của React để đảm bảo hiệu suất hiển thị tối ưu.

- **Tầng Xử lý Trung tâm (Backend - Node.js, Express.js & Socket.io):** Đóng vai trò trung tâm tiếp nhận và xử lý các yêu cầu từ Client thông qua giao thức **RESTful API**. Với kiến trúc hướng sự kiện (Event-driven) và Non-blocking I/O, Node.js đảm bảo khả năng đáp ứng lượng lớn kết nối đồng thời với độ trễ thấp. Đặc biệt, thư viện **Socket.io** thiết lập kênh truyền tin hai chiều (Full-duplex), hỗ trợ các tính năng thời gian thực như trò chuyện trực tuyến (Chat) và đẩy thông báo (Push Notification) tức thì mà không cần thực hiện tải lại trang.
- **Tầng Dịch vụ Ngoại vi và Trí tuệ nhân tạo (Google Vision, Cloudinary & Firebase):** Hệ thống ủy quyền lưu trữ tài nguyên đa phương tiện cho dịch vụ **Cloudinary**, giúp tối ưu hóa băng thông và giảm tải tài nguyên cho máy chủ gốc. Song song đó, việc tích hợp **Google Vision API** cho phép hệ thống tự động nhận diện hình ảnh và trích xuất các nhãn đặc trưng (Tags). Quá trình này giúp chuẩn hóa dữ liệu đầu vào, tăng độ chính xác cho thuật toán đối khớp vật phẩm và khắc phục triệt để các hạn chế của phương thức nhập liệu thủ công. Ngoài ra, **Firebase Cloud Messaging (FCM)** được sử dụng để duy trì kết nối thông báo ổn định đến người dùng.
- **Tầng Lưu trữ và Truy vấn Không gian (PostgreSQL & PostGIS):** Hệ thống sử dụng cơ sở dữ liệu **PostgreSQL** để lưu trữ dữ liệu có cấu trúc, đảm bảo tính toàn vẹn và an toàn thông tin (ACID). Mấu chốt công nghệ cốt lõi là tiện ích mở rộng **PostGIS**, hỗ trợ xử lý chuyên sâu dữ liệu tọa độ địa lý. Thông qua hàm **ST_DWithin** phối hợp với cơ chế chỉ mục không gian (**Spatial Indexing**), hệ thống có khả năng thực hiện các truy vấn lọc tin theo bán kính với tốc độ cực nhanh, mang lại trải nghiệm tìm kiếm vị trí chính xác và tức thì cho người dùng.

Việc áp dụng mô hình phát triển hướng thành phần tại Frontend và kiến trúc API-First tại Backend không chỉ đảm bảo hệ thống có cấu trúc bền vững, dễ dàng bảo trì mà còn tạo tiền đề vững chắc cho việc mở rộng đa nền tảng trong tương lai.

2.2. Phương pháp xây dựng phần mềm

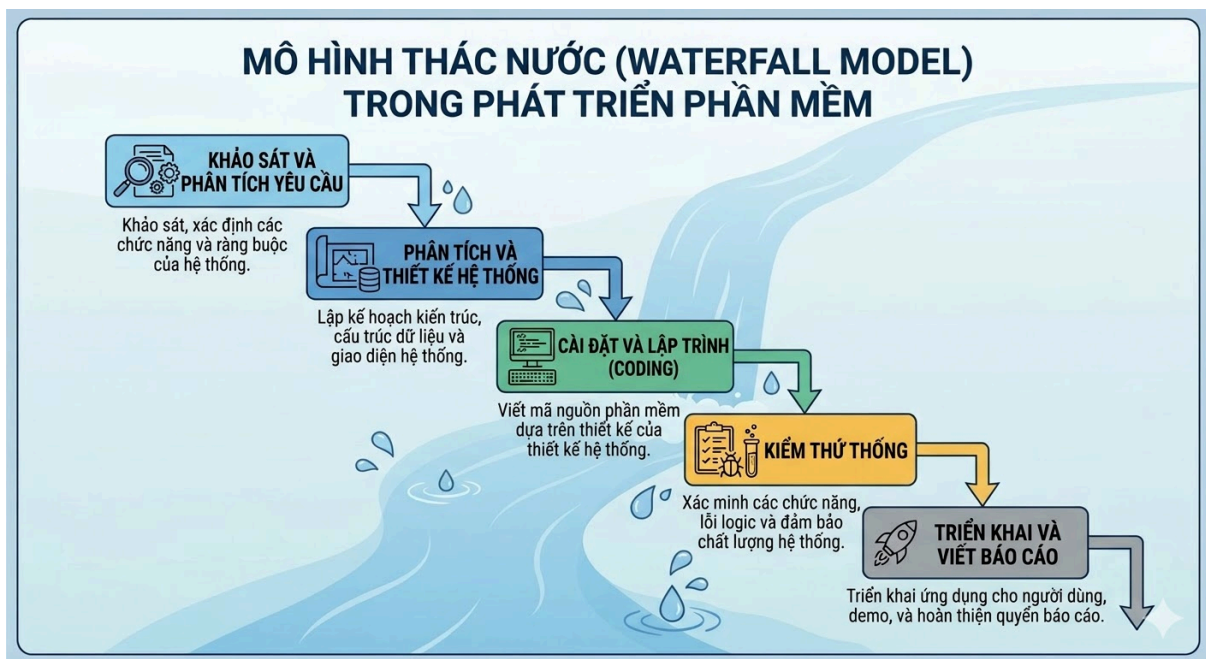
Để đối phó với sự phức tạp của việc tích hợp nhiều luồng công nghệ (Bản đồ, AI, Real-time), hệ thống áp dụng hai phương pháp tiếp cận thiết kế cốt lõi trong Kỹ thuật phần mềm hiện đại:

- **Phương pháp phát triển Hướng thành phần (Component-Based Development - CBD):** Áp dụng triệt để tại lớp Frontend. Thay vì xây

dựng giao diện nguyên khối, UI được chia nhỏ thành các thành phần (Components) có tính độc lập và khả năng tái sử dụng cao. Phương pháp này giúp cô lập lỗi (isolation), dễ dàng bảo trì và tối ưu hóa quá trình kiểm thử giao diện.

- **Phương pháp thiết kế ưu tiên API (API-First Approach):** Hệ thống tách biệt hoàn toàn sự phụ thuộc giữa Frontend và Backend. Thay vì Backend render mã HTML trả về trình duyệt như các hệ thống cũ, Backend trong dự án này chỉ đóng vai trò cung cấp các điểm cuối (Endpoints) theo chuẩn RESTful, trả về dữ liệu định dạng JSON. Phương pháp này thiết lập một "hợp đồng giao tiếp" (API Contract) rõ ràng, giúp hệ thống có khả năng mở rộng (Scalability) linh hoạt, tạo tiền đề để tích hợp đa nền tảng (như phát triển thêm Mobile App) mà không cần can thiệp lại kiến trúc Backend.

2.3. Mô hình phát triển phần mềm



Quá trình xây dựng và quản lý vòng đời phát triển phần mềm (SDLC) của dự án được thực hiện theo **Mô hình Thác nước (Waterfall)**. Đây là phương pháp tiếp cận tuyến tính, tuần tự, trong đó mỗi giai đoạn của quá trình phát triển phải được hoàn thành trọn vẹn và đóng băng tài liệu trước khi bước sang giai đoạn tiếp theo.

Lý do lựa chọn mô hình này xuất phát từ đặc thù của đồ án: các yêu cầu nghiệp vụ (như tích hợp AI, bản đồ PostGIS, nhấn tin thời gian thực) đã được xác định rõ ràng, ít có sự thay đổi ngay từ đầu. Việc áp dụng Thác nước giúp nhóm nghiên cứu xây dựng

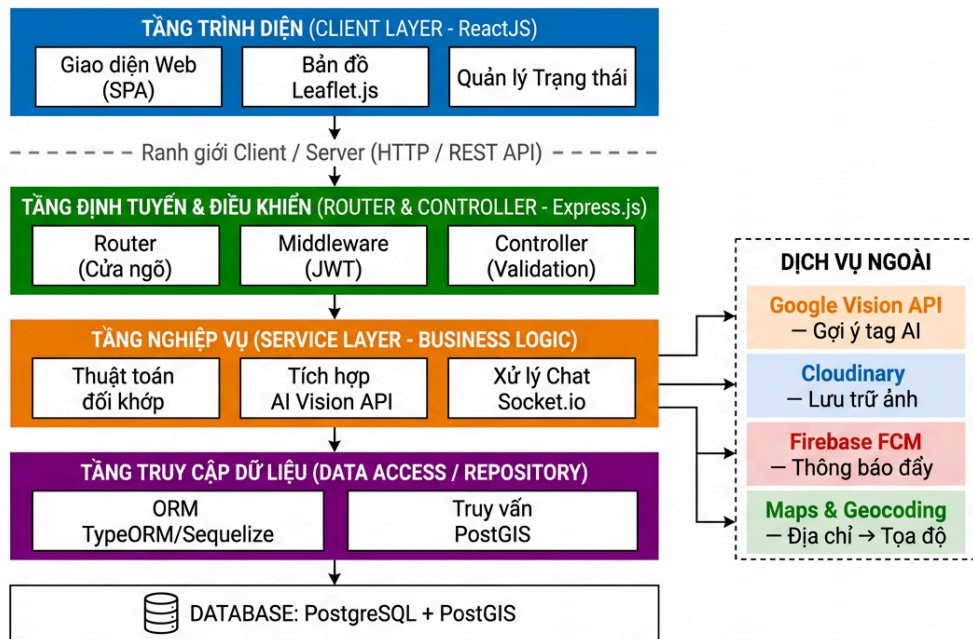
được một nền tảng kiến trúc phần mềm và cơ sở dữ liệu vững chắc trước khi tiến hành lập trình, đồng thời bám sát cấu trúc báo cáo học thuật truyền thống.

Toàn bộ quỹ thời gian 12 tuần của đồ án được phân bổ nghiêm ngặt thành 5 giai đoạn nối tiếp nhau:

- **Giai đoạn 1: Khảo sát và Phân tích yêu cầu (Tuần 1 - 2):** Tập trung thu thập thông tin, định hình bài toán tìm đồ thất lạc, phân tích các giải pháp hiện có trên thị trường và lập tài liệu đặc tả yêu cầu phần mềm (SRS).
- **Giai đoạn 2: Phân tích và Thiết kế hệ thống (Tuần 3 - 4):** Biến các yêu cầu thành bản vẽ kỹ thuật. Giai đoạn này tập trung vào việc mô hình hóa hệ thống bằng UML (Use Case, Sequence), thiết kế chi tiết cơ sở dữ liệu quan hệ (ERD với 16 bảng), thiết kế giao diện người dùng (UI/UX Mockups trên Figma) và đặc tả hợp đồng giao tiếp API. Mọi thiết kế phải được chốt chặt (Freeze) tại đây.
- **Giai đoạn 3: Cài đặt và Lập trình (Tuần 5 - 9):** Đây là giai đoạn chuyển hóa bản vẽ thiết kế thành mã nguồn thực tế. Tiến hành xây dựng các module độc lập: Xây dựng Backend (Node.js) xử lý logic nghiệp vụ và tương tác với PostgreSQL/PostGIS; Phát triển giao diện Frontend (ReactJS); và tích hợp các dịch vụ bên thứ 3 (Cloudinary, AI Vision API, Socket.io).
- **Giai đoạn 4: Kiểm thử hệ thống (Tuần 10 - 11):** Sau khi toàn bộ mã nguồn được hoàn thiện, hệ thống bước vào giai đoạn kiểm thử tổng thể (System Testing) để phát hiện và khắc phục các lỗi logic, lỗi giao diện, và kiểm tra khả năng chịu tải của tính năng nhấn tin thời gian thực.
- **Giai đoạn 5: Triển khai và Viết báo cáo (Tuần 12):** Đóng gói phần mềm, triển khai ứng dụng lên môi trường máy chủ đám mây (Cloud Deployment) để demo thực tế, đồng thời hoàn thiện và tinh chỉnh quyền báo cáo đồ án.

Nhờ việc tuân thủ nghiêm ngặt quy trình này, dự án đảm bảo tính thống nhất cao giữa tài liệu thiết kế và mã nguồn thực tế, giảm thiểu tối đa việc phải tái cấu trúc cơ sở dữ liệu (Refactoring) trong quá trình lập trình.

2.4. Kiến trúc phần mềm được áp dụng trong triển khai lập trình



Về mặt tổ chức mã nguồn, hệ thống áp dụng sự kết hợp giữa **kiến trúc MVC (Model - View - Controller)** và **kiến trúc đa tầng (N-tier / Layered Architecture)** ở phía backend nhằm tuân thủ nghiêm ngặt nguyên lý tách biệt các mối quan tâm (Separation of Concerns). Backend được phân rã thành 4 tầng chức năng rành mạch:

- **Tầng Định tuyến (Router Layer):** Đóng vai trò là cửa ngõ (Gateway) tiếp nhận các HTTP Request. Tầng này phân tích phương thức (GET, POST, PUT, DELETE) và đường dẫn (URL) để điều hướng dòng điều khiển tới Controller tương ứng, đồng thời tích hợp các Middleware để kiểm tra tính hợp lệ của Token bảo mật (JWT).
- **Tầng Điều khiển (Controller Layer):** Chịu trách nhiệm tiếp nhận dữ liệu và kiểm tra tính hợp lệ đầu vào (Input Validation). Nếu dữ liệu hợp lệ, Controller sẽ gọi các hàm nghiệp vụ ở tầng dưới và định dạng lại phản hồi (HTTP Response) dưới dạng JSON để trả về cho Client. Tầng này tuyệt đối không chứa các câu lệnh truy vấn trực tiếp vào CSDL.
- **Tầng Dịch vụ (Service / Business Logic Layer):** Đây là nơi đóng gói toàn bộ logic thuật toán của hệ thống như: mã hóa mật khẩu, xử lý logic gọi AI nhận diện hình ảnh, hay thuật toán đối khớp dữ liệu bài đăng.
- **Tầng Truy cập dữ liệu (Data Access / Repository Layer & Model):** Trực tiếp tương tác với Hệ quản trị CSDL. Tầng này sử dụng kỹ thuật ORM (Object-Relational Mapping) hoặc các truy vấn SQL thuần túy để thực hiện các thao tác CRUD và các truy vấn định vị không gian phức tạp.

2.5. Lựa chọn công nghệ phù hợp để triển khai hệ thống

Quá trình lựa chọn công nghệ được cân nhắc kỹ lưỡng dựa trên yêu cầu về hiệu năng xử lý bất đồng bộ, khả năng tính toán không gian và giới hạn tài nguyên của dự án.

a) Lựa chọn công nghệ Frontend

- **ReactJS:** Được lựa chọn thay thế cho các thư viện truyền thống nhờ cơ chế **Virtual DOM** (Mô hình đối tượng tài liệu ảo). Khi người dùng thực hiện các thao tác tương tác như lọc bản đồ, React chỉ tính toán và cập nhật những điểm ghim (Marker) có sự thay đổi thay vì tải lại toàn bộ trang, mang lại trải nghiệm người dùng mượt mà.
- **Leaflet.js:** Là thư viện bản đồ mã nguồn mở được ưu tiên lựa chọn nhờ đặc tính cực kỳ nhẹ và không giới hạn mức truy vấn (Quota). Leaflet dễ dàng tích hợp với React, đáp ứng hoàn hảo yêu cầu hiển thị trực quan các tọa độ vật phẩm thất lạc.

b) Lựa chọn công nghệ Backend

- **Node.js và Express.js:** Khác với các nền tảng đa luồng truyền thống, Node.js vận hành trên kiến trúc đơn luồng kết hợp với **Vòng lặp sự kiện (Event Loop)** để xử lý I/O không đồng bộ (Non-blocking I/O). Đặc tính này giúp hệ thống duy trì hàng ngàn kết nối đồng thời với mức tiêu thụ tài nguyên thấp, là nền tảng lý tưởng cho module trao đổi thông tin thời gian thực.

c) Lựa chọn công nghệ Cơ sở dữ liệu và Xử lý Không gian

- **PostgreSQL kết hợp PostGIS:** Đây là mắt xích quan trọng nhất của hệ thống. Dự án lựa chọn PostgreSQL để đảm bảo tính toàn vẹn dữ liệu (ACID) cho các giao dịch và tin nhắn. Tiện ích mở rộng **PostGIS** cho phép lưu trữ kiểu dữ liệu hình học (Geometry) và sử dụng thuật toán đánh chỉ mục không gian (Spatial Indexing), giúp hàm truy vấn ST_DWithin có thể tính toán khoảng cách tọa độ và lọc hàng ngàn bài đăng trong bán kính cụ thể chỉ trong vài mili-giây.

d) Lựa chọn công nghệ Giao tiếp, Lưu trữ và Trí tuệ nhân tạo

- **Socket.io (WebSocket):** Để đáp ứng yêu cầu nhắn tin tức thời, hệ thống sử dụng Socket.io để tạo kênh giao tiếp **Full-duplex** (hai chiều đồng thời). Điều này cho phép Server chủ động đẩy tin nhắn về phía Client ngay lập tức, khắc phục hoàn toàn độ trễ của giao thức HTTP truyền thống.
- **AI Vision API:** Hệ thống sử dụng API từ bên thứ ba (Machine Learning as a Service) để nhận diện hình ảnh tự động. Giải pháp này giúp tiết kiệm tài nguyên phần cứng, đảm bảo khả năng trích xuất các nhãn dán (Tags) mô tả đồ vật với độ tin cậy cao, phục vụ trực tiếp cho thuật toán đối khớp bài đăng.

- **Cloudinary & Firebase FCM:** Cloudinary được sử dụng để quản lý và tối ưu hóa việc lưu trữ hình ảnh minh chứng nhằm giảm tải cho máy chủ. Trong khi đó, **Firebase Cloud Messaging (FCM)** đảm nhiệm vai trò gửi thông báo đẩy (Push Notifications) đến thiết bị người dùng ổn định và chính xác.

Chương 3: PHÂN TÍCH THIẾT KẾ VÀ THỰC NGHIỆM HỆ THỐNG

3.1 Các kịch bản

a. Kịch bản đăng tải bài viết

Thành viên tạo bài đăng mới (báo mất hoặc báo nhặt được đồ) để chia sẻ lên bảng tin cộng đồng.

Luồng chính:

- **Bước 1 - Thành viên:** Tại giao diện chính, click nút "Đăng bài mới".
- **Bước 2 - Hệ thống:** Hiện thị bước 1 của form tạo bài viết, yêu cầu chọn loại bài, chọn danh mục và nhập tiêu đề.
- **Bước 3 - Thành viên:** Chọn "Mất đồ", chọn danh mục "Giấy tờ tùy thân", nhập tiêu đề "Rơi ví tại sảnh A" và click "Tiếp theo".
- **Bước 4 - Hệ thống:** Hiện thị bước 2, yêu cầu nhập mô tả chi tiết và chọn ngày tháng xảy ra sự việc.
- **Bước 5 - Thành viên:** Điền mô tả, chọn thời gian thực tế và click "Tiếp theo".
- **Bước 6 - Hệ thống:** Hiện thị bước 3 với giao diện Bản đồ số để ghim vị trí sự việc.
- **Bước 7 - Thành viên:** Nhập địa chỉ, kéo thả ghim vào vị trí chính xác trên bản đồ và click "Tiếp theo".
- **Bước 8 - Hệ thống:** Hiện thị bước 4 với khu vực tải ảnh minh chứng và khu vực hiển thị Tags gợi ý.
- **Bước 9 - Thành viên:** Tải lên tối đa 3 ảnh chụp vật phẩm từ thiết bị cá nhân.
- **Bước 10 - Hệ thống:** Gửi ảnh qua AI Vision API, phân tích hình ảnh và trả về các Tags gợi ý như #ví_da, #màu_nâu.
- **Bước 11 - Thành viên:** Kiểm tra Tags gợi ý, xóa Tags sai nếu có và click nút "Đăng bài".
- **Bước 12 - Hệ thống:** Lưu dữ liệu vào cơ sở dữ liệu với trạng thái "Chờ duyệt", điều hướng về MemberHomeView và thông báo bài viết đang chờ quản trị viên phê duyệt.

Ngoại lệ:

- **Ngoại lệ tại Bước 9 - Tải ảnh không hợp lệ:** Nếu ảnh sai định dạng hoặc quá dung lượng, hệ thống báo lỗi và yêu cầu chọn lại ảnh phù hợp.
- **Ngoại lệ tại Bước 11 - AI gợi ý Tag sai:** Thành viên click vào dấu "x" cạnh thẻ Tag để gỡ bỏ trước khi thực hiện Đăng bài.

b. Kịch bản tìm kiếm đồ vật

Thành viên tra cứu thông tin vật phẩm thất lạc thông qua bộ lọc nâng cao hoặc Bản đồ số (Map Search).

Luồng chính:

- **Bước 1 – Thành viên:**

Tại màn hình Bảng tin chung, thành viên nhập từ khóa và lựa chọn các bộ lọc cần thiết:

- Loại bài đăng: Mất đồ hoặc Nhật được.
- Danh mục.
- Khoảng thời gian xảy ra sự việc.
- Tag mô tả.

- **Bước 2 – Hệ thống:**

Hệ thống gửi yêu cầu tìm kiếm đến PostController.

- **Bước 3 – Hệ thống:**

PostController truy vấn cơ sở dữ liệu và thực hiện:

- Chi lấy các bài đăng có trạng thái ACTIVE.
- So khớp từ khóa với tiêu đề, mô tả, địa chỉ và tên tag.
- Lọc theo loại bài đăng, danh mục, thời gian và tag nếu được chọn.
- Sắp xếp kết quả theo thời gian tạo mới nhất.

- **Bước 4 – Hệ thống:**

Hệ thống hiển thị danh sách bài đăng phù hợp. Mỗi kết quả gồm:

Bảng minh họa kết quả tìm kiếm theo danh sách:

TT	Tiêu đề	Loại bài	Danh mục	Vị trí	Thời gian sự việc	Thao tác
1	Rơi ví da màu nâu tại sảnh A	Mất đồ	Giấy tờ tùy thân	Sảnh A	15/05/2026	Xem chi tiết
2	Nhật được ví có giấy tờ sinh viên	Nhật được	Giấy tờ tùy thân	Thư viện trung tâm	15/05/2026	Xem chi tiết

- **Bước 5 – Thành viên:**

Thành viên click nút “Xem dạng Bản đồ”.

- **Bước 6 – Hệ thống:**

Hệ thống chuyển đến **MapSearchPage**, gồm:

- Khung nhập địa chỉ trung tâm.
- Bộ chọn bán kính từ 1 km đến 20 km.
- Danh sách kết quả.
- Bản đồ hiển thị vị trí.

- **Bước 7 – Thành viên:**

Thành viên xác định vị trí trung tâm bằng một trong hai cách:

- Nhập địa chỉ trung tâm; hoặc
- Click trực tiếp vào một vị trí trên bản đồ.

Sau đó, thành viên chọn bán kính và click nút “Tìm kiếm”.

- **Bước 8 – Hệ thống:**

Nếu thành viên nhập địa chỉ, **MapService** gọi dịch vụ bản đồ để chuyển địa chỉ thành tọa độ.

Nếu thành viên click trên bản đồ, hệ thống:

+Lấy tọa độ được chọn.

+Reverse geocode tọa độ thành địa chỉ.

+Cập nhật vị trí trung tâm và ô địa chỉ.

- **Bước 9 – Hệ thống:**

MapSearchPage gửi tọa độ trung tâm và bán kính đến **PostController**.

- **Bước 10 – Hệ thống:**

PostController truy vấn các bài đăng **ACTIVE**, sau đó sử dụng công thức Haversine để:

- Tính khoảng cách từ vị trí trung tâm đến từng bài đăng.
- Loại bỏ các bài nằm ngoài bán kính.
- Sắp xếp kết quả từ gần đến xa.

Hệ thống hiện không sử dụng PostGIS hoặc **ST_DWithin**. Khoảng cách được tính trong backend bằng công thức Haversine.

- **Bước 11 – Hệ thống:**

Hệ thống hiển thị:

- Vị trí trung tâm.

- Vòng tròn thể hiện bán kính tìm kiếm.
- Các ghim đại diện cho bài đăng phù hợp.
- Danh sách kết quả kèm khoảng cách đến vị trí trung tâm.
- **Bước 12 – Thành viên:**
Thành viên click vào một ghim trên bản đồ.
- **Bước 13 – Hệ thống:**
Hệ thống hiển thị popup gồm:
 - Ảnh minh họa.
 - Loại bài đăng.
 - Tiêu đề.
 - Địa chỉ.
 - Nút “Xem chi tiết”.
- **Bước 14 – Thành viên:**
Thành viên click nút “Xem chi tiết”.
- **Bước 15 – Hệ thống:**
Hệ thống mở **PostDetailPage** và hiển thị:
 - Tiêu đề và mô tả.
 - Loại và trạng thái bài đăng.
 - Danh mục và tags.
 - Hình ảnh.
 - Thời gian xảy ra sự việc.
 - Địa chỉ và vị trí trên bản đồ.
 - Thông tin người đăng.

Ngoại lệ:

- **Ngoại lệ tại Bước 4 – Không có kết quả danh sách**
- **Ngoại lệ tại Bước 8 – Không thể xác định địa chỉ**
- **Ngoại lệ tại Bước 11 – Không có bài trong bán kính**

c. Kịch bản trao đổi tin nhắn (Messages)

Các thành viên nhắn tin riêng tư qua hệ thống để xác minh thông tin vật phẩm mà không lộ thông tin liên hệ cá nhân.

Luồng chính:

- **Bước 1 - Thành viên:** Đang xem giao diện chi tiết một bài đăng và click nút "Nhắn tin".
- **Bước 2 - Hệ thống:** Tìm hoặc tạo phòng chat nội bộ kết nối trực tiếp đến tác giả bài đăng, sau đó hiển thị giao diện trò chuyện.

- **Bước 3 - Thành viên:** Nhập nội dung tin nhắn xác minh vào ô nhập liệu và nhấn Enter hoặc nút "Gửi".
- **Bước 4 - Hệ thống:** Mã hóa nội dung tin nhắn, lưu lịch sử chat vào CSDL, hiển thị tin nhắn thời gian thực bằng Socket.io và đẩy thông báo sang người nhận.

Ngoại lệ:

- **Ngoại lệ tại Bước 3 - Tin nhắn rỗng:** Nếu người dùng nhấn gửi khi chưa nhập ký tự, hệ thống vô hiệu hóa hành động và không có phản hồi xảy ra.

d. Kịch bản quản lý hồ sơ cá nhân

Thành viên cập nhật thông tin cá nhân và quản lý các bài đăng của chính mình.

Luồng chính:

- **Bước 1 - Thành viên:** Truy cập vào mục "Hồ sơ cá nhân" từ menu điều hướng của hệ thống.
- **Bước 2 - Hệ thống:** Hiển thị giao diện Profile (Thông tin cá nhân) và lưới danh sách các bài viết mà thành viên đã đăng tải.

Bảng minh họa danh sách bài đã đăng:

TT	Tiêu đề	Loại bài	Danh mục	Ngày đăng	Trạng thái	Thao tác
1	Rơi ví tại sảnh A	Mất đồ	Giấy tờ tùy thân	15/05/2026	Hoạt động	Xem / Sửa đổi / Xóa
2	Nhặt được chìa khóa xe máy	Nhặt được	Phụ kiện	12/05/2026	Chờ duyệt	Xem / Sửa đổi / Xóa

- **Bước 3 - Thành viên:** Click vào tùy chọn "Sửa đổi" tại một bài đăng cũ đang ở trạng thái "Hoạt động" để cập nhật nội dung.
- **Bước 4 - Hệ thống:** Mở giao diện form chỉnh sửa bài viết với toàn bộ các thông tin cũ đã được điền sẵn vào các trường dữ liệu.
- **Bước 5 - Thành viên:** Thực hiện thay đổi nội dung mô tả vật phẩm hoặc hình ảnh và click vào nút "Lưu thay đổi".
- **Bước 6 - Hệ thống:** Ghi nhận nội dung mới vào cơ sở dữ liệu, tự động chuyển trạng thái bài viết từ "Hoạt động" về lại "Chờ duyệt", đồng thời tạm ẩn bài viết khỏi bảng tin chung để chờ Quản trị viên phê duyệt lại.

- **Bước 7 - Hệ thống:** Hiển thị thông báo "Cập nhật thành công, bài viết đang đợi duyệt lại" và quay về giao diện quản lý hồ sơ.

Ngoại lệ:

- **Ngoại lệ tại Bước 3 - Thành viên xóa bài đăng:** Thay vì chọn sửa, thành viên click vào tùy chọn "Xóa" đối với món đồ đã tìm thấy chủ nhân hoặc không còn nhu cầu đăng tin. Hệ thống hiển thị popup yêu cầu xác nhận xóa. Sau khi thành viên ấn "Đồng ý", hệ thống sẽ thực hiện xóa bài viết vĩnh viễn khỏi toàn mạng lưới và cập nhật lại lưới danh sách.

e. Kịch bản quản lý thông báo

Thành viên theo dõi các sự kiện mới nhất phát sinh liên quan đến bài viết và tin nhắn của mình.

Luồng chính:

- **Bước 1 - Thành viên:** Đang trực tuyến và nhận thấy biểu tượng chuông bật màu đỏ báo hiệu có thông báo mới.
- **Bước 2 - Hệ thống:** Đẩy thông báo thời gian thực thông qua dịch vụ Notifications và giao thức Socket.io.
- **Bước 3 - Thành viên:** Click vào biểu tượng hình quả chuông trên thanh điều hướng.
- **Bước 4 - Hệ thống:** Hiển thị danh sách các thông báo mới nhất dưới dạng danh sách thả xuống (Dropdown), trong đó các thông báo chưa đọc được hiển thị in đậm.

Bảng minh họa danh sách thông báo:

TT	Loại thông báo	Nội dung	Thời gian	Trạng thái	Hành động
1	Tin nhắn	Bạn có tin nhắn mới từ tranthiB	14:20 15/05/2026	Chưa đọc	Mở
2	Bài đăng	Bài viết của bạn đã được phê duyệt	09:10 15/05/2026	Đã đọc	Mở

- **Bước 5 - Thành viên:** Click vào một dòng thông báo chưa đọc (ví dụ: thông báo có tin nhắn mới).

- **Bước 6 - Hệ thống:** Thực hiện ghi nhận và đánh dấu trạng thái thông báo là "Đã đọc" trong cơ sở dữ liệu, sau đó tự động điều hướng người dùng đến màn hình chức năng liên quan (ví dụ: phòng chat tương ứng).

Ngoại lệ:

- **Ngoại lệ tại Bước 5 - Đánh dấu đã đọc tất cả:** Thành viên không click vào từng thông báo mà chọn nút "Đánh dấu tất cả là đã đọc". Hệ thống thực hiện cập nhật hàng loạt trạng thái của tất cả thông báo thành "Đã đọc" và tắt dấu hiệu thông báo màu đỏ tại biểu tượng chuông.

f. Kịch bản báo cáo bài viết/người dùng

Thành viên gửi phản ánh đến Quản trị viên khi phát hiện nội dung vi phạm.

Luồng chính:

- **Bước 1 - Thành viên:** Đang xem chi tiết một bài đăng và nhận thấy nội dung có dấu hiệu lừa đảo hoặc nghi vấn.
- **Bước 2 - Hệ thống:** Hiện thị tùy chọn báo cáo trong menu thao tác của bài đăng hoặc người dùng.
- **Bước 3 - Thành viên:** Click chức năng "Báo cáo vi phạm".
- **Bước 4 - Hệ thống:** Hiện thị form yêu cầu nhập mô tả chi tiết lý do vi phạm.
- **Bước 5 - Thành viên:** Nhập nội dung vi phạm thực tế và click nút "Gửi báo cáo".
- **Bước 6 - Hệ thống:** Ghi nhận yêu cầu, tạo báo cáo ở trạng thái "Chờ xử lý", đưa vào hàng chờ dành cho Quản trị viên, hiện thị thông báo cảm ơn và đóng form báo cáo.

Ngoại lệ:

- **Ngoại lệ tại Bước 5 - Để trống lý do:** Nếu thành viên không nhập mô tả chi tiết mà thực hiện gửi, hệ thống hiện thị cảnh báo yêu cầu nhập đầy đủ lý do để làm cơ sở cho Quản trị viên xác minh và xử lý.

g. Kịch bản quản lý bài đăng

Quản trị viên (Admin) vào hệ thống để kiểm duyệt các bài đăng báo mất hoặc nhạt được đồ vật từ các thành viên trước khi công khai lên mạng lưới.

Luồng chính:

- **Bước 1 - Quản trị viên:** Đăng nhập vào hệ thống và chọn chức năng "Quản lý bài đăng" từ bảng điều khiển Dashboard.
- **Bước 2 - Hệ thống:** Hiển thị danh sách các bài viết đang ở trạng thái "Chờ duyệt", gồm các thông tin như tiêu đề, loại bài, người đăng, thời gian khởi tạo và trạng thái.

Bảng minh họa danh sách bài đăng chờ duyệt:

TT	Tiêu đề	Loại bài	Người đăng	Thời gian khởi tạo	Trạng thái	Thao tác
1	Roi ví da màu nâu tại sảnh A	Mất đồ	nguyenvan A	10:30 15/05/2026	Chờ duyệt	Xem / Duyệt / Từ chối
2	Nhặt được chìa khóa xe máy Honda	Nhặt được	tranthiB	14:15 15/05/2026	Chờ duyệt	Xem / Duyệt / Từ chối

- **Bước 3 - Quản trị viên:** Click vào nút "Xem" tương ứng với một bài đăng cần kiểm tra.
- **Bước 4 - Hệ thống:** Hiển thị màn hình "Chi tiết bài đăng" gồm hình ảnh minh chứng, nội dung mô tả, vị trí ghim trên bản đồ số và danh sách Tags được trích xuất từ AI Vision API.
- **Bước 5 - Quản trị viên:** Đọc, đối soát nội dung và click nút "Duyệt" nếu bài đăng hợp lệ.
- **Bước 6 - Hệ thống:** Cập nhật trạng thái bài viết thành "Hoạt động", hiển thị bài viết lên bảng tin và bản đồ số, đồng thời gửi thông báo thời gian thực đến chủ bài đăng.
- **Bước 7 - Quản trị viên:** Xác nhận kết quả xử lý và tiếp tục kiểm duyệt bài đăng khác nếu cần.
- **Bước 8 - Hệ thống:** Điều hướng về danh sách các bài đăng còn ở trạng thái chờ phê duyệt.

Ngoại lệ:

- **Ngoại lệ tại Bước 2 - Bảng kết quả rỗng:** Hệ thống hiển thị thông báo "Không có bài đăng nào đang chờ duyệt".
- **Ngoại lệ tại Bước 5 - Nội dung vi phạm (Spam/Lừa đảo):** Quản trị viên click vào nút "Từ chối" thay vì "Duyệt". Hệ thống tiến hành xóa vĩnh viễn bài viết khỏi cơ sở dữ liệu và gửi thông báo bài viết bị từ chối đến thành viên.

h. Kịch bản quản lý báo cáo vi phạm

Quản trị viên (Admin) xử lý các khiếu nại từ cộng đồng để đảm bảo môi trường mạng lưới an toàn, minh bạch.

Luồng chính:

- **Bước 1 - Quản trị viên:** Đăng nhập và chọn chức năng "Quản lý Báo cáo" trên thanh menu.
- **Bước 2 - Hệ thống:** Hiện thị danh sách các báo cáo đang chờ xử lý, gồm người gửi, đối tượng bị báo cáo, loại báo cáo, lý do vi phạm và trạng thái.

Bảng minh họa danh sách báo cáo vi phạm:

TT	Người gửi	Đối tượng bị báo cáo	Loại báo cáo	Lý do vi phạm	Trạng thái	Thao tác
1	tranvan C	Bài viết #1024	Bài đăng	Cố tình đăng tin giả mạo	Chờ xử lý	Xử lý
2	lethiD	User: nguyenvanA	Tài khoản	Spam tin nhắn lừa đảo chuyển tiền	Chờ xử lý	Xử lý

- **Bước 3 - Quản trị viên:** Click nút "Xử lý" tại một báo cáo liên quan đến tài khoản "nguyenvanA".
- **Bước 4 - Hệ thống:** Hiện thị giao diện chi tiết báo cáo, gồm lịch sử bài đăng và nội dung đoạn chat bị khiếu nại.
- **Bước 5 - Quản trị viên:** Xác minh thông tin và chọn hành động "Khóa tài khoản" kèm lý do xử lý.
- **Bước 6 - Hệ thống:** Hiện thị popup xác nhận khóa tài khoản.
- **Bước 7 - Quản trị viên:** Click nút "Xác nhận khóa".
- **Bước 8 - Hệ thống:** Cập nhật trạng thái tài khoản thành "Bị khóa", gỡ bỏ bài đăng liên quan, cập nhật báo cáo thành "Đã giải quyết" và thông báo thành công.

Ngoại lệ:

- **Ngoại lệ tại Bước 2 - Bảng danh sách rỗng:** Hệ thống báo "Không có báo cáo vi phạm nào đang chờ xử lý".
- **Ngoại lệ tại Bước 5 - Báo cáo sai sự thật:** Quản trị viên xác minh tài khoản không vi phạm nên chọn "Bỏ qua". Hệ thống cập nhật trạng thái báo cáo thành "Đã giải quyết" mà không xử lý tài khoản.
- **Ngoại lệ tại Bước 7 - Chọn hủy ở bước xác nhận:** Hệ thống đóng popup, tài khoản chưa bị khóa và báo cáo giữ nguyên trạng thái chờ xử lý.

i. Kịch bản quản lý danh mục

Quản trị viên (Admin) vào hệ thống để thiết lập và điều chỉnh các phân loại đồ vật nhằm chuẩn hóa dữ liệu đầu vào.

Luồng chính:

- **Bước 1 - Quản trị viên:** Đăng nhập bằng tài khoản quản trị và chọn chức năng "Quản lý danh mục" trên thanh menu điều hướng.
- **Bước 2 - Hệ thống:** Hiển thị danh sách danh mục hiện có, số bài đăng thuộc từng danh mục và các nút thao tác "Thêm", "Sửa", "Xóa".

Bảng minh họa danh sách danh mục:

TT	Tên danh mục	Mô tả	Số bài đăng	Thao tác
1	Thiết bị điện tử	Điện thoại, laptop, máy tính bảng...	142	Sửa / Xóa
2	Giấy tờ tùy thân	CCCD, bằng lái xe, thẻ sinh viên...	350	Sửa / Xóa
3	Phụ kiện, Trang sức	Đồng hồ, nhẫn, dây chuyền, ví...	89	Sửa / Xóa

- **Bước 3 - Quản trị viên:** Click nút "Thêm danh mục" để bổ sung một phân loại đồ vật mới.
- **Bước 4 - Hệ thống:** Hiển thị form thêm danh mục gồm trường tên danh mục, mô tả và nút xác nhận.
- **Bước 5 - Quản trị viên:** Nhập tên danh mục "Thú cưng", điền mô tả và click nút "Thêm".
- **Bước 6 - Hệ thống:** Lưu danh mục mới vào cơ sở dữ liệu PostgreSQL, hiển thị thông báo thành công và cập nhật lại danh sách danh mục.
- **Bước 7 - Quản trị viên:** Chọn một danh mục trong danh sách và click nút "Sửa" nếu cần chỉnh thông tin.
- **Bước 8 - Hệ thống:** Hiển thị form sửa danh mục với dữ liệu hiện tại được điền sẵn.
- **Bước 9 - Quản trị viên:** Cập nhật thông tin và click nút "Lưu thay đổi".
- **Bước 10 - Hệ thống:** Cập nhật dữ liệu vào cơ sở dữ liệu, hiển thị thông báo thành công và quay lại danh sách danh mục.

Ngoại lệ:

- **Ngoại lệ tại Bước 2 - Danh sách danh mục rỗng:** Hệ thống báo không có danh mục nào trong kết quả hiển thị.
- **Ngoại lệ tại Bước 5 - Tên danh mục đã tồn tại:** Quản trị viên nhập tên danh mục đã tồn tại, hệ thống báo lỗi và yêu cầu nhập lại.
- **Ngoại lệ tại Bước 7 - Quản trị viên chọn xóa danh mục:** Quản trị viên click nút "Xóa" thay vì "Sửa". Hệ thống hiển thị popup xác nhận xóa. Nếu Quản trị viên xác nhận, hệ thống xóa bản ghi khỏi cơ sở dữ liệu và cập nhật lại bảng danh sách.

j. Kịch bản xem báo cáo thống kê

Quản trị viên (Admin) theo dõi tình hình hoạt động của hệ thống thông qua các số liệu thống kê.

Luồng chính:

- **Bước 1 - Quản trị viên:** Vào "Xem báo cáo thống kê" sau khi đăng nhập.
- **Bước 2 - Hệ thống:** Truy vấn CSDL và hiển thị các khối dữ liệu tổng quan gồm tổng số người dùng, người dùng mới bài đăng mới và bộ lọc thời gian.
- **Bước 3 - Quản trị viên:** Chọn khoảng thời gian "Tháng hiện tại" (Tháng 5/2026).
- **Bước 4 - Hệ thống:** Cập nhật số liệu và hiển thị bảng chi tiết theo từng tuần, gồm người dùng mới, lượt truy cập và bài đăng mới.

Bảng minh họa số liệu thống kê theo tuần:

TT	Tuần	Người dùng mới	Bài đăng mới
1	Tuần 1 (1-7/5)	68	205
2	Tuần 2 (8-14/5)	72	230
3	Tuần 3 (15-21/5)	55	210
4	Tuần 4 (22-28/5)	64	225
Tổng		259	870

- **Bước 5 - Quản trị viên:** Click nút "Xuất báo cáo".
- **Bước 6 - Hệ thống:** Tạo tệp Excel (.xlsx) chứa số liệu thống kê hiện tại và tải xuống thiết bị của Quản trị viên.

Ngoại lệ:

- **Ngoại lệ tại Bước 4 - Không có dữ liệu thống kê:** Nếu khoảng thời gian lọc chưa ghi nhận dữ liệu, hệ thống hiển thị thông báo "Chưa có dữ liệu thống kê cho khoảng thời gian này" và các chỉ số mặc định bằng 0.

3.2. Trích các lớp thực thể

Mô tả hệ thống trong một đoạn văn như sau:

Hệ thống quản lý thông tin về các bài đăng báo mất hoặc nhật được đồ vật, thông tin về người dùng (Thành viên và Quản trị viên). Hệ thống cho phép Thành viên đăng tải bài viết thông qua quy trình 4 bước (chọn loại bài, mô tả, ghim vị trí trên bản đồ, tải ảnh và nhận gợi ý Tag từ AI), tìm kiếm đồ vật theo danh sách hoặc trên bản đồ số, trao đổi tin nhắn nội bộ với các thành viên khác, quản lý hồ sơ cá nhân và các bài đăng của mình, nhận thông báo thời gian thực, và báo cáo vi phạm. Hệ thống cũng cho phép Quản trị viên xét duyệt bài đăng, quản lý danh mục đồ vật, xử lý các báo cáo vi phạm từ cộng đồng, và xem báo cáo thống kê về người dùng và bài đăng theo thời gian. Mỗi bài đăng được phân loại theo danh mục, gán các thẻ Tag do AI trích xuất, và lưu trữ tọa độ vị trí để hỗ trợ tìm kiếm trên bản đồ.

Như vậy, ta có các danh từ và các phân tích như sau:

- Hệ thống: danh từ chung chung --> loại.
- Thông tin: danh từ chung chung --> loại.
- Bài đăng: là đối tượng xử lý chính của hệ thống --> là 1 lớp thực thể: Post
- Đồ vật: trừu tượng, chung chung, được mô tả thông qua bài đăng --> loại (thuộc tính của Post).
- Loại bài đăng (Mất/Nhật) và trạng thái bài đăng (Chờ duyệt/Hoạt động/Hoàn tất): là thuộc tính của Post (enum). Trường hợp "Từ chối" thì bài đăng bị xóa, không lưu trạng thái.
- Thành viên: là người dùng trực tiếp của phần mềm, cùng với Quản trị viên đều được quản lý theo kiểu tài khoản người dùng --> đề xuất là 1 lớp thực thể chung: User

- Quản trị viên: là người dùng trực tiếp của phần mềm, cùng với Thành viên đều được quản lý theo kiểu tài khoản người dùng --> đề xuất là 1 lớp thực thể chung: User (phân biệt qua thuộc tính role).
- Trạng thái tài khoản (Hoạt động/Bị khóa): là thuộc tính của User (enum).
- Khách: chưa có tài khoản, chỉ thực hiện đăng ký --> không lưu trữ riêng, sau đăng ký trở thành User --> loại.
- Danh mục: là đối tượng xử lý của hệ thống, phân loại đồ vật --> là 1 lớp thực thể: Category
- Thẻ (Tag): là đối tượng xử lý của hệ thống, do AI trích xuất từ ảnh --> là 1 lớp thực thể: Tag
- PostTag: là lớp liên kết giữa Post và Tag để lưu thông tin gắn thẻ --> là 1 lớp thực thể: PostTag
- Vị trí, tọa độ, địa chỉ: là thông tin gắn liền với bài đăng, lưu trữ dưới dạng tọa độ địa lý --> loại (thuộc tính của Post, kiểu Geometry/PostGIS).
- Hình ảnh: là đối tượng đính kèm bài đăng (tối đa 3 ảnh) --> là 1 lớp thực thể: Image
- Tin nhắn: là đối tượng xử lý của hệ thống, lưu trữ nội dung trao đổi --> là 1 lớp thực thể: Message
- Phòng chat: là đối tượng nhóm các tin nhắn giữa 2 thành viên --> là 1 lớp thực thể: ChatRoom
- Thông báo: là đối tượng xử lý của hệ thống, đẩy cảnh báo đến người dùng, gồm loại sự kiện và trạng thái đã đọc/chưa đọc --> là 1 lớp thực thể: Notification
- Báo cáo vi phạm: là đối tượng xử lý của hệ thống --> là 1 lớp thực thể: Report
- Report lưu người gửi, nội dung vi phạm, trạng thái xử lý và liên kết đến Post hoặc User bị báo cáo.
- Bản đồ, AI, Socket.io: là dịch vụ bên ngoài, không thuộc phạm vi lưu trữ của phần mềm --> loại.
- Các thông tin thống kê: là dữ liệu tổng hợp theo thời gian, tách thành lớp thống kê riêng --> UserStat (theo kỳ, gồm người dùng mới/lượt truy cập), PostStat (theo kỳ, gồm bài đăng mới/tổng bài đang hoạt động).

Vậy chúng ta thu được các lớp thực thể ban đầu là: Post, User, Category, Tag, PostTag, Image, Message, ChatRoom, Notification, Report và các lớp thống kê: UserStat, PostStat.

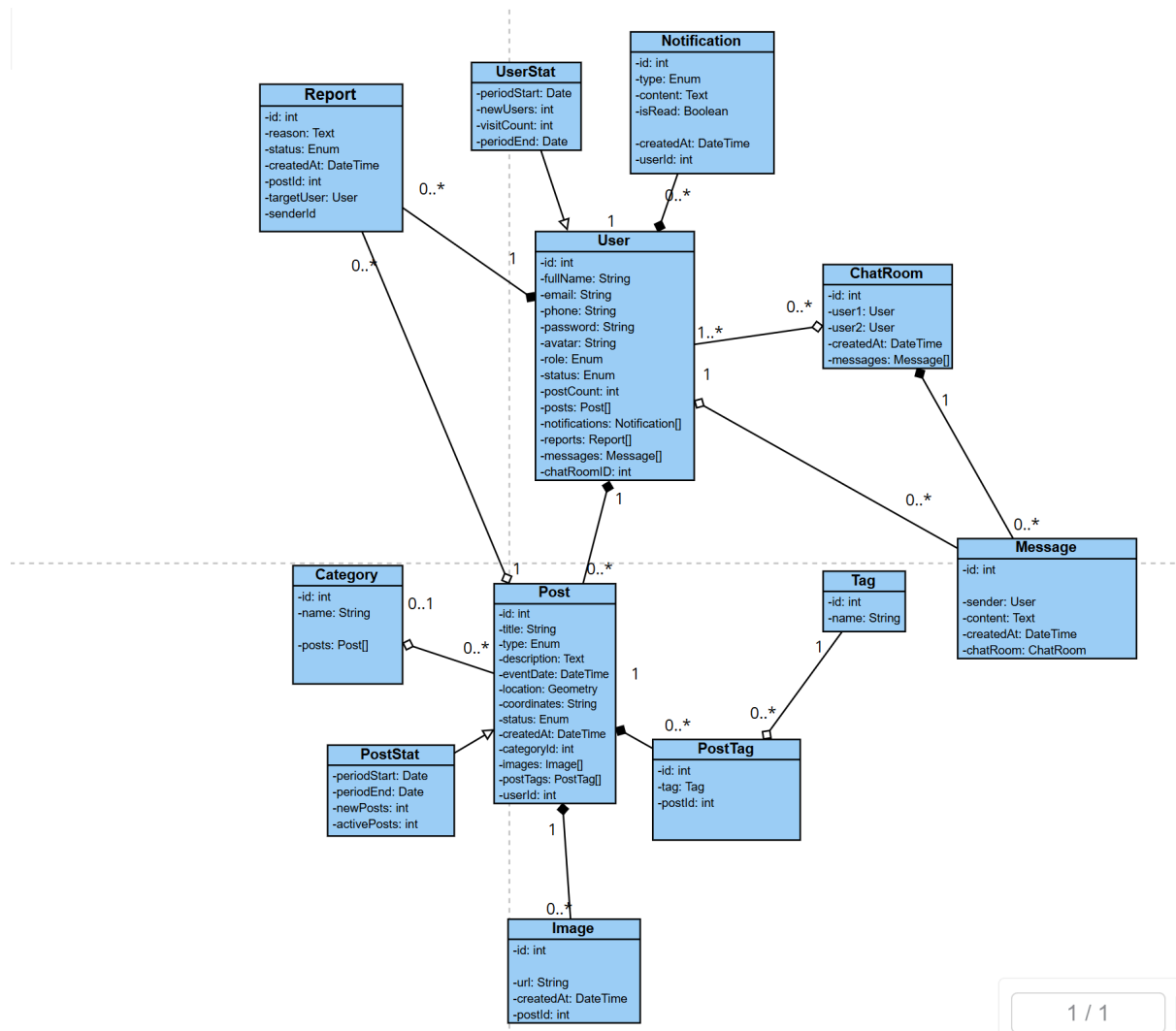
Quan hệ giữa các lớp thực thể được xác định như sau:

- Một User có thể tạo nhiều Post, một Post chỉ thuộc về một User duy nhất (người đăng). Vậy quan hệ giữa User và Post là 1-n.

- Một Category có thể chứa nhiều Post, một Post hoặc không thuộc danh mục hoặc chỉ thuộc một Category duy nhất. Vậy quan hệ giữa Category và Post là 1-n (tùy chọn).
- Một Post có thể gắn nhiều Tag, một Tag có thể thuộc nhiều Post khác nhau: quan hệ giữa Post và Tag là n-n. Do đó bổ sung một lớp thực thể liên kết giữa hai đối tượng này là PostTag (thông tin gắn thẻ cho bài đăng).
- Một Post có nhiều PostTag, một Tag có nhiều PostTag: quan hệ Post-PostTag là 1-n và Tag-PostTag là 1-n.
- Một Post có thể đính kèm nhiều Image (tối đa 3 ảnh), một Image chỉ thuộc về một Post duy nhất. Vậy quan hệ giữa Post và Image là 1-n.
- Một User có thể tham gia nhiều ChatRoom, một ChatRoom luôn có đúng 2 User tham gia: quan hệ giữa User và ChatRoom là n-n. Tuy nhiên do mỗi ChatRoom cố định 2 người, ta lưu trực tiếp 2 khóa ngoại user1_id và user2_id trong ChatRoom.
- Một ChatRoom chứa nhiều Message, một Message chỉ thuộc một ChatRoom duy nhất. Vậy quan hệ giữa ChatRoom và Message là 1-n. Mỗi Message cũng thuộc về một User (người gửi), quan hệ giữa User và Message là 1-n.
- Một User có thể nhận nhiều Notification, một Notification chỉ gửi đến một User duy nhất. Vậy quan hệ giữa User và Notification là 1-n.
- Một User có thể gửi nhiều Report, một Report chỉ thuộc về một User gửi. Vậy quan hệ giữa User và Report là 1-n. Một Report có thể liên quan đến một Post hoặc một User bị báo cáo.

Đối với các lớp thống kê, dữ liệu được lưu theo kỳ thời gian (ngày/tuần/tháng), có khóa thời gian và chỉ số tổng hợp; không kế thừa trực tiếp từ User hay Post.

Như vậy, ta thu được sơ đồ các lớp thực thể của hệ thống như sau:



3.3. Phân tích chi tiết từng module

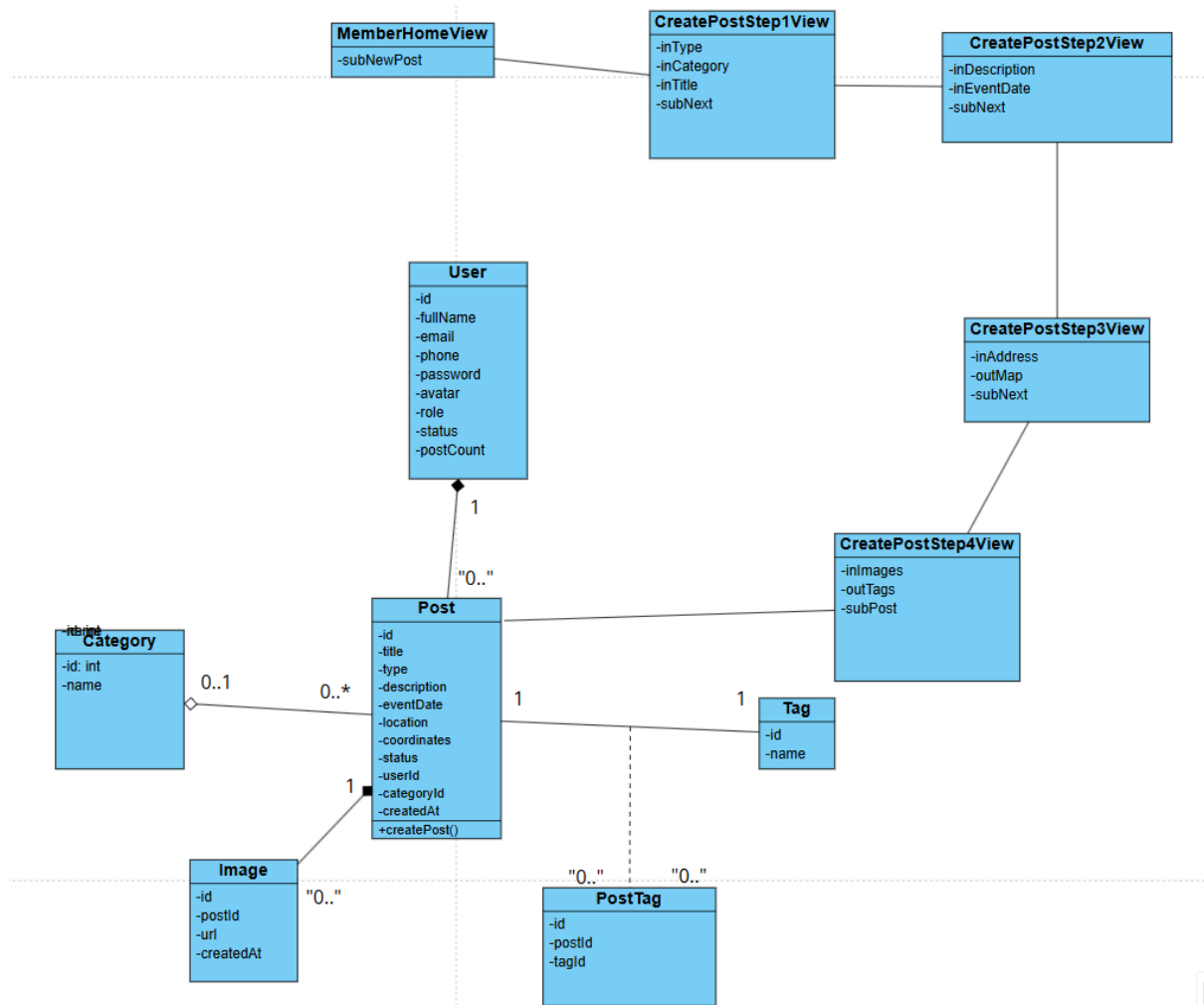
- Nội dung phần này sẽ phân tích chi tiết hoạt động từng module với hai bước: phân tích tĩnh và phân tích động.
- **Phân tích tĩnh** là phân tích các lớp biên và lớp đối tượng cần thiết trong chức năng (biểu đồ lớp).
- **Phân tích động** là phân tích các bước hoạt động của chức năng (biểu đồ tuần tự/cộng tác). Các chức năng được phân tích ở đây là: đăng tải bài viết, tìm kiếm đồ vật, xem báo cáo thống kê, trao đổi tin nhắn, xét duyệt bài đăng, quản lý báo cáo vi phạm, quản lý danh mục, quản lý hồ sơ cá nhân, quản lý thông báo, báo cáo bài viết/người dùng và đăng ký tài khoản.

a. Chức năng đăng tải bài viết

Phân tích chi tiết chức năng đăng tải bài viết diễn ra như sau:

- Vào hệ thống -> giao diện login hiện lên -> đề xuất lớp LoginView, có 2 ô nhập email, password và nút Đăng nhập.
- Nhập email/password -> hệ thống phải kiểm tra thông tin đăng nhập -> cần chức năng checkLogin() -> chức năng này là hành động của đối tượng User.
- Login thành công, hệ thống hiện giao diện chính (Bảng tin) -> đề xuất lớp MemberHomeView, có ít nhất nút "Đăng bài mới".
- Click vào nút "Đăng bài mới" -> giao diện bước 1 hiện lên -> đề xuất lớp CreatePostStep1View, có ô chọn loại bài (Mất đồ/Nhặt được), dropdown chọn danh mục, ô nhập tiêu đề, nút "Tiếp theo".
- Chọn loại bài, danh mục, nhập tiêu đề, click "Tiếp theo" -> giao diện bước 2 hiện lên -> đề xuất lớp CreatePostStep2View, có ô nhập mô tả chi tiết, ô chọn ngày tháng, nút "Tiếp theo".
- Nhập mô tả, chọn ngày, click "Tiếp theo" -> giao diện bước 3 hiện lên -> đề xuất lớp CreatePostStep3View (tích hợp Bản đồ số Leaflet.js), có ô nhập địa chỉ, bản đồ để ghim vị trí, nút "Tiếp theo".
- Ghim vị trí trên bản đồ, click "Tiếp theo" -> giao diện bước 4 hiện lên -> đề xuất lớp CreatePostStep4View, có khu vực tải ảnh, danh sách Tags gợi ý, nút "Đăng bài".
- Tải ảnh lên -> hệ thống gửi ảnh qua AI Vision API -> cần chức năng analyzeImage() -> chức năng này là hành động của dịch vụ AI bên ngoài, kết quả trả về danh sách Tag gợi ý.
- Hệ thống hiển thị Tags gợi ý lên CreatePostStep4View -> thành viên có thể xóa Tag sai.
- Click "Đăng bài" -> hệ thống lưu toàn bộ dữ liệu vào CSDL -> cần chức năng createPost() -> chức năng này là hành động của đối tượng Post.
- Hệ thống lưu các ảnh tải lên vào bảng Image (tối đa 3 ảnh) gắn với Post.
- Hệ thống đồng thời tạo các bản ghi PostTag cho các Tag được chọn.
- Lưu xong, hệ thống quay về giao diện MemberHomeView và thông báo "Bài viết đang chờ duyệt".

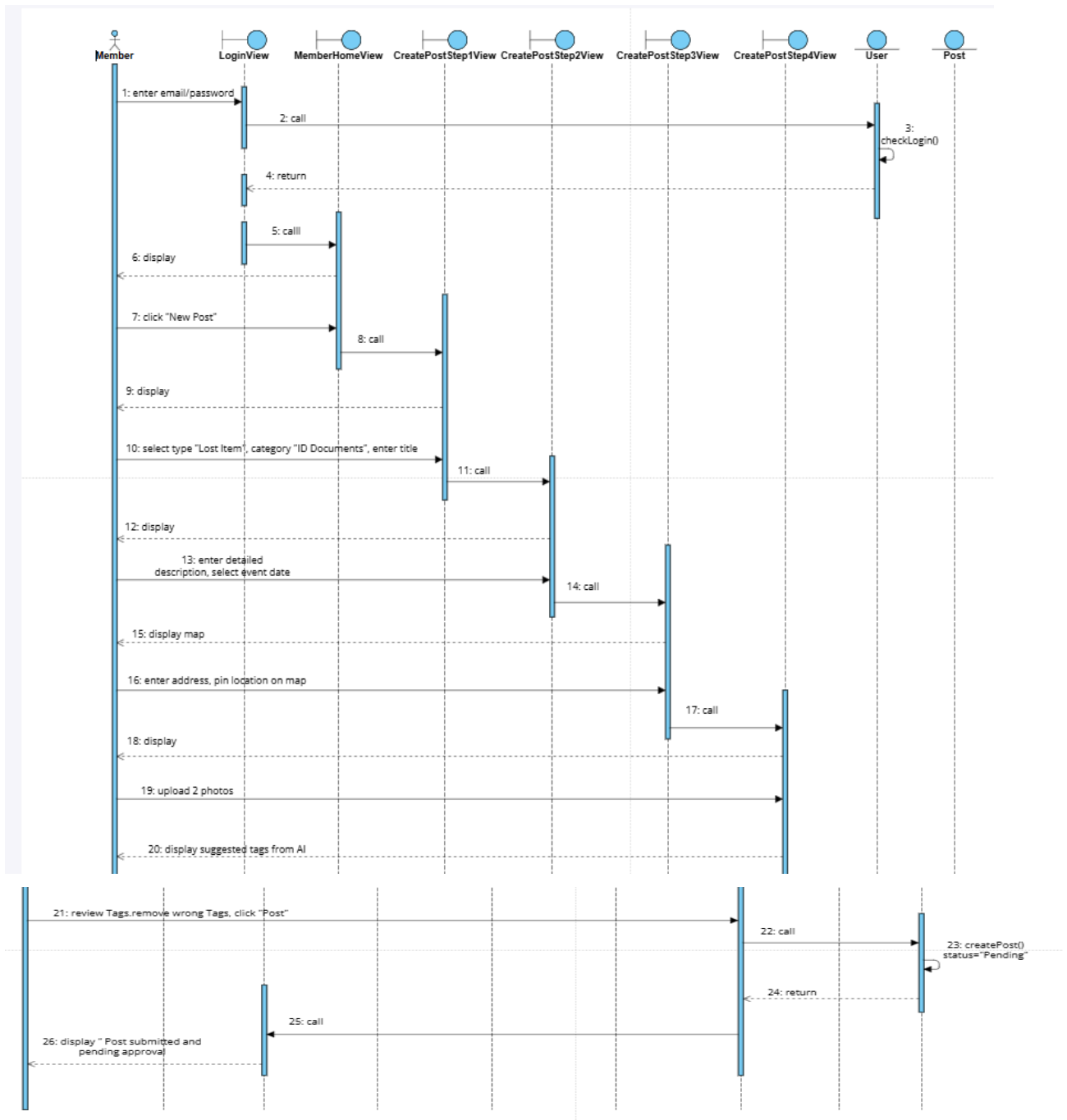
Từ các phân tích trên, biểu đồ lớp cho chức năng đăng tải bài viết cần thể hiện các lớp giao diện theo 4 bước và các lớp xử lý User, Post, Image, Tag, PostTag.



Với biểu đồ lớp như trên, kịch bản chi tiết cho chức năng đăng tải bài viết diễn ra như sau:

1. Thành viên nhập email/password vào giao diện đăng nhập và click nút Đăng nhập.
2. Lớp LoginView gọi đến lớp User để xử lí.
3. Lớp User gọi hàm kiểm tra đăng nhập. Kết quả đăng nhập thành công.
4. Lớp User gửi kết quả lại cho lớp LoginView.
5. Lớp LoginView gọi sang lớp MemberHomeView.
6. Lớp MemberHomeView hiển thị cho thành viên.
7. Thành viên click vào nút "Đăng bài".
8. Lớp MemberHomeView gọi lớp CreatePostStep1View.
9. Lớp CreatePostStep1View hiển thị cho thành viên.
10. Thành viên chọn loại bài "Mất đồ", chọn danh mục "Giấy tờ tùy thân", nhập tiêu đề và click "Tiếp theo".

11. Lớp CreatePostStep1View gọi sang lớp CreatePostStep2View.
12. Lớp CreatePostStep2View hiển thị cho thành viên.
13. Thành viên nhập mô tả chi tiết đặc điểm nhận dạng, chọn ngày xảy ra sự việc và click "Tiếp theo".
14. Lớp CreatePostStep2View gọi sang lớp CreatePostStep3View.
15. Lớp CreatePostStep3View hiển thị bản đồ cho thành viên.
16. Thành viên gõ địa chỉ, ghim vị trí trên bản đồ và click "Tiếp theo".
17. Lớp CreatePostStep3View gọi sang lớp CreatePostStep4View.
18. Lớp CreatePostStep4View hiển thị cho thành viên.
19. Thành viên tải lên 2 bức ảnh minh chứng.
20. Lớp CreatePostStep4View gọi dịch vụ AI Vision API để phân tích ảnh.
21. Dịch vụ AI Vision API trả về danh sách Tags gợi ý cho lớp CreatePostStep4View.
22. Lớp CreatePostStep4View hiển thị các Tags gợi ý lên cho thành viên.
23. Thành viên kiểm tra Tags, xóa Tag sai (nếu có) và click nút "Đăng bài".
24. Lớp CreatePostStep4View gọi lớp Post xử lý.
25. Lớp Post gọi phương thức tạo bài đăng mới với trạng thái "Chờ duyệt".
26. Lớp Post lưu danh sách ảnh vào bảng Image (tối đa 3 ảnh).
27. Lớp Post tạo các bản ghi PostTag tương ứng với các Tag đã chọn.
28. Lớp Post trả kết quả lại cho lớp CreatePostStep4View.
29. Lớp CreatePostStep4View gọi lại lớp MemberHomeView.
30. Lớp MemberHomeView hiển thị thông báo "Bài viết đã được gửi và đang chờ duyệt" cho thành viên.



b. Chức năng tìm kiếm đồ vật

Phân tích chi tiết chức năng tìm kiếm đồ vật (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

Sau khi đăng nhập thành công, giao diện chính Bảng tin hiện ra -> đề xuất lớp MemberHomeView, có thanh tìm kiếm, bộ lọc danh mục/loại bài/trạng thái/thời gian và nút "Bản đồ (Map Search)".

Nếu tìm theo danh sách, thành viên nhập từ khóa và chọn bộ lọc -> hệ thống truy vấn CSDL -> cần chức năng searchByKeywordAndFilter() -> chức năng này là hành động của đối tượng Post.

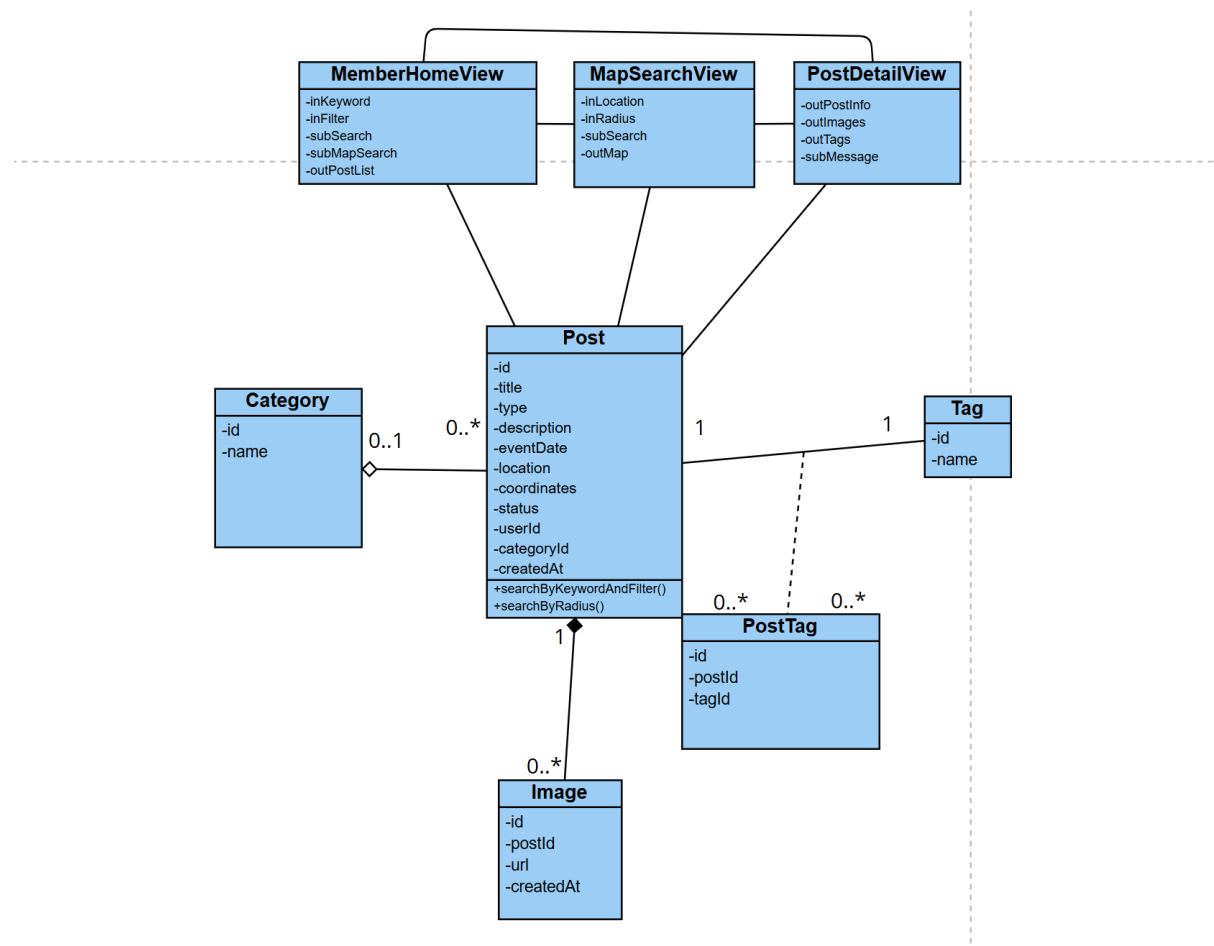
Kết quả tìm theo danh sách được hiển thị trên MemberHomeView, mỗi dòng/thẻ bài đăng có thể mở sang PostDetailView.

Nếu tìm theo bản đồ, thành viên click nút "Bản đồ" -> giao diện bản đồ toàn màn hình hiện ra -> đề xuất lớp MapSearchView, có ô nhập vị trí trọng tâm, ô chọn bán kính quét, bản đồ hiển thị ghim và nút tìm.

Thành viên nhập vị trí trọng tâm và chọn bán kính quét (VD: 5km), click nút tìm -> hệ thống truy vấn không gian PostGIS -> cần chức năng searchByRadius() -> chức năng này là hành động của đối tượng Post.

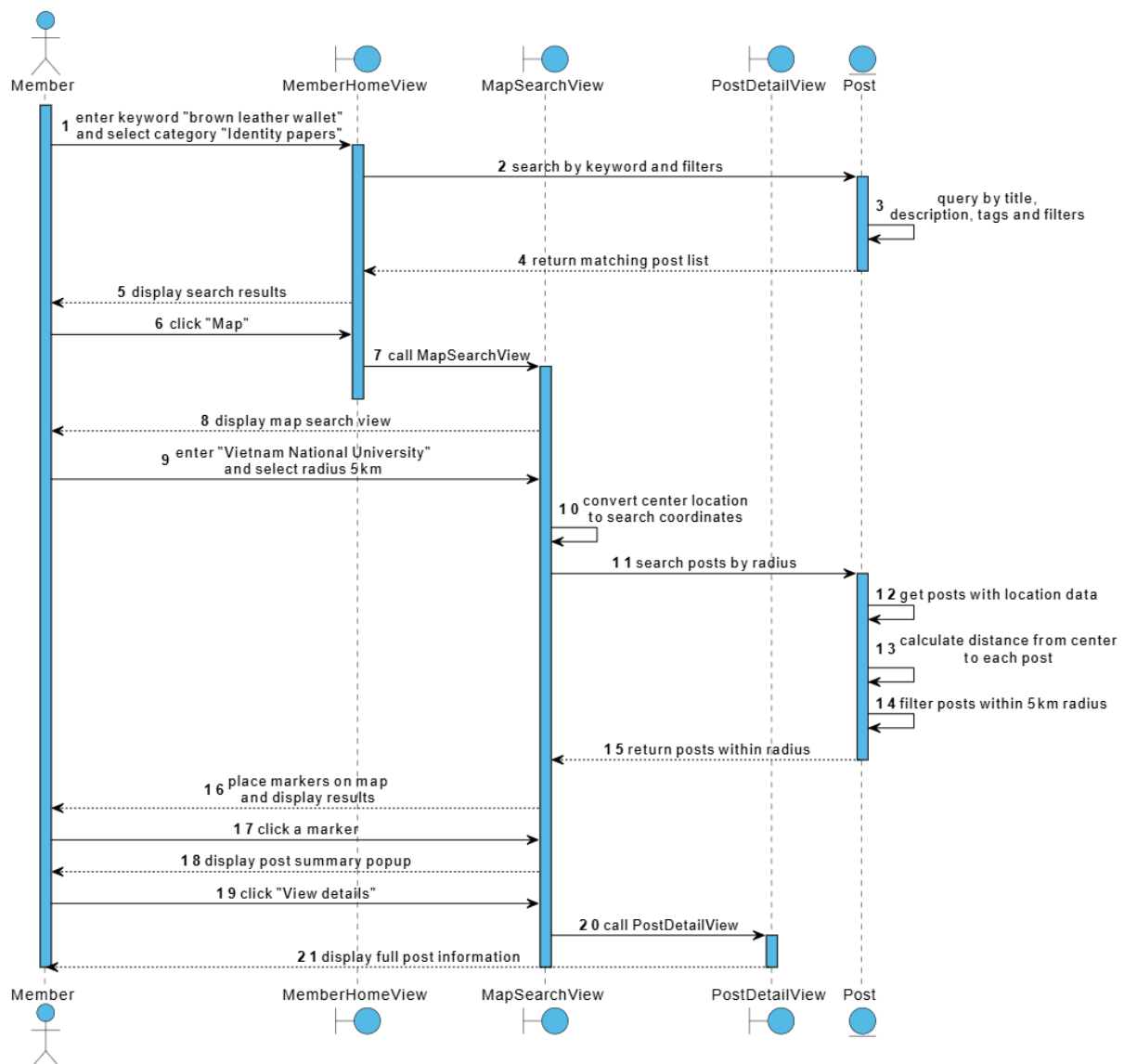
Tìm xong, danh sách các bài đăng trong bán kính được cắm ghim (Marker) lên bản đồ trong MapSearchView.

Thành viên click vào một ghim trên bản đồ -> hệ thống hiển thị popup tóm tắt bài đăng -> thành viên click "Xem chi tiết" -> giao diện chi tiết bài đăng hiện lên -> đề xuất lớp PostDetailView, có đầy đủ thông tin bài đăng (ảnh, mô tả, vị trí, Tags) và nút "Nhắn tin".



Với biểu đồ lớp này, kịch bản chi tiết cho chức năng tìm kiếm đồ vật diễn ra như sau:

1. Thành viên nhập từ khóa “ví da nâu” và chọn bộ lọc danh mục “Giấy tờ tùy thân” trên giao diện MemberHomeView.
2. Lớp MemberHomeView yêu cầu lớp Post xử lý tìm kiếm theo từ khóa và bộ lọc.
3. Lớp Post truy vấn dữ liệu theo tiêu đề, mô tả, Tags và các điều kiện lọc.
4. Lớp Post trả danh sách bài đăng phù hợp về MemberHomeView.
5. Lớp MemberHomeView hiển thị danh sách kết quả cho thành viên.
6. Thành viên tiếp tục click nút “Bản đồ”.
7. Lớp MemberHomeView gọi lớp MapSearchView.
8. Lớp MapSearchView hiển thị cho thành viên.
9. Thành viên nhập vị trí trọng tâm “Đại học Quốc gia” và chọn bán kính 5km.
10. Lớp MapSearchView chuyển vị trí trọng tâm thành tọa độ tìm kiếm.
11. Lớp MapSearchView yêu cầu lớp Post xử lý tìm kiếm bài đăng theo bán kính.
12. Lớp Post lấy danh sách bài đăng có thông tin vị trí.
13. Lớp Post tính khoảng cách giữa vị trí trọng tâm và vị trí của từng bài đăng.
14. Lớp Post lọc các bài đăng nằm trong bán kính 5km.
15. Lớp Post trả kết quả các bài đăng trong bán kính về MapSearchView.
16. Lớp MapSearchView cắm các ghim (Marker) lên bản đồ và hiển thị cho thành viên.
17. Thành viên click vào một ghim gần khu vực cần tìm.
18. Lớp MapSearchView hiển thị popup tóm tắt bài đăng.
19. Thành viên click “Xem chi tiết” từ popup.
20. Lớp MapSearchView gọi sang lớp PostDetailView.
21. Lớp PostDetailView hiển thị đầy đủ thông tin bài đăng cho thành viên.



c. Chức năng trao đổi tin nhắn

Phân tích chi tiết chức năng trao đổi tin nhắn (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

- Sau khi đăng nhập thành công, thành viên có thể bắt đầu trao đổi tin nhắn theo hai cách.

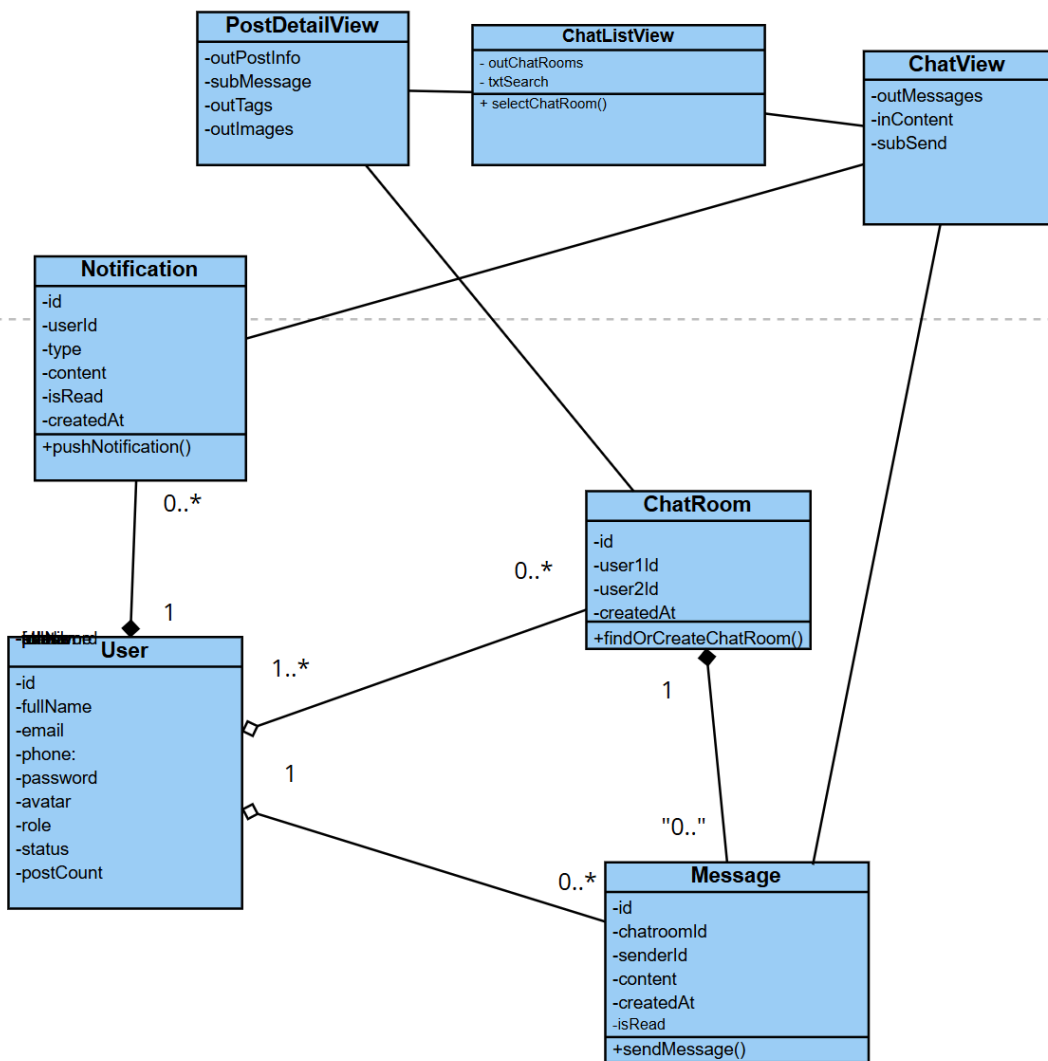
Cách thứ nhất, thành viên đang ở giao diện chi tiết bài đăng -> đề xuất lớp PostDetailView, có thông tin bài đăng và nút "Nhắn tin" cho chủ bài viết. Thành viên click nút "Nhắn tin" -> hệ thống kiểm tra giữa thành viên hiện tại và chủ bài viết đã tồn tại phòng chat hay chưa -> cần chức năng findOrCreateChatRoom() -> chức năng này là hành động của đối tượng ChatRoom. Nếu phòng chat đã tồn tại, hệ thống mở phòng chat cũ; nếu chưa tồn tại, hệ thống tạo phòng chat mới.

Cách thứ hai, thành viên truy cập mục "Tin nhắn" từ giao diện chính -> đề xuất lớp ChatListView. Giao diện này hiển thị danh sách các cuộc trò

chuyện của thành viên, gồm thông tin người đang trao đổi, tin nhắn gần nhất, thời gian gửi và trạng thái chưa đọc. Thành viên chọn một cuộc trò chuyện trong danh sách -> hệ thống mở giao diện chi tiết phòng chat tương ứng.

- Giao diện phòng chat hiện ra -> đề xuất lớp ChatView, có thông tin người đang trao đổi, khung hiển thị danh sách tin nhắn, thẻ tóm tắt bài đăng liên quan, ô nhập văn bản và nút "Gửi".
- Khi giao diện phòng chat được mở, hệ thống lấy danh sách tin nhắn cũ trong phòng chat -> cần chức năng `getMessages()` -> chức năng này là hành động của đối tượng Message.
- Thành viên nhập nội dung tin nhắn và click "Gửi" -> hệ thống lưu tin nhắn vào cơ sở dữ liệu -> cần chức năng `sendMessage()` -> chức năng này là hành động của đối tượng Message.
- Sau khi lưu tin nhắn thành công, hệ thống sử dụng Socket.io để hiển thị tin nhắn thời gian thực lên khung chat của cả hai bên. Đồng thời, hệ thống đẩy thông báo đến người nhận -> cần chức năng `pushNotification()` -> chức năng này là hành động của đối tượng Notification.

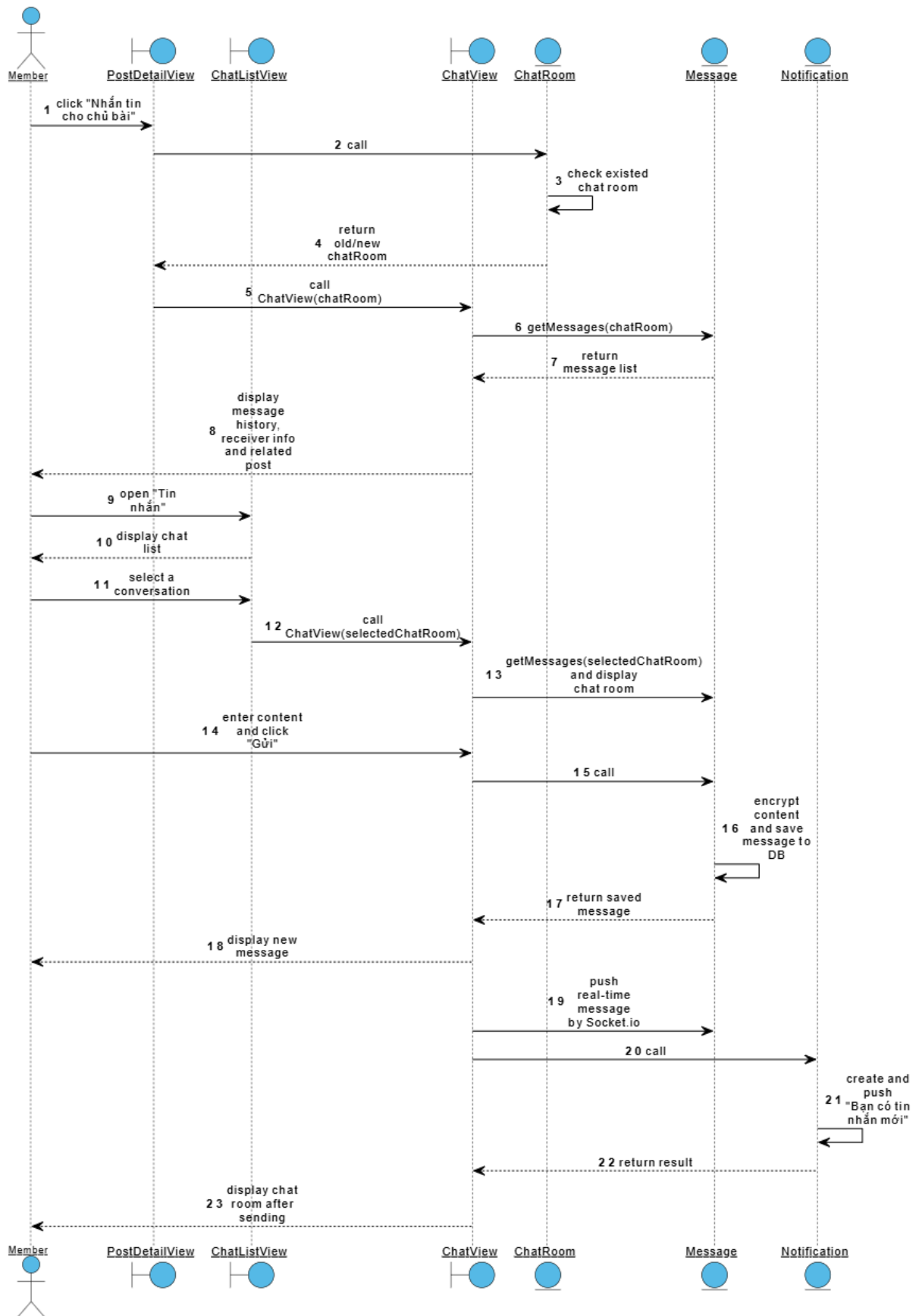
Như vậy, kết quả thu được biểu đồ lớp cho chức năng trao đổi tin nhắn như trên. Với biểu đồ lớp này, kịch bản chi tiết cho chức năng trao đổi tin nhắn diễn ra như sau:



Như vậy, kết quả thu được biểu đồ lớp cho chức năng trao đổi tin nhắn như trên. Với biểu đồ lớp này, kịch bản chi tiết cho chức năng trao đổi tin nhắn diễn ra như sau:

1. Thành viên đang ở giao diện `PostDetailView`, click nút "Nhắn tin cho chủ bài".
2. Lớp `PostDetailView` gọi lớp `ChatRoom` xử lý.
3. Lớp `ChatRoom` thực hiện kiểm tra phòng chat giữa thành viên hiện tại và chủ bài đăng đã tồn tại hay chưa.
4. Nếu phòng chat đã tồn tại, lớp `ChatRoom` trả về thông tin phòng chat cũ; nếu chưa tồn tại, lớp `ChatRoom` tạo phòng chat mới rồi trả kết quả về cho `PostDetailView`.
5. Lớp `PostDetailView` gọi sang lớp `ChatView` và truyền thông tin phòng chat vừa nhận được.
6. `ChatView` gọi lớp `Message` để lấy danh sách tin nhắn cũ của phòng chat.

7. Message trả về danh sách tin nhắn cho ChatView.
8. ChatView hiển thị lịch sử tin nhắn, thông tin người đang trao đổi và thẻ tóm tắt bài đăng liên quan.
9. Ngoài luồng mở từ chi tiết bài đăng, thành viên có thể truy cập mục "Tin nhắn" trên giao diện chính.
10. Hệ thống hiển thị ChatListView với danh sách các cuộc trò chuyện của thành viên.
11. Thành viên chọn một cuộc trò chuyện trong ChatListView.
12. ChatListView gọi sang ChatView và truyền thông tin phòng chat được chọn.
13. ChatView gọi lớp Message để lấy danh sách tin nhắn cũ và hiển thị giao diện phòng chat tương ứng.
14. Thành viên nhập nội dung "Chào bạn, cho mình hỏi chiếc ví có chứa thẻ tên Nguyễn Văn A không?" và click nút "Gửi".
15. Lớp ChatView gọi lớp Message xử lý.
16. Lớp Message thực hiện mã hóa nội dung và lưu tin nhắn vào cơ sở dữ liệu.
17. Lớp Message trả kết quả lưu tin nhắn về cho ChatView.
18. ChatView cập nhật tin nhắn mới lên khung chat của người gửi.
19. Hệ thống sử dụng Socket.io để đẩy tin nhắn thời gian thực đến người nhận, giúp tin nhắn hiển thị ngay trên khung chat của cả hai bên.
20. Lớp ChatView gọi lớp Notification xử lý.
21. Lớp Notification tạo và đẩy thông báo "Bạn có tin nhắn mới" đến người nhận.
22. Lớp Notification trả kết quả xử lý về cho ChatView.
23. Lớp ChatView hiển thị cho người xem



d. Chức năng quản lý hồ sơ cá nhân

Phân tích chi tiết chức năng quản lý hồ sơ cá nhân (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

Sau khi đăng nhập thành công, giao diện chính của thành viên hiện ra -> đề xuất lớp MemberHomeView, có mục truy cập "Hồ sơ".

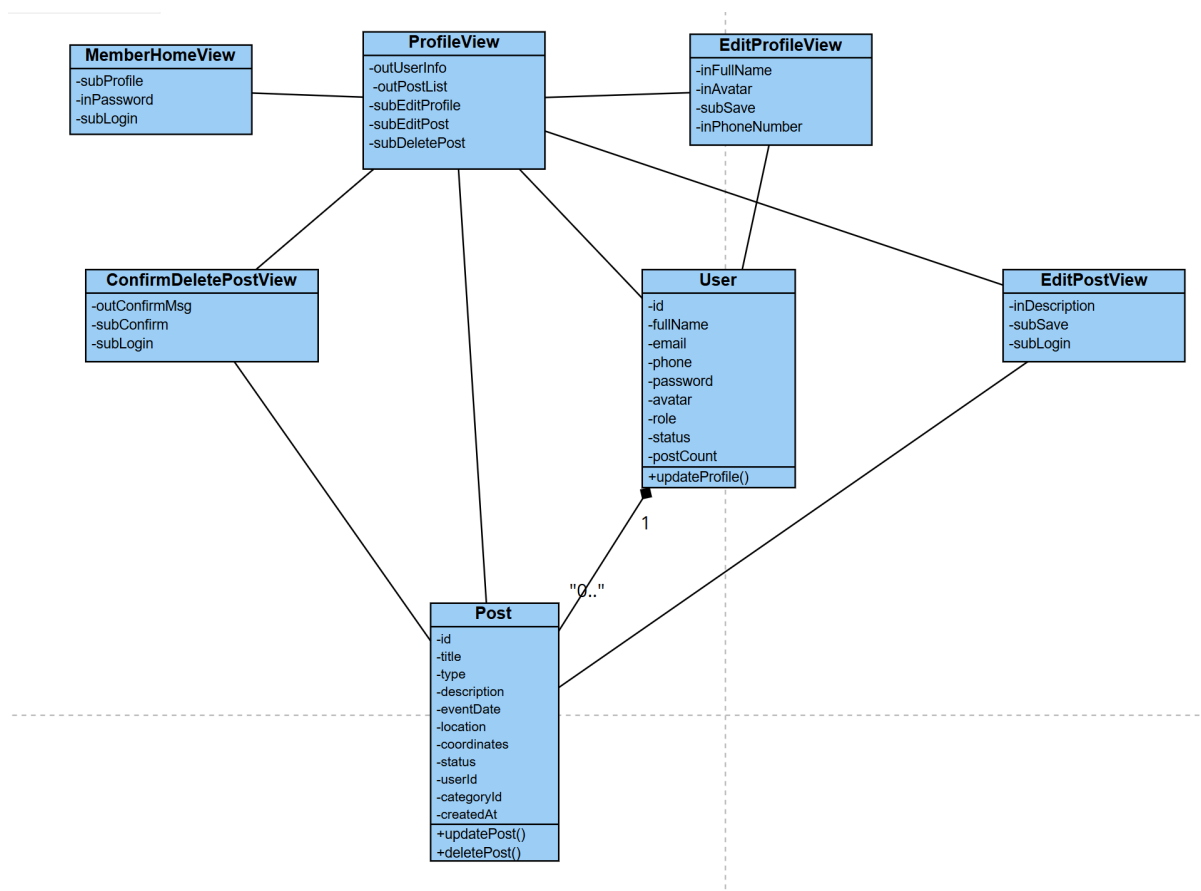
Thành viên click "Hồ sơ" -> giao diện hồ sơ hiện ra -> đề xuất lớp ProfileView, hiển thị thông tin User và danh sách bài đăng của thành viên.

Thành viên click "Sửa thông tin" -> giao diện chỉnh sửa thông tin cá nhân hiện ra -> đề xuất lớp EditProfileView, cần chức năng updateProfile() là hành động của đối tượng User.

Thành viên chọn một bài đăng cũ và click "Sửa đổi" -> giao diện chỉnh sửa bài viết hiện ra -> đề xuất lớp EditPostView, cần chức năng updatePost() là hành động của đối tượng Post.

Khi lưu thay đổi bài đăng, hệ thống cập nhật nội dung, chuyển trạng thái bài viết về "Chờ duyệt" và tạm ẩn khỏi bảng tin chung để quản trị viên xét duyệt lại.

Nếu thành viên chọn "Xóa" -> hệ thống hiển thị popup xác nhận -> đề xuất lớp ConfirmDeletePostView, cần chức năng deletePost() là hành động của đối tượng Post.



Kịch bản chi tiết cho chức năng quản lý hồ sơ cá nhân diễn ra như sau:

Thành viên click mục "Hồ sơ cá nhân" trên giao diện MemberHomeView.

Lớp MemberHomeView gọi lớp ProfileView.

Lớp ProfileView yêu cầu lớp User lấy thông tin cá nhân và yêu cầu lớp Post lấy danh sách bài đăng của thành viên.

Lớp ProfileView hiển thị thông tin hồ sơ và các bài đã đăng.

Thành viên click "Sửa đổi" tại một bài đăng đang hoạt động.

Lớp ProfileView gọi sang lớp EditPostView.

Lớp EditPostView hiển thị dữ liệu bài đăng hiện tại.

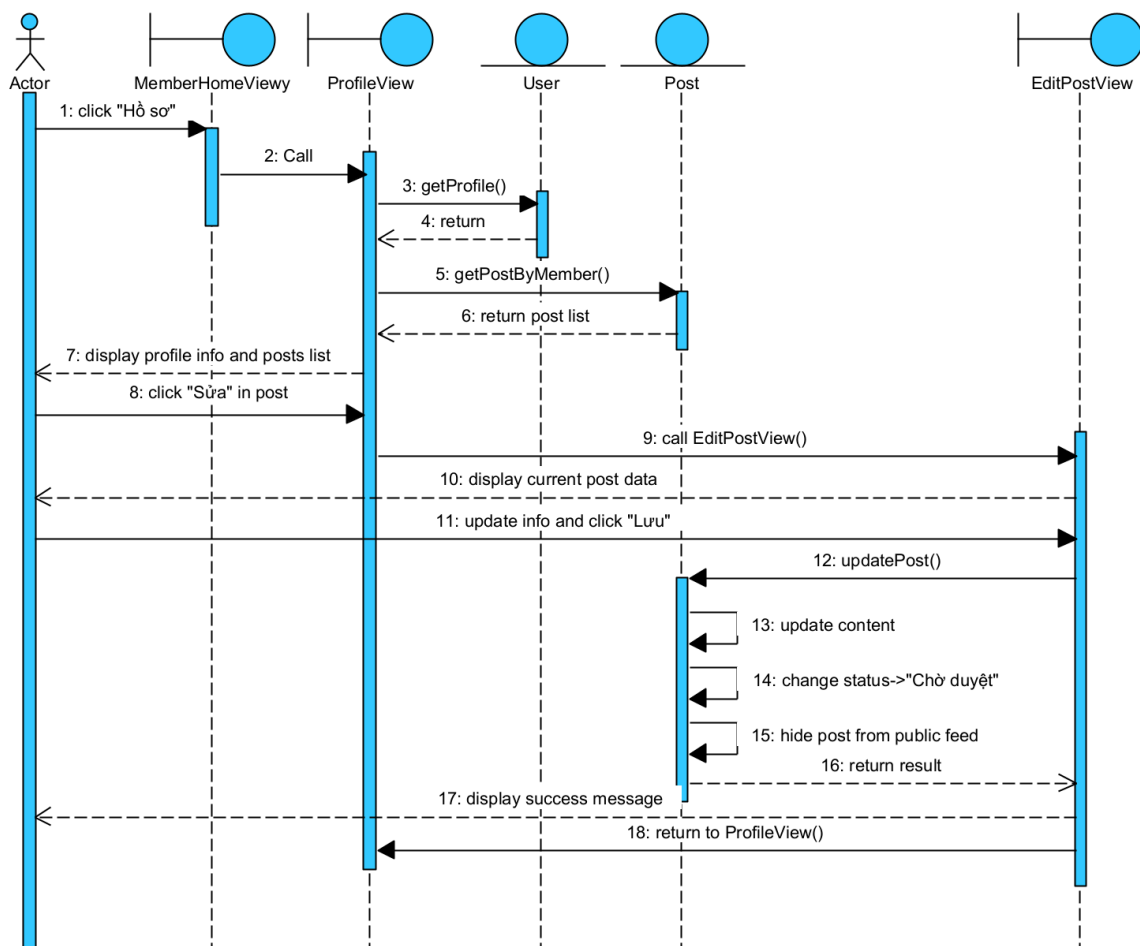
Thành viên cập nhật mô tả và click "Lưu thay đổi".

Lớp EditPostView gọi lớp Post xử lý.

Lớp Post cập nhật nội dung, chuyển trạng thái bài đăng về "Chờ duyệt" và ẩn bài khỏi bảng tin.

Lớp Post trả kết quả về cho lớp EditPostView.

Lớp EditPostView hiển thị thông báo cập nhật thành công và quay lại lớp ProfileView.



e. Chức năng quản lý thông báo

Phân tích chi tiết chức năng quản lý thông báo (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

Sau khi đăng nhập thành công, giao diện chính của thành viên hiện ra -> đề xuất lớp MemberHomeView, có mục truy cập “Thông báo”.

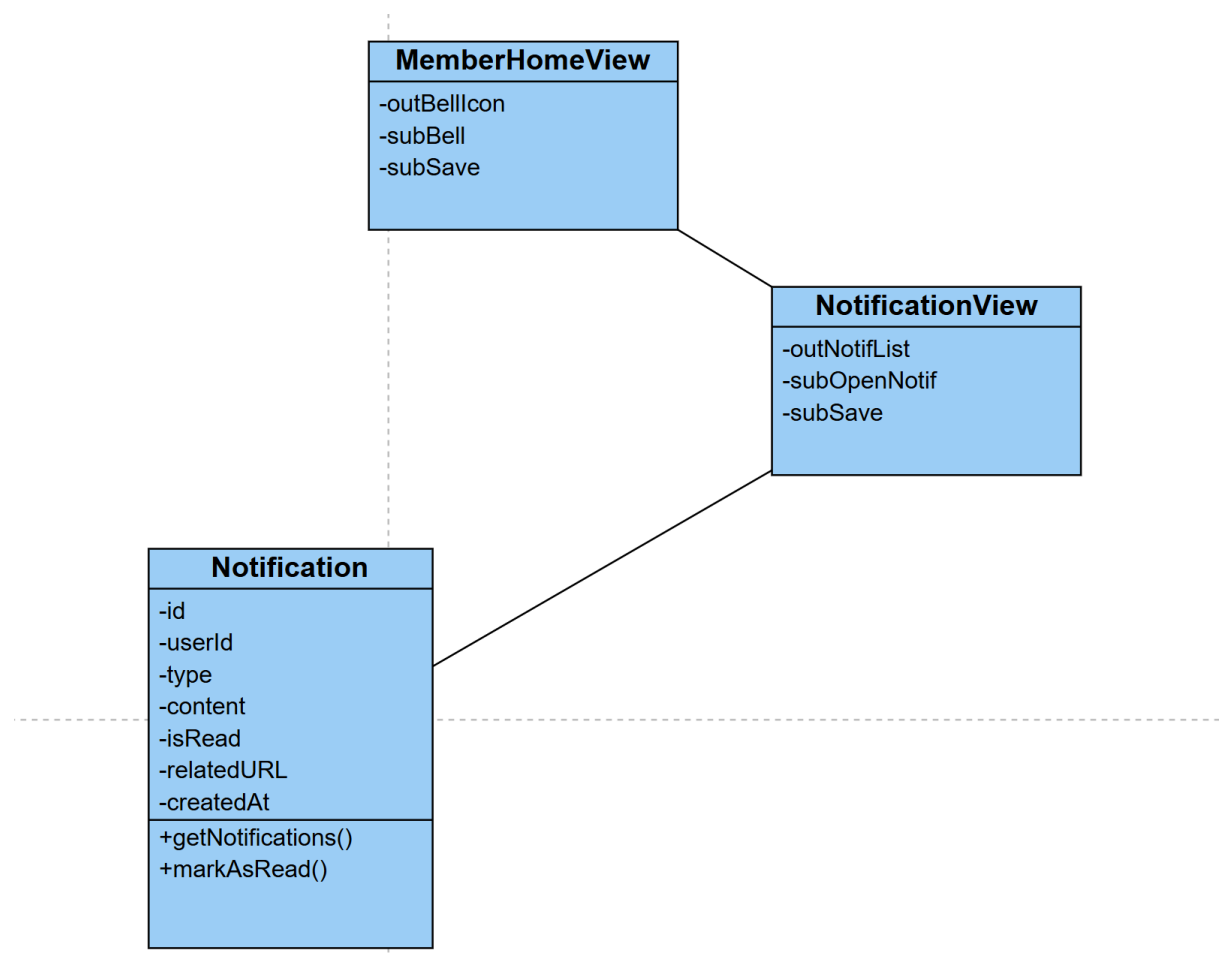
Khi có sự kiện phát sinh như bài viết được duyệt/từ chối hoặc có tin nhắn mới, hệ thống tạo bản ghi thông báo -> cần chức năng createNotification() là hành động của đối tượng Notification.

Hệ thống dùng Socket.io để đẩy thông báo thời gian thực đến thành viên đang online.

Thành viên click biểu tượng chuông -> giao diện danh sách thông báo hiện ra -> đề xuất lớp NotificationView, cần chức năng getNotification().

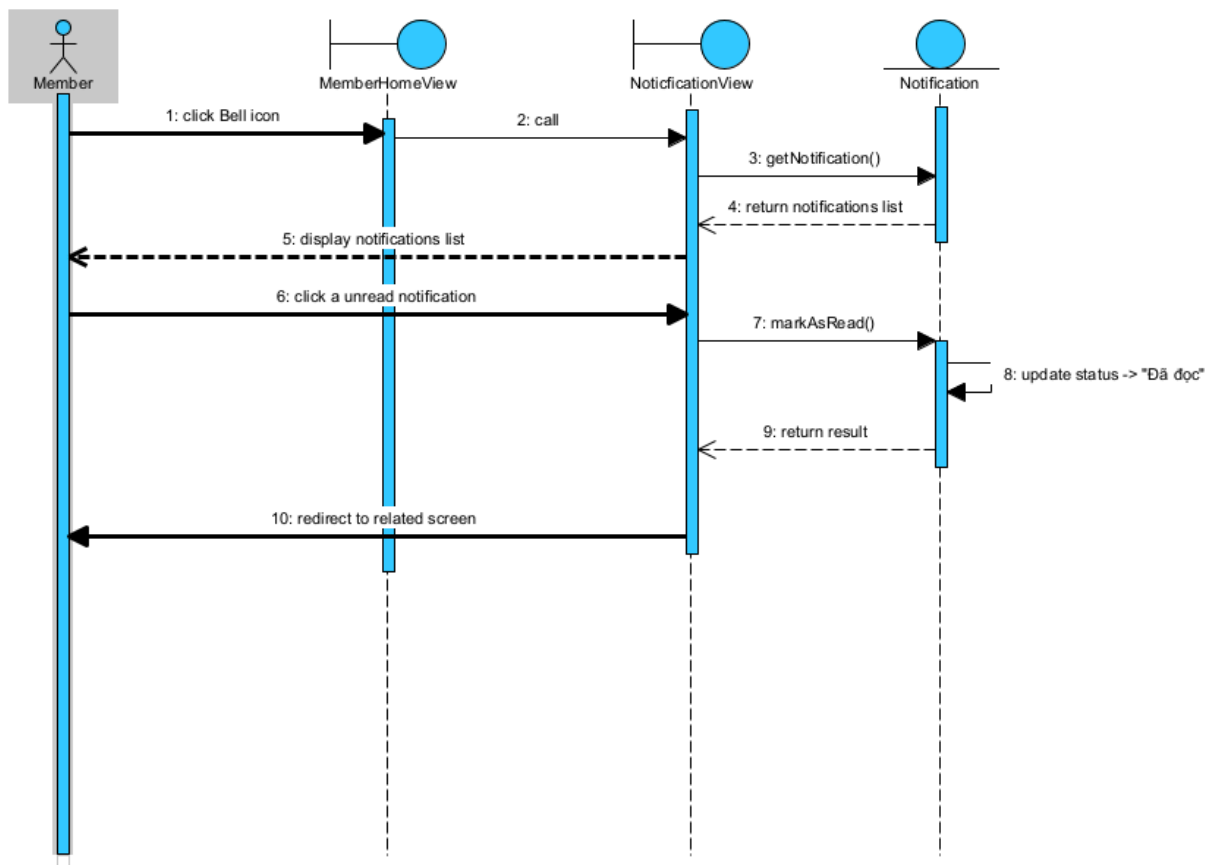
Thành viên mở một thông báo -> hệ thống đánh dấu thông báo là đã đọc -> cần chức năng markAsRead() là hành động của đối tượng Notification.

Nếu thông báo có liên kết đến phòng chat hoặc bài đăng, hệ thống điều hướng đến màn hình tương ứng.



Kịch bản chi tiết cho chức năng quản lý thông báo diễn ra như sau:

1. Thành viên đang ở giao diện chính, click vào biểu tượng chuông thông báo.
2. Lớp MemberHomeView gọi lớp NotificationView xử lý.
3. Lớp NotificationView gọi lớp Notification để yêu cầu lấy danh sách thông báo.
4. Lớp Notification truy xuất CSDL và trả danh sách thông báo về cho lớp NotificationView.
5. Lớp NotificationView hiển thị danh sách thông báo lên màn hình cho thành viên.
6. Thành viên click chọn một thông báo chưa đọc.
7. Lớp NotificationView gọi lớp Notification xử lý cập nhật trạng thái.
8. Lớp Notification cập nhật trạng thái của thông báo đó thành "Đã đọc" trong CSDL.
9. Lớp Notification trả kết quả xử lý thành công về cho lớp NotificationView.
10. Lớp NotificationView điều hướng thành viên đến phòng chat hoặc màn hình chi tiết bài đăng tương ứng.



f. Chức năng báo cáo bài viết

Phân tích chi tiết chức năng báo cáo bài viết (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

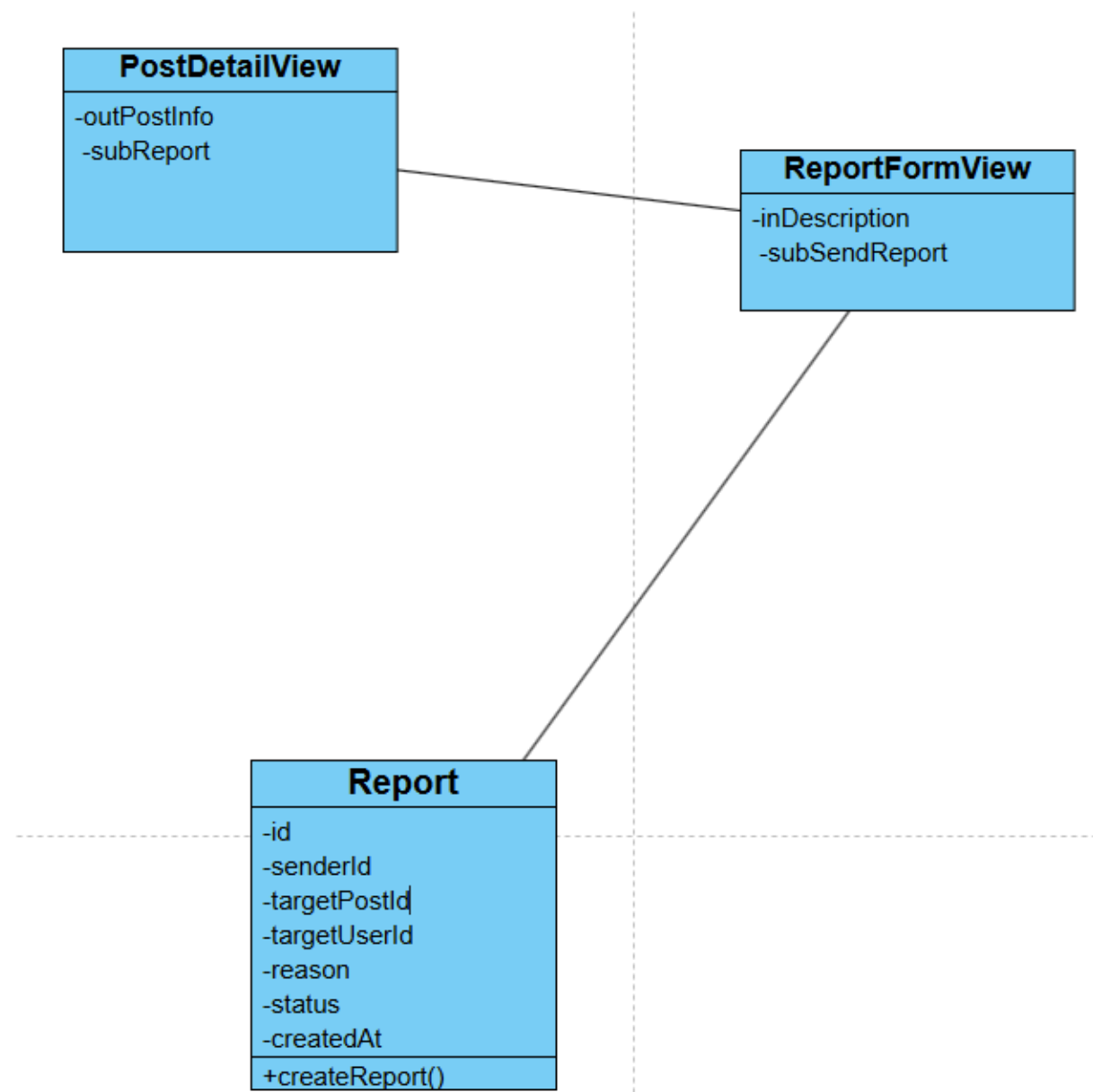
Thành viên đang ở giao diện chi tiết bài đăng -> đề xuất lớp PostDetailView, có tùy chọn "Báo cáo vi phạm" .

Thành viên click "Báo cáo vi phạm" -> giao diện nhập lý do báo cáo hiện ra -> đề xuất lớp ReportFormView, có ô nhập mô tả vi phạm và nút "Gửi báo cáo".

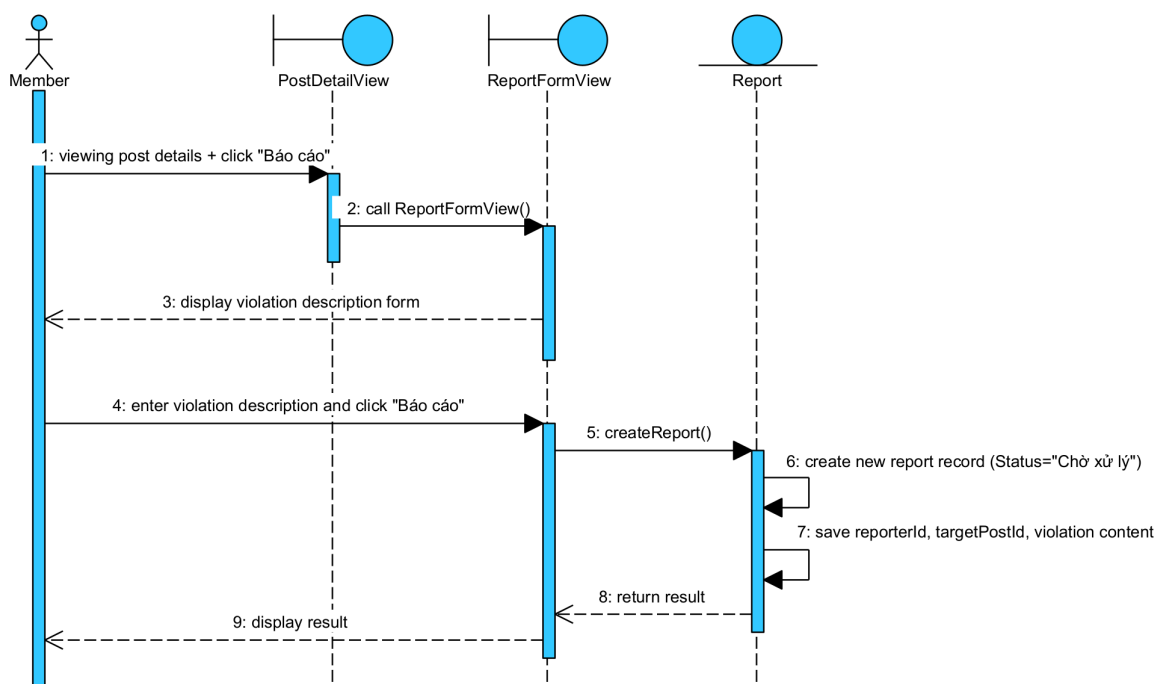
Thành viên nhập mô tả và gửi -> hệ thống tạo báo cáo mới -> cần chức năng createReport() là hành động của đối tượng Report.

Report lưu người gửi (reporterId), bài đăng bị báo cáo (targetPostId), chủ bài đăng bị báo cáo (targetUserId), nội dung vi phạm và trạng thái "Chờ xử lý".

Sau khi lưu, báo cáo xuất hiện trong danh sách chờ xử lý của chức năng Quản lý Báo cáo dành cho quản trị viên.



Kịch bản chi tiết cho chức năng báo cáo bài viết diễn ra như sau:
 Thành viên đang xem PostDetailView của một bài đăng nghi vấn.
 Thành viên click biểu tượng "3 chấm" trên bài đăng và chọn "Báo cáo vi phạm" (hệ thống không hỗ trợ xem hồ sơ người dùng khác, do đó báo cáo chỉ thực hiện được qua bài đăng).
 Lớp PostDetailView gọi lớp ReportFormView, truyền vào targetPostId.
 Lớp ReportFormView hiển thị form nhập mô tả vi phạm.
 Thành viên nhập nội dung báo cáo và click "Gửi báo cáo".
 Lớp ReportFormView gọi lớp Report xử lý.
 Lớp Report tạo bản ghi báo cáo mới, lưu reporterId, targetPostId, nội dung vi phạm với trạng thái "Chờ xử lý".
 Lớp Report trả kết quả về cho ReportFormView.
 Lớp ReportFormView hiển thị thông báo "Gửi báo cáo thành công".



g. Chức năng quản lý bài đăng

Phân tích chi tiết chức năng xét duyệt bài đăng (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

Sau khi đăng nhập thành công, giao diện chính của quản trị viên hiện ra -> đề xuất lớp AdminHomeView, có ít nhất nút chọn chức năng "Quản lý bài đăng".

Quản trị viên click chức năng "Quản lý bài đăng" -> giao diện danh sách bài đăng chờ duyệt hiện ra -> đề xuất lớp ManagePostView, có bảng danh sách bài chờ duyệt với các nút "Xem" (hình con mắt), "Duyệt" (hình dấu tích), "Từ chối" (hình dấu x).

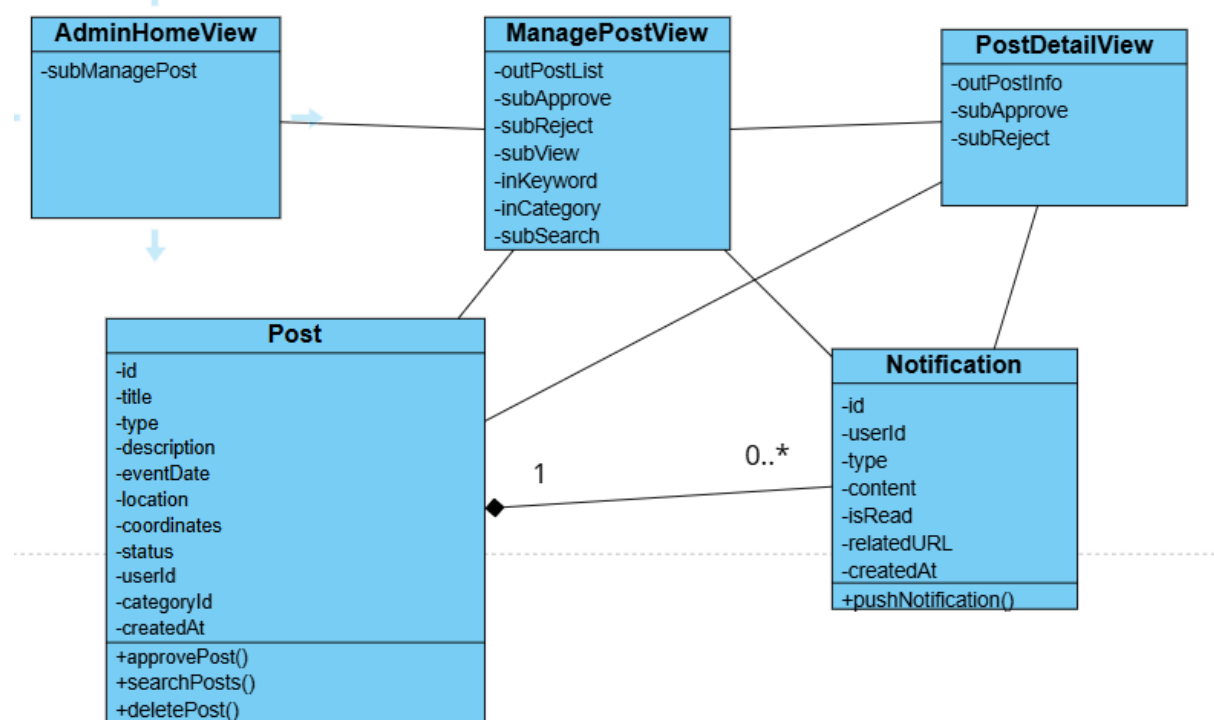
Quản trị viên click nút "Xem" -> giao diện chi tiết bài đăng hiện ra -> đề xuất lớp PostDetailView, hiển thị đầy đủ ảnh, mô tả, vị trí, Tags.

Quản trị viên đọc xong, click nút "Duyệt" -> hệ thống cập nhật trạng thái bài đăng -> cần chức năng approvePost() -> chức năng này là hành động của đối tượng Post.

Nếu chọn "Từ chối" thì hệ thống xóa bài đăng khỏi CSDL (không lưu trạng thái).

Hệ thống đẩy thông báo đến chủ bài đăng -> cần chức năng pushNotification() -> hành động của đối tượng Notification.

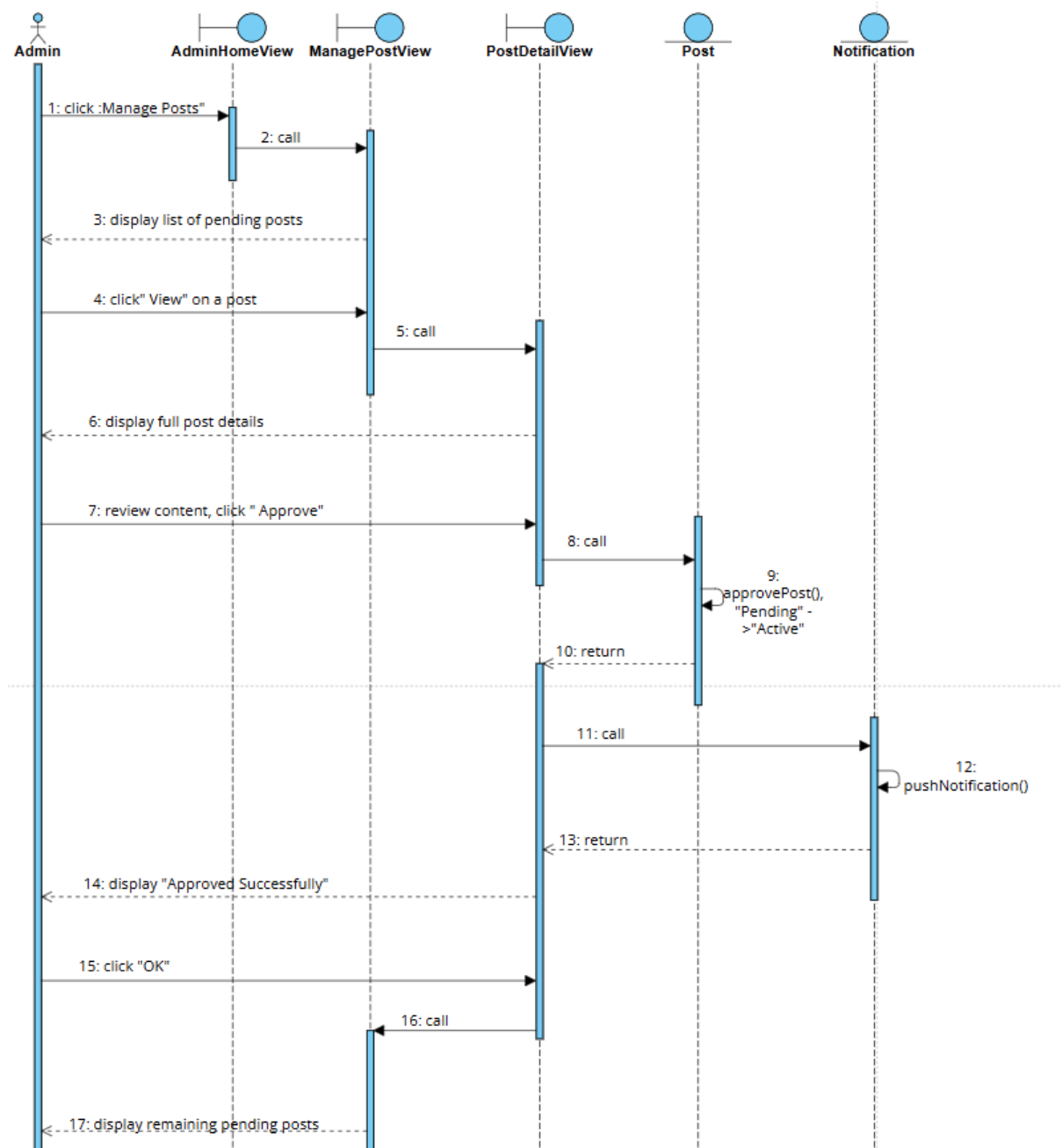
Cập nhật xong, hệ thống quay về lớp ManagePostView.



Với biểu đồ lớp này, kịch bản chi tiết cho chức năng xét duyệt bài đăng diễn ra như sau:

1. Quản trị viên click chức năng "Quản lý bài đăng" trên giao diện AdminHomeView.
2. Lớp AdminHomeView gọi lớp ManagePostView.
3. Lớp ManagePostView hiển thị danh sách bài đăng chờ duyệt cho quản trị viên.

4. Quản trị viên click nút "Xem" tại một bài đăng.
5. Lớp ManagePostView gọi sang lớp PostDetailView.
6. Lớp PostDetailView hiển thị đầy đủ thông tin bài đăng cho quản trị viên.
7. Quản trị viên đọc, xác nhận nội dung hợp lệ và click nút "Duyệt".
8. Lớp PostDetailView gọi lớp Post xử lí.
9. Lớp Post thực hiện cập nhật trạng thái từ "Chờ duyệt" sang "Hoạt động".
10. Lớp Post trả kết quả về cho lớp PostDetailView.
11. Lớp PostDetailView gọi lớp Notification xử lí.
12. Lớp Notification đẩy thông báo "Bài viết của bạn đã được phê duyệt" đến chủ bài đăng.
13. Lớp Notification trả kết quả về cho lớp PostDetailView.
14. Lớp PostDetailView thông báo duyệt thành công.
15. Quản trị viên click OK.
16. Lớp PostDetailView gọi lại lớp ManagePostView.
17. Lớp ManagePostView hiển thị danh sách bài đăng chờ duyệt còn lại cho quản trị viên.



h. Chức năng quản lý báo cáo vi phạm

Phân tích chi tiết chức năng khóa tài khoản vi phạm trong Quản lý Báo cáo (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

Sau khi đăng nhập thành công, giao diện chính của quản trị viên hiện ra -> đề xuất lớp AdminHomeView, có ít nhất nút nhấn chức năng "Quản lý Báo cáo".

Quản trị viên click chức năng "Quản lý Báo cáo" -> giao diện danh sách báo cáo hiện ra -> đề xuất lớp ManageReportView, có bảng danh sách báo cáo chờ xử lý với nút "Xử lý".

Quản trị viên click nút "Xử lý" tại một báo cáo -> giao diện chi tiết báo cáo hiện ra -> đề xuất lớp ReportDetailView, có phần hiển thị thông tin người gửi, nội dung vi phạm, đối tượng bị báo cáo, các nút hành động "Khóa tài khoản", "Xóa bài đăng", "Bỏ qua". Thông tin đối tượng bị báo cáo đã được lưu trong Report (targetPostId/targetUserId) từ lúc người dùng gửi báo cáo.

Quản trị viên xác minh vi phạm, click nút "Khóa tài khoản" -> hệ thống hiển thị popup xác nhận -> đề xuất lớp ConfirmLockView.

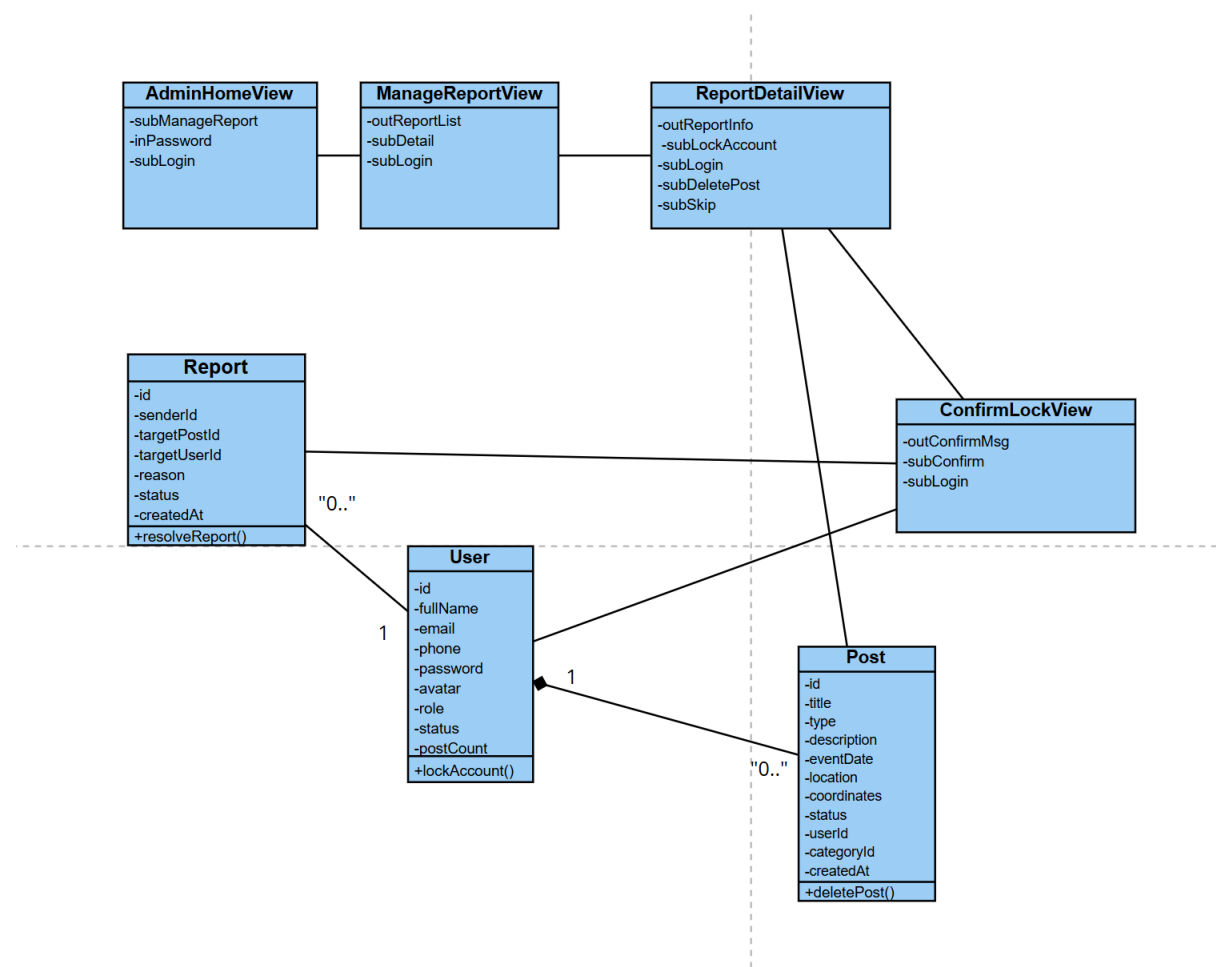
Quản trị viên click "Xác nhận khóa" -> hệ thống khóa tài khoản -> cần chức năng lockAccount() -> chức năng này là hành động của đối tượng User.

Đồng thời hệ thống cập nhật trạng thái báo cáo -> cần chức năng resolveReport() -> chức năng này là hành động của đối tượng Report.

Nếu báo cáo liên quan đến bài đăng và quản trị viên chọn "Xóa bài đăng" -> hệ thống gọi chức năng deletePost() của đối tượng Post, sau đó gọi resolveReport() để đánh dấu báo cáo đã giải quyết.

Nếu báo cáo không có vi phạm và quản trị viên chọn "Bỏ qua" -> hệ thống chỉ gọi resolveReport(), không khóa tài khoản hoặc xóa bài đăng.

Xử lý xong, hệ thống quay về lớp ManageReportView.



Kịch bản chi tiết cho chức năng quản lý báo cáo vi phạm (khóa tài khoản) diễn ra như sau:

Quản trị viên click chức năng "Quản lý Báo cáo" trên giao diện AdminHomeView.

Lớp AdminHomeView gọi lớp ManageReportView.

Lớp ManageReportView hiển thị danh sách báo cáo chờ xử lý cho quản trị viên.

Quản trị viên click nút "Xem chi tiết" tại báo cáo liên quan đến User: nguyenvanA.

Lớp ManageReportView gọi sang lớp ReportDetailView.

Lớp ReportDetailView hiển thị chi tiết báo cáo cho quản trị viên.

Quản trị viên đọc đối soát, xác minh có hành vi vi phạm và click nút "Khóa tài khoản".

Lớp ReportDetailView gọi lớp ConfirmLockView.

Lớp ConfirmLockView hiển thị popup xác nhận "Bạn có chắc chắn muốn khóa tài khoản nguyenvanA?".

Quản trị viên click "Xác nhận khóa".

Lớp ConfirmLockView gọi lớp User yêu cầu xử lý.

Lớp User thực hiện chức năng khóa tài khoản và gỡ bỏ toàn bộ bài đăng liên quan.

Lớp User trả kết quả về cho lớp ConfirmLockView.

Lớp ConfirmLockView gọi lớp Report yêu cầu xử lý.

Lớp Report thực hiện cập nhật trạng thái báo cáo thành "Đã giải quyết".

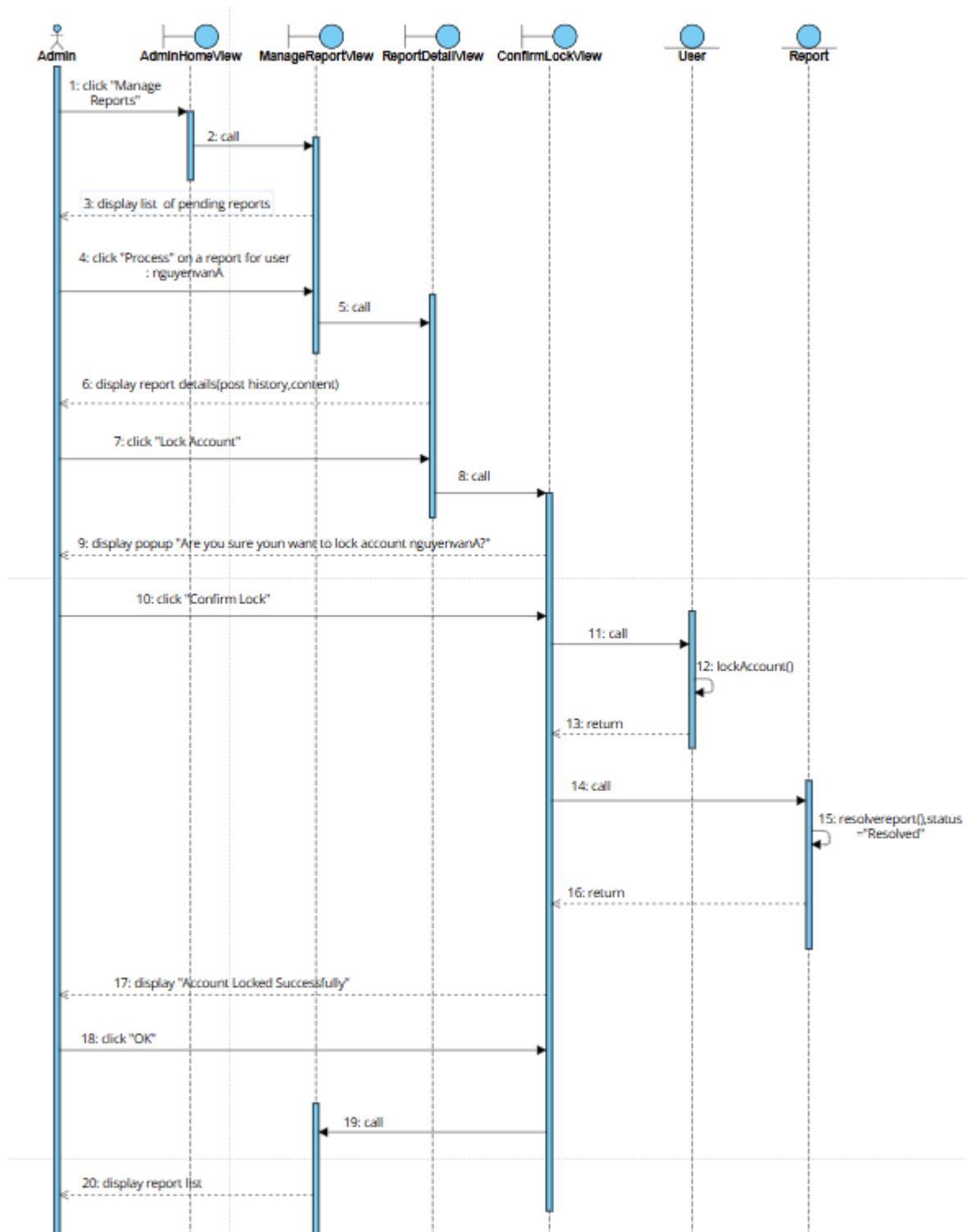
Lớp Report trả kết quả về cho lớp ConfirmLockView.

Lớp ConfirmLockView hiển thị thông báo khóa tài khoản thành công.

Quản trị viên click vào nút OK của thông báo.

Lớp ConfirmLockView gọi lại lớp ManageReportView.

Lớp ManageReportView hiển thị lại danh sách báo cáo cho quản trị viên.



i. Chức năng quản lý danh mục

Phân tích chi tiết chức năng thêm danh mục (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

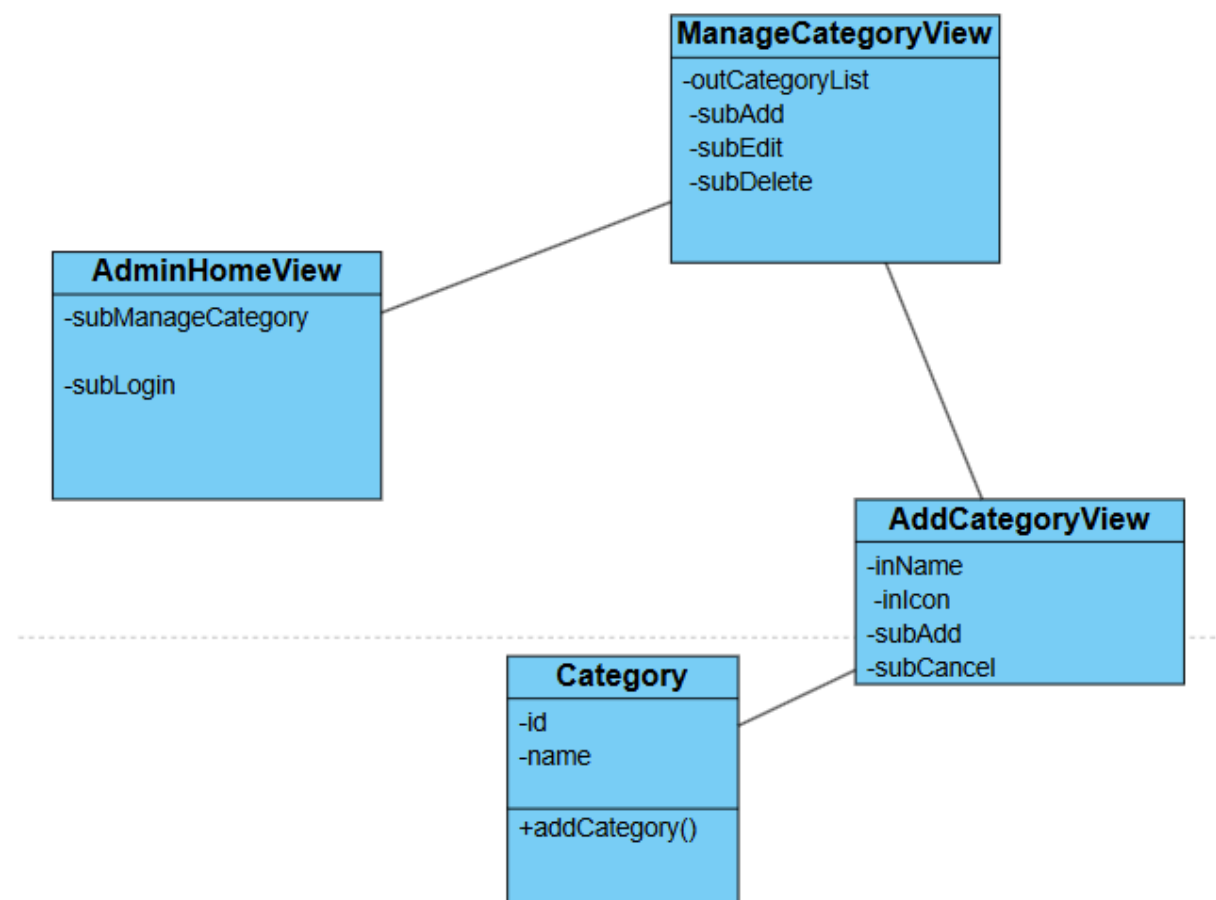
Sau khi đăng nhập thành công, giao diện chính của quản trị viên hiện ra -> đề xuất lớp AdminHomeView, có ít nhất nút nhấn chức năng "Quản lý Danh mục".

Quản trị viên click chức năng "Quản lý Danh mục" -> giao diện danh sách danh mục hiện ra -> đề xuất lớp ManageCategoryView, có bảng danh sách danh mục hiện có, nút "Thêm danh mục", các nút "Sửa" và "Xóa" tại mỗi dòng.

Quản trị viên click nút "Thêm danh mục" -> giao diện thêm danh mục hiện ra -> đề xuất lớp AddCategoryView, có ô nhập tên danh mục, ô nhập mô tả, nút "Lưu".

Quản trị viên nhập tên danh mục "Thú cưng", điền mô tả và click nút "Lưu" -> hệ thống lưu danh mục vào CSDL -> cần chức năng addCategory() -> chức năng này là hành động của đối tượng Category.

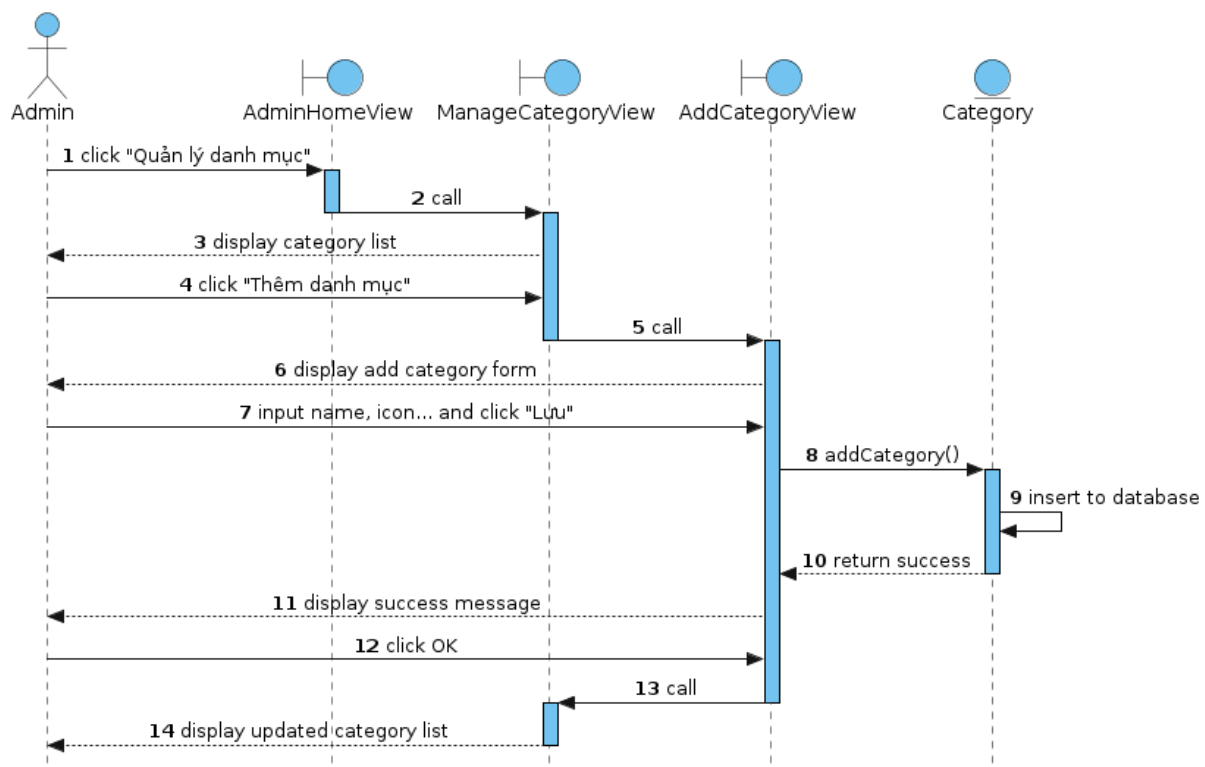
Lưu xong, hệ thống quay về lớp ManageCategoryView và cập nhật lại danh sách.



Kịch bản chi tiết cho chức năng quản lý danh mục (thêm danh mục) diễn ra như sau:

1. Quản trị viên click chức năng "Quản lý Danh mục" trên giao diện AdminHomeView.
2. Lớp AdminHomeView gọi lớp ManageCategoryView.
3. Lớp ManageCategoryView hiển thị danh sách danh mục cho quản trị viên.
4. Quản trị viên click nút "Thêm danh mục".

5. Lớp ManageCategoryView gọi sang lớp AddCategoryView.
6. Lớp AddCategoryView hiển thị cho quản trị viên.
7. Quản trị viên nhập tên danh mục "Thú cưng", điền mô tả và click nút "Thêm".
8. Lớp AddCategoryView gọi lớp Category xử lý.
9. Lớp Category thực hiện chức năng thêm danh mục mới vào CSDL.
10. Lớp Category trả kết quả về cho lớp AddCategoryView.
11. Lớp AddCategoryView hiển thị thông báo thêm danh mục thành công.
12. Quản trị viên click vào nút OK của thông báo.
13. Lớp AddCategoryView gọi lại lớp ManageCategoryView.
14. Lớp ManageCategoryView hiển thị danh sách danh mục đã cập nhật cho quản trị viên.



j. Chức năng xem báo cáo thống kê

Phân tích chi tiết chức năng xem báo cáo thống kê (bỏ qua giai đoạn đăng nhập) diễn ra như sau:

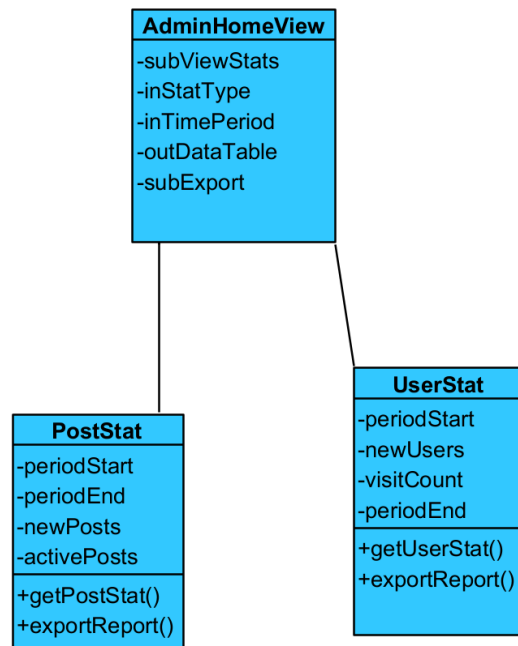
Phân tích chi tiết chức năng xem báo cáo thống kê diễn ra như sau:

Ngay sau khi đăng nhập thành công, giao diện chính AdminHomeView hiện ra và tự động yêu cầu truy vấn dữ liệu thống kê mặc định của tháng hiện tại -> cần chức năng getPostStat() và getUserStat() -> hành động của đối tượng PostStat/UserStat.

Kết quả được trả lại để AdminHomeView hiển thị sẵn các khối dữ liệu và bảng chi tiết.

[Thao tác tùy chọn]: Quản trị viên điều chỉnh bộ lọc thời gian (tuần/năm) -> hệ thống gọi lại `getPostStat()` hoặc `getUserStat()` để cập nhật dữ liệu.
 Quản trị viên click nút "Xuất báo cáo" -> hệ thống tạo file Excel với dữ liệu đang hiển thị -> cần chức năng `exportReport()` của đối tượng `PostStat/UserStat`.

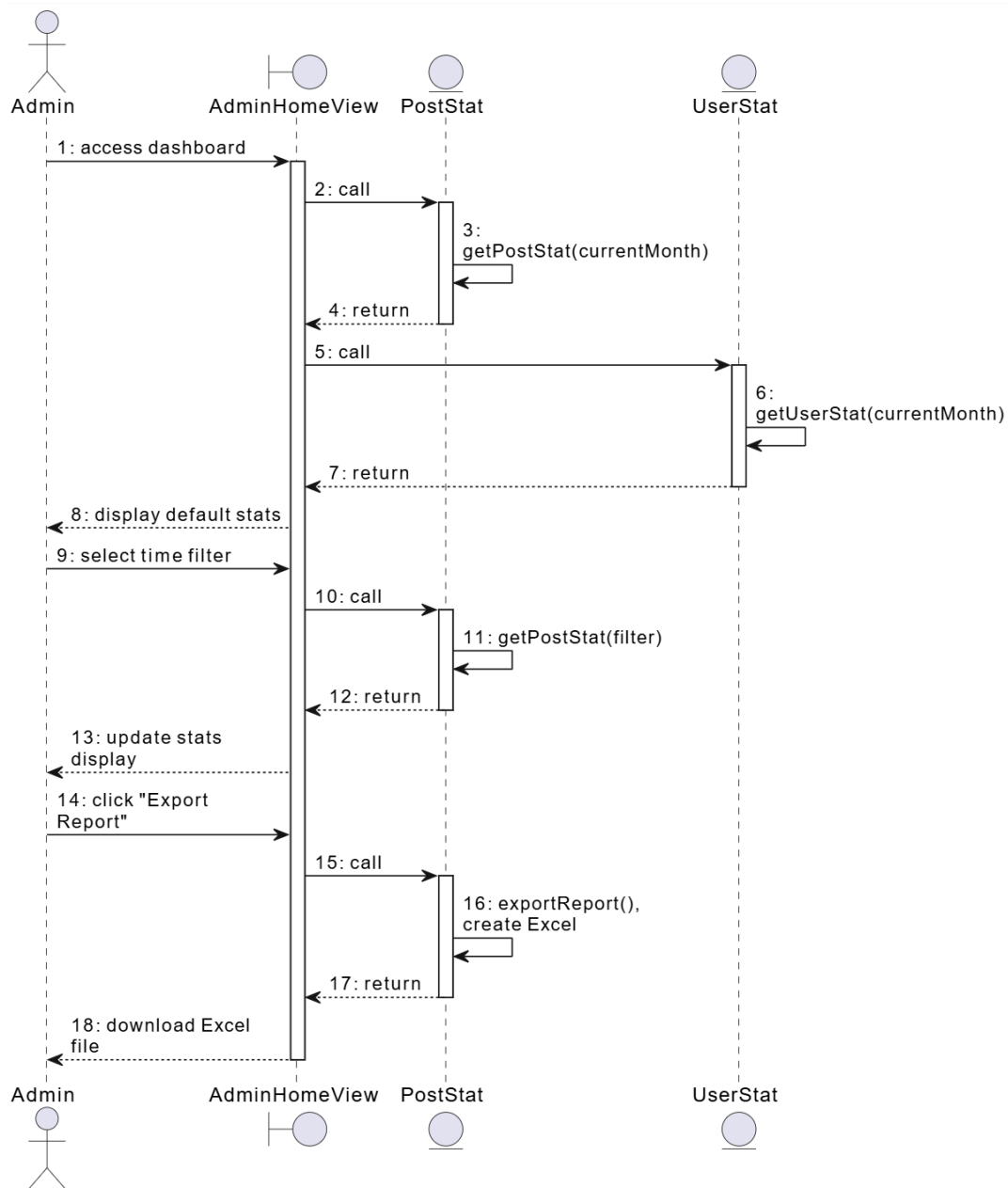
Như vậy, kết quả thu được biểu đồ lớp cho chức năng xem báo cáo thống kê như trên.



Với biểu đồ lớp này, kịch bản chi tiết cho chức năng xem báo cáo thống kê diễn ra như sau:

1. Quản trị viên đăng nhập thành công và truy cập màn hình `AdminHomeView`.
2. Lớp `AdminHomeView` tự động gọi lớp `PostStat` xử lý để lấy thống kê bài đăng mặc định.
3. Lớp `PostStat` thực hiện truy vấn số liệu bài đăng của tháng hiện tại.
4. Lớp `PostStat` trả kết quả về cho lớp `AdminHomeView`.
5. Lớp `AdminHomeView` tự động gọi lớp `UserStat` xử lý để lấy thống kê người dùng mặc định.
6. Lớp `UserStat` thực hiện truy vấn số liệu người dùng của tháng hiện tại.
7. Lớp `UserStat` trả kết quả về cho lớp `AdminHomeView`.
8. Lớp `AdminHomeView` hiển thị biểu đồ và bảng số liệu mặc định cho Quản trị viên.
9. Quản trị viên thao tác điều chỉnh bộ công cụ "Lọc theo thời gian".
10. Lớp `AdminHomeView` gọi lớp `PostStat` xử lý.

11. Lớp PostStat thực hiện truy vấn lại số liệu theo mốc thời gian mới.
12. Lớp PostStat trả kết quả về cho lớp AdminHomeView.
13. Lớp AdminHomeView hiển thị số liệu mới cập nhật cho Quản trị viên.
14. Quản trị viên click nút "Xuất báo cáo".
15. Lớp AdminHomeView gọi lớp PostStat xử lý xuất báo cáo.
16. Lớp PostStat thực hiện xuất dữ liệu đang hiển thị ra file Excel.
17. Lớp PostStat trả file Excel về cho lớp AdminHomeView.
18. Lớp AdminHomeView tải file Excel xuống thiết bị của Quản trị viên.

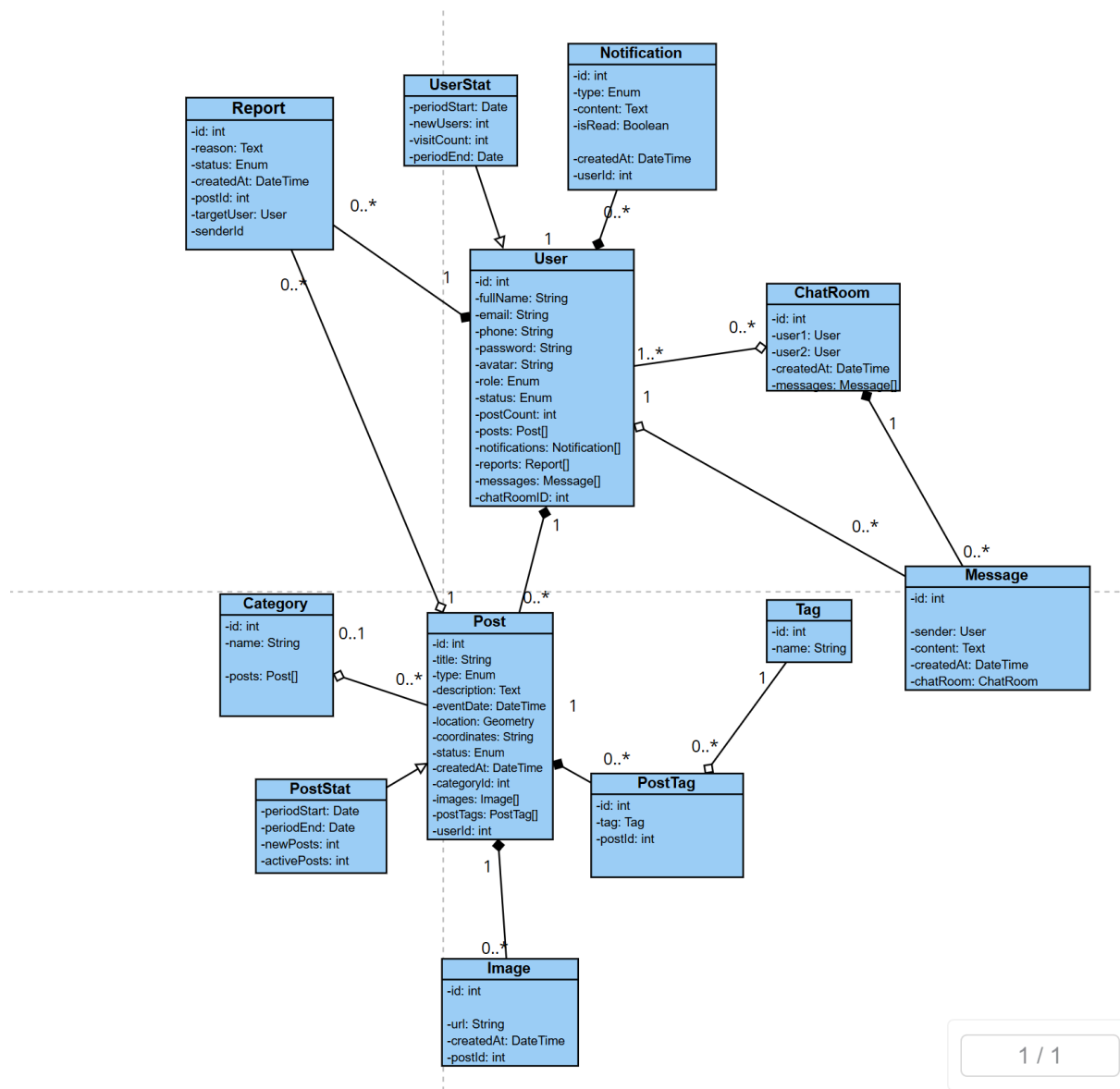


3.4. Thiết kế các lớp thực thể

Từ biểu đồ lớp thực thể ở pha phân tích, bước thiết kế được thực hiện theo các nguyên tắc sau:

- Bổ sung kiểu dữ liệu cụ thể cho toàn bộ thuộc tính của các lớp.
- Bổ sung thuộc tính định danh ``id: int`` cho các lớp thực thể không kế thừa từ lớp khác.
- Chuyển các khóa ngoại ở pha phân tích thành thuộc tính tham chiếu đối tượng trong pha thiết kế, ví dụ ``owner: User``, ``category: Category``, ``images: Image[]``.
- Chuyển các quan hệ nhiều-nhiều thành lớp liên kết có thuộc tính quan hệ rõ ràng. Trong hệ thống này, quan hệ Post + Tag được chuyển thành lớp PostTag; Post là thành phần của PostTag theo hướng bài đăng sở hữu danh sách gắn thẻ, còn Tag là đối tượng dùng chung được PostTag tham chiếu.
- Các quan hệ có vòng đời phụ thuộc được biểu diễn bằng composition. Các lớp dùng chung hoặc có thể tồn tại độc lập được biểu diễn bằng aggregation hoặc association.

Sau khi thiết kế, các lớp thực thể của hệ thống được mô tả như sau:



3.5. Thiết kế cơ sở dữ liệu

Dựa vào sơ đồ lớp thực thể đã trích được trong pha phân tích và đã thiết kế lại ở mục 3.4, chúng ta có thể đề xuất các bảng dữ liệu như sau:

- `tblUser`: lưu thông tin về người dùng của hệ thống, bao gồm: id, họ tên đầy đủ, email, số điện thoại, mật khẩu, ảnh đại diện, vai trò, trạng thái tài khoản, số lượng bài đăng và thời điểm tạo tài khoản.
- `tblCategory`: lưu thông tin các danh mục đồ vật, bao gồm: id, tên danh mục. Bảng này dùng để phân loại các bài đăng như ví tiền, giấy tờ, thiết bị điện tử, thú cưng.

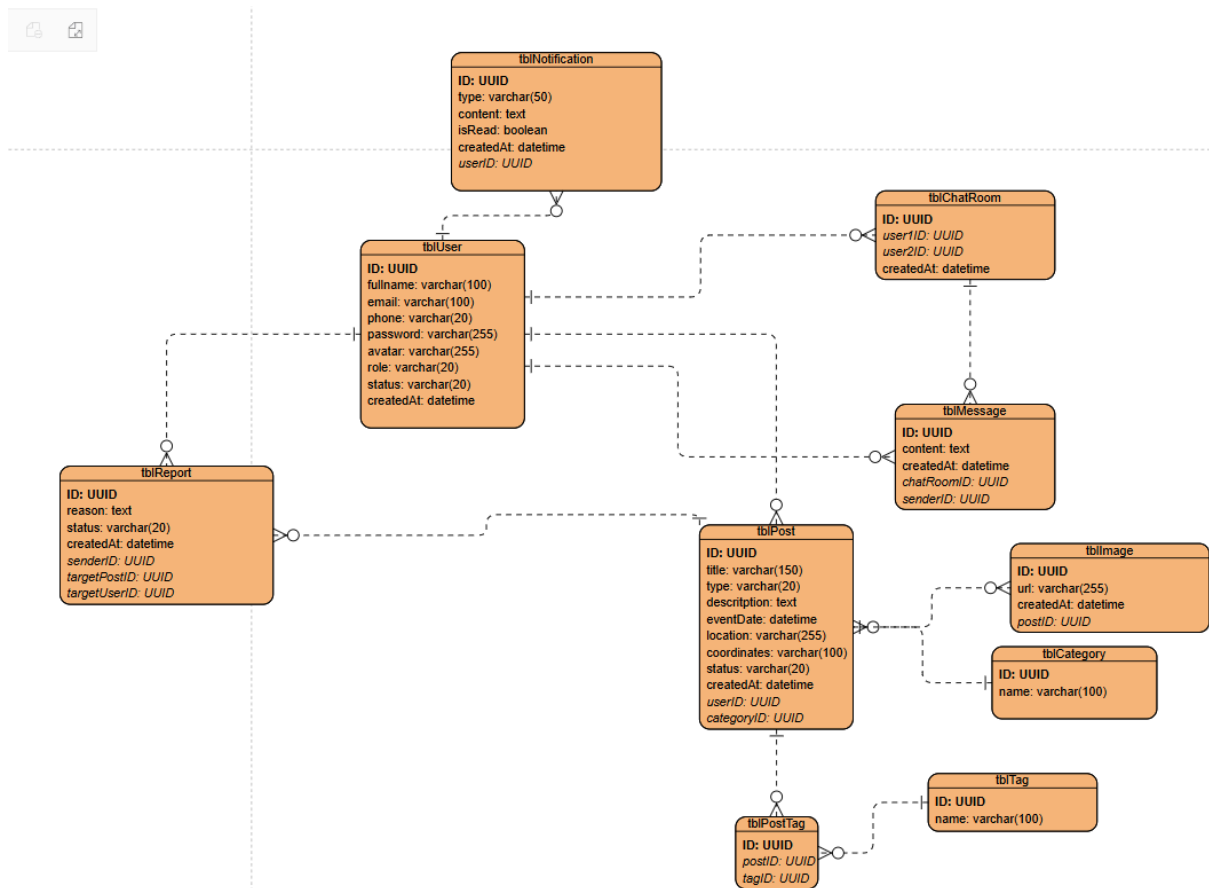
- ``tblPost``: lưu thông tin về các bài đăng báo mất hoặc nhật được, bao gồm: id, tiêu đề, loại bài đăng, mô tả, ngày xảy ra sự việc, vị trí, tọa độ, trạng thái bài đăng, thời điểm tạo bài, id người đăng và id danh mục.
- ``tblImage``: lưu thông tin các hình ảnh đính kèm bài đăng, bao gồm: id, đường dẫn ảnh, thời điểm tạo và id bài đăng tương ứng. Mỗi bài đăng có thể có nhiều ảnh, nhưng số lượng ảnh có thể được giới hạn theo yêu cầu hệ thống.
- ``tblTag``: lưu thông tin các thẻ mô tả hoặc từ khóa tìm kiếm, bao gồm: id và tên thẻ. Các thẻ này có thể được tạo thủ công hoặc gợi ý từ quá trình phân tích hình ảnh.
- ``tblPostTag``: lưu thông tin gắn thẻ cho bài đăng, bao gồm: id, id bài đăng và id thẻ. Bảng này được tách ra để biểu diễn quan hệ nhiều-nhiều giữa bài đăng và thẻ.
- ``tblChatRoom``: lưu thông tin phòng chat giữa hai người dùng, bao gồm: id, id người dùng thứ nhất, id người dùng thứ hai và thời điểm tạo phòng chat.
- ``tblMessage``: lưu thông tin các tin nhắn trong phòng chat, bao gồm: id, nội dung tin nhắn, thời điểm gửi, id phòng chat và id người gửi.
- ``tblNotification``: lưu thông tin các thông báo gửi tới người dùng, bao gồm: id, loại thông báo, nội dung thông báo, trạng thái đã đọc/chưa đọc, thời điểm tạo và id người nhận.
- ``tblReport``: lưu thông tin các báo cáo vi phạm, bao gồm: id, lý do báo cáo, trạng thái xử lý, thời điểm tạo, id người gửi báo cáo, id bài đăng bị báo cáo và id người dùng bị báo cáo.
- ``tblUserStat`` và ``tblPostStat``: là các bảng thống kê tổng hợp theo kỳ nếu hệ thống cần lưu lịch sử thống kê. ``tblUserStat`` lưu thời điểm bắt đầu kỳ, thời điểm kết thúc kỳ, số người dùng mới và số lượt truy cập. ``tblPostStat`` lưu thời điểm bắt đầu kỳ, thời điểm kết thúc kỳ, số bài đăng mới và số bài đang hoạt động.

Quan hệ giữa các bảng được mô tả như sau:

- Bảng ``tblUser`` quan hệ 1-n với bảng ``tblPost``, vì một người dùng có thể tạo nhiều bài đăng, nhưng mỗi bài đăng chỉ thuộc về một người dùng.
- Bảng ``tblCategory`` quan hệ 1-n với bảng ``tblPost``, vì một danh mục có thể chứa nhiều bài đăng, nhưng mỗi bài đăng chỉ thuộc một danh mục.

- Bảng `tblPost` quan hệ 1-n với bảng `tblImage`, vì một bài đăng có thể có nhiều hình ảnh đính kèm, nhưng mỗi hình ảnh chỉ thuộc về một bài đăng.
- Bảng `tblPost` và bảng `tblTag` quan hệ n-n thông qua bảng trung gian `tblPostTag`. Cụ thể, `tblPost` quan hệ 1-n với `tblPostTag`, và `tblTag` cũng quan hệ 1-n với `tblPostTag`.
- Bảng `tblUser` quan hệ 1-n với bảng `tblChatRoom` qua hai khóa ngoại `user1ID` và `user2ID`, vì một người dùng có thể tham gia nhiều phòng chat khác nhau.
- Bảng `tblChatRoom` quan hệ 1-n với bảng `tblMessage`, vì một phòng chat có thể chứa nhiều tin nhắn, nhưng mỗi tin nhắn chỉ thuộc về một phòng chat.
- Bảng `tblUser` quan hệ 1-n với bảng `tblMessage`, vì một người dùng có thể gửi nhiều tin nhắn, nhưng mỗi tin nhắn chỉ có một người gửi.
- Bảng `tblUser` quan hệ 1-n với bảng `tblNotification`, vì một người dùng có thể nhận nhiều thông báo, nhưng mỗi thông báo chỉ được gửi tới một người dùng.
- Bảng `tblUser` quan hệ 1-n với bảng `tblReport` theo vai trò người gửi báo cáo, vì một người dùng có thể gửi nhiều báo cáo vi phạm, nhưng mỗi báo cáo chỉ được gửi bởi một người.
- Bảng `tblPost` quan hệ 1-n với bảng `tblReport`, vì một bài đăng có thể bị báo cáo nhiều lần, nhưng mỗi báo cáo chỉ nhắm tới một bài đăng cụ thể.
- Bảng `tblUser` quan hệ 1-n với bảng `tblReport` theo vai trò người bị báo cáo, vì một người dùng có thể bị báo cáo nhiều lần, nhưng mỗi báo cáo chỉ nhắm tới một người dùng cụ thể.
- Bảng `tblUserStat` và `tblPostStat` không tham gia trực tiếp vào các quan hệ nghiệp vụ chính. Hai bảng này lấy dữ liệu tổng hợp từ `tblUser` và `tblPost` để phục vụ chức năng xem báo cáo thống kê.

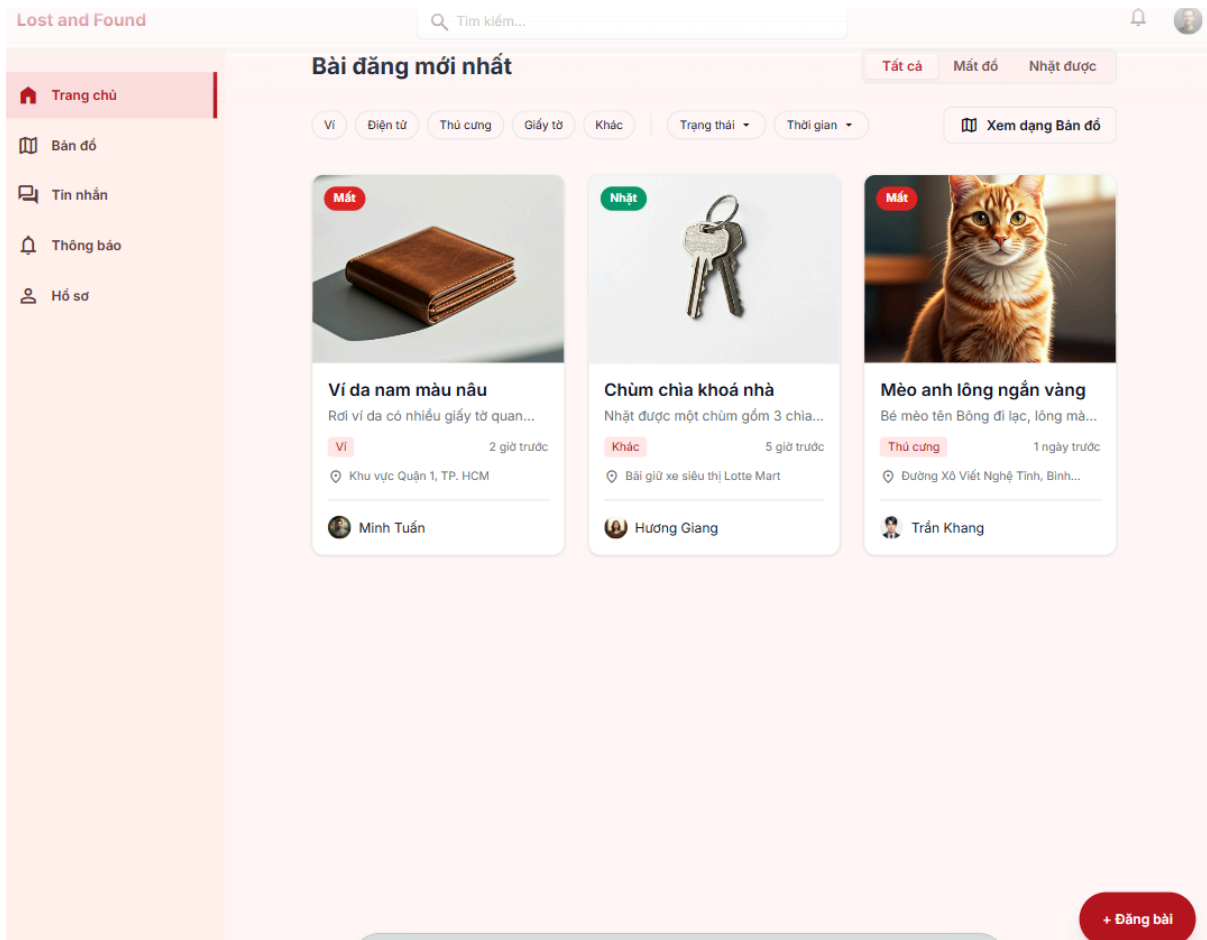
Như vậy, sơ đồ cơ sở dữ liệu của hệ thống được thể hiện như sau:



3.6. Thiết kế giao diện và sơ đồ lớp

3.6.1. Chức năng đăng tải bài viết

Giao diện trang chủ:



Giao diện đăng bài bước 1:

← Lost and Found

×

1

Loại tin

2

Mô tả

3

Vị trí

4

Hình ảnh & Tag

Bạn muốn đăng loại tin gì?

🔍

Báo mất đồ

👉

Nhặt được đồ

Danh mục (tùy chọn)

Chọn danh mục...

↕

Tiêu đề bài đăng

VD: Rơi ví da đen tại sảnh A toà nhà X

0/100

Hủy

Tiếp theo →

Giao diện đăng bài bước 2:

✓

2

3

4

1. Loại tin

2. Mô tả

3. Vị trí

4. Hình ảnh & Tag

Mô tả chi tiết món đồ

Cung cấp thông tin chi tiết giúp mọi người dễ dàng nhận ra món đồ của bạn.

Mô tả chi tiết *

Mô tả màu sắc, đặc điểm nhận dạng, hoàn cảnh, dấu hiệu riêng...

0/2000

+ Màu sắc

+ Kích thước

+ Đặc điểm riêng

Thời điểm xảy ra (Ước lượng)

10/27/2023 02:30 PM

← Quay lại

Tiếp theo →

Giao diện đăng bài bước 3:

Lost and Found

✓

✓

3

4

1. Loại tin

2. Mô tả

3. Vị trí

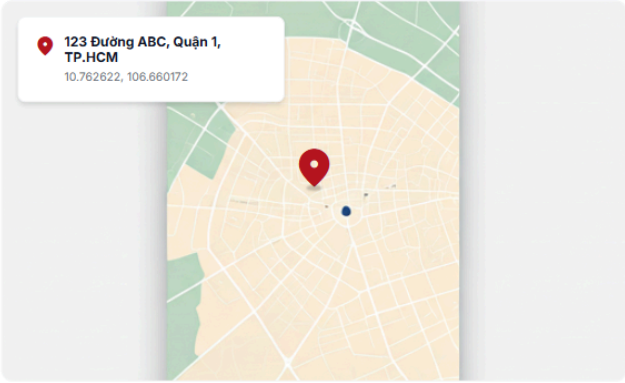
4. Hình ảnh & Tag

Ghim vị trí trên bản đồ

🔍 Nhập địa chỉ...

📍 123 Đường ABC, Quận 1, TP.HCM

10.762622, 106.660172

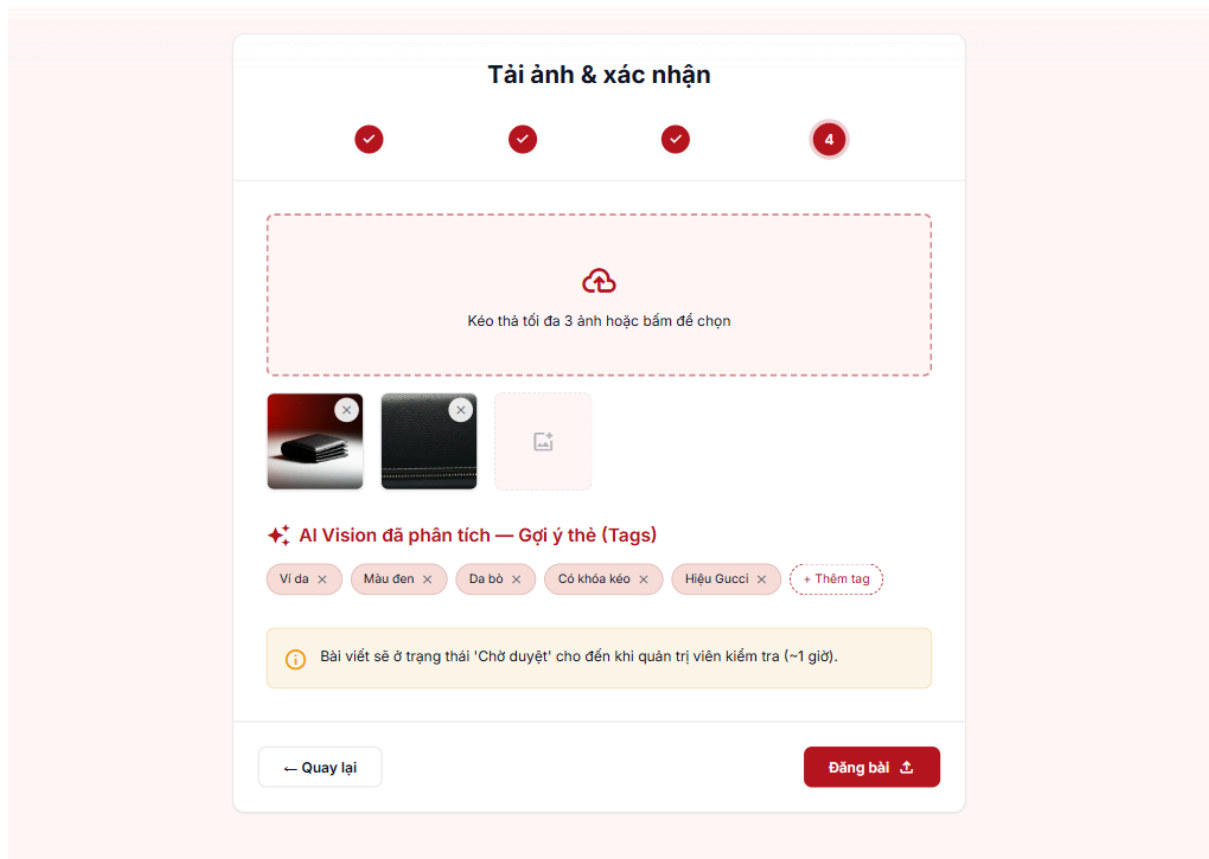


Kéo ghim để chỉnh chỉnh xác vị trí mất/nhặt được đồ

← Quay lại

Tiếp theo →

Giao diện đăng bài bước 4:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

- MemberHomePage là giao diện chính của Thành viên. Nó cần hiển thị danh sách bài đăng mới nhất và có nút "Đăng bài" để chuyển đến chức năng tạo bài đăng mới.
- CreatePostPage là giao diện tổng thể của chức năng đăng tải bài viết. Giao diện này quản lý tiến trình đăng bài gồm 4 bước: chọn loại tin, nhập mô tả, ghim vị trí và tải ảnh kèm thẻ gợi ý.
- CreatePostStep1Panel là thành phần giao diện bước 1 để nhập thông tin cơ bản của bài đăng. Nó cần một vùng chọn loại bài đăng (Mất đồ/Nhặt được), một danh sách chọn danh mục, một trường nhập tiêu đề và nút "Tiếp theo" để chuyển sang bước 2.
- CreatePostStep2Panel là thành phần giao diện bước 2 để nhập thông tin mô tả đồ vật. Nó cần một vùng nhập mô tả chi tiết, bộ chọn ngày xảy ra sự việc và nút "Tiếp theo" để chuyển sang bước chọn vị trí.

- CreatePostStep3Panel là thành phần giao diện bước 3 để chọn vị trí liên quan đến bài đăng. Nó cần một trường nhập địa chỉ, bản đồ số để ghim tọa độ và nút "Tiếp theo" để chuyển sang bước tải ảnh.
- CreatePostStep4Panel là thành phần giao diện bước 4 để tải hình ảnh minh chứng và xác nhận thông tin. Nó cần khu vực chọn ảnh, danh sách ảnh đã tải lên, danh sách thẻ gợi ý do AI Vision API trả về, công cụ xóa thẻ sai nếu cần và nút "Đăng bài" để hoàn tất quy trình.

Các lớp điều khiển:

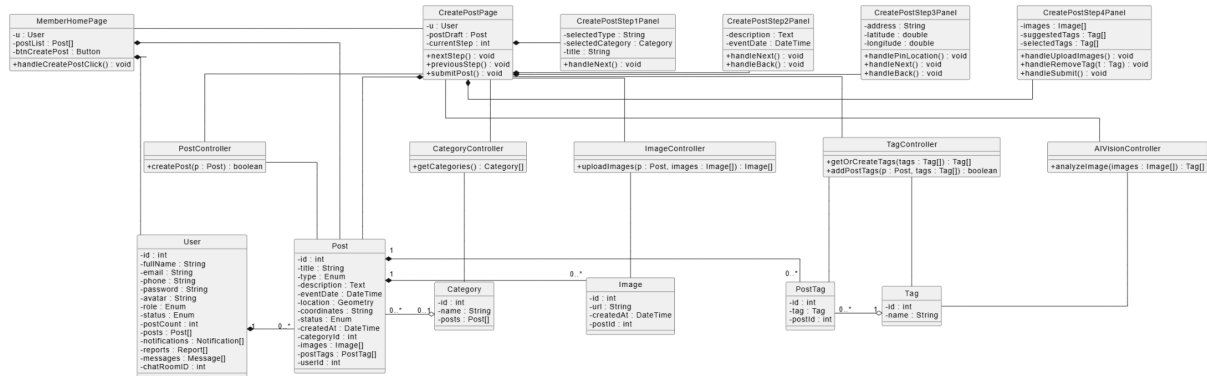
- PostController là lớp điều khiển xử lý các yêu cầu liên quan đến bài đăng. Nó có phương thức createPost() để tạo bài đăng mới với trạng thái ban đầu là "Chờ duyệt", lưu thông tin mô tả, vị trí, ảnh và danh sách thẻ của bài đăng.
- CategoryController là lớp điều khiển xử lý dữ liệu danh mục. Nó có phương thức getCategories() để lấy danh sách danh mục đồ vật hiển thị cho thành viên lựa chọn ở bước 1.
- ImageController là lớp điều khiển xử lý hình ảnh của bài đăng. Nó có phương thức uploadImages() để lưu các hình ảnh minh chứng được tải lên.
- TagController là lớp điều khiển xử lý thẻ mô tả. Nó có phương thức getOrCreateTags() để lấy thẻ đã tồn tại hoặc tạo thẻ mới nếu chưa có, và phương thức addPostTags() để gắn danh sách thẻ với bài đăng.
- AIVisionController là lớp điều khiển kết nối với dịch vụ AI Vision API. Nó có phương thức analyzeImage() để gửi ảnh tới AI Vision API và trả về danh sách thẻ gợi ý.

Các lớp thực thể:

- User là lớp thực thể lưu thông tin thành viên đăng bài.
- Post là lớp thực thể lưu thông tin bài đăng mất đồ hoặc nhặt được.
- Category là lớp thực thể lưu thông tin danh mục đồ vật.
- Image là lớp thực thể lưu thông tin ảnh minh chứng của bài đăng.
- Tag là lớp thực thể lưu thông tin thẻ mô tả đồ vật.
- PostTag là lớp thực thể trung gian dùng để gắn nhiều thẻ cho một bài đăng.

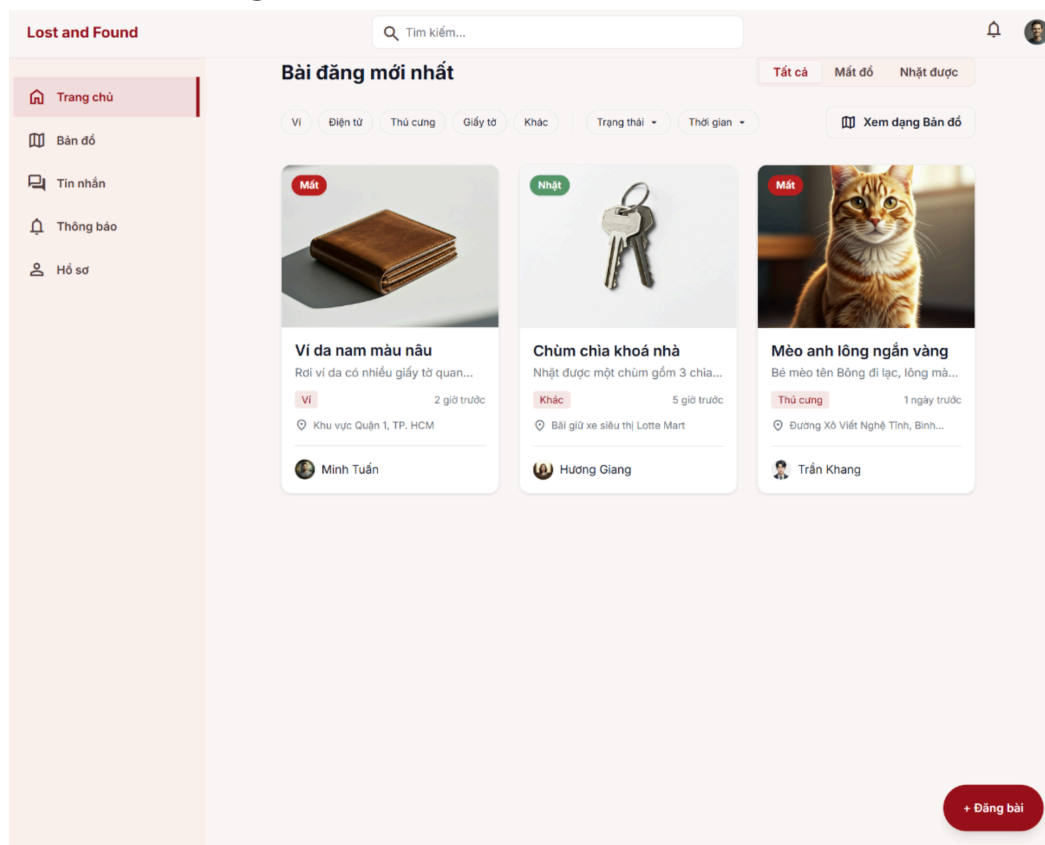
Như vậy, sơ đồ lớp cho chức năng đăng tải bài viết cần thể hiện các lớp giao diện MemberHomePage, CreatePostPage, CreatePostStep1Panel, CreatePostStep2Panel, CreatePostStep3Panel, CreatePostStep4Panel; các lớp điều khiển PostController,

CategoryController, ImageController, TagController, AIVisionController; và các lớp thực thể User, Post, Category, Image, Tag, PostTag.

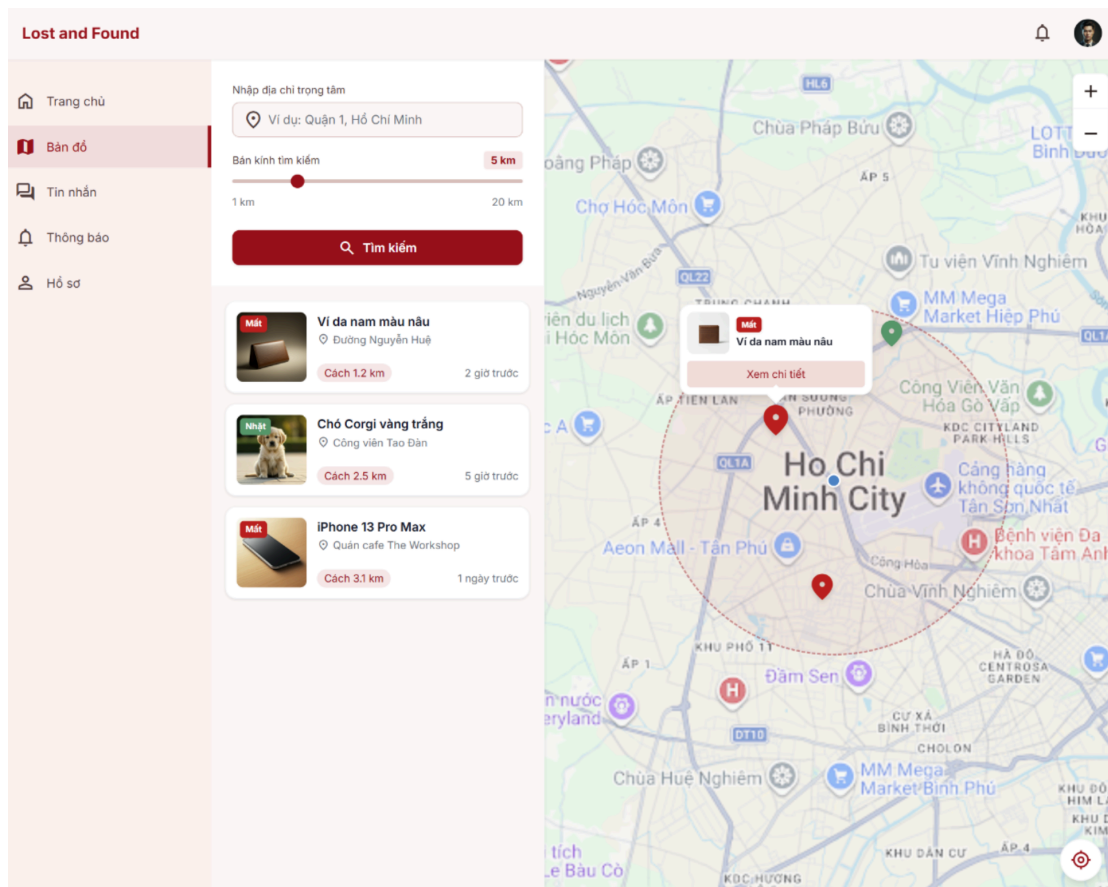


3.6.2. Chức năng tìm kiếm đồ vật

Giao diện chính gồm thanh tìm kiếm và các bộ lọc:



Giao diện tìm kiếm theo bản đồ:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

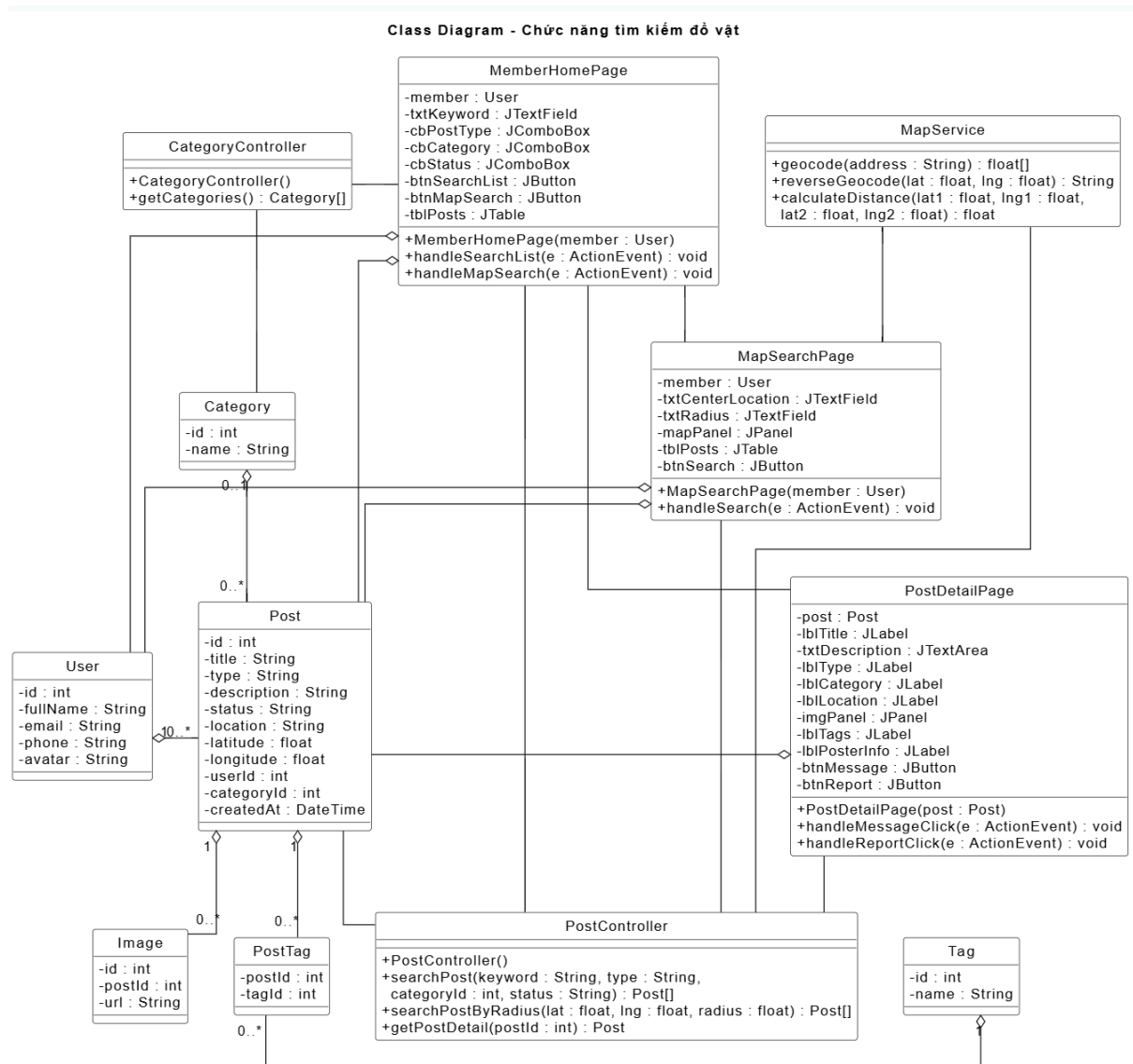
- MemberHomePage là giao diện chính của Thành viên. Nó cần ô nhập từ khóa, bộ lọc loại bài đăng, bộ lọc danh mục, bộ lọc trạng thái, nút tìm kiếm theo danh sách, nút "Tìm kiếm qua Bản đồ" và bảng hiển thị danh sách bài đăng phù hợp.
- MapSearchPage là giao diện tìm kiếm đồ vật theo bản đồ. Nó cần ô nhập vị trí trung tâm, ô nhập bán kính quét, bản đồ số hiển thị các điểm ghim (Marker) và nút tìm kiếm.
- PostDetailPage là giao diện xem chi tiết bài đăng. Nó cần hiển thị tiêu đề, mô tả, loại bài, danh mục, vị trí, hình ảnh, thẻ liên quan, thông tin người đăng và các nút "Nhắn tin", "Báo cáo vi phạm".

Các lớp điều khiển:

- PostController có ba phương thức searchPost() để tìm bài đăng theo từ khóa kết hợp bộ lọc, searchPostByRadius() để tìm bài đăng theo phạm vi tọa độ trên bản đồ và getPostDetail() để lấy chi tiết của bài đăng.

- CategoryController có phương thức getCategories() để lấy danh sách danh mục phục vụ bộ lọc.
- MapService có phương thức geocode() để chuyển địa chỉ thành tọa độ và phương thức calculateDistance() để tính khoảng cách giữa vị trí tìm kiếm và vị trí bài đăng.

Các lớp thực thể: User, Post, Category, Image, Tag và PostTag.



3.6.3. Chức năng trao đổi tin nhắn

Giao diện chi tiết bài đăng:

Lost and Found

Tìm kiếm...

Trang chủ

Bản đồ

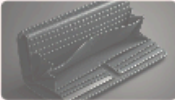
Tin nhắn

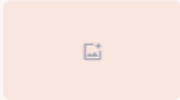
Thông báo

Hồ sơ









Mất đồ

Đang tìm

Giấy tờ

Rơi ví da đen tại sảnh A tòa nhà X

2 giờ trước

Quận 1, TP.HCM

128 lượt xem

Mô tả

Sáng nay khoảng 8h30, mình có làm rơi một chiếc ví da bò màu đen tại khu vực sảnh A tòa nhà X. Trong ví có thẻ CCCD, giấy phép lái xe và một số thẻ ngân hàng mang tên Nguyễn Văn A.

Ví có thiết kế gấp đôi, bề mặt da trơn, góc phải có logo kim loại nhỏ. Ai nhặt được xin vui lòng liên hệ mình, mình xin cảm ơn và hậu tạ.

Tags từ AI

Ví da

Màu đen

Da bò

Hiệu Gucci

Vị trí





Nguyễn Văn A

★★★★☆

Số điện thoại:

0987***321

Tham gia:

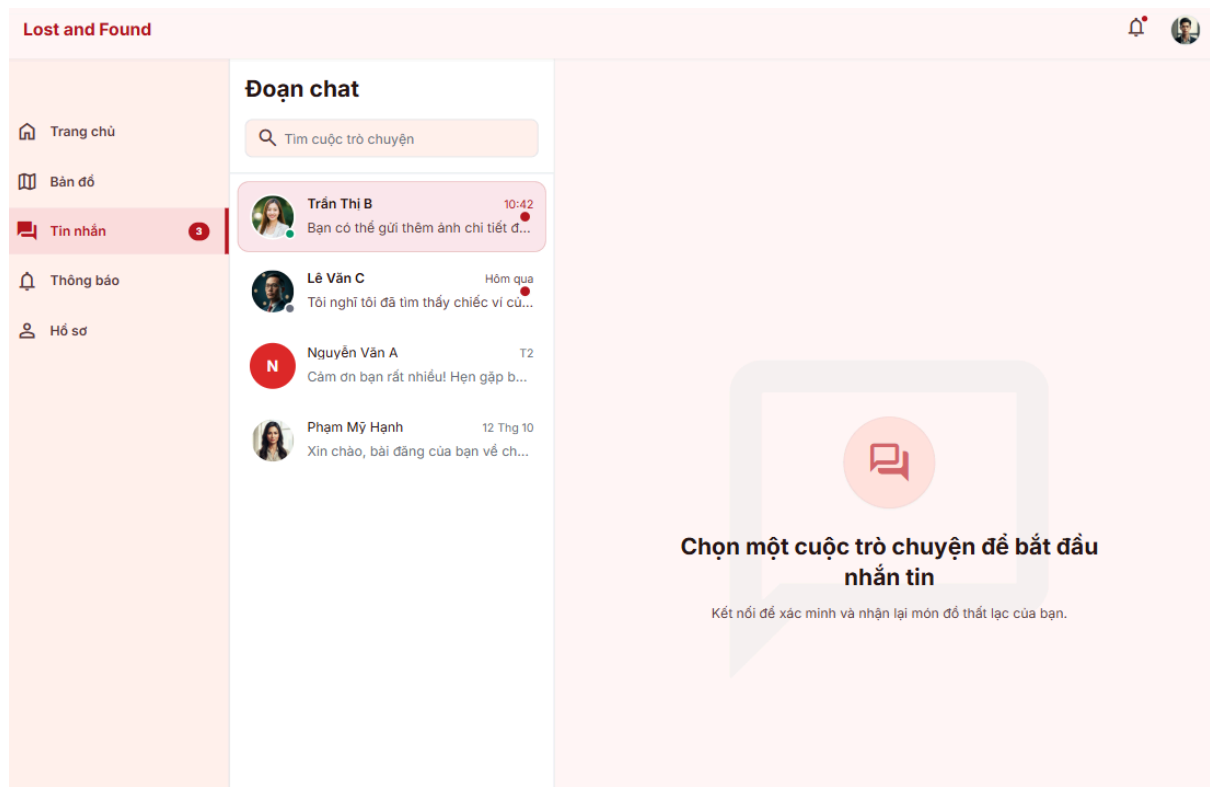
03/2024

Nhắn tin cho chủ bài

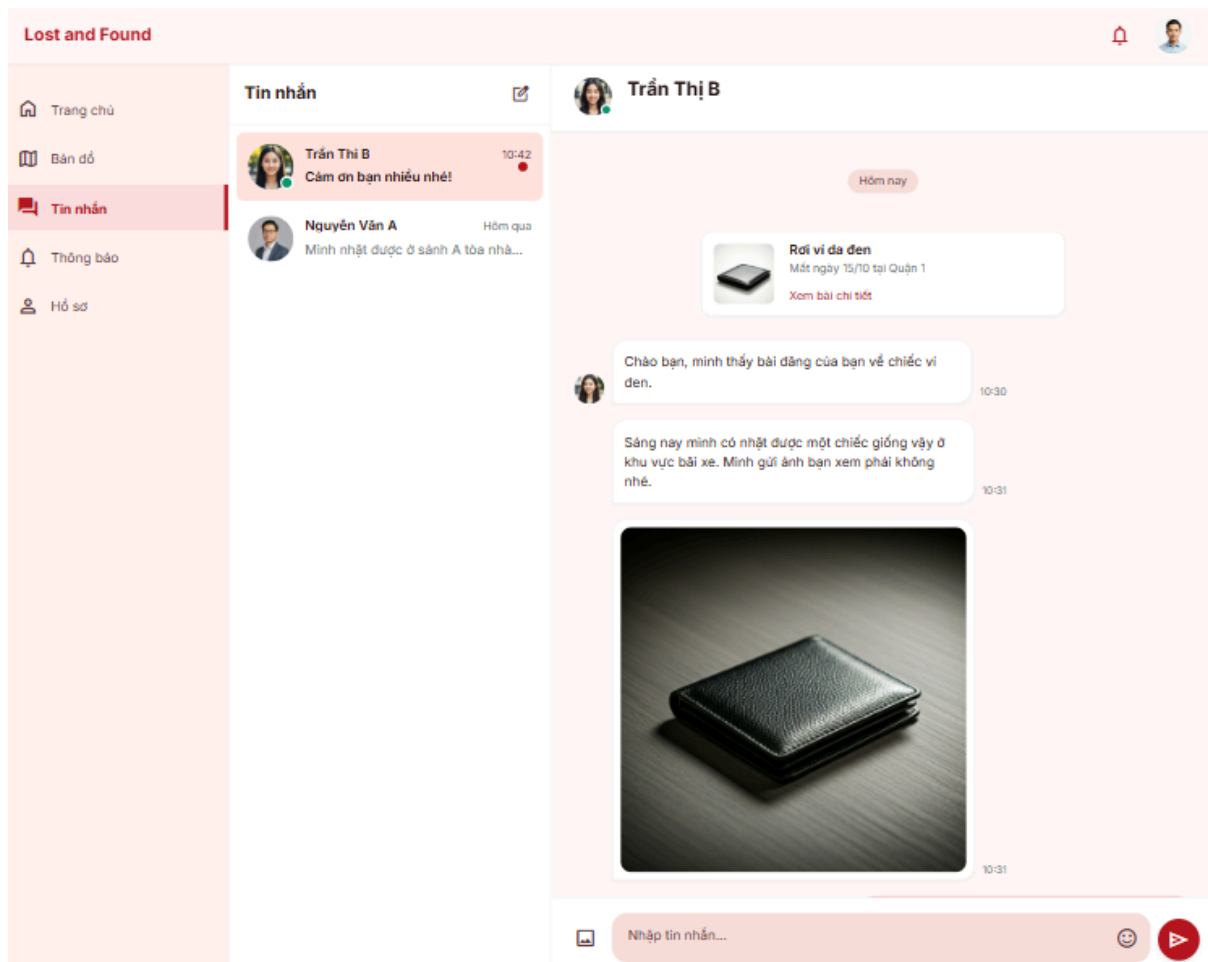
Báo cáo bài viết

Giao diện danh sách trò chuyện:

95



Giao diện chi tiết phòng chat:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

- PostDetailPage là giao diện chi tiết bài đăng. Nó cần hiển thị thông tin bài đăng gồm hình ảnh, tiêu đề, mô tả, danh sách thẻ, vị trí trên bản đồ và thông tin chủ bài đăng. Giao diện này cần có nút "Nhấn tin cho chủ bài" để thành viên bắt đầu trao đổi với người đăng bài.
- ChatListPage là giao diện danh sách trò chuyện. Nó cần hiển thị danh sách các cuộc trò chuyện của thành viên, gồm thông tin người đang trao đổi, nội dung tin nhắn gần nhất, thời gian gửi và trạng thái chưa đọc. Giao diện này cần có ô tìm kiếm cuộc trò chuyện để thành viên lọc nhanh các đoạn chat và cho phép chọn một cuộc trò chuyện để mở phòng chat tương ứng.
- ChatPage là giao diện chi tiết phòng chat. Nó cần hiển thị thông tin người đang trao đổi, danh sách tin nhắn trong phòng chat, thẻ tóm tắt bài đăng liên quan, ô nhập nội dung tin nhắn, nút "Gửi" và nút đính kèm hình ảnh nếu người dùng cần gửi minh chứng.

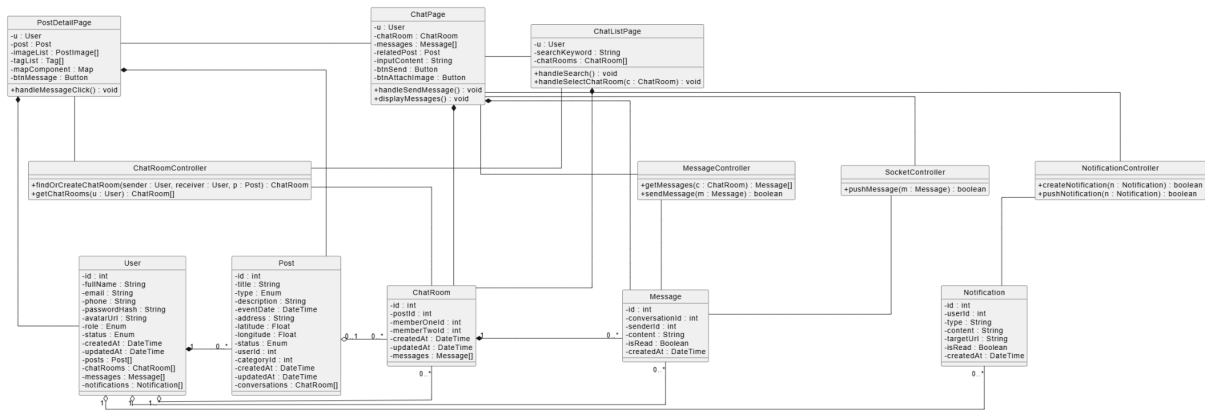
Các lớp điều khiển:

- ChatRoomController là lớp điều khiển xử lý các yêu cầu liên quan đến phòng chat. Nó có phương thức findOrCreateChatRoom() để tìm phòng chat đã tồn tại giữa hai thành viên hoặc tạo phòng chat mới nếu chưa có. Ngoài ra, lớp này có phương thức getChatRooms() để lấy danh sách phòng chat của thành viên hiển thị trên ChatListPage.
- MessageController là lớp điều khiển xử lý các yêu cầu liên quan đến tin nhắn. Nó có phương thức getMessages() để lấy danh sách tin nhắn trong một phòng chat và sendMessage() để tạo, mã hóa và lưu tin nhắn mới.
- NotificationController là lớp điều khiển xử lý thông báo khi có tin nhắn mới. Nó có phương thức createNotification() để tạo thông báo "Bạn có tin nhắn mới" cho người nhận và pushNotification() để đẩy thông báo tức thời đến người nhận đang online.
- SocketController là lớp điều khiển xử lý giao tiếp thời gian thực. Nó có phương thức pushMessage() để đẩy tin nhắn thời gian thực đến các thành viên trong phòng chat.

Các lớp thực thể:

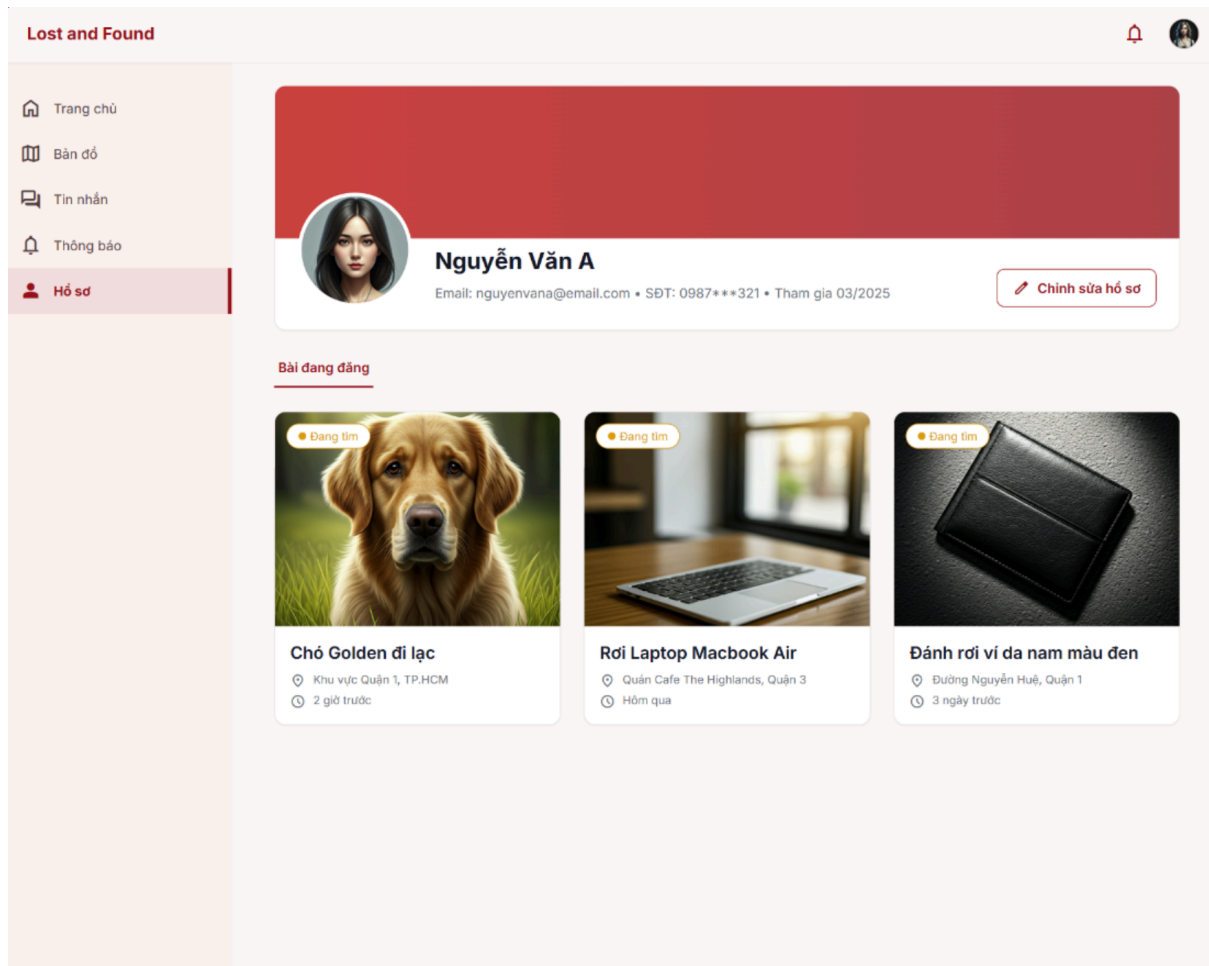
- User là lớp thực thể lưu thông tin thành viên gửi và nhận tin nhắn.
- Post là lớp thực thể lưu thông tin bài đăng liên quan đến cuộc trò chuyện.
- ChatRoom là lớp thực thể lưu thông tin phòng chat giữa hai thành viên.
- Message là lớp thực thể lưu nội dung tin nhắn trong phòng chat.
- Notification là lớp thực thể lưu thông báo gửi tới người nhận khi có tin nhắn mới.

Như vậy, sơ đồ lớp cho chức năng trao đổi tin nhắn cần thể hiện các lớp giao diện PostDetailPage, ChatListPage, ChatPage; các lớp điều khiển ChatRoomController, MessageController, NotificationController, SocketController; và các lớp thực thể User, Post, ChatRoom, Message, Notification.



3.6.4. Chức năng quản lý hồ sơ cá nhân

Giao diện hồ sơ:



Giao diện chỉnh sửa hồ sơ:

Lost and Found

Trang chủ

Bản đồ

Tin nhắn

Thông báo

Hồ sơ

Chỉnh sửa hồ sơ

Đổi ảnh đại diện

Họ và tên

Nguyễn Văn A

Email

nguyenvana@email.com

Đã xác minh

Số điện thoại

0987***321

Mật khẩu

Đổi mật khẩu

Giới thiệu bản thân

Viết một vài dòng về bạn...

Hủy

Lưu thay đổi

Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

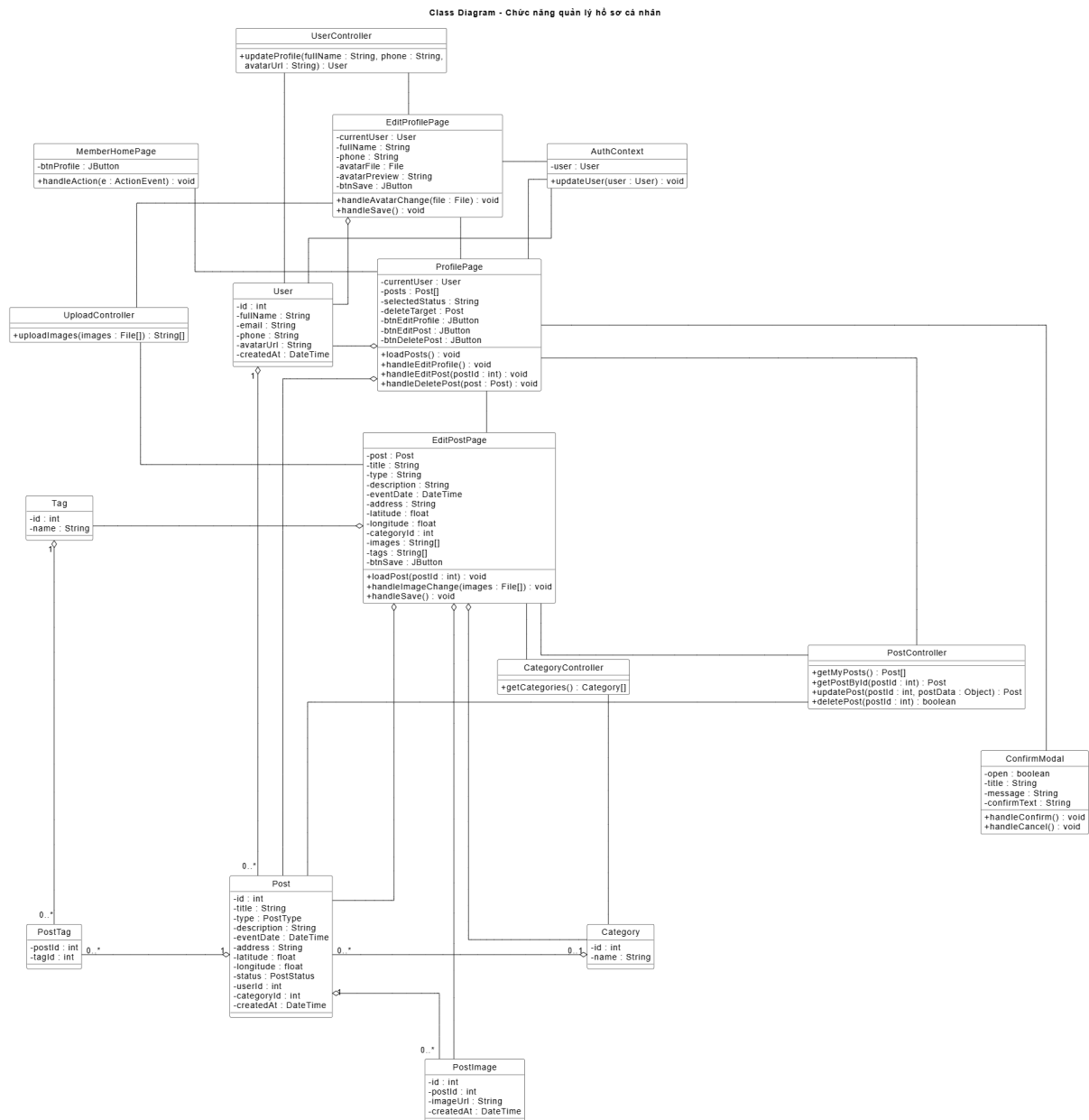
Xác định các lớp giao diện:

- MemberHomePage là giao diện chính của Thành viên. Nó cần mục truy cập "Hồ sơ cá nhân".
- ProfilePage là giao diện hiển thị hồ sơ cá nhân. Nó cần hiển thị thông tin User, ảnh đại diện, lưới danh sách bài đăng của thành viên và các nút "Sửa thông tin", "Sửa đổi", "Xóa" tương ứng với từng bài đăng.
- EditProfileModal là giao diện sửa thông tin cá nhân. Nó cần các trường nhập họ tên, email, số điện thoại, ảnh đại diện và nút "Lưu thay đổi".
- EditPostModal là giao diện sửa bài đăng. Nó cần các trường nhập tiêu đề, mô tả, ngày xảy ra sự việc, vị trí, danh mục và hình ảnh; sau khi lưu, hệ thống tự động chuyển trạng thái bài đăng về "Chờ duyệt".
- ConfirmDeletePostModal là giao diện xác nhận xóa bài đăng. Nó cần thông báo xác nhận và hai nút "Đồng ý"/"Hủy".

Các lớp điều khiển:

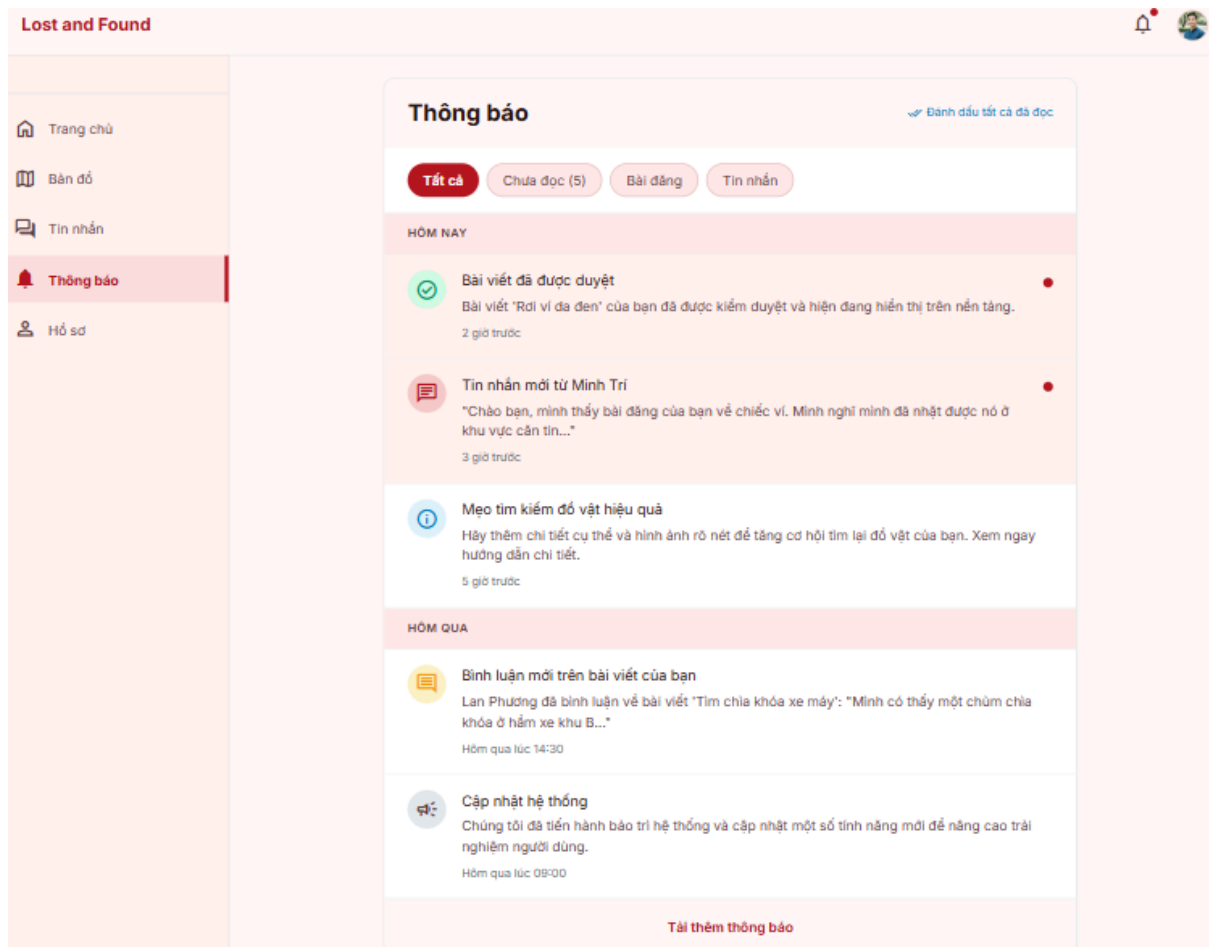
- UserController có hai phương thức `getProfile()` để lấy thông tin cá nhân và `updateProfile()` để cập nhật hồ sơ.
- PostController có bốn phương thức `getPostsByUser()` để lấy danh sách bài đăng của thành viên, `getPostDetail()` để lấy chi tiết bài đăng, `updatePost()` để cập nhật bài đăng và `deletePost()` để xóa bài đăng.
- ImageController có phương thức `updateImages()` để cập nhật danh sách ảnh đính kèm khi sửa bài đăng.
- CategoryController có phương thức `getCategories()` để lấy danh mục khi sửa bài đăng.

Các lớp thực thể: User, Post, Image và Category.



3.6.5. Chức năng quản lý thông báo

Giao diện danh sách các thông báo:



Xác định các lớp giao diện:

- MemberHomePage là giao diện chính của Thành viên. Nó cần biểu tượng quả chuông hoặc mục "Thông báo" trên thanh điều hướng để người dùng mở danh sách thông báo. Khi có thông báo chưa đọc, giao diện hiển thị dấu hiệu đỏ để nhắc người dùng.
- NotificationPage là giao diện danh sách thông báo của Thành viên. Nó cần hiển thị danh sách các thông báo gồm nội dung thông báo, loại thông báo, thời điểm tạo và trạng thái đã đọc/chưa đọc. Khi thành viên click trực tiếp vào một thông báo, hệ thống đánh dấu thông báo đó là đã đọc và điều hướng đến bài đăng hoặc phòng chat liên quan nếu thông báo có liên kết tương ứng.

Các lớp điều khiển:

- NotificationController là lớp điều khiển xử lý các yêu cầu liên quan đến thông báo. Nó có phương thức getNotifications() để lấy danh sách thông báo của thành viên, markAsRead() để đánh dấu một thông báo là đã đọc và

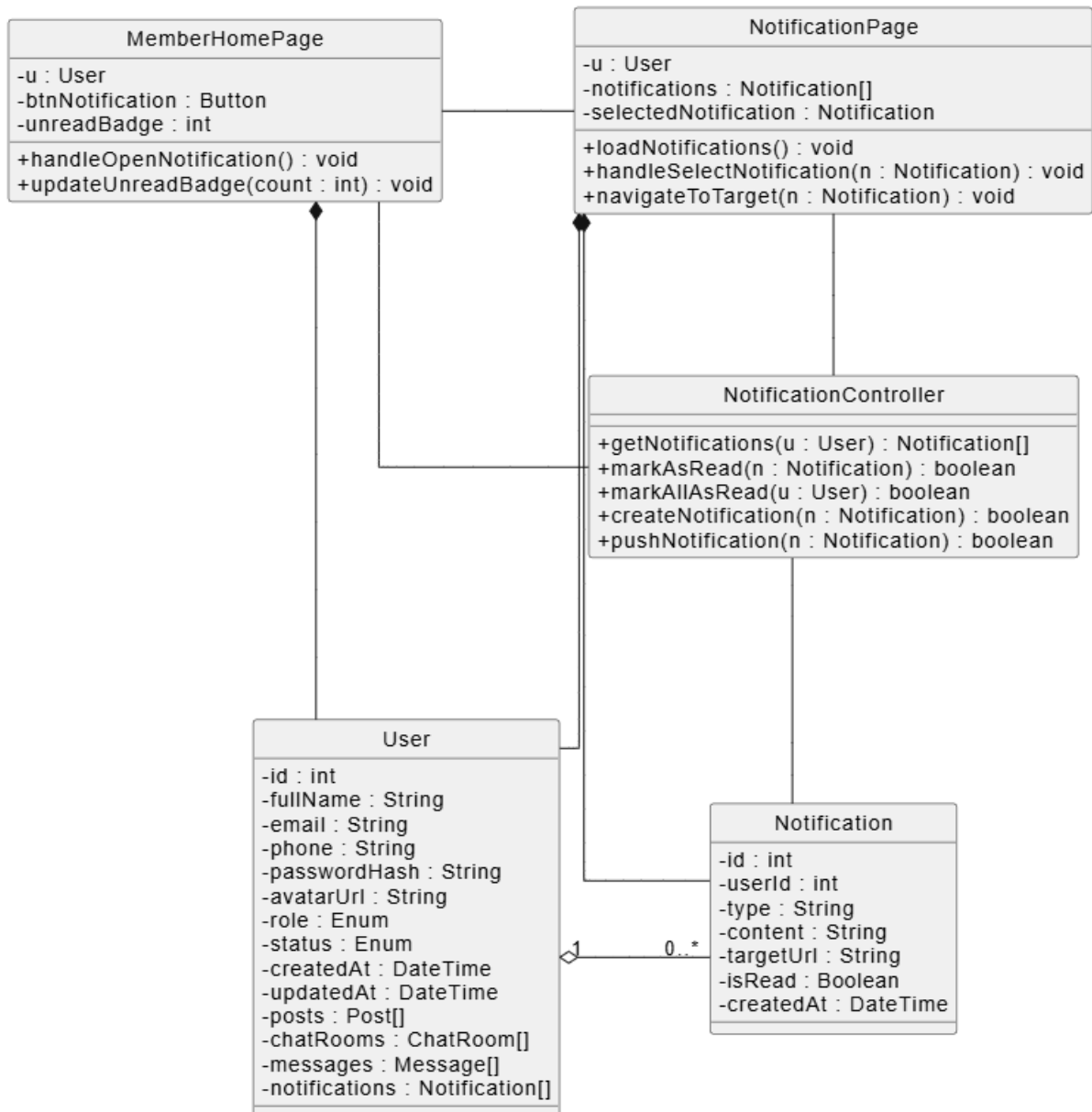
createNotification() để tạo thông báo mới khi có sự kiện phát sinh trong hệ thống.

- SocketController là lớp điều khiển xử lý thông báo thời gian thực. Nó có phương thức pushNotification() để đẩy thông báo đến người dùng đang online qua kết nối Socket.io.

Các lớp thực thể:

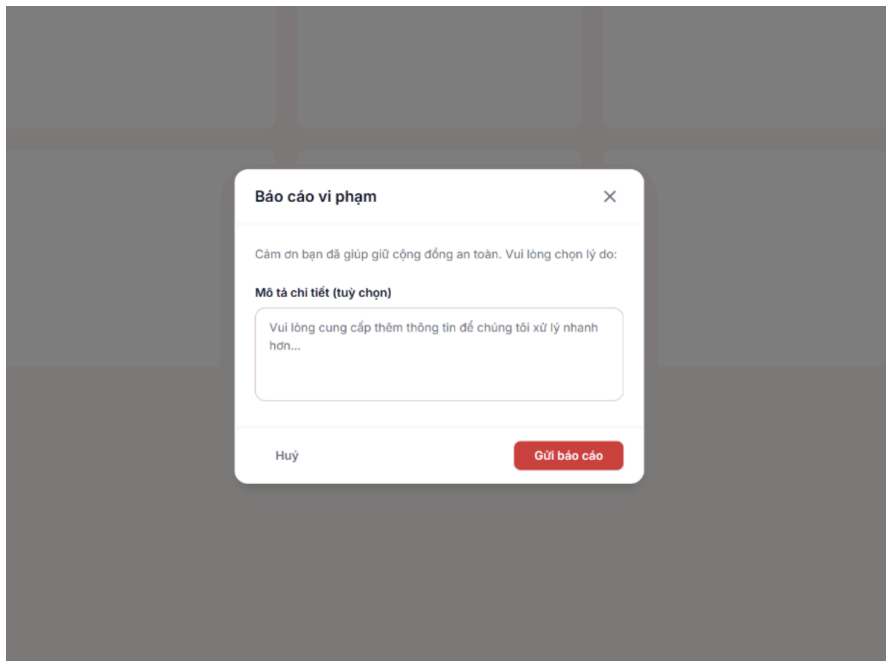
- User là lớp thực thể lưu thông tin thành viên nhận thông báo.
- Notification là lớp thực thể lưu thông tin thông báo, gồm loại thông báo, nội dung, trạng thái đọc/chưa đọc, thời điểm tạo, người nhận và đối tượng liên quan.
- Post là lớp thực thể lưu thông tin bài đăng liên quan nếu thông báo thuộc loại bài đăng.
- ChatRoom là lớp thực thể lưu thông tin phòng chat liên quan nếu thông báo thuộc loại tin nhắn.
- Message là lớp thực thể lưu thông tin tin nhắn liên quan nếu thông báo được tạo từ một tin nhắn mới.

Như vậy, sơ đồ lớp cho chức năng quản lý thông báo cần thể hiện các lớp giao diện MemberHomePage, NotificationPage; các lớp điều khiển NotificationController, SocketController; và các lớp thực thể User, Notification, Post, ChatRoom, Message.



3.6.6. Chức năng báo cáo bài viết:

Giao diện báo cáo bài viết:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

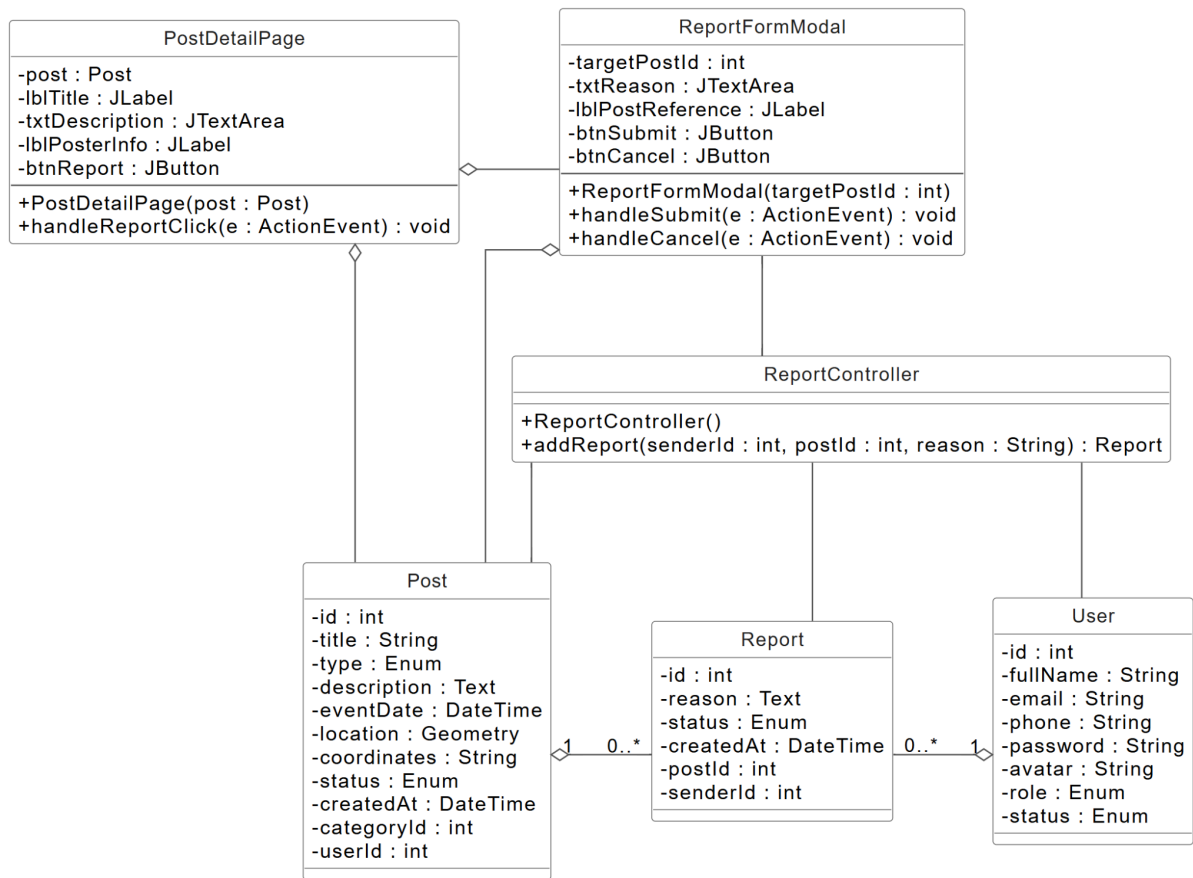
- PostDetailPage là giao diện chi tiết bài đăng. Nó cần biểu tượng "3 chấm" hoặc menu thao tác chứa lựa chọn "Báo cáo vi phạm" để mở form báo cáo, kèm theo việc truyền targetPostId của bài đăng đang xem sang form.
- ReportFormModal là giao diện nhập nội dung báo cáo. Nó cần vùng nhập lý do vi phạm, hiển thị thông tin tham chiếu của bài đăng bị báo cáo và nút "Gửi báo cáo"; sau khi gửi thành công, form hiển thị thông báo cảm ơn rồi đóng lại.

Các lớp điều khiển:

- ReportController có phương thức addReport() để tạo bản ghi báo cáo vi phạm mới với trạng thái "Chờ xử lý", lưu kèm reporterId, targetPostId và nội dung vi phạm.

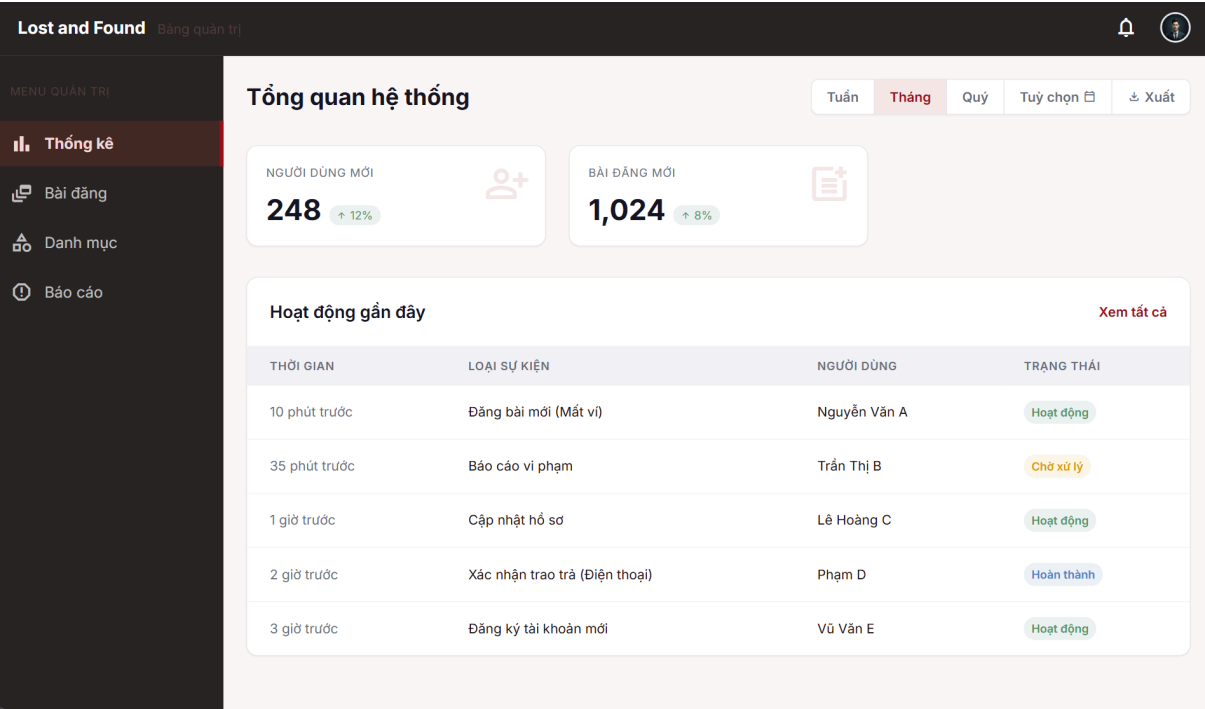
Các lớp thực thể: User, Post và Report.

Class Diagram - Chức năng báo cáo bài viết

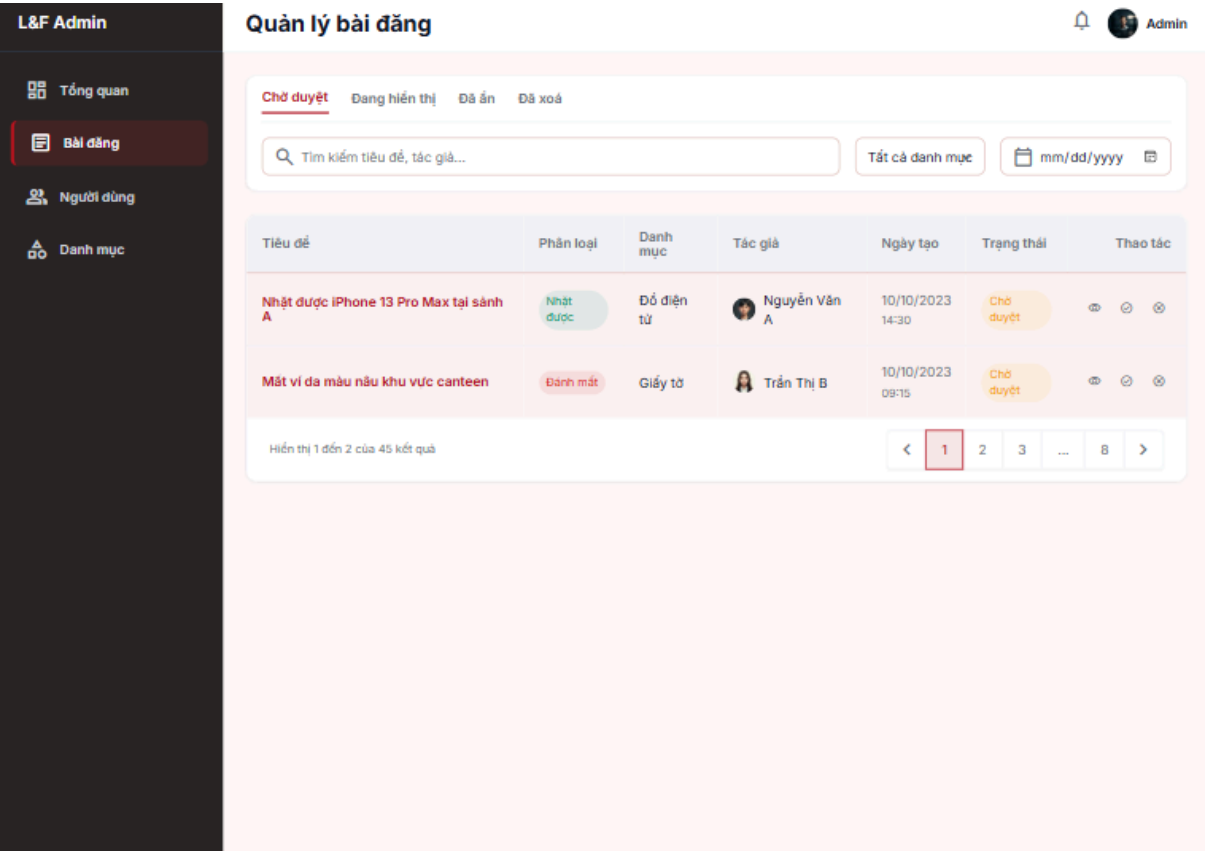


3.6.7. Chức năng quản lý bài đăng

Giao diện chính Admin:



Giao diện chính quản lý bài đăng:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

- AdminHomePage là giao diện chính của Quản trị viên. Nó cần nút hoặc mục menu để chuyển đến chức năng "Quản lý bài đăng".
- ManagePostPage là giao diện danh sách bài đăng chờ duyệt. Nó cần bảng hiển thị bài đăng kèm thông tin tiêu đề, loại bài, người đăng, thời gian khởi tạo và nút "Xem" tại mỗi dòng để mở chi tiết bài đăng.
- PostDetailPage là giao diện chi tiết bài đăng dùng cho quản trị viên xét duyệt. Nó cần hiển thị thông tin bài đăng, ảnh minh chứng, vị trí ghim trên bản đồ, danh sách Tags và các nút "Duyệt", "Từ chối". Trường hợp bài đăng bị từ chối, hệ thống tiến hành xóa bài đăng khỏi cơ sở dữ liệu mà không cần form nhập lý do riêng.

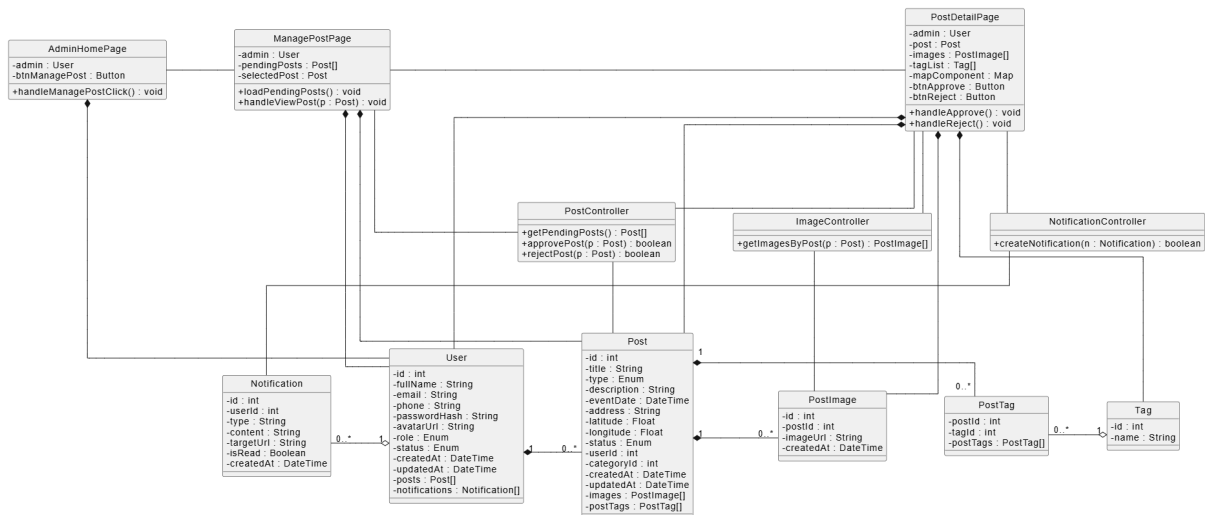
Các lớp điều khiển:

- PostController là lớp điều khiển xử lý các yêu cầu liên quan đến quản lý bài đăng. Nó có phương thức getPendingPosts() để lấy danh sách bài đăng chờ duyệt, approvePost() để chuyển trạng thái bài đăng sang "Hoạt động" và rejectPost() để xóa bài đăng vi phạm khỏi hệ thống.
- ImageController là lớp điều khiển xử lý dữ liệu hình ảnh của bài đăng. Nó có phương thức getImagesByPost() để lấy hình ảnh của bài đăng phục vụ hiển thị trong PostDetailPage.
- NotificationController là lớp điều khiển xử lý thông báo kết quả kiểm duyệt. Nó có phương thức createNotification() để gửi thông báo duyệt hoặc từ chối bài đăng cho người đăng.

Các lớp thực thể:

- User là lớp thực thể lưu thông tin quản trị viên và người đăng bài.
- Post là lớp thực thể lưu thông tin bài đăng cần kiểm duyệt.
- Image là lớp thực thể lưu thông tin hình ảnh minh chứng của bài đăng.
- Notification là lớp thực thể lưu thông báo kết quả duyệt hoặc từ chối bài đăng.

Như vậy, sơ đồ lớp cho chức năng quản lý bài đăng cần thể hiện các lớp giao diện AdminHomePage, ManagePostPage, PostDetailPage; các lớp điều khiển PostController, ImageController, NotificationController; và các lớp thực thể User, Post, Image, Notification.



3.6.8. Chức năng quản lý báo cáo vi phạm

Giao diện quản lý báo cáo vi phạm:

Lost and Found
Admin Portal

Dashboard

Người dùng

Bài đăng

7

Đăng xuất

Quản lý báo cáo vi phạm

Theo dõi và xử lý các báo cáo từ cộng đồng.

Chờ xử lý (7)

Đang xem xét

Đã giải quyết

Bỏ qua

Tắt cả mức độ

Người gửi	Bài viết	Thời Gian	Hành Động
Nguyễn Văn A	Post: Poodle...	10 phút trước	<button>Xem chi tiết</button>
Trần Thị B	1	2 giờ trước	<button>Xem chi tiết</button>
Lê Văn C	2	Hôm qua	<button>Xem chi tiết</button>
Phạm Minh D	bài đăng	Hôm qua	<button>Xem chi tiết</button>

Đang xem 1 đến 4 trong 7 báo cáo

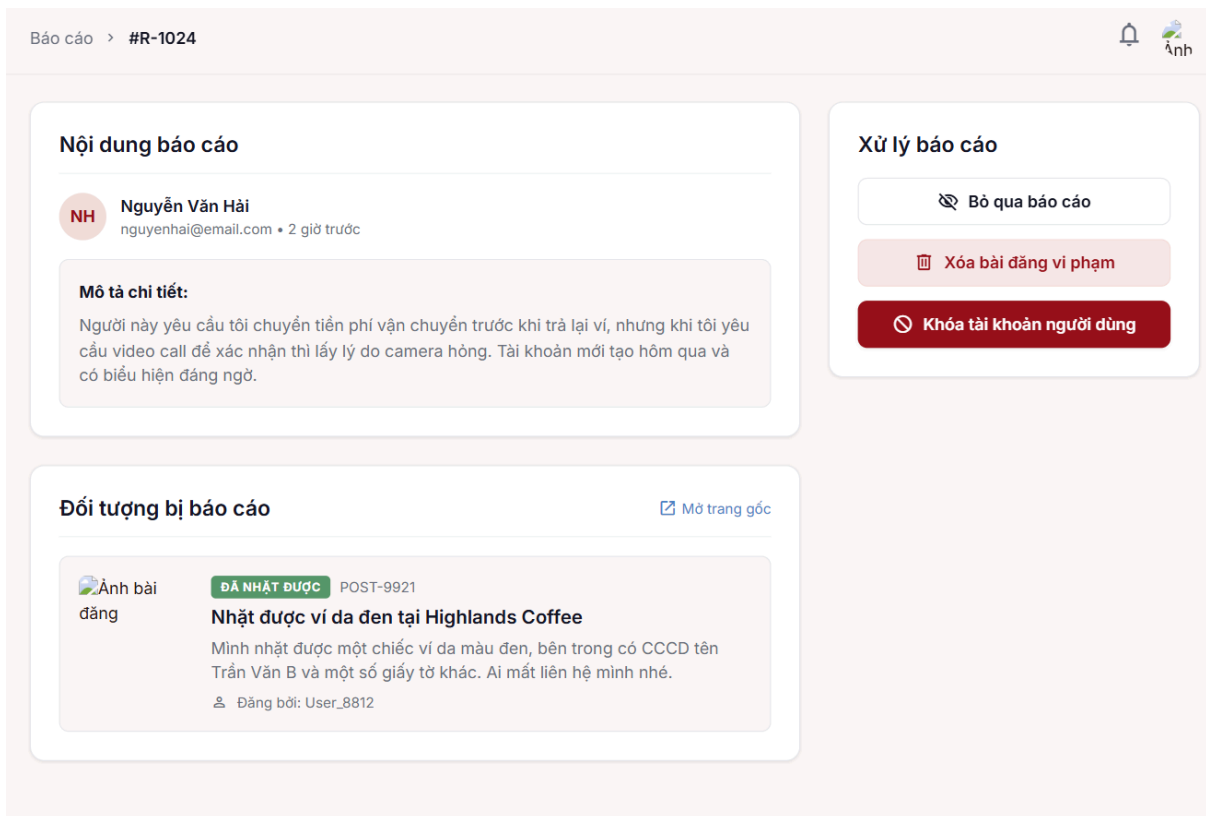
<

1

2

>

Giao diện xử lý vi phạm:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

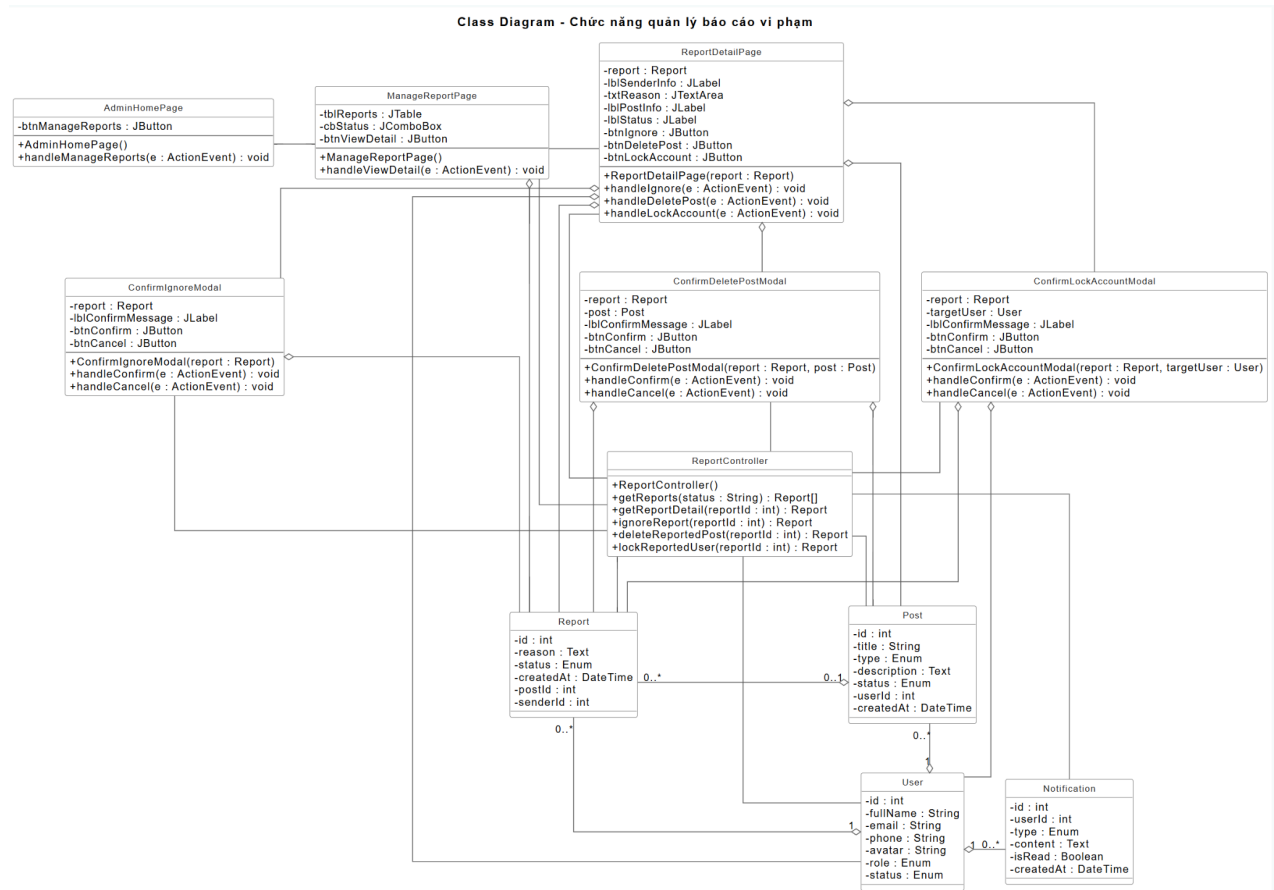
- AdminHomePage là giao diện chính của Quản trị viên. Nó cần nút hoặc mục menu để chuyển đến chức năng "Quản lý Báo cáo".
- ManageReportPage là giao diện danh sách báo cáo chờ xử lý. Nó cần bảng hiển thị bài đăng, người gửi, thời gian gửi và nút "Xem chi tiết" tại mỗi dòng.
- ReportDetailModal là giao diện chi tiết báo cáo. Nó cần hiển thị thông tin người gửi, nội dung vi phạm, đối tượng bị báo cáo (bài đăng hoặc người dùng), lịch sử liên quan và các nút hành động "Khóa tài khoản", "Xóa bài đăng", "Bỏ qua".
- ConfirmLockModal là giao diện popup xác nhận khóa tài khoản. Nó cần thông báo xác nhận có kèm tên tài khoản bị khóa và nút "Xác nhận khóa"/"Hủy".
- ConfirmDeletePostModal là giao diện popup xác nhận xóa bài đăng vi phạm. Nó cần thông báo xác nhận và nút "Đồng ý"/"Hủy".

Các lớp điều khiển:

- ReportController có hai phương thức getReports() để lấy danh sách báo cáo và resolveReport() để cập nhật trạng thái báo cáo thành "Đã giải quyết" sau khi xử lý.
- UserController có phương thức lockAccount() để khóa tài khoản vi phạm và tự động gỡ bỏ các bài đăng liên quan.

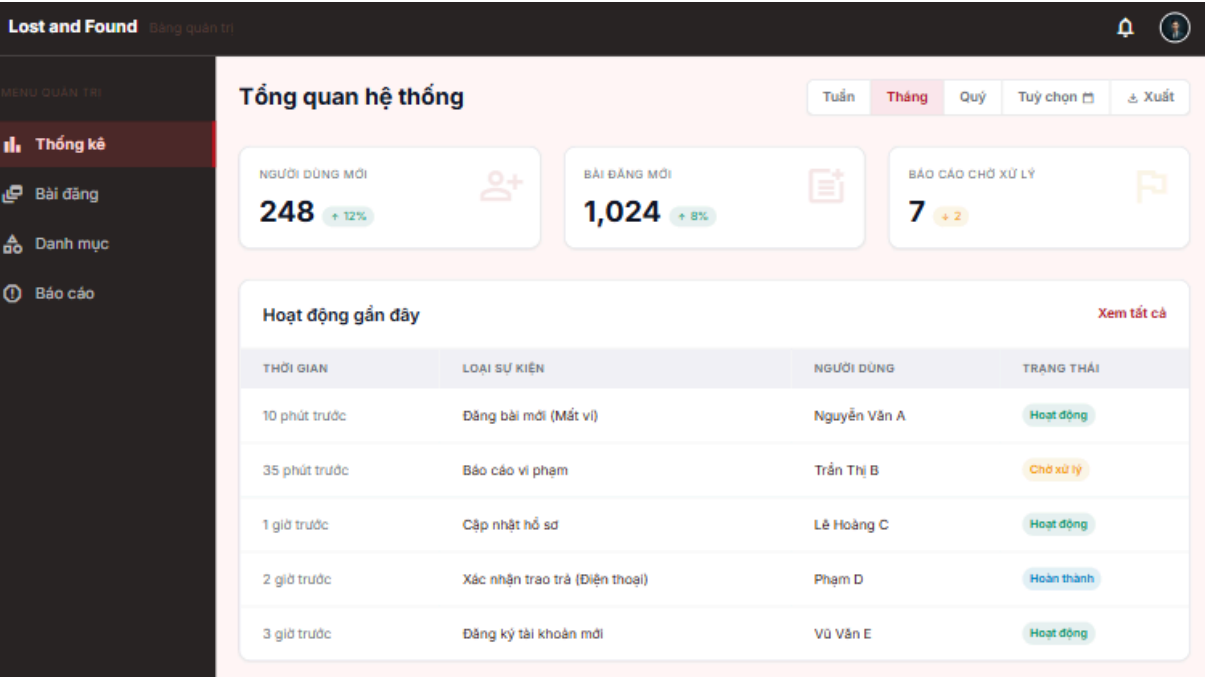
- PostController có phương thức deletePost() để xóa bài đăng vi phạm trong trường hợp chọn hành động "Xóa bài đăng".
- NotificationController có phương thức createNotification() để thông báo kết quả xử lý cho người liên quan nếu cần.

Các lớp thực thể: Report, User, Post và Notification.

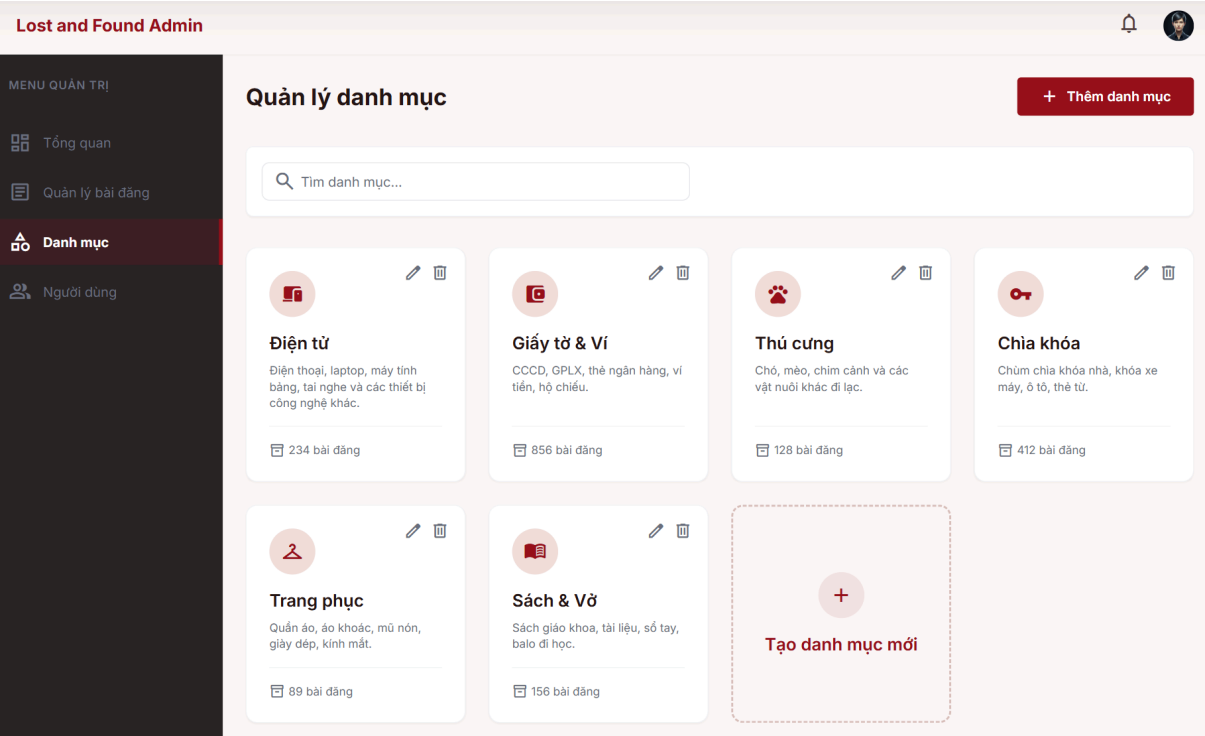


3.6.9. Chức năng quản lý danh mục

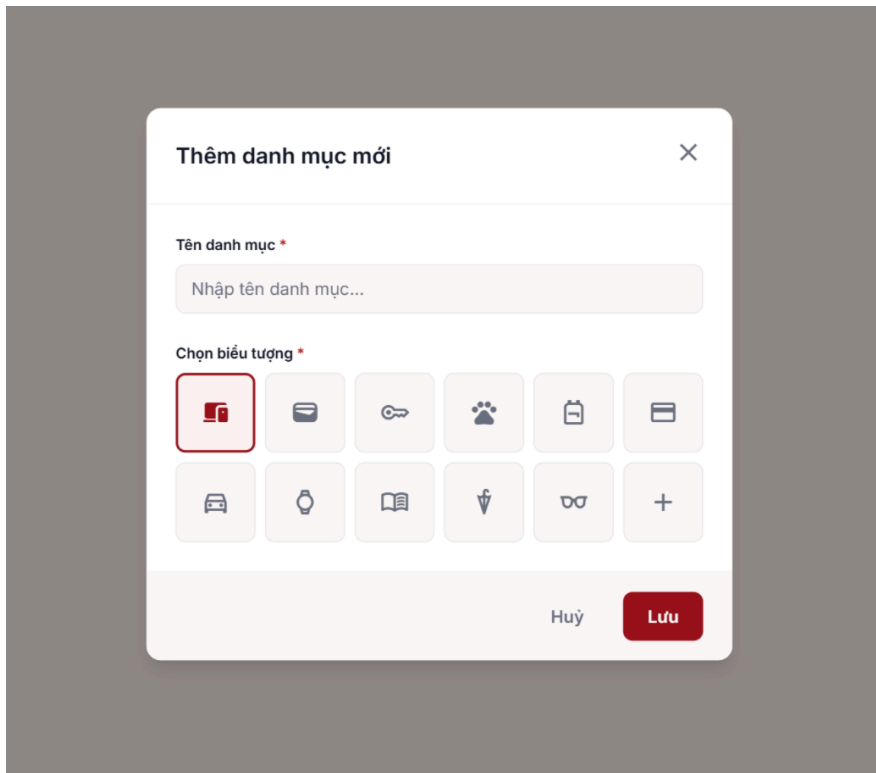
Giao diện chính Admin:



Giao diện quản lý thư mục:



Giao diện thêm thư mục:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

Xác định các lớp giao diện:

- AdminHomePage là giao diện chính của Quản trị viên. Nó cần nút hoặc mục menu để chuyển đến chức năng "Quản lý Danh mục".
- ManageCategoryPage là giao diện danh sách danh mục. Nó cần bảng hiển thị danh mục kèm số bài đăng thuộc từng danh mục, nút "Thêm danh mục", các nút "Sửa" và "Xóa" tại từng dòng.
- AddCategoryModal là giao diện thêm danh mục mới. Nó cần trường nhập tên danh mục, mô tả và nút "Thêm".
- EditCategoryModal là giao diện sửa danh mục. Nó cần hiển thị dữ liệu hiện tại của danh mục được điền sẵn vào các trường nhập và nút "Lưu thay đổi".
- ConfirmDeleteCategoryModal là giao diện popup xác nhận xóa danh mục. Nó cần thông báo xác nhận và nút "Đồng ý"/"Hủy".

Các lớp điều khiển:

- CategoryController là lớp điều khiển xử lý các yêu cầu liên quan đến danh mục. Nó có bốn phương thức getCategories() để lấy danh sách danh mục,

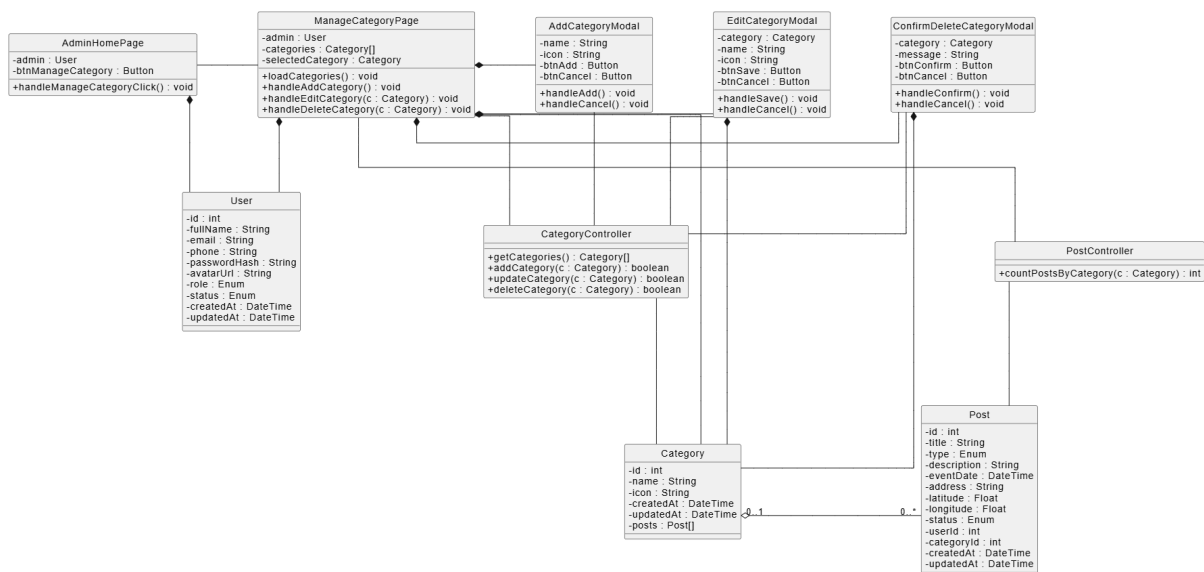
addCategory() để thêm danh mục mới, updateCategory() để cập nhật danh mục và deleteCategory() để xóa danh mục.

- PostController là lớp điều khiển xử lý dữ liệu bài đăng liên quan đến danh mục. Nó có phương thức countPostsByCategory() để kiểm tra số bài đăng đang thuộc danh mục trước khi xác nhận xóa.

Các lớp thực thể:

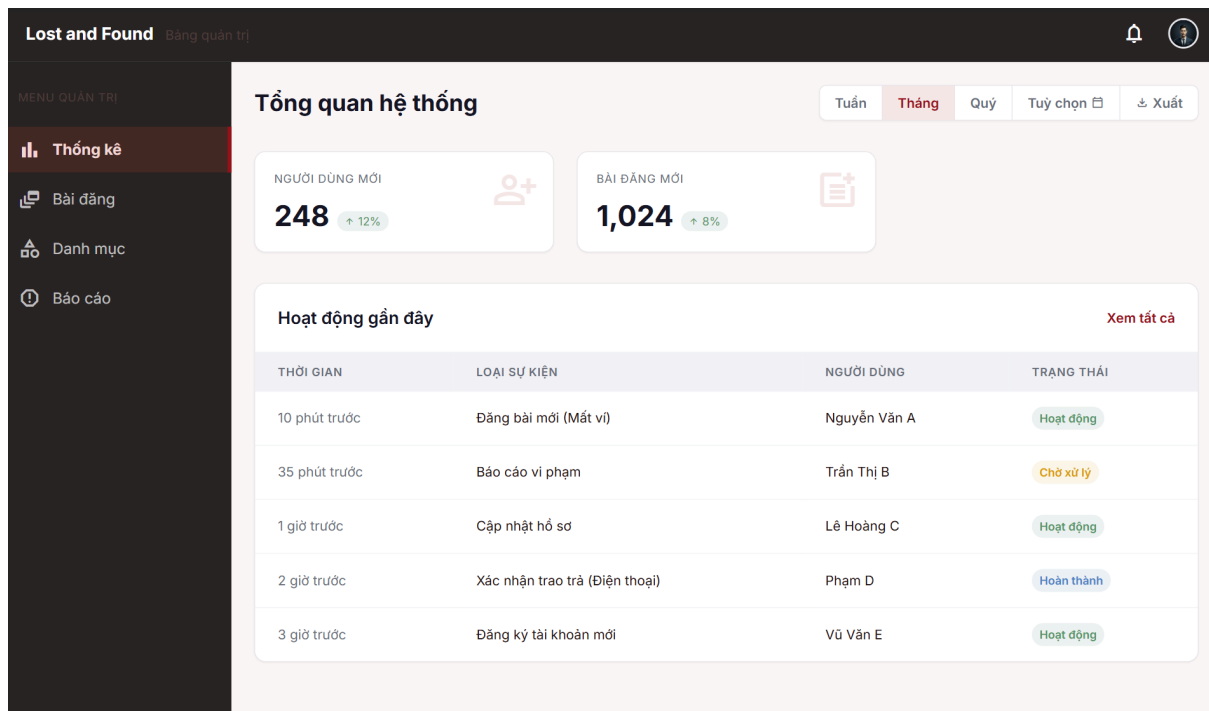
- Category là lớp thực thể lưu thông tin danh mục đồ vật.
- Post là lớp thực thể lưu thông tin bài đăng thuộc danh mục.

Như vậy, sơ đồ lớp cho chức năng quản lý danh mục cần thể hiện các lớp giao diện AdminHomePage, ManageCategoryPage, AddCategoryModal, EditCategoryModal, ConfirmDeleteCategoryModal; các lớp điều khiển CategoryController, PostController; và các lớp thực thể Category, Post



3.6.10. Chức năng xem báo cáo thống kê:

Giao diện xem báo cáo thống kê:



Trong mô-đun này, quá trình xử lý đăng nhập bị bỏ qua:

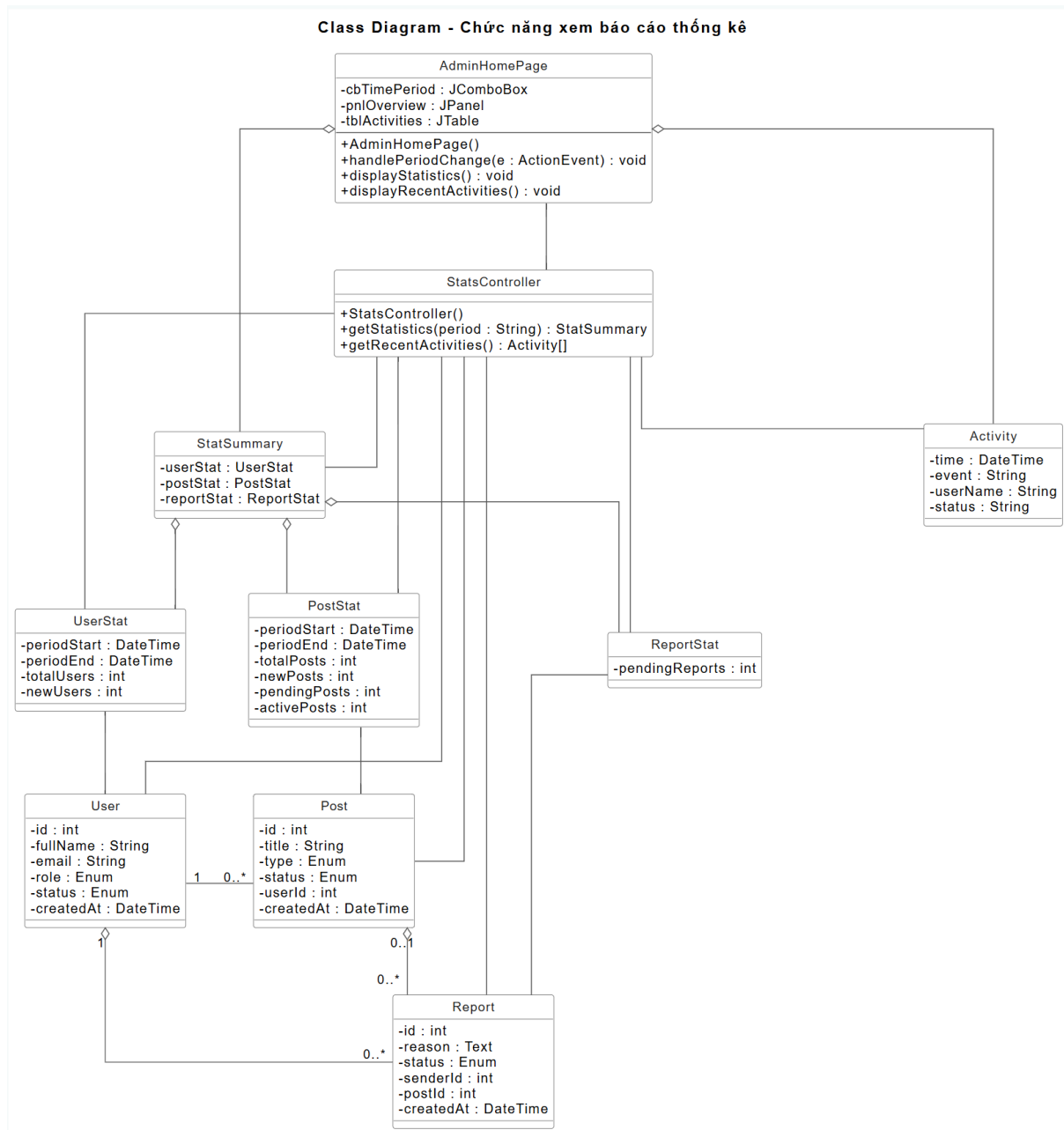
Xác định các lớp giao diện:

- AdminHomePage là giao diện chính của Quản trị viên, đồng thời đóng vai trò là giao diện thống kê tổng hợp mặc định khi đăng nhập. Nó cần ô chọn loại thống kê (người dùng mới/lượt truy cập/bài đăng mới), bộ chọn khoảng thời gian, các khối dữ liệu tổng quan, bảng số liệu chi tiết theo kỳ thời gian, biểu đồ minh họa và nút "Xuất báo cáo".
- ExportModal là giao diện xác nhận xuất báo cáo. Nó cần ô chọn định dạng file (Excel, PDF...) và nút "Xuất".

Các lớp điều khiển (DAO và dịch vụ):

- UserStatController có phương thức getUserStat() để lấy thống kê người dùng mới và lượt truy cập theo kỳ thời gian.
- PostStatController có phương thức getPostStat() để lấy thống kê bài đăng mới và bài đăng đang hoạt động theo kỳ thời gian.
- ReportExportService có phương thức exportReport() để xuất số liệu thống kê ra file.

Các lớp thực thể: UserStat, PostStat, User và Post.



3.7. Thiết kế biểu đồ tuần tự cho các chức năng

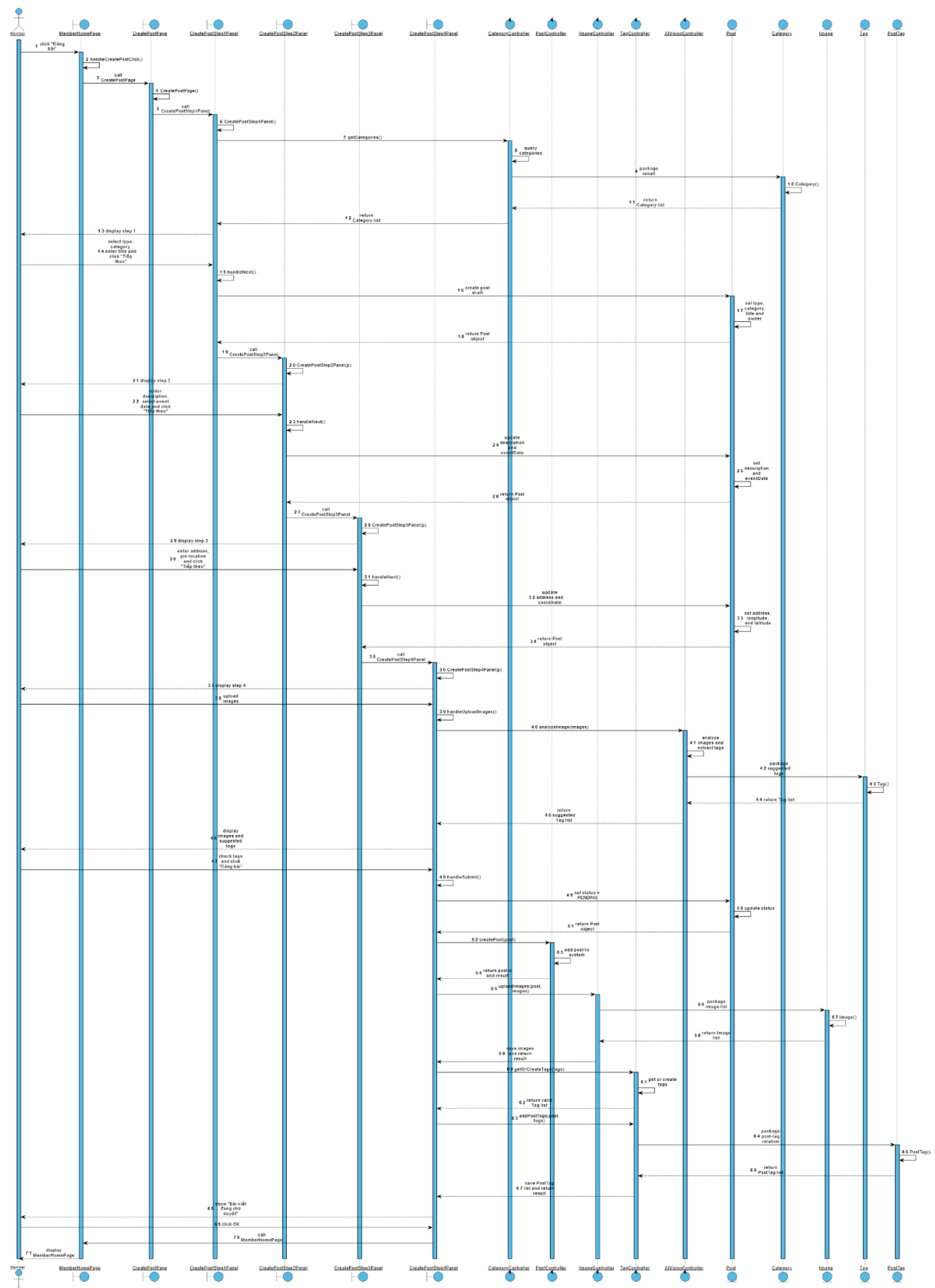
3.7.1. Chức năng đăng tải bài viết

1. Thành viên đang ở giao diện MemberHomePage và click nút "Đăng bài".
2. Phương thức handleCreatePostClick() của lớp MemberHomePage được gọi.
3. MemberHomePage gọi lớp CreatePostPage.
4. Constructor CreatePostPage() được gọi.

5. CreatePostPage gọi lớp CreatePostStep1Panel.
6. Constructor CreatePostStep1Panel() được gọi.
7. CreatePostStep1Panel gọi phương thức getCategories() của lớp CategoryController để lấy danh sách danh mục.
8. CategoryController truy vấn danh sách danh mục trong hệ thống.
9. CategoryController gọi lớp Category để đóng gói kết quả.
10. Lớp Category đóng gói từng đối tượng Category.
11. Lớp Category trả kết quả về cho CategoryController.
12. CategoryController trả danh sách Category về cho CreatePostStep1Panel.
13. Giao diện CreatePostStep1Panel được hiển thị cho thành viên.
14. Thành viên chọn loại bài đăng, chọn danh mục, nhập tiêu đề và click nút "Tiếp theo".
15. Phương thức handleNext() của lớp CreatePostStep1Panel được gọi.
16. CreatePostStep1Panel gọi lớp Post để tạo đối tượng bài đăng tạm.
17. Lớp Post đóng gói thông tin loại bài đăng, danh mục, tiêu đề và người đăng vào đối tượng Post.
18. Lớp Post trả đối tượng Post về cho CreatePostStep1Panel.
19. CreatePostStep1Panel gọi lớp CreatePostStep2Panel.
20. Constructor CreatePostStep2Panel() được gọi.
21. Giao diện CreatePostStep2Panel được hiển thị cho thành viên.
22. Thành viên nhập mô tả chi tiết, chọn thời điểm xảy ra sự việc và click nút "Tiếp theo".
23. Phương thức handleNext() của lớp CreatePostStep2Panel được gọi.
24. CreatePostStep2Panel gọi lớp Post để cập nhật mô tả và thời điểm xảy ra sự việc.
25. Lớp Post thiết lập các thuộc tính mô tả và thời điểm cho đối tượng Post.
26. Lớp Post trả kết quả về cho CreatePostStep2Panel.
27. CreatePostStep2Panel gọi lớp CreatePostStep3Panel.
28. Constructor CreatePostStep3Panel() được gọi.
29. Giao diện CreatePostStep3Panel được hiển thị cho thành viên.
30. Thành viên nhập địa chỉ, ghim vị trí trên bản đồ và click nút "Tiếp theo".
31. Phương thức handleNext() của lớp CreatePostStep3Panel được gọi.
32. CreatePostStep3Panel gọi lớp Post để cập nhật địa chỉ và tọa độ.
33. Lớp Post thiết lập địa chỉ, kinh độ, vĩ độ cho đối tượng Post.
34. Lớp Post trả kết quả về cho CreatePostStep3Panel.
35. CreatePostStep3Panel gọi lớp CreatePostStep4Panel.
36. Constructor CreatePostStep4Panel() được gọi.
37. Giao diện CreatePostStep4Panel được hiển thị cho thành viên.
38. Thành viên tải ảnh minh chứng lên giao diện.

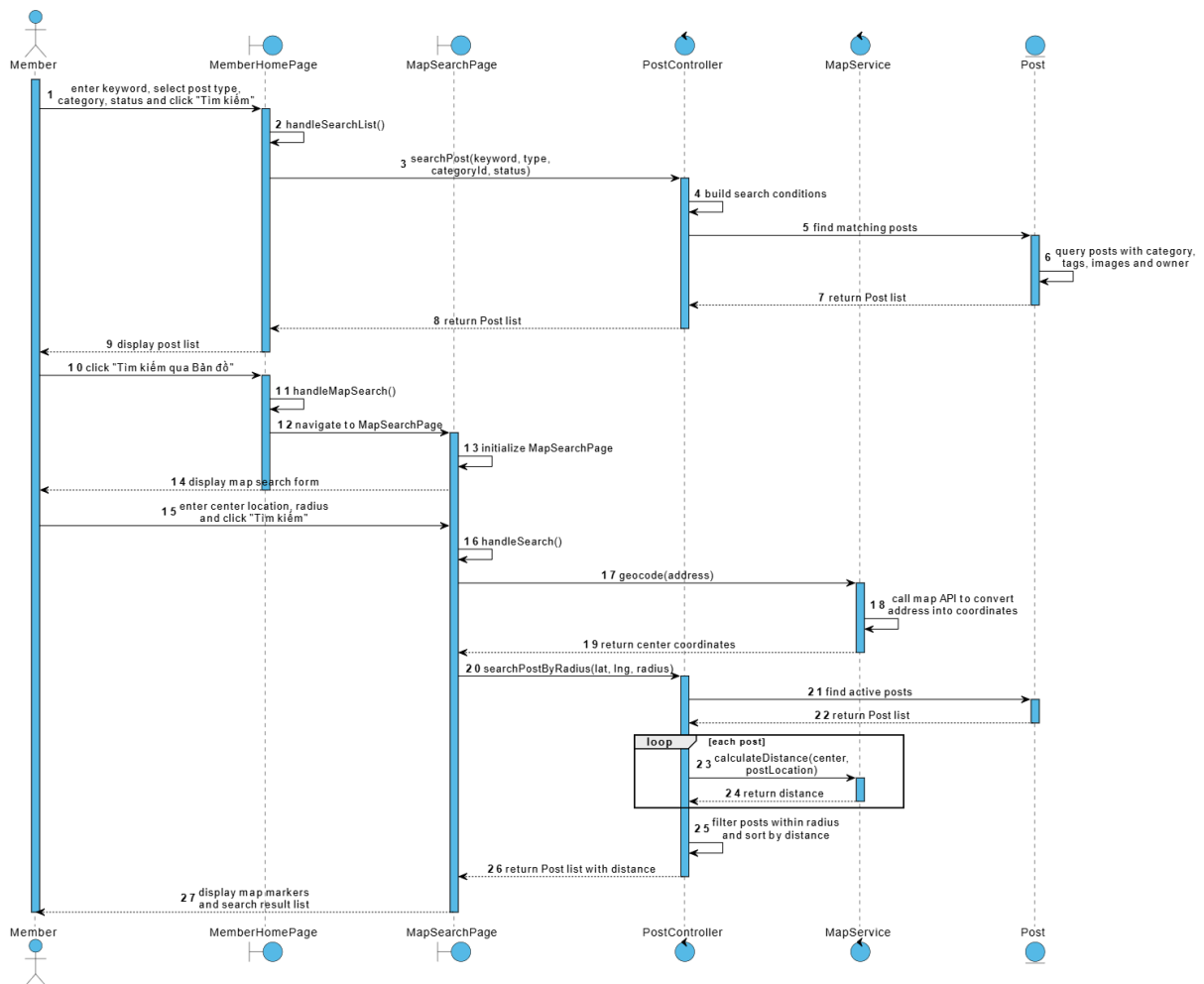
39. Phương thức `handleUploadImages()` của lớp `CreatePostStep4Panel` được gọi.
40. `CreatePostStep4Panel` gọi phương thức `analyzeImage()` của lớp `AIVisionController`.
41. `AIVisionController` phân tích ảnh và trích xuất các thẻ gợi ý.
42. `AIVisionController` gọi lớp `Tag` để đóng gói danh sách thẻ.
43. Lớp `Tag` đóng gói từng đối tượng `Tag`.
44. Lớp `Tag` trả danh sách `Tag` về cho `AIVisionController`.
45. `AIVisionController` trả danh sách `Tag` gợi ý về cho `CreatePostStep4Panel`.
46. `CreatePostStep4Panel` hiển thị ảnh đã tải và danh sách thẻ gợi ý cho thành viên.
47. Thành viên kiểm tra thẻ, xóa thẻ sai nếu có và click nút "Đăng bài".
48. Phương thức `handleSubmit()` của lớp `CreatePostStep4Panel` được gọi.
49. `CreatePostStep4Panel` gọi lớp `Post` để thiết lập trạng thái ban đầu là "Chờ duyệt".
50. Lớp `Post` cập nhật trạng thái cho đối tượng `Post`.
51. Lớp `Post` trả kết quả về cho `CreatePostStep4Panel`.
52. `CreatePostStep4Panel` gọi phương thức `createPost()` của lớp `PostController`.
53. `PostController` thêm bài đăng mới vào hệ thống.
54. `PostController` trả kết quả và id bài đăng về cho `CreatePostStep4Panel`.
55. `CreatePostStep4Panel` gọi phương thức `uploadImages()` của lớp `ImageController`.
56. `ImageController` gọi lớp `Image` để đóng gói danh sách ảnh.
57. Lớp `Image` đóng gói từng đối tượng `Image`.
58. Lớp `Image` trả danh sách `Image` về cho `ImageController`.
59. `ImageController` lưu danh sách ảnh và trả kết quả về cho `CreatePostStep4Panel`.
60. `CreatePostStep4Panel` gọi phương thức `getOrCreateTags()` của lớp `TagController`.
61. `TagController` lấy thẻ đã tồn tại hoặc tạo thẻ mới nếu chưa có.
62. `TagController` trả danh sách `Tag` hợp lệ về cho `CreatePostStep4Panel`.
63. `CreatePostStep4Panel` gọi phương thức `addPostTags()` của lớp `TagController`.
64. `TagController` gọi lớp `PostTag` để đóng gói quan hệ giữa bài đăng và thẻ.
65. Lớp `PostTag` đóng gói danh sách `PostTag`.
66. Lớp `PostTag` trả kết quả về cho `TagController`.
67. `TagController` lưu danh sách `PostTag` và trả kết quả về cho `CreatePostStep4Panel`.

- 68. CreatePostStep4Panel hiển thị thông báo "Bài viết đã được gửi và đang chờ duyệt".
- 69. Thành viên click nút OK của thông báo.
- 70. CreatePostStep4Panel gọi lại lớp MemberHomePage.
- 71. Giao diện MemberHomePage được hiển thị lại cho thành viên.



3.7.2. Chức năng tìm kiếm đồ vật

1. Thành viên đang ở MemberHomePage, nhập từ khóa, chọn loại bài đăng, danh mục, trạng thái và click “Tìm kiếm”.
2. Phương thức handleSearchList() của MemberHomePage được gọi.
3. MemberHomePage gọi searchPost() của PostController.
4. PostController tạo điều kiện tìm kiếm từ từ khóa và các bộ lọc.
5. PostController gọi entity Post để tìm các bài đăng phù hợp.
6. Entity Post truy vấn dữ liệu bài đăng cùng danh mục, tags, ảnh và người đăng.
7. Entity Post trả danh sách Post về cho PostController.
8. PostController trả danh sách Post về cho MemberHomePage.
9. MemberHomePage hiển thị danh sách bài đăng cho thành viên.
10. Thành viên click nút “Tìm kiếm qua Bản đồ”.
11. Phương thức handleMapSearch() của MemberHomePage được gọi.
12. MemberHomePage chuyển đến MapSearchPage.
13. Giao diện MapSearchPage được khởi tạo.
14. MapSearchPage hiển thị biểu mẫu tìm kiếm qua bản đồ.
15. Thành viên nhập vị trí trung tâm, chọn bán kính và click “Tìm kiếm”.
16. Phương thức handleSearch() của MapSearchPage được gọi.
17. MapSearchPage gọi geocode(address) của MapService.
18. MapService gọi API bản đồ để chuyển địa chỉ thành tọa độ.
19. MapService trả tọa độ trung tâm về cho MapSearchPage.
20. MapSearchPage gọi searchPostByRadius(lat, lng, radius) của PostController.
21. PostController gọi entity Post để lấy danh sách bài đăng đang hoạt động.
22. Entity Post truy vấn dữ liệu và trả danh sách Post về cho PostController.
23. Với mỗi Post, PostController gọi calculateDistance() của MapService.
24. MapService tính và trả khoảng cách về cho PostController.
25. PostController loại bỏ các bài nằm ngoài bán kính và sắp xếp kết quả từ gần đến xa.
26. PostController trả danh sách Post kèm khoảng cách về cho MapSearchPage.
27. MapSearchPage hiển thị các bài đăng dưới dạng ghim và danh sách kết quả.

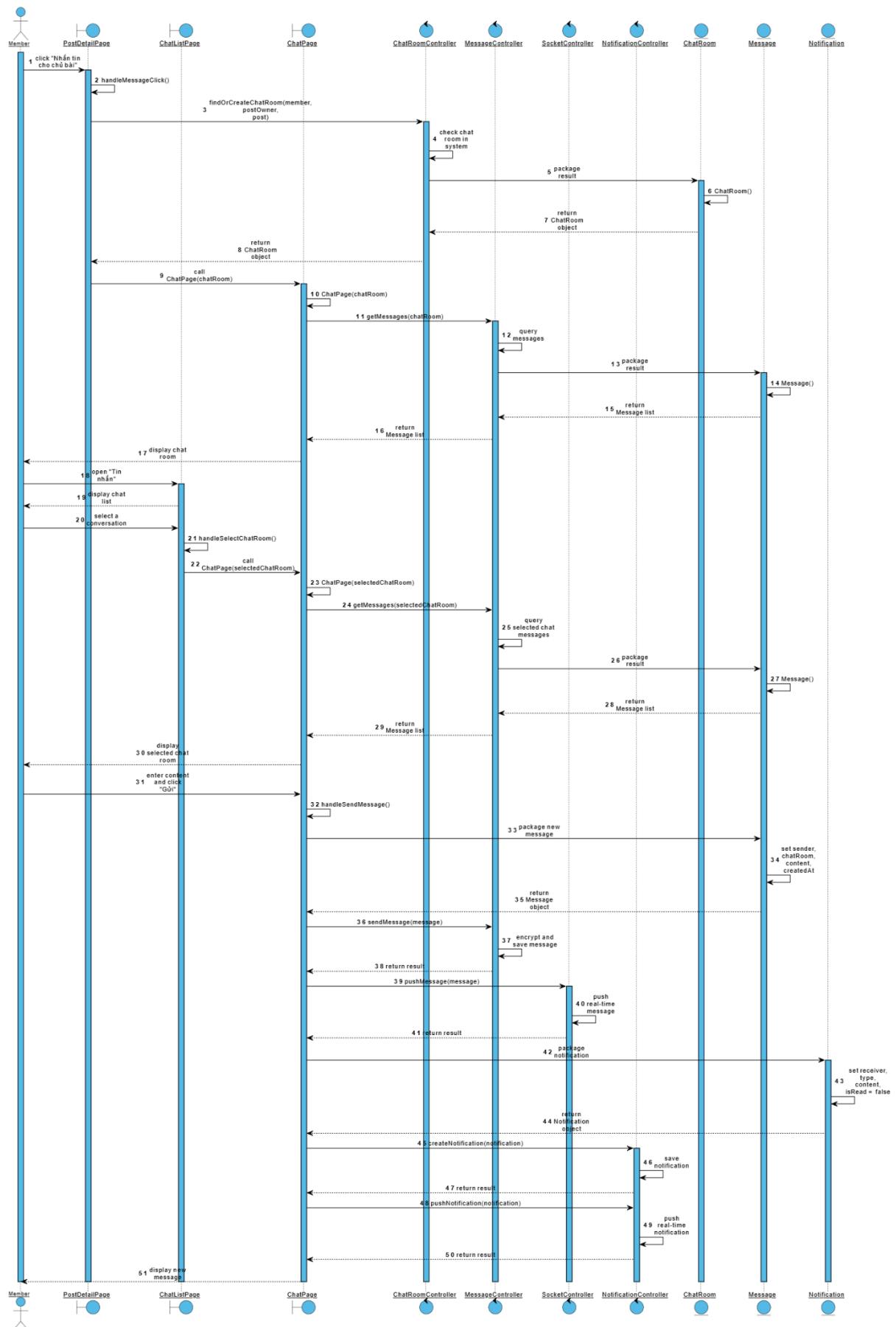


3.7.3. Chức năng trao đổi tin nhắn

1. Thành viên đang ở giao diện PostDetailPage và click nút "Nhắn tin cho chủ bài".
2. Phương thức handleMessageClick() của lớp PostDetailPage được gọi.
3. Phương thức handleMessageClick() gọi phương thức findOrCreateChatRoom() của lớp ChatRoomController.
4. Phương thức findOrCreateChatRoom() kiểm tra phòng chat giữa thành viên hiện tại và chủ bài đăng trong hệ thống.
5. ChatRoomController gọi lớp ChatRoom để đóng gói kết quả.
6. Lớp ChatRoom đóng gói thông tin phòng chat.
7. Lớp ChatRoom trả đối tượng ChatRoom về cho ChatRoomController.
8. ChatRoomController trả đối tượng ChatRoom về cho PostDetailPage.
9. PostDetailPage gọi lớp ChatPage và truyền thông tin phòng chat vừa nhận được.
10. Constructor ChatPage() được gọi.
11. ChatPage gọi phương thức getMessages() của lớp MessageController để lấy danh sách tin nhắn cũ.

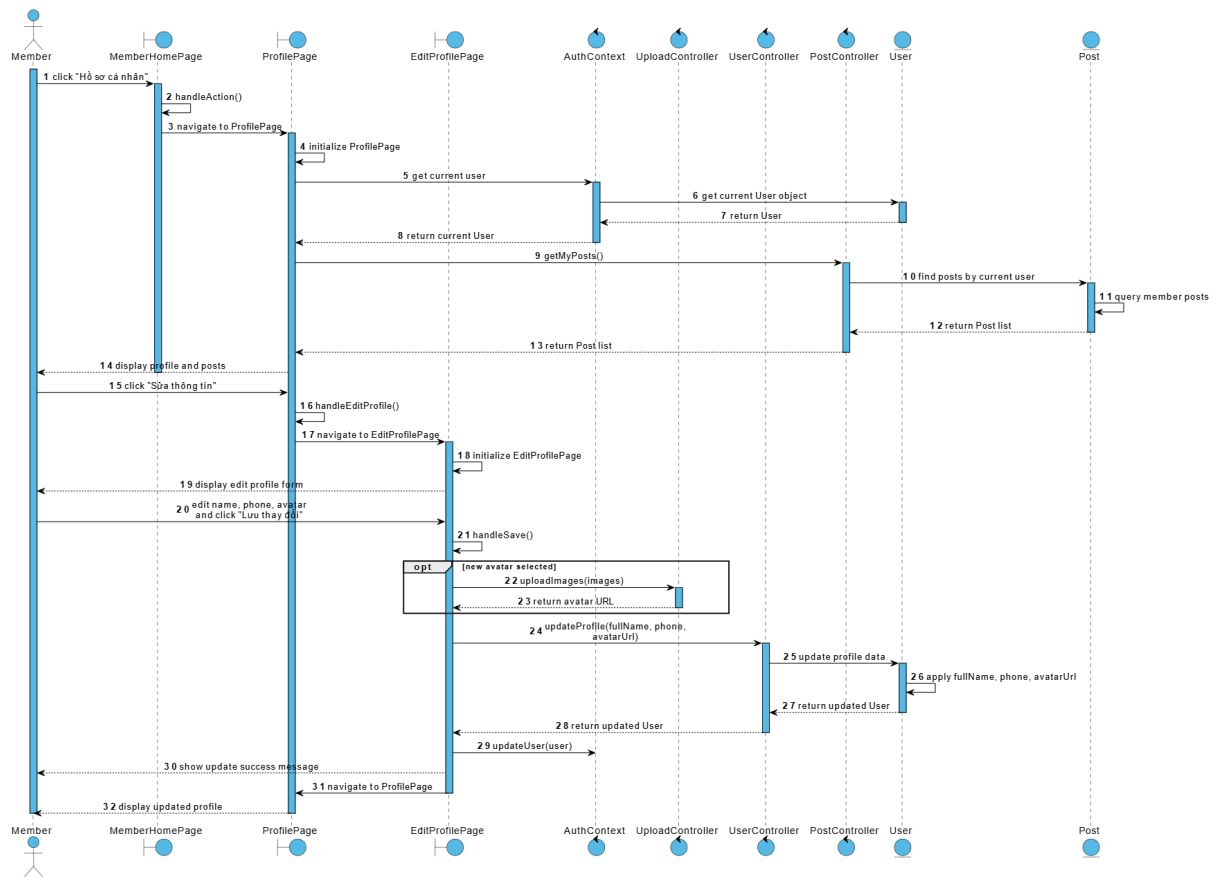
12. MessageController truy vấn danh sách tin nhắn trong hệ thống.
13. MessageController gọi lớp Message để đóng gói kết quả.
14. Lớp Message đóng gói từng đối tượng Message.
15. Lớp Message trả danh sách Message về cho MessageController.
16. MessageController trả danh sách Message về cho ChatPage.
17. Giao diện ChatPage hiển thị lịch sử tin nhắn, thông tin người đang trao đổi và bài đăng liên quan cho thành viên.
18. Ngoài ra, thành viên có thể mở danh sách trò chuyện từ mục "Tin nhắn".
19. Giao diện ChatListPage được hiển thị cho thành viên.
20. Thành viên chọn một cuộc trò chuyện trong danh sách.
21. Phương thức handleSelectChatRoom() của lớp ChatListPage được gọi.
22. ChatListPage gọi lớp ChatPage và truyền phòng chat được chọn.
23. Constructor ChatPage() được gọi.
24. ChatPage gọi phương thức getMessages() của lớp MessageController.
25. MessageController truy vấn danh sách tin nhắn của phòng chat được chọn.
26. MessageController gọi lớp Message để đóng gói kết quả.
27. Lớp Message đóng gói danh sách tin nhắn.
28. Lớp Message trả danh sách Message về cho MessageController.
29. MessageController trả danh sách Message về cho ChatPage.
30. Giao diện ChatPage hiển thị phòng chat tương ứng cho thành viên.
31. Thành viên nhập nội dung tin nhắn và click nút "Gửi".
32. Phương thức handleSendMessage() của lớp ChatPage được gọi.
33. ChatPage gọi lớp Message để đóng gói tin nhắn mới.
34. Lớp Message thiết lập người gửi, phòng chat, nội dung và thời điểm gửi.
35. Lớp Message trả đối tượng Message về cho ChatPage.
36. ChatPage gọi phương thức sendMessage() của lớp MessageController.
37. MessageController mã hóa nội dung và lưu tin nhắn vào hệ thống.
38. MessageController trả kết quả lưu tin nhắn về cho ChatPage.
39. ChatPage gọi phương thức pushMessage() của lớp SocketController.
40. SocketController đẩy tin nhắn thời gian thực đến người nhận đang online.
41. SocketController trả kết quả về cho ChatPage.
42. ChatPage gọi lớp Notification để đóng gói thông báo.
43. Lớp Notification thiết lập người nhận, loại thông báo, nội dung và trạng thái chưa đọc.
44. Lớp Notification trả đối tượng Notification về cho ChatPage.
45. ChatPage gọi phương thức createNotification() của lớp NotificationController.
46. NotificationController lưu thông báo mới vào hệ thống.

47. NotificationController trả kết quả về cho ChatPage.
48. ChatPage gọi phương thức pushNotification() của lớp NotificationController.
49. NotificationController đẩy thông báo thời gian thực đến người nhận đang online.
50. NotificationController trả kết quả về cho ChatPage.
51. ChatPage hiển thị tin nhắn mới trên khung chat của thành viên.



3.7.4. Chức năng quản lý hồ sơ cá nhân

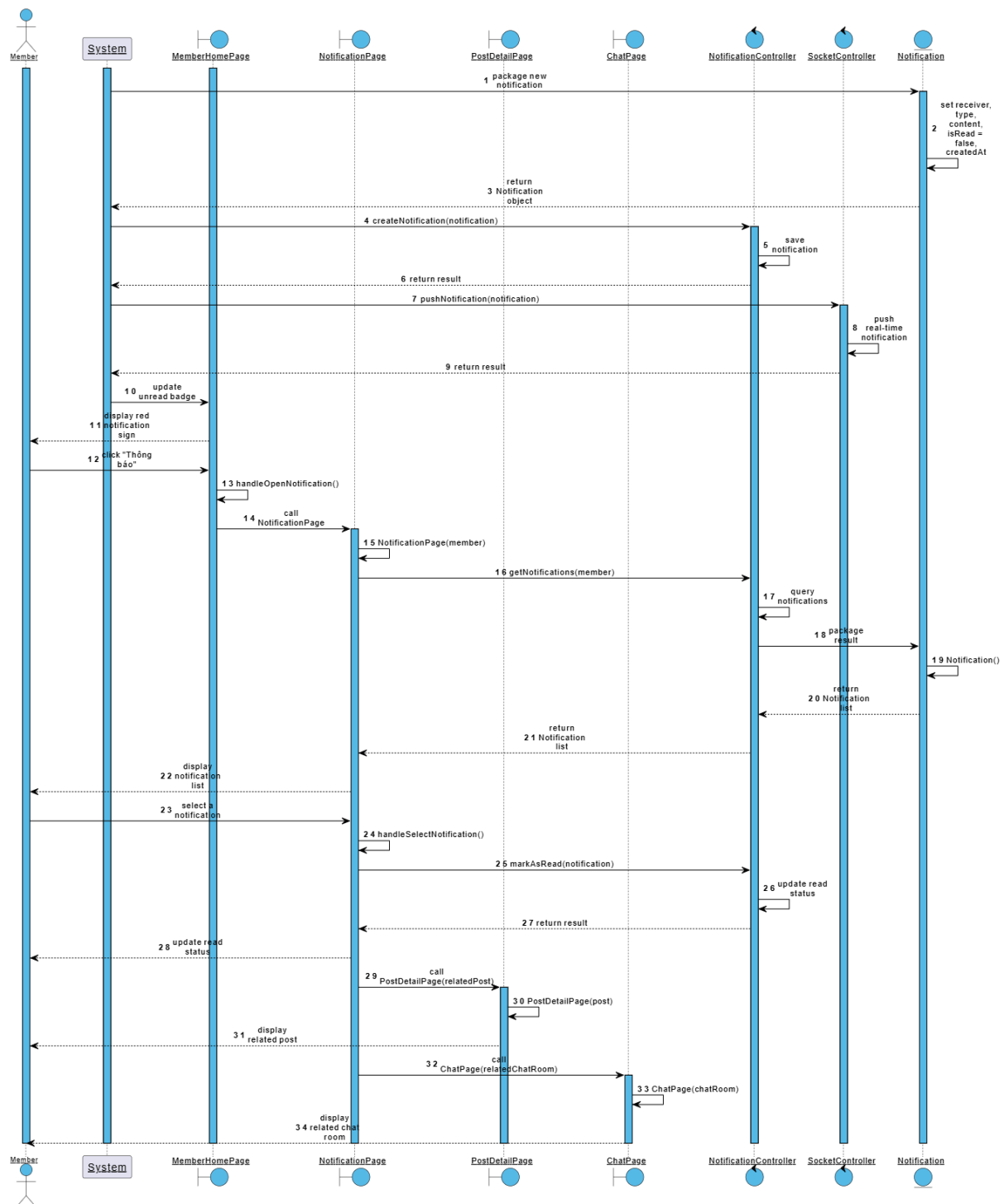
1. Thành viên đang ở giao diện MemberHomePage và click “Hồ sơ cá nhân”.
2. Phương thức `handleAction()` của MemberHomePage được gọi.
3. MemberHomePage chuyển đến ProfilePage.
4. Giao diện ProfilePage được khởi tạo.
5. ProfilePage lấy thông tin người dùng hiện tại từ AuthContext.
6. AuthContext lấy đối tượng User hiện tại.
7. User trả thông tin người dùng hiện tại về cho AuthContext.
8. AuthContext trả đối tượng User hiện tại về cho ProfilePage.
9. ProfilePage gọi phương thức `getMyPosts()` của PostController.
10. PostController gọi lớp Post để lấy danh sách bài đăng của thành viên.
11. Post truy vấn danh sách bài đăng của thành viên trong hệ thống.
12. Post trả danh sách Post về cho PostController.
13. PostController trả danh sách Post về cho ProfilePage.
14. ProfilePage hiển thị hồ sơ cá nhân và danh sách bài đăng cho thành viên.
15. Thành viên click nút “Sửa thông tin”.
16. Phương thức `handleAction()` của ProfilePage được gọi.
17. ProfilePage chuyển đến EditProfilePage.
18. Giao diện EditProfilePage được khởi tạo.
19. EditProfilePage hiển thị biểu mẫu sửa hồ sơ.
20. Thành viên thay đổi tên, số điện thoại, ảnh đại diện và click “Lưu thay đổi”.
21. Phương thức `handleAction()` của EditProfilePage được gọi.
22. Nếu thành viên chọn ảnh mới, EditProfilePage gọi `uploadImages()` của UploadController.
23. UploadController tải ảnh và trả URL ảnh đại diện về cho EditProfilePage.
24. EditProfilePage gọi `updateProfile(fullName, phone, avatarUrl)` của UserController.
25. UserController gọi lớp User để cập nhật thông tin hồ sơ.
26. User cập nhật tên, số điện thoại và URL ảnh đại diện.
27. User trả đối tượng User đã cập nhật về cho UserController.
28. UserController trả đối tượng User đã cập nhật về cho EditProfilePage.
29. EditProfilePage cập nhật thông tin người dùng trong AuthContext.
30. EditProfilePage hiển thị thông báo cập nhật thành công.
31. EditProfilePage tự động chuyển về ProfilePage.
32. ProfilePage hiển thị hồ sơ đã được cập nhật.



3.7.5. Chức năng quản lý thông báo

1. Thành viên đang ở giao diện MemberHomePage.
2. Khi có sự kiện phát sinh trong hệ thống, hệ thống gọi lớp Notification để đóng gói thông báo mới.
3. Lớp Notification thiết lập người nhận, loại thông báo, nội dung, trạng thái chưa đọc và thời điểm tạo.
4. Lớp Notification trả đối tượng Notification về cho hệ thống.
5. Hệ thống gọi phương thức createNotification() của lớp NotificationController.
6. NotificationController lưu thông báo mới vào hệ thống.
7. NotificationController trả kết quả về cho hệ thống.
8. Hệ thống gọi phương thức pushNotification() của lớp SocketController.
9. SocketController đẩy thông báo thời gian thực đến thành viên đang online.
10. SocketController trả kết quả về cho hệ thống.
11. MemberHomePage hiển thị dấu hiệu đỏ trên biểu tượng thông báo.
12. Thành viên click biểu tượng thông báo hoặc mục "Thông báo".
13. Phương thức handleOpenNotification() của lớp MemberHomePage được gọi.
14. MemberHomePage gọi lớp NotificationPage.
15. Constructor NotificationPage() được gọi.

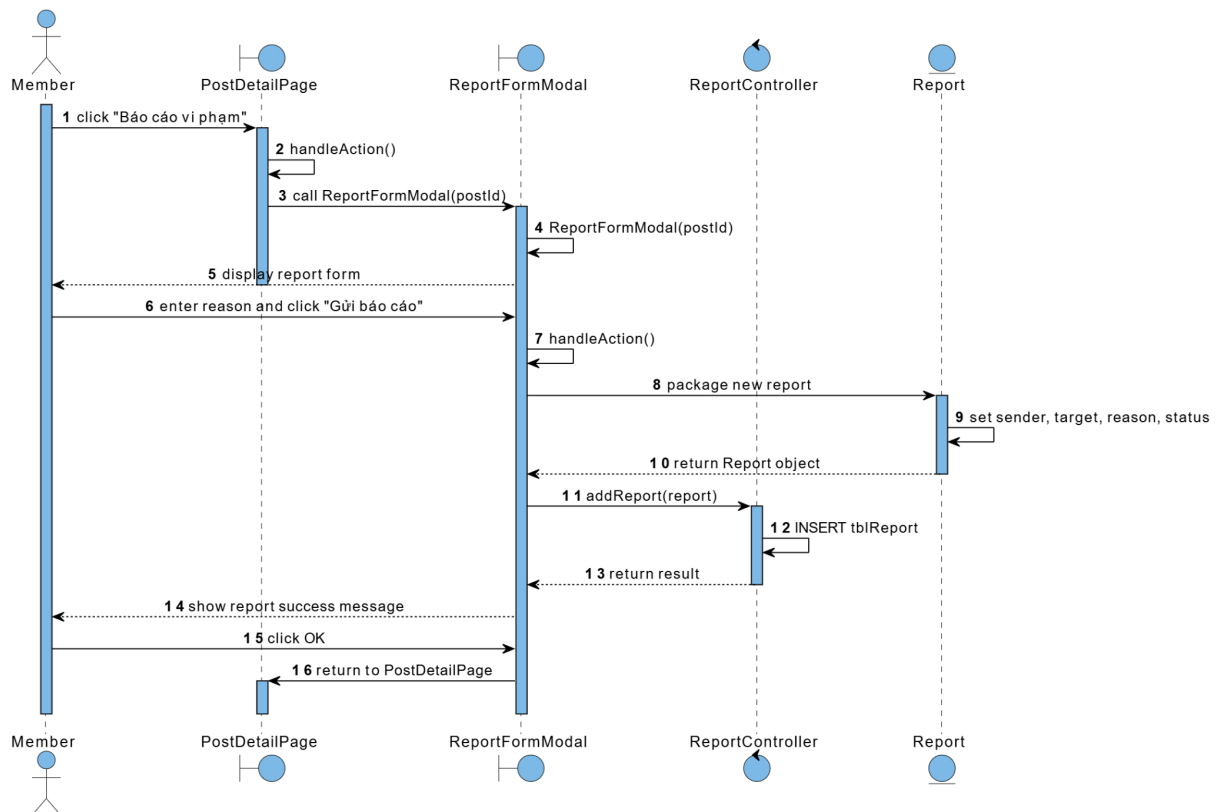
16. NotificationPage gọi phương thức getNotifications() của lớp NotificationController.
17. NotificationController truy vấn danh sách thông báo của thành viên trong hệ thống.
18. NotificationController gọi lớp Notification để đóng gói kết quả.
19. Lớp Notification đóng gói từng đối tượng Notification.
20. Lớp Notification trả danh sách Notification về cho NotificationController.
21. NotificationController trả danh sách Notification về cho NotificationPage.
22. Giao diện NotificationPage hiển thị danh sách thông báo cho thành viên.
23. Thành viên click chọn một thông báo trong danh sách.
24. Phương thức handleSelectNotification() của lớp NotificationPage được gọi.
25. NotificationPage gọi phương thức markAsRead() của lớp NotificationController.
26. NotificationController cập nhật trạng thái thông báo thành đã đọc trong hệ thống.
27. NotificationController trả kết quả về cho NotificationPage.
28. NotificationPage cập nhật trạng thái đã đọc trên giao diện.
29. Nếu thông báo liên quan đến bài đăng, NotificationPage gọi lớp PostDetailPage.
30. Constructor PostDetailPage() được gọi.
31. Giao diện PostDetailPage hiển thị bài đăng liên quan cho thành viên.
32. Nếu thông báo liên quan đến tin nhắn, NotificationPage gọi lớp ChatPage.
33. Constructor ChatPage() được gọi.
34. Giao diện ChatPage hiển thị phòng chat liên quan cho thành viên.



3.7.6. Chức năng báo cáo bài viết/người dùng

1. Thành viên đang ở giao diện PostDetailPage và click "Báo cáo vi phạm".
2. Phương thức handleAction() của lớp PostDetailPage được gọi.

3. PostDetailPage gọi lớp ReportFormModal với tham số id bài đăng (postId).
4. Constructor ReportFormModal() được gọi.
5. Giao diện ReportFormModal hiển thị biểu mẫu báo cáo cho thành viên.
6. Thành viên nhập lý do báo cáo và click "Gửi báo cáo".
7. Phương thức handleAction() của lớp ReportFormModal được gọi.
8. ReportFormModal gọi lớp Report để đóng gói báo cáo mới.
9. Lớp Report thiết lập thông tin người gửi, đối tượng bị báo cáo, lý do và trạng thái chờ xử lý.
10. Lớp Report trả đối tượng Report về cho ReportFormModal.
11. ReportFormModal gọi phương thức addReport() của lớp ReportController.
12. ReportController lưu bản ghi báo cáo mới vào hệ thống.
13. ReportController trả kết quả về cho ReportFormModal.
14. ReportFormModal hiển thị thông báo gửi báo cáo thành công cho thành viên.
15. Thành viên click nút OK.
16. ReportFormModal quay lại giao diện PostDetailPage.

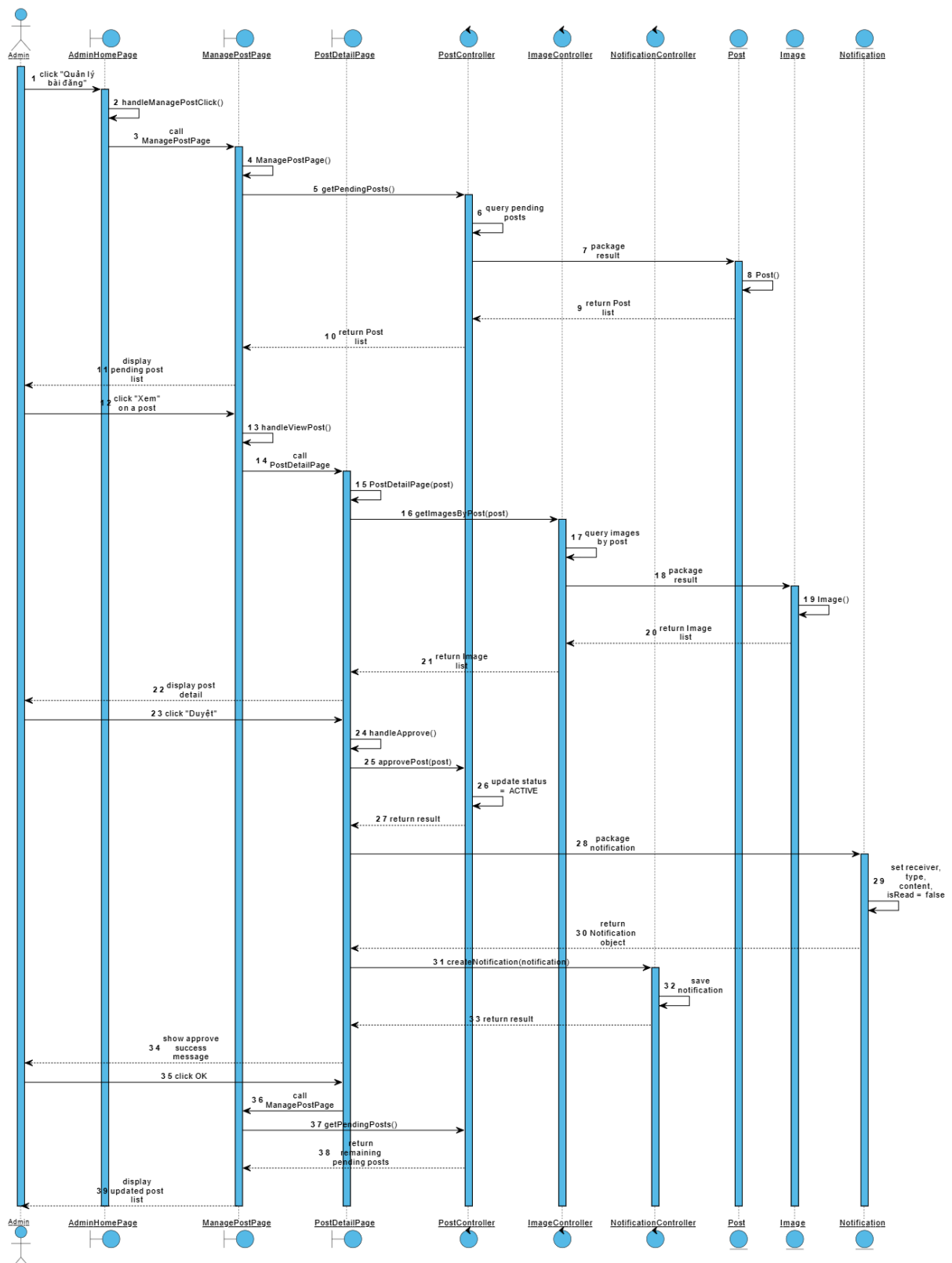


3.7.7. Chức năng quản lý bài đăng

1. Quản trị viên đang ở giao diện AdminHomePage và click chức năng "Quản lý bài đăng".
2. Phương thức handleManagePostClick() của lớp AdminHomePage được gọi.
3. Phương thức handleManagePostClick() gọi lớp ManagePostPage.
4. Constructor ManagePostPage() được gọi.

5. ManagePostPage gọi phương thức getPendingPosts() của lớp PostController để lấy danh sách bài đăng đang chờ duyệt.
6. PostController truy vấn danh sách bài đăng có trạng thái "Chờ duyệt" trong hệ thống.
7. PostController gọi lớp Post để đóng gói kết quả.
8. Lớp Post đóng gói từng đối tượng Post.
9. Lớp Post trả danh sách Post về cho PostController.
10. PostController trả danh sách Post về cho ManagePostPage.
11. Giao diện ManagePostPage hiển thị danh sách bài đăng chờ duyệt cho quản trị viên.
12. Quản trị viên click nút "Xem" tại một bài đăng cần kiểm duyệt.
13. Phương thức handleViewPost() của lớp ManagePostPage được gọi.
14. ManagePostPage gọi lớp PostDetailPage.
15. Constructor PostDetailPage() được gọi.
16. PostDetailPage gọi phương thức getImagesByPost() của lớp ImageController để lấy hình ảnh minh chứng của bài đăng.
17. ImageController truy vấn danh sách hình ảnh theo bài đăng trong hệ thống.
18. ImageController gọi lớp Image để đóng gói kết quả.
19. Lớp Image đóng gói từng đối tượng Image.
20. Lớp Image trả danh sách Image về cho ImageController.
21. ImageController trả danh sách Image về cho PostDetailPage.
22. Giao diện PostDetailPage hiển thị chi tiết bài đăng cho quản trị viên.
23. Quản trị viên kiểm tra nội dung và click nút "Duyệt".
24. Phương thức handleApprove() của lớp PostDetailPage được gọi.
25. PostDetailPage gọi phương thức approvePost() của lớp PostController.
26. PostController cập nhật trạng thái bài đăng từ "Chờ duyệt" sang "Hoạt động" trong hệ thống.
27. PostController trả kết quả về cho PostDetailPage.
28. PostDetailPage gọi lớp Notification để đóng gói thông báo duyệt bài.
29. Lớp Notification thiết lập người nhận, loại thông báo, nội dung và trạng thái chưa đọc.
30. Lớp Notification trả đối tượng Notification về cho PostDetailPage.
31. PostDetailPage gọi phương thức createNotification() của lớp NotificationController.
32. NotificationController lưu thông báo mới vào hệ thống.
33. NotificationController trả kết quả về cho PostDetailPage.
34. PostDetailPage hiển thị thông báo duyệt bài thành công cho quản trị viên.
35. Quản trị viên click nút OK của thông báo.
36. PostDetailPage gọi lại lớp ManagePostPage.

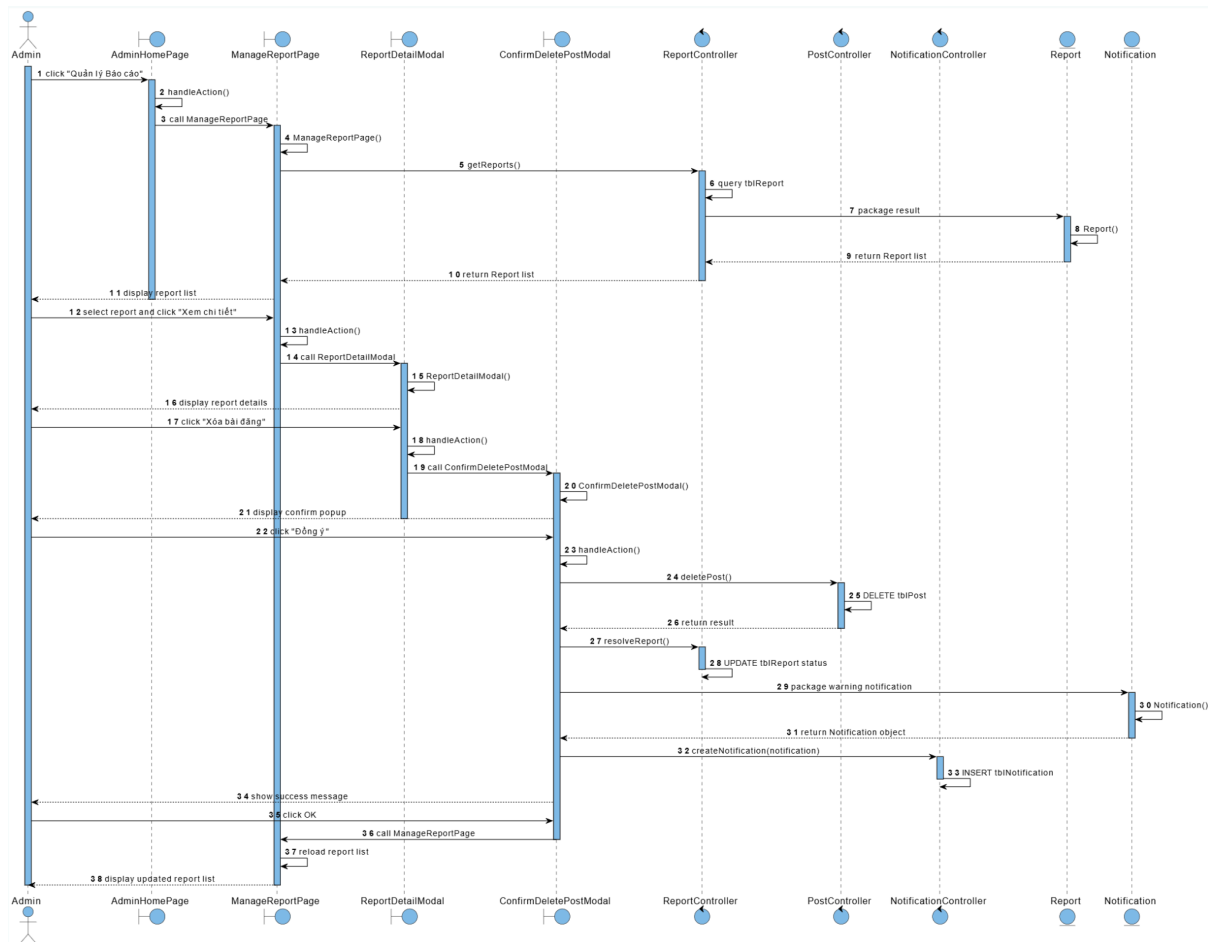
37. ManagePostPage gọi phương thức getPendingPosts() của lớp PostController để tải lại danh sách bài đăng chờ duyệt.
38. PostController trả danh sách bài đăng chờ duyệt còn lại về cho ManagePostPage.
39. Giao diện ManagePostPage hiển thị danh sách bài đăng đã cập nhật cho quản trị viên.



3.7.8. Chức năng quản lý báo cáo vi phạm

1. Quản trị viên đang ở giao diện AdminHomePage và click chức năng "Quản lý Báo cáo".
2. Phương thức `handleAction()` của lớp AdminHomePage được gọi.
3. AdminHomePage gọi lớp ManageReportPage.
4. Constructor `ManageReportPage()` được gọi.
5. ManageReportPage gọi phương thức `getReports()` của lớp ReportController để lấy danh sách báo cáo.
6. ReportController truy vấn danh sách báo cáo trong hệ thống.
7. ReportController gọi lớp Report để đóng gói kết quả.
8. Lớp Report đóng gói từng đối tượng Report.
9. Lớp Report trả danh sách Report về cho ReportController.
10. ReportController trả danh sách Report về cho ManageReportPage.
11. Giao diện ManageReportPage hiển thị danh sách báo cáo cho quản trị viên.
12. Quản trị viên chọn một báo cáo và click nút "Xem chi tiết".
13. Phương thức `handleAction()` của lớp ManageReportPage được gọi.
14. ManageReportPage gọi lớp ReportDetailModal.
15. Constructor `ReportDetailModal()` được gọi.
16. Giao diện ReportDetailModal hiển thị chi tiết báo cáo cho quản trị viên.
17. Quản trị viên click nút "Xóa bài đăng".
18. Phương thức `handleAction()` của lớp ReportDetailModal được gọi.
19. ReportDetailModal gọi lớp ConfirmDeletePostModal.
20. Constructor `ConfirmDeletePostModal()` được gọi.
21. Giao diện ConfirmDeletePostModal hiển thị cửa sổ xác nhận cho quản trị viên.
22. Quản trị viên click "Đồng ý".
23. Phương thức `handleAction()` của lớp ConfirmDeletePostModal được gọi.
24. ConfirmDeletePostModal gọi phương thức `deletePost()` của lớp PostController.
25. PostController xóa bản ghi bài đăng trong hệ thống.
26. PostController trả kết quả về cho ConfirmDeletePostModal.
27. ConfirmDeletePostModal gọi phương thức `resolveReport()` của lớp ReportController.
28. ReportController cập nhật trạng thái báo cáo đã giải quyết trong hệ thống.
29. ConfirmDeletePostModal gọi lớp Notification để đóng gói thông báo cảnh báo.
30. Lớp Notification đóng gói thông báo gửi đến chủ bài đăng.
31. Lớp Notification trả đối tượng Notification về cho ConfirmDeletePostModal.
32. ConfirmDeletePostModal gọi phương thức `createNotification()` của lớp NotificationController.
33. NotificationController lưu thông báo vào hệ thống.
34. ConfirmDeletePostModal hiển thị thông báo xử lý thành công cho quản trị viên.

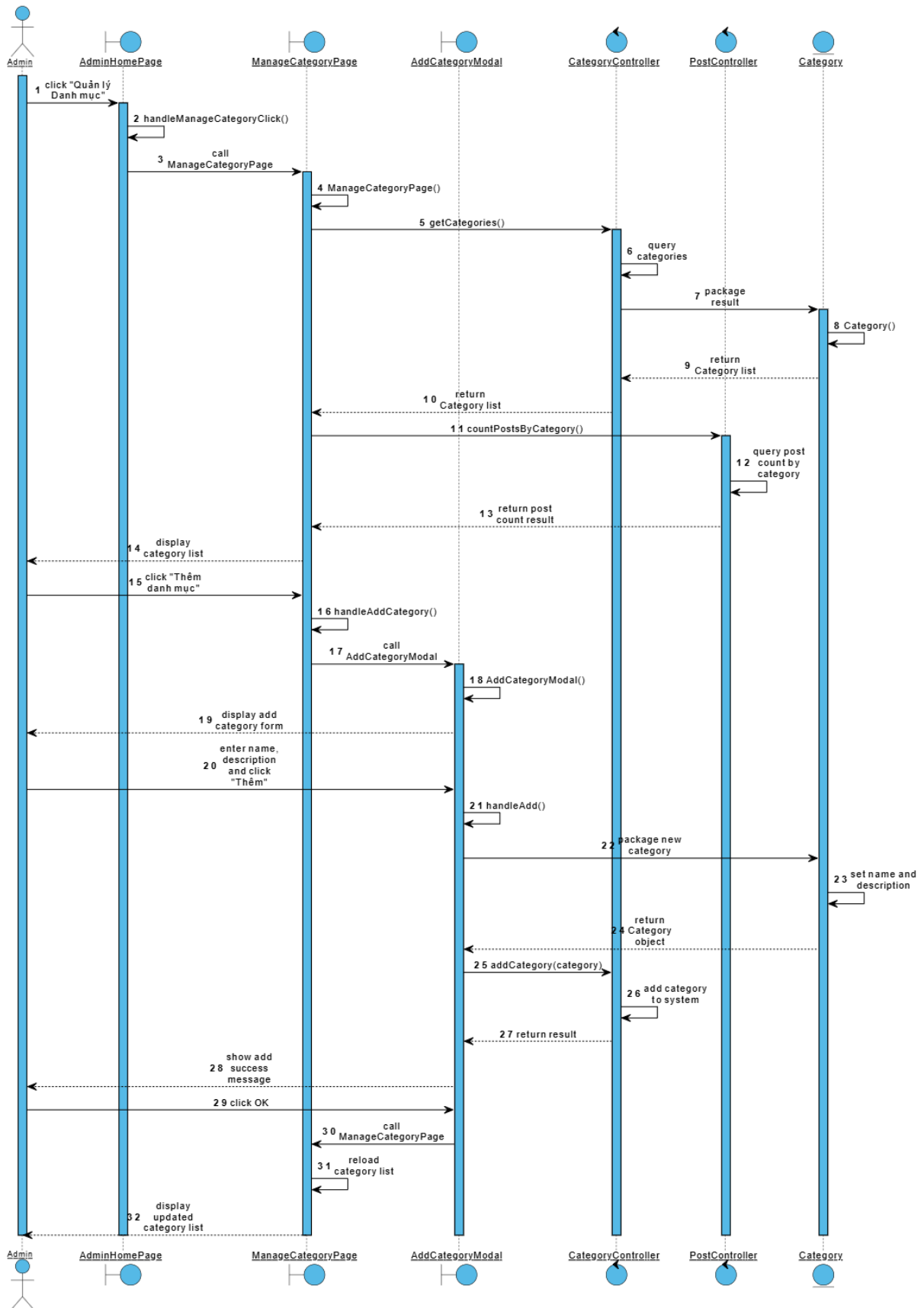
35. Quản trị viên click nút OK.
36. ConfirmDeletePostModal gọi lại lớp ManageReportPage.
37. ManageReportPage tải lại danh sách báo cáo chờ xử lý.
38. Giao diện ManageReportPage hiển thị danh sách báo cáo đã cập nhật cho quản trị viên.



3.7.9. Chức năng quản lý danh mục (Thêm danh mục)

1. Quản trị viên đang ở giao diện AdminHomePage và click chức năng "Quản lý Danh mục".
2. Phương thức handleManageCategoryClick() của lớp AdminHomePage được gọi.
3. Phương thức handleManageCategoryClick() gọi lớp ManageCategoryPage.
4. Constructor ManageCategoryPage() được gọi.
5. ManageCategoryPage gọi phương thức getCategories() của lớp CategoryController để lấy danh sách danh mục.
6. CategoryController truy vấn danh sách danh mục trong hệ thống.
7. CategoryController gọi lớp Category để đóng gói kết quả.
8. Lớp Category đóng gói từng đối tượng Category.

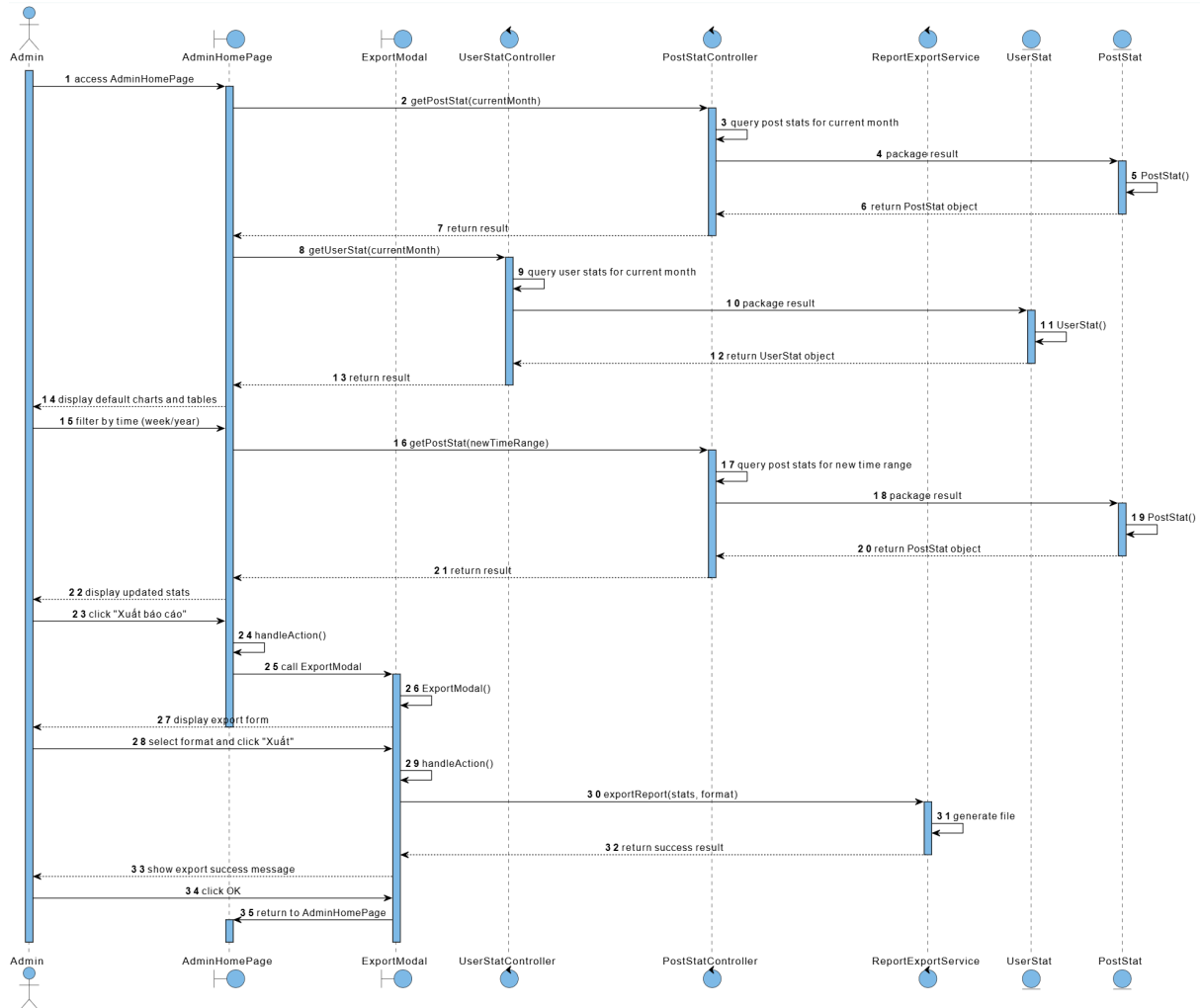
9. Lớp Category trả danh sách Category về cho CategoryController.
10. CategoryController trả danh sách Category về cho ManageCategoryPage.
11. ManageCategoryPage gọi phương thức countPostsByCategory() của lớp PostController để lấy số bài đăng thuộc từng danh mục.
12. PostController truy vấn số lượng bài đăng theo từng danh mục trong hệ thống.
13. PostController trả kết quả thống kê về cho ManageCategoryPage.
14. Giao diện ManageCategoryPage hiển thị danh sách danh mục và số bài đăng thuộc từng danh mục cho quản trị viên.
15. Quản trị viên click nút "Thêm danh mục".
16. Phương thức handleAddCategory() của lớp ManageCategoryPage được gọi.
17. ManageCategoryPage gọi lớp AddCategoryModal.
18. Constructor AddCategoryModal() được gọi.
19. Giao diện AddCategoryModal được hiển thị cho quản trị viên.
20. Quản trị viên nhập tên danh mục, mô tả và click nút "Thêm".
21. Phương thức handleAdd() của lớp AddCategoryModal được gọi.
22. AddCategoryModal gọi lớp Category để đóng gói thông tin danh mục mới.
23. Lớp Category thiết lập tên danh mục và mô tả.
24. Lớp Category trả đối tượng Category về cho AddCategoryModal.
25. AddCategoryModal gọi phương thức addCategory() của lớp CategoryController.
26. CategoryController thêm danh mục mới vào hệ thống.
27. CategoryController trả kết quả về cho AddCategoryModal.
28. AddCategoryModal hiển thị thông báo thêm danh mục thành công.
29. Quản trị viên click nút OK của thông báo.
30. AddCategoryModal gọi lại lớp ManageCategoryPage.
31. ManageCategoryPage tải lại danh sách danh mục.
32. Giao diện ManageCategoryPage hiển thị danh sách danh mục đã cập nhật.



3.7.10. Chức năng xem báo cáo thống kê

1. Quản trị viên truy cập vào giao diện AdminHomePage.
2. AdminHomePage gọi phương thức getPostStat() của lớp PostStatController với tham số là tháng hiện tại.
3. PostStatController thực hiện truy vấn số liệu thống kê bài đăng của tháng hiện tại.
4. PostStatController gọi lớp PostStat để đóng gói kết quả.
5. Lớp PostStat đóng gói đối tượng PostStat.
6. Lớp PostStat trả đối tượng PostStat về cho PostStatController.
7. PostStatController trả kết quả về cho AdminHomePage.
8. AdminHomePage gọi phương thức getUserStat() của lớp UserStatController với tham số là tháng hiện tại.
9. UserStatController thực hiện truy vấn số liệu thống kê người dùng của tháng hiện tại.
10. UserStatController gọi lớp UserStat để đóng gói kết quả.
11. Lớp UserStat đóng gói đối tượng UserStat.
12. Lớp UserStat trả đối tượng UserStat về cho UserStatController.
13. UserStatController trả kết quả về cho AdminHomePage.
14. Giao diện AdminHomePage hiển thị các biểu đồ và bảng số liệu mặc định cho quản trị viên.
15. Quản trị viên thao tác lọc dữ liệu theo thời gian (tuần/năm).
16. AdminHomePage gọi phương thức getPostStat() của lớp PostStatController với khoảng thời gian mới.
17. PostStatController thực hiện truy vấn lại số liệu thống kê bài đăng theo khoảng thời gian mới.
18. PostStatController gọi lớp PostStat để đóng gói kết quả.
19. Lớp PostStat đóng gói đối tượng PostStat.
20. Lớp PostStat trả đối tượng PostStat về cho PostStatController.
21. PostStatController trả kết quả về cho AdminHomePage.
22. Giao diện AdminHomePage hiển thị số liệu đã được cập nhật cho quản trị viên.
23. Quản trị viên click nút "Xuất báo cáo".
24. Phương thức handleAction() của lớp AdminHomePage được gọi.
25. AdminHomePage gọi lớp ExportModal.
26. Constructor ExportModal() được gọi.
27. Giao diện ExportModal hiển thị biểu mẫu xuất báo cáo cho quản trị viên.
28. Quản trị viên chọn định dạng file và click nút "Xuất".
29. Phương thức handleAction() của lớp ExportModal được gọi.
30. ExportModal gọi phương thức exportReport() của lớp ReportExportService.
31. ReportExportService tạo tệp tin chứa dữ liệu thống kê.
32. ReportExportService trả kết quả xuất file thành công về cho ExportModal.

33. ExportModal hiển thị thông báo xuất báo cáo thành công cho quản trị viên.
34. Quản trị viên click nút OK.
35. ExportModal quay lại giao diện AdminHomePage.



KẾT LUẬN

Sau thời gian nghiêm túc nghiên cứu và thực hiện đề tài dưới sự hướng dẫn tận tình của TS. Đỗ Thị Liên, nhóm đã hoàn thành quá trình phân tích, thiết kế và xây dựng hệ thống tìm kiếm đồ thất lạc BeaconFound. Đề tài hướng đến giải quyết bài toán thất lạc tài sản trong đời sống hằng ngày, đặc biệt tại các khu vực đông người, nơi người mất đồ và người nhặt được thường thiếu một kênh kết nối chính thống, an toàn và hiệu quả.

Trong quá trình thực hiện đề tài, nhóm đã đạt được một số kết quả chính như sau:

- Về mặt lý thuyết và phương pháp luận: Nhóm đã vận dụng các kiến thức về phân tích thiết kế hệ thống, mô hình hóa UML và thiết kế cơ sở dữ liệu để xây dựng tài liệu phân tích tương đối đầy đủ cho hệ thống. Các chức năng chính như đăng tải bài viết, tìm kiếm đồ vật, trao đổi tin nhắn, quản lý thông báo, quản lý bài đăng và quản lý danh mục đã được mô tả thông qua các kịch bản nghiệp vụ, sơ đồ lớp và sơ đồ trình tự.
- Về mặt công nghệ: Nhóm đã nghiên cứu và lựa chọn các công nghệ phù hợp với bài toán xây dựng nền tảng web, bao gồm ReactJS cho phía giao diện, Node.js và Express.js cho phía xử lý backend, Socket.io cho các chức năng thời gian thực, PostgreSQL kết hợp PostGIS cho lưu trữ và truy vấn dữ liệu vị trí, đồng thời tích hợp các dịch vụ như Cloudinary, Firebase Cloud Messaging và AI Vision API để hỗ trợ lưu trữ hình ảnh, gửi thông báo và gợi ý thẻ từ hình ảnh.
- Về mặt sản phẩm thực tiễn: Hệ thống đã định hình được một nền tảng hỗ trợ người dùng đăng tin báo mất hoặc nhặt được đồ vật, tìm kiếm thông tin theo danh sách và bản đồ, trao đổi qua tin nhắn nội bộ, nhận thông báo thời gian thực và hỗ trợ quản trị viên kiểm duyệt nội dung. Việc kết hợp bản đồ số, phân loại danh mục và thẻ gợi ý giúp quá trình tìm kiếm đồ thất lạc trở nên trực quan, có tổ chức và an toàn hơn so với các phương thức đăng tin rời rạc trên mạng xã hội.
- Mặc dù đã đáp ứng được các mục tiêu cơ bản đặt ra, hệ thống vẫn còn một số hạn chế nhất định. Các tính năng như đối sánh tự động giữa bài đăng mất đồ và nhặt được, tối ưu truy vấn khi dữ liệu lớn, kiểm duyệt nội dung thông minh và trải nghiệm trên thiết bị di động vẫn cần tiếp tục được hoàn thiện. Bên cạnh đó, việc đảm bảo an toàn thông tin cá nhân và phòng tránh hành vi lừa đảo cũng là vấn đề cần được quan tâm sâu hơn khi hệ thống được triển khai thực tế.

Trong tương lai, nhóm định hướng phát triển hệ thống theo các hướng sau:

- Tối ưu hóa chức năng tìm kiếm và truy vấn bản đồ, đặc biệt với các bài đăng có dữ liệu vị trí lớn và phạm vi tìm kiếm rộng.
- Phát triển tính năng gợi ý đối sánh tự động giữa bài đăng báo mất và bài đăng nhặt được dựa trên danh mục, vị trí, thời gian, mô tả và thẻ nhận diện từ hình ảnh.
- Tăng cường các cơ chế bảo mật, xác thực người dùng, kiểm duyệt nội dung và phát hiện hành vi lừa đảo nhằm xây dựng môi trường trao đổi an toàn hơn.
- Hoàn thiện giao diện trên thiết bị di động, cải thiện trải nghiệm người dùng và mở rộng khả năng triển khai thành ứng dụng di động trong tương lai.
- Nghiên cứu tích hợp thêm các mô hình trí tuệ nhân tạo để nhận diện đồ vật chính xác hơn, tự động trích xuất đặc điểm nhận dạng và hỗ trợ quản trị viên trong quá trình kiểm duyệt nội dung.

Nhìn chung, đề tài BeaconFound đã giúp nhóm vận dụng tổng hợp các kiến thức đã học vào một bài toán thực tế, từ phân tích yêu cầu, thiết kế hệ thống đến lựa chọn công nghệ triển khai. Đây là nền tảng quan trọng để nhóm tiếp tục hoàn thiện sản phẩm và phát triển các hệ thống phần mềm có tính ứng dụng cao hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] T. Đ. Quế và N. M. Hùng, Bài giảng Nhập môn Công nghệ phần mềm, Học viện Công nghệ Bưu chính Viễn thông.
- [2] S. Schach, Object-Oriented and Classical Software Engineering, 8th ed. New York, NY, USA: McGraw-Hill, 2012.
- [3] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.
- [4] G. Booch, J. Rumbaugh và I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [5] React, “React Documentation,” [Trực tuyến]. Khả dụng: <https://react.dev/learn/>.
- [6] Node.js, “Node.js Documentation,” [Trực tuyến]. Khả dụng: <https://nodejs.org/docs/latest/api/>.
- [7] Express.js, “Express - Node.js web application framework,” [Trực tuyến]. Khả dụng: <https://expressjs.com/>.
- [8] Socket.IO, “Socket.IO Documentation (v4),” [Trực tuyến]. Khả dụng: <https://socket.io/docs/v4/>.
- [9] Leaflet, “Leaflet Documentation,” [Trực tuyến]. Khả dụng: <https://leafletjs.com/reference.html>.
- [10] PostgreSQL, “PostgreSQL Documentation,” [Trực tuyến]. Khả dụng: <https://www.postgresql.org/docs/>.
- [11] PostGIS, “PostGIS Documentation,” [Trực tuyến]. Khả dụng: <https://postgis.net/documentation/>.
- [12] Cloudinary, “Cloudinary Upload API Reference,” [Trực tuyến]. Khả dụng: https://cloudinary.com/documentation/image_upload_api_reference.
- [13] Firebase, “Firebase Cloud Messaging Documentation,” [Trực tuyến]. Khả dụng: <https://firebase.google.com/docs/cloud-messaging>.
- [14] Google Cloud, “Cloud Vision API Documentation,” [Trực tuyến]. Khả dụng: <https://cloud.google.com/vision/docs>.
- [15] Google Cloud, “Detect Labels - Cloud Vision API,” [Trực tuyến]. Khả dụng: <https://cloud.google.com/vision/docs/labels>.

PHỤ LỤC CÀI ĐẶT, TRIỂN KHAI VÀ KIỂM THỬ:

+ **Thiết lập môi trường:** Máy triển khai cần cài đặt Node.js, npm, PostgreSQL, tiện ích mở rộng PostGIS và trình duyệt web hiện đại. Các dịch vụ ngoài hệ thống gồm Cloudinary, Firebase Cloud Messaging và AI Vision API cần được cấu hình khóa truy cập tương ứng trong biến môi trường của backend.

+ **Cài đặt triển khai hệ thống:** Tiến hành cài đặt thư viện cho frontend và backend, cấu hình kết nối cơ sở dữ liệu PostgreSQL/PostGIS, khởi tạo các bảng dữ liệu chính, sau đó chạy backend API và frontend web client. Khi triển khai thật, hệ thống cần cấu hình HTTPS, tên miền, biến môi trường production và cơ chế sao lưu dữ liệu định kỳ.

+ Hình ảnh sản phẩm.

File thiết kế tương tác (Figma,...):

<https://stitch.withgoogle.com/projects/852331883998363845>

File UML + Source code + file cài build:

<https://github.com/nguyenhuuniem12022005/BeacondFound-T-m-Ki-m-Th-t-L-c>

Kiểm thử hệ thống

1. Phương pháp và kỹ thuật kiểm thử

Nhóm sử dụng kiểm thử hộp đen cho các chức năng nghiệp vụ chính, kết hợp kỹ thuật phân vùng tương đương và kiểm thử giá trị biên để đánh giá khả năng xử lý dữ liệu đầu vào của hệ thống. Các ca kiểm thử bao phủ 10 chức năng chính đã phân tích trong hệ thống BeaconFound, gồm: đăng tải bài viết, tìm kiếm đồ vật, xem báo cáo thống kê, trao đổi tin nhắn, quản lý bài đăng, quản lý báo cáo vi phạm, quản lý danh mục, quản lý hồ sơ cá nhân, quản lý thông báo và báo cáo bài viết/người dùng.

2. Thiết kế và thực hiện ca kiểm thử

a. Chức năng đăng tải bài viết

Đầu vào gồm loại bài đăng, danh mục, tiêu đề, mô tả, thời gian xảy ra sự việc, vị trí liên quan và danh sách ảnh minh chứng. Sau khi dữ liệu đúng định dạng, hệ thống lưu bài viết với trạng thái "Chờ duyệt" để Quản trị viên kiểm tra trước khi hiển thị công khai.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Loại bài đăng	Mất đồ; Nhặt được	Bỏ trống; giá trị ngoài danh sách	-
Danh mục	Danh mục tồn tại trong hệ thống	Bỏ trống; danh mục không tồn tại	-
Tiêu đề	Có ít nhất 1 ký tự hữu ích	Chuỗi rỗng; chỉ chứa khoảng trắng	0 / 1 ký tự

Mô tả	Có ít nhất 1 ký tự hữu ích	Chuỗi rỗng; chỉ chứa khoảng trắng	0 / 1 ký tự
Số lượng ảnh	0 đến 3 ảnh	Nhiều hơn 3 ảnh	0 / 3 / 4 ảnh
Định dạng ảnh	PNG, JPG, JPEG	GIF, BMP, PDF, SVG	-

Bảng PL.1. Phân vùng tương đương và giá trị biên chức năng đăng tải bài viết

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC0 1	Loại "Mất đồ", danh mục "Giấy tờ tùy thân", tiêu đề và mô tả hợp lệ, ghim vị trí hợp lệ, 3 ảnh PNG/JPG	Tạo bài viết thành công, trạng thái "Chờ duyệt", chuyển về giao diện chính	Pass
TC0 2	Bỏ trống loại bài đăng, các trường còn lại hợp lệ	Báo lỗi yêu cầu chọn loại bài đăng, không cho gửi bài	Pass
TC0 3	Không chọn danh mục, các trường còn lại hợp lệ	Báo lỗi yêu cầu chọn danh mục, không cho gửi bài	Pass
TC0 4	Tiêu đề rỗng, các trường còn lại hợp lệ	Báo lỗi tiêu đề không được để trống	Pass
TC0 5	Tiêu đề gồm 1 ký tự hữu ích, các trường còn lại hợp lệ	Chấp nhận dữ liệu, cho phép chuyển bước hoặc gửi bài	Pass
TC0 6	Mô tả chỉ chứa khoảng trắng	Báo lỗi mô tả không hợp lệ, không cho gửi bài	Pass
TC0 7	Tải lên 4 ảnh minh chứng	Báo lỗi vượt quá số lượng ảnh tối đa, không cho gửi bài	Pass
TC0 8	Tải lên tệp GIF làm ảnh minh chứng	Báo lỗi định dạng ảnh không được hỗ trợ	Pass

Bảng PL.2. Ca kiểm thử chức năng đăng tải bài viết

b. Chức năng tìm kiếm đồ vật

Đầu vào gồm từ khóa, bộ lọc danh mục, loại bài đăng, trạng thái, thời gian và thông tin tìm kiếm trên bản đồ. Với tìm kiếm theo bản đồ, người dùng cần cung cấp vị trí trọng tâm và bán kính quét hợp lệ để hệ thống thực hiện truy vấn không gian.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Từ khóa	Chuỗi rỗng hoặc có ký tự tìm kiếm hợp lệ	Chỉ chứa ký tự đặc biệt không có ý nghĩa tìm kiếm	0 / 1 ký tự
Danh mục	Bỏ trống hoặc chọn danh mục tồn tại	Danh mục không tồn tại	-
Loại bài đăng	Bỏ trống; Mất đồ; Nhật được	Giá trị ngoài danh sách	-
Vị trí trọng tâm	Có địa chỉ hoặc tọa độ hợp lệ khi tìm trên bản đồ	Bỏ trống khi dùng tìm kiếm bản đồ	-
Bán kính quét	Số dương	Bằng 0; số âm; không phải số	0 / 1 km

Bảng PL.3. Phân vùng tương đương và giá trị biên chức năng tìm kiếm đồ vật

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC0 1	Từ khóa "ví da nâu", danh mục "Giấy tờ tùy thân"	Hiển thị danh sách bài đăng phù hợp với từ khóa và bộ lọc	Pass
TC0 2	Từ khóa hợp lệ nhưng không có bài đăng phù hợp	Hiển thị thông báo không tìm thấy kết quả	Pass
TC0 3	Từ khóa rỗng, chọn danh mục "Thiết bị điện tử"	Hiển thị các bài đăng thuộc danh mục đã chọn	Pass
TC0 4	Loại bài đăng có giá trị ngoài danh sách	Báo lỗi bộ lọc không hợp lệ hoặc bỏ qua giá trị sai	Pass
TC0 5	Tìm trên bản đồ với vị trí hợp lệ và bán kính 5 km	Hiển thị các Marker bài đăng nằm trong bán kính	Pass
TC0 6	Tìm trên bản đồ nhưng bỏ trống vị trí trọng tâm	Báo lỗi yêu cầu nhập vị trí hoặc chọn tọa độ	Pass
TC0 7	Tìm trên bản đồ với bán kính 0 km	Báo lỗi bán kính không hợp lệ, không thực hiện truy vấn	Pass

Bảng PL.4. Ca kiểm thử chức năng tìm kiếm đồ vật

c. Chức năng xem báo cáo thống kê

Đầu vào gồm khoảng thời gian thống kê và yêu cầu xuất báo cáo. Hệ thống cần truy vấn số liệu người dùng, bài đăng và hiển thị kết quả theo kỳ được chọn.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
---------	------------	------------------	--------------

Khoảng thời gian	Ngày, tuần, tháng hoặc khoảng ngày hợp lệ	Khoảng ngày bắt đầu lớn hơn ngày kết thúc; bỏ trống khi yêu cầu lọc tùy chỉnh	-
Dữ liệu thống kê	Có hoặc không có dữ liệu phát sinh	Dữ liệu truy vấn lỗi hoặc không phản hồi	0 bản ghi / nhiều bản ghi
Định dạng xuất	Excel, PDF	Định dạng ngoài danh sách	-

Bảng PL.5. Phân vùng tương đương và giá trị biên chức năng xem báo cáo thống kê

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC0 1	Chọn thống kê tháng hiện tại, hệ thống có dữ liệu	Hiển thị biểu đồ và bảng số liệu tương ứng	Pass
TC0 2	Chọn khoảng thời gian không có dữ liệu	Hiển thị thông báo chưa có dữ liệu và các chỉ số bằng 0	Pass
TC0 3	Chọn ngày bắt đầu lớn hơn ngày kết thúc	Báo lỗi khoảng thời gian không hợp lệ	Pass
TC0 4	Chọn xuất báo cáo định dạng Excel	Tạo và tải xuống file báo cáo Excel	Pass
TC0 5	Chọn định dạng xuất ngoài danh sách	Báo lỗi định dạng xuất không được hỗ trợ	Pass

Bảng PL.6. Ca kiểm thử chức năng xem báo cáo thống kê

d. Chức năng trao đổi tin nhắn

Đầu vào là yêu cầu mở phòng chat và nội dung tin nhắn. Tin nhắn rỗng hoặc chỉ chứa khoảng trắng sẽ không được gửi; tin nhắn hợp lệ phải được lưu và đồng bộ tới người nhận.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Bài đăng liên quan	Bài đăng tồn tại và đang hoạt động	Bài đăng không tồn tại; bài đăng đã bị ẩn/xóa	-
Người nhận	Chủ bài đăng hợp lệ	Tài khoản bị khóa hoặc không tồn tại	-
Nội dung tin nhắn	Có ít nhất 1 ký tự hữu ích	Chuỗi rỗng; chỉ chứa khoảng trắng	0 / 1 ký tự

Bảng PL.7. Phân vùng tương đương và giá trị biên chức năng trao đổi tin nhắn

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC01	Click "Nhắn tin" từ bài đăng đang hoạt động	Tạo hoặc mở phòng chat với chủ bài đăng	Pass
TC02	Gửi nội dung "Minh nhật được ví giống mô tả của bạn"	Gửi tin nhắn thành công, đồng bộ tới người nhận theo thời gian thực	Pass
TC03	Gửi nội dung "A" (1 ký tự)	Gửi tin nhắn thành công	Pass
TC04	Gửi nội dung rỗng hoặc chỉ có khoảng trắng	Không gửi tin nhắn, nút gửi bị vô hiệu hoặc hệ thống bỏ qua thao tác	Pass
TC05	Mở chat với bài đăng đã bị xóa	Báo lỗi bài đăng không còn khả dụng	Pass

Bảng PL.8. Ca kiểm thử chức năng trao đổi tin nhắn

e. Chức năng quản lý bài đăng

Đầu vào là bài đăng đang chờ duyệt và thao tác xử lý của Quản trị viên. Hệ thống cập nhật trạng thái bài đăng và gửi thông báo phù hợp đến chủ bài viết.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Trạng thái bài đăng	Chờ duyệt	Hoạt động; Đã xóa; Bị ẩn	-
Thao tác xử lý	Xem chi tiết; Duyệt; Từ chối	Thao tác ngoài quyền Quản trị viên	-
Lý do từ chối	Có nội dung khi từ chối	Bỏ trống lý do khi hệ thống yêu cầu	0 / 1 ký tự

Bảng PL.9. Phân vùng tương đương và giá trị biên chức năng quản lý bài đăng

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC01	Quản trị viên mở danh sách bài đăng chờ duyệt	Hiển thị danh sách bài đăng chờ xử lý	Pass
TC02	Duyệt một bài đăng hợp lệ	Chuyển trạng thái bài đăng sang "Hoạt động" và gửi thông báo cho chủ bài	Pass

TC0 3	Từ chối bài đăng có nội dung vi phạm kèm lý do	Bài đăng bị từ chối/xóa, chủ bài nhận thông báo	Pass
TC0 4	Danh sách bài đăng chờ duyệt rỗng	Hiển thị thông báo không có bài đăng nào đang chờ duyệt	Pass
TC0 5	Thành viên thường truy cập chức năng quản lý bài đăng	Từ chối truy cập do không đủ quyền	Pass

Bảng PL.10. Ca kiểm thử chức năng quản lý bài đăng

f. Chức năng quản lý báo cáo vi phạm

Đầu vào là báo cáo vi phạm do thành viên gửi và thao tác xử lý của Quản trị viên. Hệ thống cần hỗ trợ xem chi tiết, bỏ qua, xóa bài đăng hoặc khóa tài khoản vi phạm.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Báo cáo vi phạm	Báo cáo tồn tại, trạng thái chờ xử lý	Báo cáo không tồn tại; đã xử lý	-
Thao tác xử lý	Bỏ qua; Xóa bài đăng; Khóa tài khoản	Thao tác không hợp lệ hoặc không đủ quyền	-
Lý do xử lý	Có nội dung khi khóa/xóa	Bỏ trống nếu hệ thống yêu cầu lý do	0 / 1 ký tự

Bảng PL.11. Phân vùng tương đương và giá trị biên chức năng quản lý báo cáo vi phạm

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC0 1	Quản trị viên mở danh sách báo cáo chờ xử lý	Hiển thị danh sách báo cáo vi phạm	Pass
TC0 2	Mở chi tiết một báo cáo hợp lệ	Hiển thị người gửi, đối tượng bị báo cáo và nội dung vi phạm	Pass
TC0 3	Chọn "Bỏ qua" với báo cáo sai sự thật	Cập nhật trạng thái báo cáo thành "Đã giải quyết"	Pass
TC0 4	Chọn "Xóa bài đăng" với báo cáo hợp lệ	Bài đăng bị xóa/ẩn và báo cáo chuyển sang "Đã giải quyết"	Pass
TC0 5	Chọn "Khóa tài khoản" với tài khoản vi phạm	Tài khoản bị khóa và báo cáo chuyển sang "Đã giải quyết"	Pass

Bảng PL.12. Ca kiểm thử chức năng quản lý báo cáo vi phạm

g. Chức năng quản lý danh mục

Đầu vào gồm tên danh mục, mô tả và thao tác thêm/sửa/xóa của Quản trị viên. Tên danh mục không được bỏ trống và không được trùng với danh mục đã tồn tại.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Tên danh mục	Có ít nhất 1 ký tự hữu ích, chưa tồn tại	Bỏ trống; chỉ chứa khoảng trắng; trùng tên	0 / 1 ký tự
Mô tả	Có thể bỏ trống hoặc có nội dung	Nội dung vượt giới hạn nếu hệ thống đặt giới hạn	-
Thao tác	Thêm; Sửa; Xóa	Thao tác ngoài quyền Quản trị viên	-

Bảng PL.13. Phân vùng tương đương và giá trị biên chức năng quản lý danh mục

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC01	Thêm danh mục "Thú cưng", tên chưa tồn tại	Tạo danh mục mới và cập nhật danh sách	Pass
TC02	Thêm danh mục với tên rỗng	Báo lỗi tên danh mục không được để trống	Pass
TC03	Thêm danh mục với tên đã tồn tại	Báo lỗi danh mục đã tồn tại	Pass
TC04	Sửa mô tả của danh mục hợp lệ	Cập nhật thông tin danh mục thành công	Pass
TC05	Xóa danh mục chưa có bài đăng liên quan	Xóa danh mục và cập nhật danh sách	Pass
TC06	Thành viên thường truy cập chức năng quản lý danh mục	Từ chối truy cập do không đủ quyền	Pass

Bảng PL.14. Ca kiểm thử chức năng quản lý danh mục

h. Chức năng quản lý hồ sơ cá nhân

Đầu vào gồm thông tin cá nhân và thao tác quản lý bài đăng của chính thành viên. Khi sửa bài đăng đã hoạt động, hệ thống cần chuyển bài về trạng thái "Chờ duyệt" để Quản trị viên kiểm duyệt lại.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Họ tên	Có ít nhất 1 ký tự hữu ích	Bỏ trống; chỉ chứa khoảng trắng	0 / 1 ký tự
Email/Số điện thoại	Đúng định dạng	Sai định dạng; trùng với tài khoản khác	-
Bài đăng cần sửa/xóa	Thuộc người dùng hiện tại	Không tồn tại; thuộc người dùng khác	-
Nội dung cập nhật bài đăng	Dữ liệu hợp lệ	Tiêu đề/mô tả rỗng; ảnh sai định dạng	0 / 1 ký tự

Bảng PL.15. Phân vùng tương đương và giá trị biên chức năng quản lý hồ sơ cá nhân

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC01	Thành viên mở hồ sơ cá nhân	Hiện thị thông tin cá nhân và danh sách bài đã đăng	Pass
TC02	Cập nhật họ tên hợp lệ	Lưu thông tin mới và hiện thị thông báo thành công	Pass
TC03	Cập nhật email sai định dạng	Báo lỗi định dạng email, không lưu thay đổi	Pass
TC04	Sửa bài đăng của chính mình với dữ liệu hợp lệ	Cập nhật bài viết, chuyển trạng thái về "Chờ duyệt"	Pass
TC05	Xóa bài đăng của chính mình sau khi xác nhận	Xóa bài đăng và cập nhật danh sách hồ sơ	Pass

Bảng PL.16. Ca kiểm thử chức năng quản lý hồ sơ cá nhân

i. Chức năng quản lý thông báo

Đầu vào là các thông báo phát sinh từ bài đăng, tin nhắn hoặc xử lý của Quản trị viên. Hệ thống cần hiển thị trạng thái đọc/chưa đọc và điều hướng đúng màn hình liên quan.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
---------	------------	------------------	--------------

Thông báo	Thuộc người dùng hiện tại	Không tồn tại; thuộc người dùng khác	-
Trạng thái	Chưa đọc; Đã đọc	Giá trị trạng thái ngoài danh sách	-
Liên kết điều hướng	Bài đăng hoặc phòng chat còn tồn tại	Liên kết tới đối tượng đã bị xóa	-

Bảng PL.17. Phân vùng tương đương và giá trị biên chức năng quản lý thông báo

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC01	Có thông báo tin nhắn mới	Hiển thị dấu hiệu thông báo chưa đọc trên biểu tượng chuông	Pass
TC02	Click một thông báo chưa đọc	Đánh dấu đã đọc và điều hướng đến màn hình liên quan	Pass
TC03	Chọn "Đánh dấu tất cả là đã đọc"	Cập nhật toàn bộ thông báo của người dùng thành "Đã đọc"	Pass
TC04	Click thông báo liên kết tới bài đăng đã bị xóa	Báo đối tượng không còn khả dụng, không gây lỗi hệ thống	Pass

Bảng PL.18. Ca kiểm thử chức năng quản lý thông báo

j. Chức năng báo cáo bài viết/người dùng

Đầu vào gồm đối tượng bị báo cáo và nội dung mô tả vi phạm. Người dùng chỉ gửi được báo cáo khi có lý do hợp lệ; báo cáo sau khi gửi được đưa vào hàng chờ xử lý của Quản trị viên.

Tham số	Lớp hợp lệ	Lớp không hợp lệ	Giá trị biên
Đối tượng báo cáo	Bài viết hoặc người dùng tồn tại	Đối tượng không tồn tại; đã bị xóa	-
Nội dung báo cáo	Có ít nhất 1 ký tự hữu ích	Chuỗi rỗng; chỉ chứa khoảng trắng	0 / 1 ký tự
Người gửi báo cáo	Tài khoản đang hoạt động	Chưa đăng nhập; tài khoản bị khóa	-

Bảng PL.19. Phân vùng tương đương và giá trị biên chức năng báo cáo bài viết/người dùng

Mã ca	Mô tả đầu vào	Kết quả mong đợi	Trạng thái
TC01	Thành viên báo cáo bài viết đang hoạt động với lý do hợp lệ	Tạo báo cáo ở trạng thái "Chờ xử lý"	Pass
TC02	Nội dung báo cáo rỗng	Báo lỗi yêu cầu nhập lý do báo cáo	Pass
TC03	Nội dung báo cáo gồm 1 ký tự hữu ích	Chấp nhận và tạo báo cáo	Pass
TC04	Báo cáo bài viết đã bị xóa	Báo đối tượng không còn tồn tại, không tạo báo cáo	Pass
TC05	Người dùng chưa đăng nhập gửi báo cáo	Yêu cầu đăng nhập trước khi báo cáo	Pass

Bảng PL.20. Ca kiểm thử chức năng báo cáo bài viết/người dùng

3. Đánh giá kết quả kiểm thử

Chức năng	Số ca kiểm thử	Đạt (Pass)	Không đạt (Fail)
Đăng tải bài viết	8	8	0
Tìm kiếm đồ vật	7	7	0
Xem báo cáo thống kê	5	5	0
Trao đổi tin nhắn	5	5	0
Quản lý bài đăng	5	5	0
Quản lý báo cáo vi phạm	5	5	0
Quản lý danh mục	6	6	0
Quản lý hồ sơ cá nhân	5	5	0
Quản lý thông báo	4	4	0
Báo cáo bài viết/người dùng	5	5	0
Tổng cộng	55	55	0

Bảng PL.21. Tổng hợp kết quả kiểm thử

